



UNIVERSIDAD PRIVADA DEL VALLE
FACULTAD DE INFORMÁTICA Y ELECTRÓNICA
CARRERA DE LICENCIATURA EN INGENIERÍA
SISTEMAS

SISTEMA WEB XXXXXXXXXXXXXXXXXXXX

PROYECTO DE GRADO PARA
OPTAR AL TÍTULO DE
LICENCIATURA EN INGENIERÍA
DE SISTEMAS

POSTULANTE: XXXXXXXXX
TUTOR: XXXXXXXXXXXXXXXX

Santa Cruz – Bolivia

DEDICATORIA

A mis padres, que con su presencia silenciosa pero firme han moldeado la manera en que enfrento la vida. Su amor y su ejemplo me enseñaron que la verdadera fortaleza no está en no caer, sino en aprender a levantarse con dignidad.

A mi familia y amigos, quienes dieron color a este camino con su compañía. Cada conversación, cada gesto de apoyo y cada pausa compartida fue un recordatorio de que el crecimiento nunca es un viaje solitario.

A mis docentes, que no solo transmiten conocimientos, sino también miradas distintas sobre el mundo. Gracias por desafiar mis certezas, por enseñarme a cuestionar con respeto y por mostrar que el rigor académico también puede convivir con la empatía.

A la Universidad Privada del Valle, por ser un espacio donde descubrí capacidades que desconocía y donde aprendí que el aprendizaje es un proceso continuo, hecho tanto de aciertos como de tropiezos. Aquí encontré oportunidades que transformaron no solo mi formación profesional, sino también mi manera de comprenderme a mí mismo.

AGRADECIMIENTOS

A la Universidad Privada del Valle, por ofrecer un espacio académico que favoreció mi crecimiento profesional, así como por los recursos, desafíos y oportunidades que hicieron posible culminar esta etapa formativa

Al director de la carrera de Ingeniería de Sistemas Informáticos, el Ing. Sebastián Arze Pando, por su liderazgo cercano, su amistad sincera y el apoyo constante que brindó en cada momento decisivo. Su acompañamiento fue un pilar fundamental durante la carrera.

A mi tutor, el Ing. XXXX, por su orientación clara y comprometida, por señalar con precisión los aspectos a mejorar y por exigir siempre con justicia, permitiendo que este proyecto alcanzará un nivel superior.

Al docente de Taller de Sistemas, la Ing. XXXXXXXX, por su acompañamiento académico, por proveernos todas las herramientas y guías necesarias, y por guiarnos con paciencia, impulsando siempre una visión metodológica ordenada y crítica.

A la empresa xzxxxxxxx por abrirme las puertas de su institución y permitirme comprender de cerca sus procesos. Su disposición para compartir información y necesidades reales fue clave para orientar el proyecto hacia soluciones aplicables y pertinentes.

A mis amigos y compañeros XXXX,XXXXXX por el apoyo mutuo, las conversaciones que aclararon rutas y la compañía constante que hizo más ligero el camino.

A mi familia, por su paciencia inagotable, su aliento permanente y su confianza absoluta. Su presencia fue la fuerza que me motivó a perseverar hasta lograr esta meta.

RESUMEN

Palabras clave: XX.

ABSTRACT

XXXXXXXXXXXXXXXXXX

Keywords: XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

ÍNDICE DE CONTENIDO

CAPÍTULO I MARCO GENERAL

1.1. ANTECEDENTES	1
1.2. PLANTEAMIENTO DEL PROBLEMA	2
1.2.1. FORMULACIÓN DEL PROBLEMA	3
1.2.2. ÁRBOL DEL PROBLEMA	4
1.3. OBJETIVOS	4
1.3.1. OBJETIVO GENERAL	4
1.3.2. OBJETIVOS ESPECÍFICOS	4
1.4. JUSTIFICACIÓN	5
1.4.1. JUSTIFICACIÓN TÉCNICA	5
1.4.2. JUSTIFICACIÓN ECONÓMICA	6
1.4.3. JUSTIFICACIÓN SOCIAL	6
1.5. METODOLOGÍA	7
1.5.1. ENFOQUE DE LA INVESTIGACIÓN	7
1.5.2. TIPO DE INVESTIGACIÓN	7
1.5.3. MÉTODO DE INVESTIGACIÓN	8
1.5.4. TÉCNICAS E INSTRUMENTOS DE INVESTIGACIÓN	8
1.5.5. FUENTES DE INFORMACIÓN	9
1.5.6. PROPUESTA	9

CAPÍTULO II MARCO TEÓRICO

2.1. METODOLOGÍAS DE DESARROLLO DE SOFTWARE	11
2.1.1. EL MODELO INCREMENTAL	11
2.2. ANÁLISIS Y MÉTODOS DE INVESTIGACIÓN	11
2.2.1. ANÁLISIS DE ENTREVISTAS Y REVISIÓN DOCUMENTAL	11
2.2.2. REQUERIMIENTOS FUNCIONALES Y NO FUNCIONALES	12
2.2.2.1. APLICACIÓN GENERAL DE LOS REQUERIMIENTOS EN EL DESARROLLO DE SISTEMAS	13
2.3. MODELADO DE SISTEMAS	13
2.3.1. INTRODUCCIÓN AL UML	13
2.3.2. CASOS DE USO E HISTORIAS DE USUARIO	14
2.3.2.1. DEFINICIÓN Y CARACTERÍSTICAS	14

2.3.2.2. APLICACIÓN GENERAL DE CASOS DE USO Y SU IMPORTANCIA EN EL DISEÑO DE SISTEMAS	15
2.3.3. DIAGRAMA DE SECUENCIA	15
2.3.3.1. DEFINICIÓN Y USO EN EL MODELADO DE SISTEMAS	15
2.3.3.2. EJEMPLO DE APLICACIÓN EN LA GESTIÓN DE PROCESOS	16
2.3.4. MODELO C4 DE ARQUITECTURA DE SOFTWARE	16
2.4. ARQUITECTURA DE SISTEMAS	17
2.4.1. MODELO CLIENTE-SERVIDOR	17
2.4.1.1. DEFINICIÓN Y CARACTERÍSTICAS	17
2.4.1.2. APLICACIÓN GENERAL DEL MODELO EN SISTEMAS WEB	17
2.4.2. ARQUITECTURA DE SOFTWARE EN APLICACIONES WEB	18
2.4.2.1. ARQUITECTURA MONOLÍTICA	18
2.4.2.2. ARQUITECTURA DE MICROSERVICIOS	18
2.4.2.3. ARQUITECTURA EN CAPAS (N-TIER ARCHITECTURE)	19
2.4.2.4. ARQUITECTURA LIMPIA (CLEAN ARCHITECTURE)	20
2.5. BACKEND Y FRONTEND	21
2.5.1. BACKEND: FRAMEWORKS Y CARACTERÍSTICAS	21
2.5.1.1. FRAMEWORKS MONOLÍTICOS	21
2.5.1.2. FRAMEWORKS DE MICROSERVICIOS	22
2.5.1.3. ARQUITECTURAS DE COMUNICACIÓN DE API: REST Y GraphQL	23
2.5.2. FRONTEND: FRAMEWORKS Y CARACTERÍSTICAS	24
2.5.2.1. DESARROLLO BASADO EN COMPONENTES	24
2.5.2.2. ECOSISTEMA DE NEXT.JS Y ESTRATEGIAS DE RENDERIZADO	25
2.5.2.3. GESTIÓN DE ESTADO Y HOOKS EN EL CLIENTE	25
2.6. EXPERIENCIA DE USUARIO (UX) Y ACCESIBILIDAD	26
2.6.1. DISEÑO CENTRADO EN EL USUARIO (USER-CENTERED DESIGN)	26
2.6.2. APLICACIÓN DE UX EN LA INTERACCIÓN DE PLATAFORMAS DIGITALES	27
2.6.3. ACCESIBILIDAD WEB	27
2.6.4. NORMATIVAS Y BUENAS PRÁCTICAS PARA GARANTIZAR LA ACCESIBILIDAD EN PLATAFORMAS	27
2.7. TECNOLOGÍAS DE DESARROLLO	28
2.7.1. BASE DE DATOS Y PERSISTENCIA DE DATOS	28
2.7.1.1. BASES DE DATOS RELACIONALES	28
2.7.1.2. BASES DE DATOS NoSQL	29
2.7.2. TECNOLOGÍAS DE PERSISTENCIA Y SUS PRINCIPIOS	29

2.7.2.1. <i>MAPEO OBJETO-RELACIONAL (ORM)</i>	29
2.7.2.2. <i>ECOSISTEMA DE PRISMA ORM</i>	30
2.7.3. <i>FRAMEWORKS PARA DESARROLLO WEB</i>	31
2.7.3.1. <i>NESTJS</i>	31
2.7.3.2. <i>EXPRESS.JS (NODE.JS)</i>	32
2.7.4. <i>DESPLIEGUE Y CONTENEDORES</i>	32
2.7.4.1. <i>DOCKER</i>	32
2.7.4.2. <i>KUBERNETES</i>	33

CAPÍTULO III INGENIERÍA DEL PROYECTO

3.1. ANÁLISIS DE CONTEXTO OPERATIVO	35
3.1.1. <i>RECOLECCIÓN DE EVIDENCIAS</i>	35
3.1.1.1. <i>Entrevistas y observación directa</i>	35
3.1.1.2. <i>Documentación operativa</i>	36
3.1.1.3. <i>Integración de hallazgos</i>	36
3.1.2.1. <i>Roles y responsabilidades</i>	38
3.1.2.2. <i>Dificultades operativas identificadas</i>	39
3.1.2.3. <i>Limitaciones del proceso manual</i>	39
3.1.3. <i>FLUJO PROPUESTO</i>	40
3.1.3.1. <i>Descripción del proceso objetivo</i>	41
3.1.3.2. <i>Actores y roles propuestos</i>	43
3.1.4. <i>REQUERIMIENTOS FUNCIONALES</i>	44
3.1.5. <i>REQUERIMIENTOS NO FUNCIONALES</i>	47
3.1.5.1. <i>Rendimiento</i>	48
3.1.5.2. <i>Seguridad y privacidad</i>	48
3.1.5.3. <i>Usabilidad y accesibilidad</i>	48
3.1.5.4. <i>Mantenibilidad</i>	49
3.1.6. <i>ALCANCE Y DELIMITACIONES</i>	49
3.1.6.1. <i>Escenarios límites</i>	49
3.1.6.2. <i>Supuestos operativos</i>	50
3.2. SELECCIÓN DE HERRAMIENTAS PARA EL DESARROLLO	50
3.2.1. <i>BACKEND</i>	51
3.2.1.1. <i>Comparación de alternativas</i>	51
3.2.1.2. <i>Decisión y justificación</i>	52

3.2.2. <i>FRONTEND</i>	53
3.2.2.1. <i>Comparación de alternativas</i>	53
3.2.2.2. <i>Decisión y justificación</i>	54
3.2.3. <i>BASES DE DATOS</i>	54
3.2.3.1. <i>Tipos</i>	55
3.2.3.2. <i>Bases de datos relacionales</i>	56
3.2.4. <i>HERRAMIENTAS COMPLEMENTARIAS</i>	57
3.2.5. <i>ENFOQUE METODOLÓGICO ADOPTADO</i>	58
3.2.5.1. <i>Comparación de metodologías</i>	58
3.2.5.2. <i>Elección de la metodología</i>	59
3.3. DISEÑO DEL SISTEMA	59
3.3.1. <i>ARQUITECTURA DE LA SOLUCION</i>	59
3.3.1.1. <i>Vista de contexto</i>	61
3.3.1.2. <i>Vista de contenedores</i>	62
3.3.1.3. <i>Paquetes y organización del código</i>	63
3.3.2. <i>MODELO DE DATOS</i>	70
3.3.2.1. <i>Esquema de base de datos relacional</i>	70
3.3.3. <i>INTERACCIÓN USUARIO-SISTEMA</i>	74
3.3.3.1. <i>Identificación de actores</i>	75
3.3.3.2. <i>Diagramas de secuencia</i>	75
3.3.4. <i>SEGURIDAD Y CUMPLIMIENTO</i>	83
3.3.4.1. <i>Gestión de seguridad de acuerdo a roles</i>	84
3.3.5. <i>PLANIFICACIÓN DE INCREMENTOS PARA LA CONSTRUCCIÓN DEL SISTEMA</i>	84
3.3.5.1. <i>Alcance y objetivo por incremento</i>	85
3.3.5.2. <i>Resumen de cobertura por incremento</i>	87
3.3.5.3. <i>Criterios de cierre de incrementos</i>	88
3.4. CONSTRUCCIÓN DEL SISTEMA	89
3.4.1. <i>INCREMENTO 1: IDENTIDAD, ACCESOS Y PARAMETRIZACIÓN INICIAL</i>	89
3.4.1.1. <i>Alcance del incremento</i>	89
3.4.1.2. <i>Plan de trabajo y estimaciones</i>	90
3.4.1.3. <i>Análisis funcional detallado</i>	91
3.4.1.4. <i>Diseño de solución</i>	94
3.4.1.5. <i>Desarrollo del Incremento</i>	100
3.4.1.6. <i>Entrega y validación</i>	120

3.4.2. INCREMENTO 2: ACTIVACIÓN OPERATIVA Y DISPONIBILIDAD EN TIEMPO REAL	121
3.4.2.1. Alcance del incremento	121
3.4.2.2. Plan de trabajo y estimaciones	122
3.4.2.3. Análisis funcional detallado	123
3.4.2.4. Diseño de solución	125
3.4.2.5. Desarrollo del Incremento	130
3.4.2.6. Entrega y validación	149
3.4.3. INCREMENTO 3: HISTORIAL DE SESIONES DE BÚSQUEDA	150
3.4.3.1. Alcance del incremento	150
3.4.3.2. Plan de trabajo y estimaciones	150
3.4.3.3. Análisis funcional detallado	152
3.4.3.4. Diseño de solución	155
3.4.3.5. Desarrollo del Incremento	159
3.4.3.6. Entrega del Incremento	171
3.4.4. INCREMENTO 4: REPORTES Y LIQUIDACIONES	171
3.4.4.1. Alcance del incremento	172
3.4.4.2. Plan de trabajo y estimaciones	172
3.4.4.3. Análisis funcional detallado	173
3.4.4.4. Diseño de solución	175
3.4.4.5. Desarrollo del incremento	178
3.5. EVALUACIÓN DEL SISTEMA	188
3.5.1. PRUEBAS FUNCIONALES	188
3.5.1.1. Cobertura de casos de uso por incremento	188
3.5.1.2. Reglas de negocio y autorizaciones	189
3.5.2. PRUEBAS TÉCNICAS	190
3.5.2.1. Rendimiento	190
3.5.3. PRUEBAS DE USUARIO	194
CAPÍTULO IV CONCLUSIONES Y RECOMENDACIONES	
4.1. CONCLUSIONES	250
4.2. RECOMENDACIONES	251

ÍNDICE DE FIGURAS

Ilustración 1	Fotografía de planilla manual de ventas	3
Ilustración 2	Diagrama del árbol del problema	4
Ilustración 3	Diagrama de actividad del flujo actual de investigación	38
Ilustración 4	Diagrama de actividad del flujo propuesto con carriles de responsabilidad	42
Ilustración 5	Diagrama de arquitectura en capas de la solución	61
Ilustración 6	Diagrama de contexto (C4 – Nivel 1) de la solución	62
Ilustración 7	Diagrama de contenedores (C4 – Nivel 2) de la solución	63
Ilustración 8	Diagrama de paquetes generales del backend	65
Ilustración 9	Diagrama de paquetes generales del backend	66
Ilustración 10	Diagrama de paquetes generales del frontend	68
Ilustración 11	Diagrama de paquetes generales del frontend	70
Ilustración 12	Modelo entidad-relación del sistema	72
Ilustración 13	Modelo de datos lógico del sistema	74
Ilustración 14	Diagrama de secuencia de gestión de sesión	76
Ilustración 15	Diagrama de secuencia de Disponibilidad y Asignación de Rutas	77
Ilustración 16	Diagrama de secuencia de Selección de Asientos y Creación de Orden	78
Ilustración 17	Diagrama de secuencia de Creación de Venta Rápida	79
Ilustración 18	Diagrama de secuencia de Reembolso de Tickets	80
Ilustración 19	Diagrama de gestión de XXXXX	81
Ilustración 20	Diagrama de gestión de XXXXXXXXXXXXX	82
Ilustración 21	Diagrama de XXXXXXXXXXXXX	83
Ilustración 22	Casos de uso XXXXX XXXXXXXXXXXX	91
Ilustración 27	Servicio de Autenticación	103
Ilustración 28	Servicio login Asincrónico	104
Ilustración 29	Exportación de sesión de Redis	105
Ilustración 30	Método createForUser() con switch de roles	106
Ilustración 31	Servicio de Creación de Rutas	107

Ilustración 32	Servicio de Creación de Rutas	109
Ilustración 33	Reporte de pruebas unitarias de Servicio Usuarios	110
Ilustración 34	Reporte de pruebas unitarias de Servicios de Autenticación	111
Ilustración 35	Reporte de pruebas unitarias de Servicios de Rutas	112
Ilustración 36	Reporte de pruebas unitarias de Servicios de bus	113
Ilustración 37	Pantalla de inicio de sesión en la interfaz web	114
Ilustración 38	Pantalla de inicio de sesión con Credenciales en la interfaz web	114
Ilustración 39	Redirección de Éxito de login	115
Ilustración 40	Credenciales inválidas en la interfaz web	116
Ilustración 41	Listado de usuarios en la interfaz web	117
Ilustración 42	Formulario de creación de usuario en la interfaz web	117
Ilustración 43	Cambio de estado (habilitar/deshabilitar) de un usuario en la interfaz web	118
Ilustración 44	Listado bus layouts	119
Ilustración 45	Creación de un nuevo bus layout	119
Ilustración 46	Creación de Ruta	120
Ilustración 47	Casos de uso cubiertos en el segundo incremento	124
Ilustración 48	Diagrama de paquetes de api del segundo incremento	126
Ilustración 49	Modelo de datos físico para el segundo incremento	128
Ilustración 50	Diagrama de Robustez del Incremento 2	128
Ilustración 51	Método de Generación de Ruta	131
Ilustración 52	Creación Transaccional de Trips y Journeys con Logging	132
Ilustración 53	Ejecución de Pruebas Unitarias de Activación de Ruta	133
Ilustración 54	Interfaz Gráfica de Activación de Rutas	134
Ilustración 55	Selección de Bus para Activar Ruta	134
Ilustración 56	Servicio de Reasignación de Bus y Asientos	136
Ilustración 57	Métodos de Validaciones	138
Ilustración 58	Ejecución de Pruebas Unitarias de Reasignación	139
Ilustración 59	Interfaz de Reasignación de Bus	140
Ilustración 60	Interfaz de Reasignación de asientos	140
Ilustración 61	Método de Consulta de Viaje	142
Ilustración 62	Metodo de Busqueda de Viaje	142
Ilustración 63	Servicio de Consulta de Disponibilidad de Asientos	144

LISTA DE SIGLAS Y ABREVIATURAS

API: Application Programming Interface (interfaz de programación de aplicaciones)

JSON: JavaScript Object Notation (formato de intercambio de datos)

JWT: JSON Web Token (token web en formato JSON)

RBAC: Role-Based Access Control (control de acceso basado en roles)

REST: Representational State Transfer (estilo de arquitectura para servicios web)

SQL: Structured Query Language (lenguaje de consultas estructurado)

UX: User Experience (experiencia de usuario)

CAPÍTULO II

MARCO TEÓRICO

CAPÍTULO II

MARCO TEÓRICO

2.1. METODOLOGÍAS DE DESARROLLO DE SOFTWARE

El desarrollo de sistemas de información requiere la adopción de un proceso estructurado que permita transformar los requisitos del usuario en un producto tecnológico funcional. Según Pressman (2020), un proceso de software proporciona un marco de trabajo que permite la gestión, el diseño y la construcción de software de manera técnica y organizada.

2.1.1. EL MODELO INCREMENTAL

El modelo incremental combina elementos de los flujos de trabajo lineales y paralelos. Se fundamenta en la filosofía de "construir para luego entregar", donde el proyecto se descompone en una serie de entregas denominadas incrementos. Cada incremento proporciona una porción de la funcionalidad total del sistema (Sommerville, 2006).

Características y Ciclo de Vida En este enfoque, el primer incremento suele ser el "núcleo" del sistema, el cual cubre los requisitos básicos. Los incrementos subsiguientes añaden características y funcionalidades adicionales. Cada etapa de un incremento sigue un ciclo de actividades estructurales:

- **Comunicación y Planeación:** Definición de los requisitos específicos del incremento.
- **Modelado (Análisis y Diseño):** Estructuración técnica del módulo.
- **Construcción (Código y Pruebas):** Desarrollo de la lógica y validación de errores.
- **Despliegue y Retroalimentación:** Entrega del incremento al usuario para su evaluación.

2.2. ANÁLISIS Y MÉTODOS DE INVESTIGACIÓN

2.2.1. ANÁLISIS DE ENTREVISTAS Y REVISIÓN DOCUMENTAL

El análisis de entrevistas es una técnica cualitativa utilizada para obtener información detallada y contextualizada sobre un tema específico, en este caso, los procesos de gestión de talleres. Las entrevistas permiten explorar las experiencias, perspectivas y opiniones de los participantes, lo cual es esencial para comprender los procesos y mejorar la toma de decisiones dentro de una organización.

Existen diferentes tipos de entrevistas, siendo las semiestructuradas una de las más comunes en investigaciones sociales y organizacionales. Este tipo de entrevista permite al investigador guiar la conversación hacia ciertos temas clave, pero también ofrece la flexibilidad necesaria para explorar áreas emergentes durante la conversación (Patton, M. Q. 2015).

La revisión documental implica el análisis de documentos existentes que contienen información relevante sobre el objeto de estudio. Esta técnica es esencial para obtener una comprensión profunda del contexto histórico y organizacional, además de ayudar a identificar las políticas, procedimientos y flujos de trabajo previos que afectan el área en cuestión. Los documentos pueden incluir manuales, informes, actas y otros materiales relevantes.

La revisión de documentos proporciona un marco teórico y contextual que complementa la información obtenida a través de entrevistas, ofreciendo una visión más completa y precisa de la situación (Bowen, G. A. 2009).

2.2.2. REQUERIMIENTOS FUNCIONALES Y NO FUNCIONALES

Los requerimientos funcionales son las características y comportamientos específicos que un sistema debe exhibir para cumplir con las expectativas y necesidades de los usuarios y stakeholders. Se refieren a las funciones que el sistema debe ser capaz de realizar para satisfacer los objetivos del proyecto. En términos simples, los requerimientos funcionales describen qué debe hacer el sistema.

Estos requerimientos suelen expresarse de manera detallada, indicando tanto las acciones específicas como las condiciones bajo las cuales estas acciones deben ser ejecutadas. Los requerimientos funcionales son la base sobre la que se construye la lógica del sistema, y su cumplimiento es esencial para asegurar que el producto final cumpla con su propósito (Sommerville, 2006).

Por otro lado, los requerimientos no funcionales se refieren a las características del sistema que no están directamente relacionadas con las funciones que realiza, sino con la manera en que se desempeña. Estos requerimientos describen aspectos como el rendimiento, la usabilidad, la fiabilidad, la seguridad o la escalabilidad. Por ejemplo, dentro de los no funcionales se consideran el tiempo de respuesta del sistema, la facilidad de uso de la plataforma, la protección de los datos sensibles o la capacidad del sistema para adaptarse a un número creciente de usuarios sin perder eficiencia. A diferencia de los funcionales, estos son más

abstractos y se enfocan en atributos globales que afectan la calidad de la experiencia del usuario y la eficiencia general del sistema (Wiegers & Beatty, 2013).

2.2.2.1. Aplicación general de los requerimientos en el desarrollo de sistemas

Los requerimientos, tanto funcionales como no funcionales, son esenciales en las fases iniciales del desarrollo de cualquier sistema. Durante el proceso de desarrollo, es crucial que se identifiquen, documenten y comprendan los requerimientos de manera clara para guiar todas las fases del ciclo de vida del proyecto, desde el análisis hasta la implementación y prueba.

En una primera etapa se lleva a cabo la identificación de requerimientos, que consiste en recoger información mediante entrevistas, revisiones documentales, encuestas a los usuarios y análisis de las necesidades del negocio (Sommerville, 2006). Posteriormente, se procede a la documentación de requerimientos, donde se registran de forma clara y precisa, utilizando formatos que resulten comprensibles tanto para los desarrolladores como para los stakeholders (Wiegers & Beatty, 2013). Una vez documentados, se realiza la especificación de requerimientos, que implica definir con detalle las funcionalidades exactas que debe ofrecer el sistema, así como los límites y expectativas relacionadas con atributos como el rendimiento o la fiabilidad. Finalmente, se llevan a cabo actividades de verificación y validación, destinadas a comprobar que los requerimientos han sido correctamente implementados y que el sistema cumple con las expectativas técnicas y de los usuarios (Sommerville, 2006).

2.3. MODELADO DE SISTEMAS

2.3.1. INTRODUCCIÓN AL UML

El Lenguaje Unificado de Modelado (UML) es un lenguaje estándar utilizado en el desarrollo de sistemas orientados a objetos. UML proporciona un conjunto de diagramas para especificar, visualizar, construir y documentar los aspectos de un sistema de software, permitiendo a los desarrolladores representar tanto la estructura estática como el comportamiento dinámico del sistema. UML es un lenguaje estándar que se utiliza para modelar sistemas de software mediante una variedad de diagramas que abarcan diferentes perspectivas del sistema, desde la estructura hasta la interacción dinámica (Booch et al., 2005)

2.3.2. CASOS DE USO E HISTORIAS DE USUARIO

Los casos de uso son una herramienta esencial en el modelado de sistemas, ya que describen cómo los usuarios interactúan con un sistema para alcanzar un objetivo específico. Los casos de uso permiten describir cómo un sistema interactúa con sus usuarios y actores externos, proporcionando una manera de visualizar los requisitos de manera práctica y comprensible (Fowler, 2004). Estos casos de uso son fundamentales para el desarrollo de los requerimientos funcionales, ya que se centran en las interacciones de los usuarios con el sistema.

Por otro lado, las historias de usuario proporcionan una descripción concisa de una funcionalidad desde la perspectiva del usuario final, como explica Patton (2015), "Las historias de usuario proporcionan una descripción concisa de una funcionalidad desde la perspectiva de un usuario final, permitiendo a los desarrolladores comprender mejor lo que el sistema debe hacer". Este enfoque ágil permite a los desarrolladores captar las necesidades del usuario de forma directa y sencilla.

2.3.2.1. Definición y características

Un caso de uso es una técnica que describe cómo un sistema interactúa con actores externos para lograr un objetivo específico. Un caso de uso define una interacción específica entre un actor y el sistema, describiendo cómo el sistema debe comportarse en respuesta a las acciones del actor (Booch et al., 2005). Un caso de uso se centra en una funcionalidad desde la perspectiva del usuario, representando un conjunto de acciones o pasos que el sistema realiza para satisfacer una necesidad o alcanzar un objetivo determinado.

Las características clave de los casos de uso se centran principalmente en la interacción entre el sistema y el usuario, ya que estos describen cómo el sistema debe responder a las acciones realizadas por el actor. Además, los casos de uso contemplan tanto los escenarios principales como las variaciones o excepciones que pueden presentarse durante la interacción, lo cual permite anticipar diferentes posibilidades en el flujo de trabajo. Finalmente, constituyen una herramienta eficaz para proyectos dirigidos por usuarios, puesto que facilitan la captura de los requisitos funcionales desde su propia perspectiva, garantizando que el sistema refleje de manera adecuada sus necesidades y expectativas.

2.3.2.2. Aplicación general de casos de uso y su importancia en el diseño de sistemas

Los casos de uso son fundamentales en el diseño de sistemas, ya que permiten entender cómo interactúan los usuarios con el sistema y qué funcionalidades deben implementarse. Estos casos proporcionan una visión clara de las necesidades y expectativas del usuario, lo cual es crucial para garantizar que el sistema cumpla con los requisitos funcionales. Según Fowler (2004), "los casos de uso no solo ayudan a identificar los requisitos funcionales, sino que también sirven como base para la planificación de pruebas y validación del sistema".

La aplicación de los casos de uso en el diseño de sistemas abarca diversos ámbitos. En primer lugar, resultan de gran utilidad durante las etapas iniciales del desarrollo de software, ya que permiten identificar y documentar las funcionalidades del sistema de forma estructurada. Asimismo, sirven como base para la planificación de pruebas, pues a partir de ellos es posible diseñar escenarios que validen si el sistema cumple con los requisitos establecidos. Finalmente, los casos de uso contribuyen al diseño de interfaces de usuario más adecuadas, en la medida en que facilitan la creación de interacciones alineadas con las necesidades reales de los usuarios finales.

2.3.3. DIAGRAMA DE SECUENCIA

2.3.3.1. Definición y uso en el modelado de sistemas

El diagrama de secuencia es un tipo de diagrama UML que muestra cómo los objetos del sistema interactúan entre sí a lo largo del tiempo. En un diagrama de secuencia, los objetos o actores del sistema se representan como líneas verticales y las interacciones entre ellos se representan como flechas horizontales. El diagrama de secuencia representa la secuencia temporal de los mensajes intercambiados entre objetos en el sistema (Booch et al., 2005).

El uso de los diagramas de secuencia es crucial en el modelado de sistemas porque permiten visualizar las interacciones temporales, mostrando cómo fluyen los mensajes entre los objetos y en qué orden ocurren. También resultan valiosos para validar procesos complejos, dado que ayudan a representar con detalle la interacción entre los diferentes componentes que intervienen en un procedimiento específico. Además, son una herramienta eficaz para detallar los escenarios de ejecución, ya que describen de manera precisa el flujo de control y las interacciones que se producen durante la ejecución de una función dentro del sistema.

2.3.3.2. *Ejemplo de aplicación en la gestión de procesos*

En el contexto de un sistema de gestión de talleres, un diagrama de secuencia podría modelar el proceso de inscripción de un participante en un taller. En este caso, el diagrama de secuencia podría mostrar cómo el usuario (actor) interactúa con la plataforma (sistema) para registrarse en un taller. El diagrama representaría los pasos, como la selección del taller, el ingreso de los datos del participante, y la confirmación de la inscripción. Los diagramas de secuencia son esenciales para modelar escenarios interactivos y validar que los procesos ocurren de manera correcta y en el orden adecuado (Booch et al., 2005).

Este tipo de diagrama permite identificar posibles cuellos de botella en la interacción y ayuda a garantizar que el sistema responda adecuadamente en todas las situaciones posibles.

2.3.4. *MODELO C4 DE ARQUITECTURA DE SOFTWARE*

El modelo C4 es un enfoque ligero para describir la arquitectura de un sistema a distintos niveles de detalle ("niveles de zoom"). Su objetivo es comunicar con claridad qué es el sistema, quién interactúa con él, cómo se estructura internamente y cómo colaboran sus partes, sin imponer una notación rígida. C4 no sustituye a UML: lo complementa. Mientras UML ofrece un conjunto amplio de diagramas para representar estructura y comportamiento, C4 prioriza la comprensión arquitectónica progresiva y el límite del sistema, de lo externo a lo interno.

C4 organiza la descripción en cuatro niveles. El diagrama de contexto (C1) muestra el sistema como una "caja" dentro de su entorno: actores humanos y sistemas externos que interactúan con él, junto con los flujos de comunicación principales. El diagrama de contenedores (C2) descompone el sistema en sus aplicaciones y almacenes de datos (por ejemplo, cliente web, API, servicios internos y base de datos), define responsabilidades y tecnologías a alto nivel y explicita cómo se comunican. El diagrama de componentes (C3) baja un nivel para describir la estructura interna de un contenedor: componentes lógicos, responsabilidades y relaciones entre ellos. El cuarto nivel, código (C4), es opcional y se utiliza cuando se requiere detallar la implementación de un componente concreto (por ejemplo, clases o paquetes) para decisiones finas de diseño o revisión.

2.4. ARQUITECTURA DE SISTEMAS

2.4.1. MODELO CLIENTE-SERVIDOR

El modelo cliente-servidor es un paradigma de arquitectura de sistemas en el que los procesos se dividen en dos roles principales: cliente y servidor. El cliente es el que inicia las solicitudes de servicios o recursos, mientras que el servidor responde a dichas solicitudes, procesa la información y proporciona los resultados correspondientes. Este modelo permite la centralización de los recursos y facilita la escalabilidad y seguridad del sistema (Bass et al., 2021).

2.4.1.1. Definición y características

Las principales características del modelo cliente-servidor incluyen, en primer lugar, la separación de responsabilidades, donde el cliente se encarga de la presentación y la interacción con el usuario, mientras que el servidor asume el procesamiento de datos y el almacenamiento (Bass et al., 2021). Este modelo destaca también por su escalabilidad, al permitir que múltiples clientes sean atendidos simultáneamente mediante servidores robustos o el uso de balanceadores de carga. Otra característica importante es la centralización de datos, ya que toda la información y procesos críticos se manejan en el servidor, garantizando así la consistencia y seguridad del sistema. Asimismo, ofrece flexibilidad al posibilitar cambios, actualizaciones o mejoras en el servidor sin necesidad de modificar los clientes, lo que facilita el mantenimiento. Finalmente, la comunicación entre cliente y servidor se establece mediante protocolos de red estandarizados, tales como HTTP, TCP/IP o sockets, lo que asegura la interoperabilidad entre distintos entornos tecnológicos.

2.4.1.2. Aplicación general del modelo en sistemas web

En aplicaciones web, el cliente suele ser un navegador o aplicación móvil que solicita recursos al servidor, el cual procesa las solicitudes y devuelve los resultados. Por ejemplo, cuando un usuario accede a un portal web para registrarse en un taller, el cliente envía la información al servidor, que valida los datos, actualiza la base de datos y devuelve una confirmación al usuario. Según Sommerville (2006), "El modelo cliente-servidor facilita la separación entre la interfaz de usuario y la lógica de negocio, permitiendo que los servidores manejen las peticiones de múltiples usuarios simultáneamente y centralizando el control de los datos".

2.4.2. ARQUITECTURA DE SOFTWARE EN APLICACIONES WEB

2.4.2.1. *Arquitectura Monolítica*

La arquitectura monolítica es un modelo tradicional de diseño de sistemas donde todos los componentes de la aplicación (backend, frontend, base de datos, etc.) están integrados en un solo bloque de código. Este enfoque es adecuado para aplicaciones pequeñas o medianas que no requieren una alta escalabilidad.

La arquitectura monolítica presenta ventajas y desventajas que deben considerarse en el desarrollo de sistemas. Entre sus beneficios, se destaca la simplicidad de desarrollo, ya que todos los componentes de la aplicación se encuentran integrados en un único bloque de código, lo que facilita tanto la implementación inicial como la gestión de la infraestructura. Al no requerir múltiples servicios ni redes para la comunicación interna, la administración resulta más sencilla. Además, en aplicaciones pequeñas, esta arquitectura ofrece un mejor rendimiento debido a la interacción directa entre los componentes, lo que evita la latencia de red.

Sin embargo, también presenta limitaciones importantes. Su escalabilidad es reducida, ya que al crecer la aplicación se vuelve complejo aumentar la capacidad sin escalar todos los componentes al mismo tiempo. El mantenimiento puede volverse difícil, ya que la modificación de una parte del sistema puede afectar al resto de la aplicación. A largo plazo, este enfoque puede dificultar la innovación, pues la rigidez de la arquitectura monolítica complica la incorporación de nuevas tecnologías o cambios en los requisitos. En este sentido, Sommerville (2006) señala que una arquitectura monolítica puede resultar eficiente para sistemas pequeños y medianos, pero su mantenimiento y escalabilidad se complican conforme aumenta la complejidad del sistema.

2.4.2.2. *Arquitectura de Microservicios*

La arquitectura de microservicios divide la aplicación en servicios pequeños e independientes, cada uno con su propia lógica y base de datos, que se comunican entre sí a través de interfaces bien definidas, generalmente a través de APIs. Este modelo es adecuado para aplicaciones grandes y distribuidas que requieren una gran escalabilidad y flexibilidad.

La arquitectura de microservicios también presenta ventajas y desventajas significativas. Entre sus beneficios destaca la posibilidad de escalar de manera independiente cada microservicio según las necesidades, lo que permite adaptarse con mayor facilidad a un crecimiento en el

número de usuarios o en el volumen de datos. Este modelo también promueve un desarrollo ágil, ya que diferentes equipos pueden trabajar de forma autónoma en distintos microservicios, acelerando así la implementación de nuevas características. Otro aspecto positivo es su resiliencia, pues si un microservicio falla, el resto del sistema puede seguir funcionando, lo que incrementa la disponibilidad y tolerancia a fallos.

Por otra parte, la arquitectura de microservicios conlleva desafíos. Su gestión es más compleja debido a la necesidad de coordinar la comunicación entre múltiples servicios y administrar diversas bases de datos. La dependencia de la red introduce latencia en las interacciones entre microservicios, lo cual puede afectar el rendimiento en ciertos escenarios. Asimismo, requiere herramientas avanzadas de monitoreo para poder seguir el comportamiento de cada servicio de manera individual. Tal como indica Newman (2021), esta arquitectura proporciona flexibilidad, escalabilidad y resistencia al permitir que cada componente del sistema evolucione de forma independiente, aunque exige un mayor esfuerzo de gestión y supervisión.

2.4.2.3. *Arquitectura en Capas (N-Tier Architecture)*

La arquitectura en capas organiza el sistema en niveles horizontales de responsabilidad, donde cada capa tiene un rol específico y se comunica únicamente con la capa inmediata inferior. Este enfoque se basa en el principio de "separación de preocupaciones", permitiendo que los cambios en una capa tengan un impacto mínimo en las demás (Pressman & Maxim, 2020).

En el desarrollo de sistemas empresariales modernos, las capas fundamentales suelen dividirse en:

- Capa de Presentación: Gestiona la interfaz y la interacción con el usuario.
- Capa de Aplicación o Negocio: Contiene la lógica de los procesos y las reglas operativas.
- Capa de Infraestructura o Datos: Responsable de la persistencia y comunicación con recursos externos.

Entre sus beneficios destaca la mantenibilidad, ya que permite que diferentes equipos trabajen de forma independiente en el frontend y el backend, facilitando la evolución del sistema. Asimismo, ofrece una mayor seguridad por aislamiento, debido a que la capa de datos no tiene exposición directa con la interfaz, obligando a que toda petición sea validada por la lógica de negocio.

Como desventaja, este modelo puede introducir una latencia adicional debido al flujo de datos a través de múltiples niveles (efecto cascada). Además, en sistemas muy básicos, puede generar una sobrecarga de desarrollo innecesaria al requerir la creación de múltiples estructuras para tareas simples de consulta.

2.4.2.4. *Arquitectura Limpia (Clean Architecture)*

La Arquitectura Limpia, propuesta por Martin (2017), es un modelo de diseño que busca el desacoplamiento total de la lógica de negocio respecto a los detalles técnicos, como los frameworks, la base de datos o la interfaz de usuario. Su principio fundamental es la Regla de Dependencia, que establece que las dependencias de código solo pueden apuntar hacia el núcleo o centro del sistema (la lógica de mayor nivel).

Los niveles concéntricos definidos por Martin (2017) incluyen:

- Entidades: Reglas de negocio globales que no cambian ante modificaciones externas.
- Casos de Uso: Orquestan el flujo de datos y ejecutan las reglas específicas de la aplicación.
- Adaptadores de Interfaz: Convierten los datos del formato más conveniente para las entidades al formato requerido por agentes externos (APIs, UI).

Su principal fortaleza es la independencia del framework, lo que garantiza que el núcleo del sistema sea agnóstico a las herramientas tecnológicas (como NestJS o Prisma), permitiendo actualizaciones de librerías sin afectar la lógica central. Además, maximiza la testabilidad, al permitir realizar pruebas unitarias sobre los procesos críticos del negocio sin necesidad de contar con una base de datos o un servidor activo.

La implementación de esta arquitectura conlleva una curva de aprendizaje elevada y una mayor complejidad inicial en el código, debido a la necesidad de definir interfaces y adaptadores para mantener el desacoplamiento, lo que puede percibirse como redundancia en proyectos de menor escala.

2.5. BACKEND Y FRONTEND

2.5.1. BACKEND: FRAMEWORKS Y CARACTERÍSTICAS

El backend es la parte del sistema que maneja la lógica de negocio, la gestión de bases de datos y la comunicación con el frontend. Los frameworks de backend proporcionan un conjunto de herramientas y bibliotecas que facilitan el desarrollo de estas funcionalidades. Según Sommerville (2006), los frameworks de backend permiten gestionar tareas repetitivas como el enrutamiento de solicitudes, la interacción con bases de datos y la implementación de medidas de seguridad, lo que reduce la necesidad de escribir código desde cero y permite a los desarrolladores concentrarse en la lógica de negocio.

Características principales de los frameworks de backend incluyen la capacidad para gestionar rutas de solicitudes HTTP, realizar operaciones en bases de datos mediante ORMs (Object-Relational Mapping), y ofrecer funcionalidades para la autenticación y la protección de datos (Wiegers & Beatty, 2013). Estos marcos de trabajo permiten una gestión eficiente de las solicitudes y respuestas, lo que facilita la creación de aplicaciones web robustas y seguras.

2.5.1.1. *Frameworks Monolíticos*

Los frameworks monolíticos son aquellos en los que todos los componentes de una aplicación (frontend y backend) están fuertemente integrados dentro de una misma unidad. Este enfoque centralizado facilita el desarrollo y la implementación, especialmente en proyectos más pequeños. Frameworks como Django (Python) o Ruby on Rails siguen esta filosofía, proporcionando herramientas listas para usar para gestionar desde la base de datos hasta la interfaz de usuario.

Los frameworks monolíticos presentan diversas ventajas para el desarrollo de sistemas. Una de las más destacadas es la posibilidad de un desarrollo rápido, ya que al integrar todas las funciones dentro de una misma estructura, los desarrolladores pueden avanzar de manera más ágil y eficiente, sobre todo en aplicaciones pequeñas o medianas. Asimismo, este enfoque ofrece simplicidad en la gestión del proyecto, dado que al estar todo centralizado no es necesario coordinar múltiples servicios o componentes independientes, lo que facilita la comunicación entre las distintas partes del sistema. Otra ventaja importante radica en que generan una menor sobrecarga de infraestructura, puesto que al trabajar con un único bloque de código se evita la configuración y el mantenimiento de numerosos servicios, reduciendo así la complejidad técnica.

No obstante, también presentan limitaciones. Una de ellas es la escalabilidad reducida, ya que cuando la aplicación crece en usuarios o funcionalidades resulta complejo aumentar la capacidad de manera selectiva, dado que todos los componentes deben escalarse al mismo tiempo aunque solo uno requiera más recursos. A ello se suma un mantenimiento más complicado a largo plazo, pues a medida que se incorporan nuevas características, la estructura monolítica tiende a volverse más rígida y difícil de gestionar. Los cambios en una parte del sistema pueden impactar en otras, aumentando la posibilidad de errores. Finalmente, la adopción de nuevas tecnologías suele ser más difícil dentro de este modelo, ya que introducir modificaciones significativas requiere reestructurar grandes secciones de la aplicación. En palabras de Sommerville (2006), este enfoque es apropiado para proyectos pequeños o medianos que no demandan alta escalabilidad, pero puede tornarse problemático conforme la aplicación crece.

2.5.1.2. *Frameworks de Microservicios*

Los frameworks de microservicios se basan en una arquitectura distribuida donde una aplicación se divide en varios servicios pequeños e independientes. Cada servicio realiza una tarea específica y se comunica con otros a través de APIs. Frameworks como Spring Boot (Java) o Express (Node.js) son ampliamente utilizados para construir aplicaciones basadas en microservicios, permitiendo que cada servicio evolucione de manera independiente.

Los frameworks de microservicios aportan importantes beneficios en el desarrollo de sistemas complejos. Uno de los principales es la escalabilidad independiente, ya que cada servicio puede ajustarse de manera aislada en función de su demanda, lo que permite optimizar el uso de los recursos sin necesidad de escalar toda la aplicación. También destacan por su flexibilidad, puesto que cada microservicio puede ser diseñado, implementado y mantenido por distintos equipos utilizando las tecnologías más adecuadas para cada caso, lo que facilita la incorporación de innovaciones tecnológicas. Además, ofrecen una mayor resiliencia, dado que si un servicio falla los demás pueden continuar funcionando, asegurando la estabilidad global del sistema y permitiendo una recuperación más rápida frente a errores.

Sin embargo, esta arquitectura también conlleva desafíos. La gestión de múltiples servicios genera una complejidad considerable, ya que es necesario contar con una infraestructura robusta para manejar la comunicación entre ellos mediante APIs o sistemas de mensajería distribuida, lo cual puede ser costoso y demandante. Otro inconveniente es la latencia, ya que la interacción entre servicios a través de la red puede ralentizar el rendimiento de la aplicación. A

esto se suma que la implementación inicial suele ser más exigente, requiriendo mayor tiempo y esfuerzo para organizar los diferentes componentes. Como señala Newman (2021), la arquitectura de microservicios resulta idónea para sistemas grandes y complejos porque permite escalar cada servicio de forma independiente y facilita la incorporación de nuevas características, aunque su complejidad operativa es más alta que la de los modelos monolíticos.

2.5.1.3. *Arquitecturas de Comunicación de API: REST y GraphQL*

La comunicación entre el backend y el frontend se gestiona a través de interfaces de programación de aplicaciones (APIs), las cuales definen las reglas y protocolos para el intercambio de datos. En el desarrollo moderno, las dos arquitecturas predominantes son REST (*Representational State Transfer*) y GraphQL. Mientras que REST se basa en recursos identificados por URLs únicas y métodos HTTP estandarizados, GraphQL propone un lenguaje de consulta que permite al cliente definir con precisión la estructura de los datos requeridos.

Para el funcionamiento de GraphQL, son fundamentales los siguientes componentes:

- Esquema (Schema): Actúa como un contrato fuertemente tipado que define las operaciones permitidas y las relaciones entre los tipos de datos disponibles en el servidor.
- Resolvers: Funciones encargadas de vincular los campos del esquema con las fuentes de datos reales, permitiendo la resolución de consultas complejas de forma modular.

La principal ventaja de GraphQL es la optimización en la transferencia de datos, ya que elimina los problemas de *over-fetching* (recepción de datos innecesarios) y *under-fetching* (insuficiencia de datos en una sola petición). En entornos con estructuras de datos relacionales complejas o profundamente anidadas, esta arquitectura permite consolidar la información en una única transacción de red, reduciendo significativamente la latencia y el consumo de ancho de banda. Asimismo, su sistema de tipos estáticos mejora la consistencia del desarrollo y facilita la documentación automática del contrato de comunicación.

Como contraparte, GraphQL introduce una mayor complejidad operativa en el servidor en comparación con la naturaleza simplificada de REST. La implementación de mecanismos de caché a nivel de red resulta más sofisticada debido a que las peticiones suelen utilizar un único punto de entrada (*endpoint*). Además, requiere un control estricto de la profundidad de las

consultas para evitar una sobrecarga excesiva en el procesamiento del servidor ante peticiones mal formadas o excesivamente extensas.

Según Banks y Porcello (2022), "GraphQL desplaza el control de los datos del servidor hacia el cliente, permitiendo que las aplicaciones frontend sean más rápidas y resilientes al solicitar solo la información estrictamente necesaria para su renderizado".

2.5.2. *FRONTEND: FRAMEWORKS Y CARACTERÍSTICAS*

El frontend es la parte del sistema con la que los usuarios interactúan directamente. Los frameworks de frontend proporcionan las herramientas necesarias para construir interfaces de usuario interactivas y dinámicas. Según Garrett (2010), los frameworks frontend permiten a los desarrolladores crear interfaces reactivas, es decir, aquellas que responden en tiempo real a las interacciones del usuario sin necesidad de recargar la página.

Características principales de frameworks frontend incluyen la capacidad de dividir la interfaz en componentes reutilizables, lo que mejora la organización y el mantenimiento del código (Garrett, 2010). Los frameworks como React.js o Vue.js son particularmente efectivos en este aspecto, permitiendo a los desarrolladores crear aplicaciones que pueden adaptarse a los cambios de estado sin tener que reconstruir la interfaz completa.

La experiencia de usuario (UX) es clave en el diseño frontend, ya que determina cómo los usuarios interactúan con la interfaz. Un diseño centrado en el usuario se enfoca en garantizar que la interacción con la plataforma sea intuitiva, eficiente y agradable. Según Norman (2013), "El diseño UX mejora la conectividad emocional entre el usuario y el producto, lo que se traduce en mayor satisfacción y lealtad del usuario". La accesibilidad y la facilidad de navegación son factores cruciales para asegurar que la plataforma sea útil para una amplia gama de usuarios.

2.5.2.1. *Desarrollo Basado en Componentes*

El desarrollo basado en componentes es un paradigma de diseño de interfaces de usuario que consiste en dividir la aplicación en piezas pequeñas, independientes y reutilizables. Cada componente encapsula su propia estructura (HTML), estilo (CSS) y lógica de comportamiento (JavaScript), lo que permite construir interfaces complejas mediante la composición de unidades simples.

La principal ventaja de este enfoque es la reutilización de código, permitiendo que un mismo elemento de la interfaz sea empleado en múltiples secciones del sistema, lo que reduce la redundancia y asegura la consistencia visual. Asimismo, facilita el mantenimiento modular, ya que las modificaciones realizadas en un componente específico no afectan al resto de la aplicación, agilizando los procesos de actualización y corrección de errores.

Como desventaja, requiere una planificación arquitectónica más rigurosa para definir correctamente la jerarquía y comunicación entre componentes. En aplicaciones pequeñas, este modelo puede generar una fragmentación excesiva del código, aumentando la complejidad técnica innecesariamente.

2.5.2.2. Ecosistema de Next.js y Estrategias de Renderizado

Next.js es un framework de React que optimiza el rendimiento de las aplicaciones web mediante diversas estrategias de renderizado y una estructura de enrutamiento avanzada. A diferencia de las aplicaciones tradicionales de una sola página (SPA), Next.js permite el Renderizado del lado del Servidor (SSR), donde el contenido HTML se genera en el servidor para cada solicitud, mejorando el posicionamiento en buscadores (SEO) y el tiempo de carga inicial.

Características de Enrutamiento:

- App Router: Modelo moderno basado en componentes de servidor (React Server Components) que permite una gestión de rutas más eficiente y una carga de datos optimizada a nivel de layout.
- Pages Router: Modelo tradicional basado en el sistema de archivos donde cada archivo en una carpeta específica corresponde a una ruta de la aplicación.

Destaca por su capacidad de optimización automática, que incluye la división de código (code splitting) y la carga perezosa (lazy loading), cargando únicamente el JavaScript necesario para la página que el usuario está visualizando. Esto se traduce en una experiencia de usuario fluida y un rendimiento superior en dispositivos con recursos limitados.

2.5.2.3. Gestión de Estado y Hooks en el Cliente

La gestión de estado se refiere a la administración de los datos que cambian a lo largo del tiempo durante la interacción del usuario. React emplea los Hooks (como `useState` y `useEffect`)

como funciones especiales que permiten a los desarrolladores "engancharse" al ciclo de vida y al estado de los componentes sin necesidad de escribir clases.

Para la gestión de información compartida entre múltiples niveles de la aplicación, se utiliza la Context API. Esta herramienta permite centralizar el estado global, evitando el paso manual de propiedades a través de componentes intermedios (prop drilling).

Proporciona una reactividad eficiente, asegurando que la interfaz de usuario se actualice automáticamente ante cualquier cambio en los datos subyacentes. La centralización mediante contextos mejora la predictibilidad del flujo de datos, facilitando la depuración y garantizando que la información (como el estado de una sesión o preferencias de usuario) sea consistente en toda la plataforma.

2.6. EXPERIENCIA DE USUARIO (UX) Y ACCESIBILIDAD

2.6.1. DISEÑO CENTRADO EN EL USUARIO (USER-CENTERED DESIGN)

El Diseño Centrado en el Usuario (User-Centered Design, UCD) es un enfoque que coloca al usuario como eje fundamental durante todo el proceso de desarrollo de un producto. Este modelo asegura que las decisiones de diseño se fundamenten en las necesidades, expectativas y limitaciones de los usuarios, con el fin de crear soluciones intuitivas, fáciles de utilizar y eficaces. Más allá de optimizar la funcionalidad, también refuerza la conexión emocional entre el usuario y el producto, lo que genera confianza y una mayor satisfacción (Norman, 2013).

Los principios del diseño de experiencia de usuario se basan en varios aspectos esenciales. En primer lugar, la usabilidad establece que el producto debe ser fácil de aprender y de utilizar, minimizando la complejidad en la interacción. A esto se suma la satisfacción del usuario, que busca garantizar que la solución cumpla de manera eficiente con sus necesidades y expectativas. Finalmente, la accesibilidad ocupa un lugar central, ya que asegura que el producto pueda ser utilizado por la mayor cantidad de personas posible, independientemente de sus capacidades o limitaciones.

Adoptar este enfoque conlleva beneficios significativos. Entre ellos se encuentra la mejora en la satisfacción y la retención de usuarios, la reducción de los costos de soporte y capacitación, así como un incremento en la lealtad hacia el producto, ya que los usuarios perciben que han sido tomados en cuenta en el diseño y se sienten comprendidos en sus necesidades.

2.6.2. APLICACIÓN DE UX EN LA INTERACCIÓN DE PLATAFORMAS DIGITALES

La experiencia de usuario desempeña un papel fundamental en las plataformas digitales, ya que permite garantizar una interacción eficiente, efectiva y satisfactoria. Un diseño adecuado de UX se traduce en interfaces intuitivas, en las que los usuarios pueden desenvolverse sin necesidad de invertir tiempo excesivo en comprender cómo utilizar el sistema.

Dentro de los aspectos más relevantes en la aplicación de UX se encuentra el diseño de interacción, orientado a construir interfaces claras y fáciles de usar, en las que las tareas puedan completarse sin esfuerzo ni confusión. Otro aspecto esencial es la accesibilidad, que asegura que todas las personas, incluyendo aquellas con discapacidades, puedan utilizar la plataforma de manera efectiva. Asimismo, la realización de pruebas con usuarios reales constituye una práctica necesaria para evaluar la efectividad de la interfaz y realizar mejoras continuas. Como señala Garrett (2010), la esencia del diseño UX radica en comprender las necesidades y comportamientos de los usuarios para que la interfaz resulte intuitiva, eficiente y satisfactoria.

2.6.3. ACCESIBILIDAD WEB

La accesibilidad web implica garantizar que las plataformas digitales puedan ser utilizadas por todas las personas, incluidas aquellas con algún tipo de discapacidad. Para ello es necesario adaptar los elementos de la interfaz y, cuando corresponde, incorporar tecnologías de asistencia que faciliten la interacción de usuarios con limitaciones visuales, auditivas, motoras o cognitivas.

Este enfoque se sostiene en varios conceptos generales. El diseño inclusivo busca que las plataformas sean utilizables por una amplia gama de personas sin necesidad de adaptaciones posteriores. La adaptabilidad, por su parte, exige que las plataformas permitan ajustes en la visualización, como el aumento de texto o la modificación de colores, así como opciones de navegación alternativas. Finalmente, las tecnologías de asistencia, entre las que se incluyen los lectores de pantalla o teclados especializados, cumplen un rol esencial en permitir el acceso equitativo al contenido digital.

2.6.4. *NORMATIVAS Y BUENAS PRÁCTICAS PARA GARANTIZAR LA ACCESIBILIDAD EN PLATAFORMAS*

Con el fin de estandarizar y promover la accesibilidad, existen normativas internacionales como las WCAG (Web Content Accessibility Guidelines), que ofrecen lineamientos específicos para asegurar que las plataformas web sean inclusivas. Estas normas contemplan varios principios. La perceptibilidad exige que el contenido sea presentado de manera accesible, por ejemplo, mediante subtítulos en videos o textos alternativos en imágenes. La operabilidad se centra en que los usuarios puedan interactuar con la interfaz a través de diferentes herramientas, como teclados alternativos o dispositivos adaptados. La comprensibilidad establece que la navegación y el contenido deben ser claros y coherentes para evitar confusión. Finalmente, la robustez hace referencia a la necesidad de que las plataformas funcionen correctamente en distintos dispositivos y sean compatibles con diversas tecnologías de asistencia.

Un ejemplo concreto de estas directrices es la exigencia de que toda imagen incorpore un texto alternativo (alt-text), de modo que las personas con discapacidad visual puedan acceder a la información mediante lectores de pantalla. Asimismo, los formularios deben ser diseñados de manera que resulten fáciles de navegar y completar sin requerir interacciones visuales complejas. En este sentido, las WCAG constituyen un marco esencial para garantizar que el contenido digital sea accesible para una amplia diversidad de usuarios (W3C, 2018).

2.7. *TECNOLOGÍAS DE DESARROLLO*

2.7.1. *BASE DE DATOS Y PERSISTENCIA DE DATOS*

La persistencia de datos se refiere a la capacidad de un sistema para almacenar información de manera duradera, incluso después de que el sistema haya sido cerrado o reiniciado. En el desarrollo de plataformas web, se utilizan bases de datos para gestionar y almacenar esta información de manera eficiente.

2.7.1.1. *Bases de datos Relacionales*

Las bases de datos relacionales son el tipo más tradicional y común de base de datos. Organizan los datos en tablas con filas y columnas, donde cada fila representa un registro y cada columna contiene atributos del registro. Estas bases de datos utilizan el lenguaje SQL (Structured Query Language) para realizar consultas y manipulaciones de datos.

Las bases de datos relacionales son ideales cuando se necesita mantener integridad referencial entre los datos. Por ejemplo, en una plataforma de gestión de talleres, podrías tener una tabla de participantes y una tabla de talleres, donde la relación entre ambos se maneja mediante claves foráneas (foreign keys).

Una característica clave de las bases de datos relacionales es que la estructura de los datos está definida de manera rígida. Esto garantiza que los datos sean consistentes y bien organizados, pero también puede hacer que sea más difícil manejar datos no estructurados o cambian frecuentemente. Ejemplos comunes de bases de datos relacionales incluyen MySQL, PostgreSQL, y Oracle.

2.7.1.2. *Bases de datos NoSQL*

Las bases de datos NoSQL son una alternativa a las bases de datos relacionales, especialmente cuando se trata de manejar grandes volúmenes de datos no estructurados o semi-estructurados. A diferencia de las bases de datos relacionales, NoSQL no requiere un esquema fijo para almacenar los datos, lo que ofrece mayor flexibilidad para adaptarse a cambios frecuentes en la estructura de los datos.

Existen varios tipos de bases de datos NoSQL, cada una diseñada para manejar distintos tipos de datos. Por ejemplo, las bases de datos de documentos, como MongoDB, almacenan la información en formato JSON, lo que permite estructuras flexibles y dinámicas. Las bases de datos de clave-valor, como Redis, organizan la información en pares clave-valor, lo cual es especialmente útil para operaciones rápidas y almacenamiento en caché. Finalmente, las bases de datos orientadas a columnas, como Cassandra, registran los datos en columnas en lugar de filas, lo que optimiza las consultas sobre grandes volúmenes de información.

Las bases de datos NoSQL ofrecen una gran flexibilidad y escalabilidad, lo que las hace adecuadas para aplicaciones que requieren alta disponibilidad y manejo de grandes volúmenes de datos no estructurados (Sommerville, 2006).

2.7.2. *TECNOLOGÍAS DE PERSISTENCIA Y SUS PRINCIPIOS*

Las tecnologías de persistencia son herramientas que facilitan el almacenamiento y la recuperación de datos de las bases de datos, proporcionando una interfaz de programación más fácil de usar que interactúa directamente con el sistema de base de datos.

2.7.2.1. Mapeo Objeto-Relacional (ORM)

El ORM (*Object-Relational Mapping*) es una técnica de programación que permite convertir datos entre sistemas de tipo incompatibles utilizando lenguajes de programación orientados a objetos. Esta capa de abstracción crea una "base de datos virtual de objetos", permitiendo a los desarrolladores interactuar con las tablas de una base de datos relacional como si fueran objetos locales del lenguaje (Sadalage & Fowler, 2012).

Entre sus beneficios destaca la productividad, al automatizar la escritura de consultas CRUD (Crear, Leer, Actualizar, Borrar) y reducir el código repetitivo. Asimismo, mejora la seguridad, ya que la mayoría de los ORMs integran protecciones nativas contra ataques de inyección SQL mediante la parametrización automática de consultas. Finalmente, facilita la portabilidad, permitiendo cambiar el motor de base de datos (por ejemplo, de MySQL a PostgreSQL) con ajustes mínimos en el código de la aplicación.

Como desventaja, el uso de un ORM puede introducir una sobrecarga de rendimiento en consultas extremadamente complejas o masivas debido a la capa de traducción. Además, la abstracción puede ocultar lo que sucede a nivel de base de datos, lo que en ocasiones dificulta la optimización de consultas específicas que un administrador de base de datos podría realizar manualmente.

2.7.2.2. Ecosistema de Prisma ORM

Prisma es un ORM de última generación diseñado para Node.js y TypeScript que se diferencia de los modelos tradicionales por su enfoque basado en esquemas declarativos y su fuerte tipado. A diferencia de otros ORMs que definen modelos en clases de código, Prisma utiliza un archivo de configuración central (*schema.prisma*) para definir la estructura de datos y las relaciones.

Características Principales:

- Enfoque Schema-first: La estructura de la base de datos se define en un lenguaje de modelado de datos (DML) legible, que sirve como la única fuente de verdad para el esquema de la base de datos y los tipos del lenguaje de programación.
- Prisma Client: Un generador de consultas que proporciona un autocompletado total y seguridad de tipos en tiempo de compilación, garantizando que las consultas coincidan exactamente con la estructura de la base de datos.

- Prisma Migrate: Una herramienta de migración declarativa que transforma automáticamente el esquema de modelado en comandos SQL de alteración de tablas, manteniendo un historial de cambios rastreable.

Su principal fortaleza es la seguridad de tipos (Type-safety), que previene errores comunes de desarrollo al validar las consultas antes de que se ejecuten. Además, su capacidad de gestión de migraciones automatizada asegura que todos los entornos (desarrollo, prueba y producción) mantengan una estructura de datos idéntica, facilitando el despliegue continuo.

Requiere un paso adicional de generación de código cada vez que el esquema cambia, lo que puede ralentizar ligeramente el flujo de trabajo inicial. Al ser una tecnología más reciente, su comunidad y ecosistema de complementos, aunque crecientes, son menores en comparación con herramientas más veteranas como TypeORM o Sequelize.

2.7.3. FRAMEWORKS PARA DESARROLLO WEB

En el desarrollo web, existen diversos frameworks que simplifican la creación de aplicaciones web. Estos marcos proporcionan una estructura que facilita tareas comunes, como la gestión de bases de datos, la autenticación, y el enrutamiento de solicitudes, lo que permite a los desarrolladores concentrarse en la lógica de negocio.

2.7.3.1. NestJS

NestJS es un framework para construir aplicaciones del lado del servidor sobre Node.js utilizando TypeScript. Propone una arquitectura modular con inversión de control e inyección de dependencias, inspirada en patrones empresariales que favorecen la separación por dominios y la mantenibilidad a largo plazo. Funciona por defecto sobre Express y permite, cuando el rendimiento es una prioridad, usar Fastify como adaptador.

El framework organiza la aplicación en módulos, controladores y servicios, y añade elementos transversales como pipes, guards, interceptors y filters. Con estos mecanismos se cubren necesidades habituales: validación de datos con esquemas (por ejemplo, class-validator), autenticación con JWT y Passport, autorización basada en roles (RBAC), transformación y saneamiento de entradas, manejo uniforme de errores, registro de actividades y observabilidad. La integración con el ecosistema de datos es directa: se trabaja con Prisma, TypeORM o Mongoose, y se dispone de utilidades oficiales para la gestión de configuración, cacheo y documentación de la API con OpenAPI/Swagger. La CLI acelera el scaffolding y el framework

se acompaña bien con Jest para pruebas unitarias e integración. Además, cuando el caso de uso lo requiere, incorpora modelos de comunicación para microservicios (gRPC, Kafka, RabbitMQ) y soporte de WebSockets.

Entre sus ventajas destacan la arquitectura modular y escalable, el tipado estático que reduce errores en tiempo de compilación, la productividad derivada de convenciones claras y herramientas de scaffolding, y la facilidad para implantar buenas prácticas de seguridad y calidad desde el inicio. Como contrapartida, presenta una curva de aprendizaje más pronunciada que Express “puro”, un arranque con algo más de boilerplate y una ligera sobrecarga por sus capas de abstracción; no obstante, estos costos iniciales se compensan a medida que el proyecto crece en funcionalidades y equipo.

2.7.3.2. *Express.js (Node.js)*

Express.js es un framework minimalista para Node.js que proporciona las herramientas básicas necesarias para desarrollar aplicaciones web y APIs. Express es ligero y flexible, permitiendo que los desarrolladores personalicen completamente la aplicación según sus necesidades.

Express.js presenta características clave como el uso de middleware para gestionar solicitudes y respuestas HTTP, lo que permite personalizar el flujo de datos dentro de la aplicación. También ofrece un sistema de enrutamiento sencillo para manejar las rutas y se integra fácilmente tanto con bases de datos relacionales como con bases de datos NoSQL.

Entre sus ventajas destacan la flexibilidad y ligereza, que lo hacen ideal para aplicaciones escalables y de alto rendimiento. Asimismo, es rápido y eficiente en la gestión de solicitudes. No obstante, presenta como desventaja la necesidad de configurar manualmente muchas de las tareas comunes que en otros frameworks ya están resueltas, lo que puede dificultar su mantenimiento en aplicaciones de gran escala debido a su naturaleza minimalista.

2.7.4. *DESPLIEGUE Y CONTENEDORES*

El despliegue es el proceso mediante el cual las aplicaciones se ponen en marcha en un entorno de producción, lo que permite a los usuarios acceder a ellas.

2.7.4.1. *Docker*

Docker es una plataforma que permite crear, desplegar y ejecutar aplicaciones dentro de contenedores. Un contenedor es una unidad de software que empaqueta el código y todas sus dependencias, permitiendo que la aplicación se ejecute de manera consistente en cualquier entorno, sin importar el sistema operativo subyacente.

Docker se caracteriza por su portabilidad, ya que los contenedores pueden ejecutarse en cualquier sistema operativo compatible, lo que facilita trasladar aplicaciones entre diferentes entornos. También proporciona aislamiento de las aplicaciones respecto al entorno en el que se ejecutan, lo cual incrementa la seguridad y la estabilidad. Asimismo, es más eficiente que las máquinas virtuales tradicionales, al requerir menos recursos y ofrecer un rendimiento optimizado.

Las ventajas más notables de Docker son el aislamiento de las aplicaciones, que refuerza la seguridad; la consistencia entre los entornos de desarrollo, prueba y producción; y la capacidad de escalar aplicaciones de manera horizontal. Sin embargo, requiere conocimientos técnicos para gestionar contenedores y puede generar complejidad cuando se administran aplicaciones con múltiples contenedores.

2.7.4.2. *Kubernetes*

Kubernetes es una plataforma de orquestación de contenedores que automatiza el despliegue, la gestión y el escalado de aplicaciones contenedorización. Con Kubernetes, los desarrolladores pueden administrar aplicaciones distribuidas de forma más eficiente.

Kubernetes se distingue por la automatización del despliegue y escalado de aplicaciones, ya que gestiona la ejecución de contenedores en múltiples servidores y ajusta la capacidad de manera dinámica según la demanda. También optimiza el uso de recursos de hardware, como CPU y memoria, y asegura la continuidad del servicio al garantizar redundancia y alta disponibilidad, incluso ante fallas de nodos o contenedores.

Entre sus ventajas, ofrece una gestión automatizada de contenedores a gran escala, incrementa la disponibilidad y resiliencia de las aplicaciones, y facilita el escalado eficiente de sistemas distribuidos. Como contrapartida, requiere una infraestructura compleja para su implementación y demanda conocimientos avanzados en orquestación de contenedores, lo que puede dificultar su configuración y mantenimiento (Merkel, 2014).

CAPÍTULO III

INGENIERÍA DEL PROYECTO

CAPÍTULO III

INGENIERÍA DEL PROYECTO

3.1. ANÁLISIS DE CONTEXTO OPERATIVO

3.1.1. RECOLECCIÓN DE EVIDENCIAS

La recopilación de información para caracterizar el proceso actual se llevó a cabo mediante técnicas cualitativas complementadas con el análisis de documentos y registros operativos. Esta recolección se contrastó sistemáticamente con conceptos del marco teórico en gestión de procesos y logística del transporte, con el propósito de garantizar consistencia, validez y claridad en los resultados. Se dio prioridad a la observación del entorno real de trabajo, involucrando a los actores clave jefes de línea, conductores, vendedores y usuarios, así como al examen de los soportes manuales utilizados en la operación diaria, tales como pizarras, planillas físicas y comunicaciones internas vía WhatsApp. Toda la información obtenida fue organizada en bitácoras y matrices de seguimiento, lo que permitió asegurar la trazabilidad de los hallazgos y la pertinencia de los requerimientos identificados.

3.1.1.1. Entrevistas y observación directa

Se aplicó una entrevista semiestructurada a los líderes de ruta, conductores y vendedores de la Cooperativa la Perla del Oriente con experiencia directa en la coordinación de salidas y en la venta manual de pasajes. El propósito fue caracterizar el flujo real de coordinación, sus variaciones y los principales puntos de fricción, además de identificar los criterios utilizados para asignar turnos, verificar disponibilidad y registrar ventas. La guía empleada, que abarcó temas relacionados con coordinación previa, comunicación interna, gestión de planillas, manejo de incidencias, trazabilidad y cierre de jornada, se detalla en el Apéndice A.1. Los hallazgos consolidados utilizados para validar el diagrama del proceso actual y derivar los requisitos del sistema se presentan en el Apéndice A.2.

En síntesis, la entrevista evidenció como fricciones principales la dispersión de información entre pizarras, notas y mensajes de WhatsApp, la dependencia de consultas manuales entre trabajadores para confirmar disponibilidad, la falta de un registro centralizado que unifique las ventas, las variaciones generadas por incidentes operativos que obligan a ajustes improvisados

y la dificultad para consolidar información entre distintos puntos de origen y destino. Como métricas aproximadas, se estimó que la verificación manual de disponibilidad puede tomar entre tres y diez minutos por consulta, que la coordinación por WhatsApp exige revisar numerosos mensajes y que un vendedor puede realizar varias actualizaciones informales por turno antes de registrar una venta válida.

3.1.1.2. Documentación operativa

Se realizó una revisión sistemática de la documentación utilizada en la operación diaria, incluyendo planillas físicas, pizarras de turnos, notas internas y el historial de comunicaciones en el grupo de WhatsApp. Estos elementos se presentan en forma de imágenes en el Apéndice A.3. El objetivo fue mapear el circuito de información empleado para coordinar buses y registrar ventas, identificar diferencias en los formatos y criterios de registro, y conformar un conjunto de evidencias que respaldan el análisis del proceso y las pruebas preliminares del sistema propuesto. Los insumos obtenidos de esta revisión permiten describir con precisión el flujo actual de coordinación y complementan los hallazgos de la entrevista presentados en el Apéndice A.2, garantizando coherencia y trazabilidad entre la evidencia recopilada y los requisitos definidos.

3.1.1.3. Integración de hallazgos

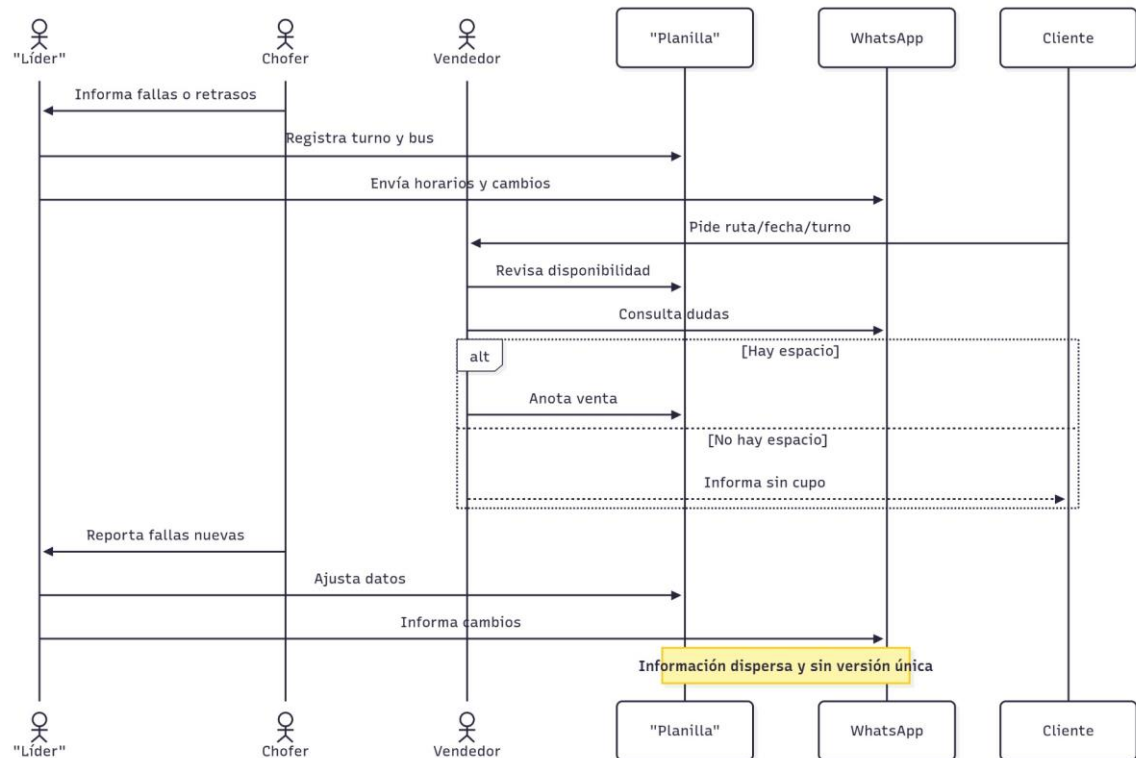
La triangulación integró tres insumos principales: la información empírica interna procedente de entrevistas y registros operativos, la revisión de documentación utilizada en la práctica diaria (planillas, anotaciones y comunicaciones por WhatsApp) y el marco teórico-técnico relacionado con gestión de flotas, diseño de sistemas de información, trazabilidad digital y comunicación en operaciones distribuidas. El contraste se desarrolló atendiendo a criterios metodológicos como trazabilidad, replicabilidad y consistencia; a criterios operativos como oportunidad de la información, confiabilidad y centralización; y a criterios técnicos como integración de datos, disponibilidad en tiempo real, reducción de duplicaciones y mecanismos de validación interna. El procedimiento permitió mapear cada hallazgo a fundamentos teóricos y técnicos, garantizando que las conclusiones respondan tanto a la evidencia práctica como a principios reconocidos en la disciplina.

Los resultados evidenciaron una fuerte convergencia entre las fricciones identificadas en campo, información dispersa entre distintos soportes, ausencia de centralización, verificación manual de disponibilidad, comunicaciones informales y dificultades para consolidar ventas, y las soluciones recomendadas por la literatura, que incluyen la digitalización del flujo operativo, la centralización de datos en un sistema único, la estandarización de registros, la generación automática de trazabilidad y la comunicación estructurada en tiempo real. En síntesis, las implicaciones para el diseño del sistema prioriza una plataforma integrada que permita consultar disponibilidad en tiempo real, registrar ventas de manera unificada, gestionar incidencias sin pérdida de información y mantener historial verificable de todas las operaciones. En lo no funcional, destacan la necesidad de transparencia en los cambios, auditabilidad del flujo de información y accesibilidad para todos los actores involucrados.

3.1.2. FLUJO ACTUAL DE INVESTIGACIÓN

El flujo operativo actual se encuentra distribuido entre múltiples canales informales, principalmente una planilla editable (véase Anexo 5) y comunicaciones por WhatsApp (véase Anexo 6), lo que impide contar con una versión única y confiable del estado de los turnos, las ventas y las incidencias. Esta fragmentación obliga a los actores involucrados a revisar manualmente la disponibilidad, confirmar cambios por mensajería y registrar las ventas en momentos y formatos distintos, lo que genera inconsistencias y duplicación de esfuerzos. Como consecuencia, el proceso depende de la memoria y del criterio individual, aumenta la probabilidad de errores y dificulta el seguimiento claro de las decisiones tomadas. Las entrevistas internas y la observación directa del trabajo diario, corroboradas además por los hallazgos descritos en el Apéndice 2, confirmaron que las fricciones más frecuentes se originan en la falta de una fuente centralizada de datos y en la necesidad constante de verificar y comparar información dispersa entre la planilla y los mensajes de WhatsApp, lo cual ralentiza la operación y afecta la coordinación entre los actores del flujo.

Ilustración 3 Diagrama de actividad del flujo actual de investigación



Fuente: Elaboración propia (2025).

3.1.2.1. Roles y responsabilidades

El líder de línea coordina la operación diaria; verifica la disponibilidad de los buses, recibe los reportes de estado de cada unidad y asigna los turnos de salida. Los conductores informan las condiciones del vehículo, incluidos posibles retrasos o fallas, y comunican cualquier incidencia que pueda afectar el servicio. Los vendedores atienden a los usuarios, revisan la disponibilidad consultando la planilla actualizada por el líder y registran las ventas de forma manual. En muchos casos, utilizan planillas que no siempre son propias, lo que genera duplicación de registros y pérdida de control operativo. Debido a la ausencia de un sistema centralizado, toda la coordinación depende de actualizaciones manuales y de información distribuida, lo que incrementa la carga de trabajo y dificulta mantener una versión única y confiable del estado de los turnos y las ventas.

3.1.2.2. *Dificultades operativas identificadas*

En el flujo actual de coordinación de buses y venta de pasajes se identifican tres problemas recurrentes. El primero es la dispersión de la información, ya que la disponibilidad de turnos, los cambios operativos y las ventas se registran en planillas manuales que circulan entre distintos vendedores, lo que genera duplicación de apuntes, versiones contradictorias y riesgo de pérdida de datos. El segundo problema es la verificación manual de disponibilidad, puesto que los vendedores dependen exclusivamente de la planilla física actualizada por el líder de línea, lo que provoca demoras cuando varios actores requieren confirmar la misma información al mismo tiempo. El tercer problema es la falta de trazabilidad, dado que las razones por las cuales se asigna un turno, se cambia un bus o se modifica un asiento quedan registradas solo en mensajes informales o en notas personales, dificultando reconstruir decisiones y auditar el proceso.

En cuanto a los cuellos de botella, destacan principalmente dos. El primero es la necesidad de revisar repetidamente la planilla o solicitar confirmación verbal ante cualquier duda, lo que incrementa el tiempo necesario para atender a cada cliente y genera interrupciones constantes en el flujo de trabajo. El segundo es la circulación de planillas entre distintos vendedores, que obliga a ajustar o reinterpretar registros ajenos como y provoca inconsistencias cuando varias personas modifican la misma información sin un control central (véase Apéndice A.4). Ambos factores elevan el costo operativo, aumentan el riesgo de errores y retrasan la construcción de un registro confiable del estado real de los turnos y las ventas.

3.1.2.3. *Limitaciones del proceso manual*

Cuadro 3.1. Acciones, características y deficiencias del flujo actual de investigación

Acciones (paso del flujo)	Características actuales	Deficiencias identificadas
Asignar turno al bus	El líder decide el turno según disponibilidad y experiencia previa.	No existen criterios estandarizados ni historial de decisiones que permita justificar cambios o reasignaciones.
Registrar turno en planilla	El líder anota manualmente en una planilla el bus y el turno asignado.	Errores de escritura, enmiendas y registros desactualizados; no hay versión única ni control de cambios.
Comunicar horarios al personal	El líder informa horarios y turnos por mensaje o de manera verbal.	Riesgo de omisiones y malentendidos; algunos actores no

Acciones (paso del flujo)	Características actuales	Deficiencias identificadas
		reciben o no revisan a tiempo la información.
Atender la solicitud de pasaje del cliente	El cliente indica ruta, fecha y turno al vendedor en ventanilla o punto de venta.	La atención se ve afectada si el vendedor no tiene acceso inmediato a la planilla actualizada.
Verificar disponibilidad en planilla	El vendedor consulta la planilla manual para ver asientos disponibles.	La planilla puede estar incompleta, en uso por otro vendedor o desactualizada; alta probabilidad de inconsistencias.
Registrar la venta y el pago	El vendedor anota la venta en una planilla disponible y cobra en efectivo o por QR.	Se utilizan planillas “prestadas” o compartidas; se duplican registros y se pierde el rastro de quién vendió qué.
Ajustar por incidencias en el servicio	Ante falla o retraso, el chofer informa al líder y este decide cambios en turno o bus.	No hay procedimiento formal ni registro detallado; los cambios se comunican de forma fragmentada y tardía.
Conservar y usar planillas durante la jornada	Cada vendedor mantiene o utiliza la planilla que tenga a mano para seguir vendiendo.	No existe consolidación central; aparecen versiones distintas del mismo viaje y no se sabe con certeza la ocupación.
Cerrar y revisar el día de trabajo	Se revisan planillas físicas para estimar ventas y ocupación de los buses.	Proceso lento y propenso a errores; difícil reconstruir el estado real de cada viaje y realizar auditorías.

Fuente: Elaboración propia, 2025.

3.1.3. FLUJO PROPUESTO

El flujo objetivo, validado con el personal operativo (véase Apéndice A.2), concentra toda la coordinación de turnos y ventas en una única plataforma accesible para el líder de línea, conductores y vendedores. El ingreso se realiza mediante credenciales verificadas, garantizando control de acceso y uso adecuado del sistema. La consulta sobre disponibilidad de buses, turnos y asientos se formula directamente en la interfaz, que responde desde una base de datos centralizada y actualizada en tiempo real. Los resultados se presentan de manera clara, con opciones para visualizar la información detallada del viaje, revisar el historial de movimientos y registrar retroalimentación.

El usuario puede continuar la operación dentro de la misma sesión, ya sea consultando disponibilidad, registrando ventas o gestionando incidencias, o puede iniciar una nueva interacción cuando sea necesario. El sistema unifica toda la información en un solo entorno,

eliminando la circulación de planillas físicas y disminuyendo significativamente el riesgo de errores, omisiones y duplicaciones.

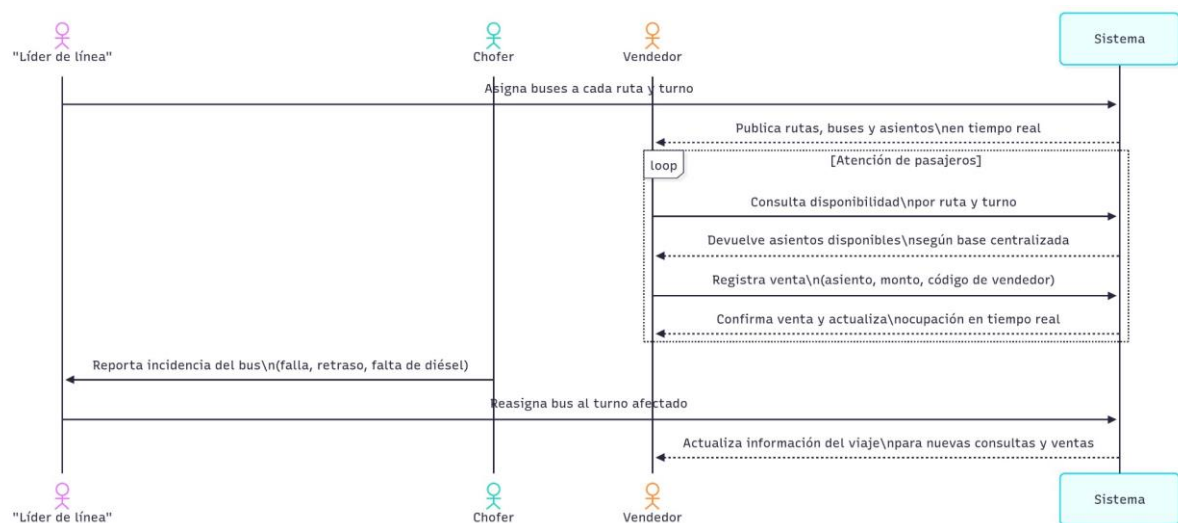
3.1.3.1. *Descripción del proceso objetivo*

- Acceso a la plataforma: Para ingresar al sistema, el usuario escribe su correo y contraseña. La plataforma verifica estos datos contra las credenciales previamente registradas por el equipo técnico.
 - No permitido → se muestra un mensaje de error de credenciales.
 - Permitido → el acceso se habilita inmediatamente y el usuario puede comenzar a operar en la plataforma.
- Administración de accesos y configuración (equipo técnico): El equipo técnico es responsable de:
 - Registrar y habilitar usuarios.
 - Cargar la información base: rutas, turnos, buses, capacidad, precios y reglas operativas.
 - Configurar parámetros y actualizaciones necesarias para el funcionamiento diario.
 - Modificar accesos, actualizar datos o incorporar nueva información cuando se requiera.
- Operación del sistema por parte del usuario: Una vez autenticado, el usuario —ya sea líder de línea, vendedor o chofer— puede consultar información directamente desde la interfaz utilizando lenguaje natural, sin comandos especiales. Entre las consultas disponibles se encuentran:
 - Rutas y turnos programados.
 - Buses asignados y capacidad disponible.
 - Estado de ocupación de asientos.
 - Cambios bus por incidencia y historial de acciones por asiento.
- Atención de pasajeros y registro de ventas (Vendedor): Dentro del ciclo operativo, el vendedor:
 - Consulta la disponibilidad de un asiento para una ruta y turno específicos → El sistema devuelve los asientos libres según la base centralizada.
 - Registra la venta indicando asiento, monto y su código de vendedor → El sistema confirma la venta y actualiza la ocupación en tiempo real.

- Gestión de incidencias operativas (Chofer y Líder de línea): El chofer reporta al líder de línea cualquier incidencia del bus (fallas, retrasos, falta de combustible, etc.). El líder de línea actualiza la operación reasignando un nuevo bus al turno afectado. El sistema actualiza al instante la información de viaje, para que los vendedores realicen consultas y ventas con datos correctos.
- Operaciones y registro de acciones:
 - o Modificación de precios.
 - o Gestión de liquidaciones, asociadas a ventas por vendedor y por viaje.
- Historial y trazabilidad: Se registran consultas, respuestas, historial de pasajeros por fechas.

Para una representación conjunta de las etapas y de las responsabilidades asignadas a cada carril (Vendedor, Sistema, Chofer y Líder de Línea) puede consultarse la figura 3.2, donde se muestra la secuencia completa del flujo propuesto. En dicha figura se observan los puntos de control asociados al acceso con credenciales habilitadas, el ciclo operativo de consulta, respuesta y registro, así como las decisiones relacionadas con la continuidad del proceso dentro de la misma sesión o con la apertura de un nuevo flujo de trabajo.

Ilustración 4 Diagrama de actividad del flujo propuesto con carriles de responsabilidad



Fuente: Elaboración propia, 2025.

3.1.3.2. Actores y roles propuestos

a. Vendedor

Encargado de la atención comercial en la terminal donde se encuentra asignado. Realiza consultas sobre disponibilidad por ruta y turno, registra ventas y recibe confirmación del sistema en tiempo real. A diferencia del chofer, su ámbito de venta está limitado a su terminal de origen, lo que garantiza control operativo y trazabilidad en el punto de atención. Su actividad se concentra en el ciclo de consulta-venta dentro de la operación diaria de pasajeros.

b. Chofer

Responsable operativo del bus durante el turno asignado. Reporta incidencias al Líder de Línea, como fallas mecánicas, retrasos o falta de combustible, desencadenando los procesos correctivos necesarios para asegurar la continuidad del servicio. Además, el chofer está habilitado para realizar ventas desde cualquier terminal de origen, independientemente de su ubicación actual. Aunque no gestiona consultas complejas ni procesos administrativos, su capacidad de venta flexible y su rol operativo lo convierten en un actor esencial para la eficiencia y continuidad del flujo de trabajo.

c. Líder de Línea

Encargado de la gestión operativa de las rutas. Inicialmente asigna buses a cada ruta y turno, y durante la operación atiende las incidencias reportadas por los choferes. Tiene la autoridad para reasignar buses cuando se ve afectado un servicio, lo que obliga al sistema a actualizar la información visible para los vendedores. Su rol es crítico para la coordinación en campo y la estabilidad del flujo operativo.

d. Sistema

Plataforma centralizada que administra y coordina todo el flujo operativo. Publica rutas, buses y asientos en tiempo real; procesa consultas de disponibilidad; registra ventas y actualiza ocupación de forma inmediata; integra las reasignaciones de buses realizadas por el Líder de

Línea; y difunde los cambios a los vendedores para mantener la operación sincronizada. Además, registra las interacciones para efectos de trazabilidad y soporte.

e. Dueño

Actor con acceso total a la información consolidada de montos de venta por terminal, pudiendo visualizar datos diarios, semanales y mensuales. Su función se centra en el monitoreo financiero de la operación, revisando comparativos entre terminales, tendencias de ingresos y variaciones en los montos vendidos en cada periodo. No participa en la operación ni en la gestión de buses o ventas individuales; su rol está orientado exclusivamente al análisis global de resultados comerciales.

3.1.4. REQUERIMIENTOS FUNCIONALES

RF-01	Autenticación de usuario	
Descripción	El sistema permite que un usuario ingrese su correo y contraseña para acceder a la plataforma. El sistema valida las credenciales contra los registros habilitados por el equipo técnico.	
Actores	Vendedor, Chofer, Líder de Línea, Dueño	
Precondiciones	El usuario debe estar previamente registrado y habilitado por el equipo técnico.	
Criterios de aceptación	Si las credenciales son correctas, el acceso queda habilitado inmediatamente. Si las credenciales no coinciden, se muestra un mensaje de error.	

RF-02	Administración de usuarios y accesos	
Descripción	El sistema permite que el equipo técnico registre, habilite, modifique o deshabilite usuarios. También gestiona la carga base (rutas, turnos, buses, capacidad, precios y reglas operativas).	
Actores	Equipo Técnico	
Precondiciones	Administrador técnico autenticado.	
Criterios de aceptación	Registro o modificación de usuario.	

RF-03	Asignación de bus a ruta y turno (Activación operativa)	
Descripción	El Líder de Línea asigna un bus a una ruta y horario previamente creados para activar esa ruta.	
Actores	Líder de Línea.	
Precondiciones	Rutas, turnos y buses cargados por el equipo técnico.	

Criterios de aceptación	La asignación activa la ruta para operaciones. La asignación se refleja inmediatamente en cuentas de vendedores y choferes. El sistema registra la asignación con hora, ruta y bus. Si un bus se reasigna, la actualización reemplaza la configuración existente en tiempo real.
--------------------------------	--

RF-04	Ajuste de precios por ruta activa	
Descripción	El Líder de Línea puede modificar el precio asociado a rutas ya activadas mediante la asignación de bus.	
Actores	Líder de Línea.	
Precondiciones	Ruta activa (bus asignado). Líder autenticado.	
Criterios de aceptación	El nuevo precio se refleja para vendedores y choferes. Las ventas posteriores usan el precio actualizado.	

RF-05	Consulta de disponibilidad de asientos	
Descripción	El sistema permite consultar disponibilidad en tiempo real por ruta y turno.	
Actores	El sistema permite consultar disponibilidad en tiempo real por ruta y turno.	
Precondiciones	La ruta debe estar activada por el Líder de Línea. Usuario autenticado.	
Criterios de aceptación	El sistema devuelve asientos libres desde la base centralizada.	

RF-06	Gestión de incidencias	
Descripción	El chofer reporta incidencias; el Líder de Línea puede reasignar un bus dentro del Sistema.	
Actores	Chofer, Líder de Línea.	
Precondiciones	Ruta activa.	
Criterios de aceptación	Reasignación del bus actualiza disponibilidad y datos visibles para vendedores y choferes. La actualización es inmediata.	

RF-07	Liquidaciones y control de ventas
Descripción	El sistema reporta ventas por vendedor, chofer y viaje.
Actores	Dueño.
Precondiciones	Ventas existentes y viajes
Criterios de aceptación	Reportes por viaje. Totales claros por monto y terminal.

RF-08	Visualización de montos consolidados por terminal
Descripción	El dueño visualiza montos de ventas por terminal en períodos específicos.
Actores	Dueño.
Precondiciones	Ventas registradas.
Criterios de aceptación	Información accesible diaria, semanal y mensual. Comparativos por terminal y por periodo. Datos exactos y consistentes con el historial.

RF-09	Venta de Boletos con Selección de Asientos
Descripción	El sistema permite a los vendedores seleccionar asientos específicos en el layout del bus y registrar información de pasajeros para cada asiento seleccionado.
Actores	Vendedor, Chofer
Precondiciones	Ruta activa con bus asignado usuario autenticado Disponibilidad de asientos
Criterios de aceptación	Visualización del layout del bus con asientos disponibles/ocupados Selección múltiple de asientos Registro de datos de pasajeros (nombre, apellido, CI, teléfono) por asiento Cálculo automático del precio total Actualización en tiempo real de disponibilidad

RF-10	Modo de Venta Rápida
Descripción	El sistema permite realizar ventas rápidas sin completar toda la información del pasajero, permitiendo completar los datos posteriormente.
Actores	Vendedor, Chofer
Precondiciones	Ruta activa con bus asignado usuario autenticado Disponibilidad de asientos
Criterios de aceptación	Venta de boletos con información mínima del pasajero Identificación temporal automática Actualización posterior de datos del pasajero Visualización de boletos pendientes de completar información impresión de listas de pasajeros con datos completos

RF-11	Procesamiento de Pagos
Descripción	El sistema procesa pagos mediante efectivo o código QR, con autenticación del vendedor mediante PIN.
Actores	Usuario
Precondiciones	Boletos seleccionados Información de pasajeros completa (o modo venta rápida) Vendedor autenticado
Criterios de aceptación	autenticación del vendedor mediante PIN Selección de método de pago (efectivo/QR) Procesamiento de pago exitoso Generación de orden de compra Actualización inmediata de disponibilidad de asientos

RF-12	Reembolso de Boletos
Descripción	El sistema permite reembolsar boletos vendidos, liberando el asiento y registrando la transacción.
Actores	Vendedor, Líder de Línea
Precondiciones	Boleto vendido Orden identificada
Criterios de aceptación	Visualización de detalles de la orden Liberación del asiento Registro de la transacción de reembolso Actualización inmediata de disponibilidad

RF-13	Historial de Asientos
Descripción	El sistema permite visualizar detalles completos de órdenes vendidas, incluyendo pasajeros, asientos, montos y método de pago.
Actores	Vendedor, chofer, Líder de Línea, Dueño
Precondiciones	Asiento identificado Usuario autenticado
Criterios de aceptación	Consulta de historial por número de asiento. Visualización de eventos (venta, cancelación, reembolso, reasignación). Información de pasajero, operador, terminal, ruta y monto. Fechas y horas de cada evento.

3.1.5. REQUERIMIENTOS NO FUNCIONALES

Los siguientes requerimientos no funcionales (RNF) fijan objetivos verificables de calidad para el sistema (rendimiento, disponibilidad, seguridad, UX y operación). Su verificación se sustenta en métricas instrumentadas y en pruebas técnicas. Los siguientes requerimientos no

funcionales (RNF) fijan objetivos verificables de calidad para el sistema (rendimiento, disponibilidad, seguridad, UX y operación). Su verificación se sustenta en métricas instrumentadas y en pruebas técnicas.

3.1.5.1. *Rendimiento*

- Objetivos de latencia (p95):
 - Carga inicial de dashboard < 2 s con autenticación válida.
 - Consulta de disponibilidad de asientos < 500 ms.
 - Actualización de disponibilidad mediante WebSocket < 200 ms desde evento en servidor.
 - Procesamiento de venta (checkout) < 1.5 s incluyendo validación y actualización de disponibilidad.
 - Consulta de historial de asientos < 800 ms.

3.1.5.2. *Seguridad y privacidad*

- Acceso y control:
 - Autenticación mediante cookies HTTP-only con validación en cada request.
 - RBAC con mínimo privilegio: roles definidos con permisos específicos por funcionalidad.
 - Autenticación de empleados mediante PIN (4 dígitos) para procesamiento de transacciones de venta.
 - Rate limiting en endpoints de autenticación (máximo 5 intentos por minuto por IP).
 - Validación de sesión en cada navegación con redirección automática a login si expira.
 - Protección CSRF mediante SameSite cookies y validación de origen.

3.1.5.3. *Usabilidad y accesibilidad*

- Claridad:
 - Un usuario nuevo puede completar una venta básica sin asistencia tras 5 minutos de uso.
 - Notificaciones de error/éxito clara y contextual en español.

- o Indicadores visuales de estado (asientos disponibles/ocupados/seleccionados) con leyenda visible.
- Idioma y Contexto:
 - o Interfaz y contenido en español (es), inglés (en) y portugués (pt) con detección automática de idioma del navegador.
 - o UI optimizada para PC/laptop (1024px+) y dispositivos móviles (320px+).
 - o Formato de fechas, horas y montos según configuración regional del usuario.

3.1.5.4. *Mantenibilidad*

- Estructura del proyecto:
 - o Código con tipado estricto.
 - o Separación clara de responsabilidades: componentes UI, hooks, stores, contextos, API.
 - o Convenciones de nomenclatura consistentes.

3.1.6. *ALCANCE Y DELIMITACIONES*

El sistema se limita a asistir la gestión operativa y comercial del proceso de venta de boletos de transporte terrestre mediante terminales POS (Point of Sale). Su funcionalidad abarca la selección de asientos, el registro de datos de pasajeros, el procesamiento de pagos, la administración de ventas por terminal u operador, la actualización en tiempo real de la disponibilidad y la generación de reportes consolidados para líderes de línea, choferes y dueños. El objetivo del sistema es apoyar las operaciones de venta en terminales físicas y proporcionar visibilidad operativa y comercial en tiempo real.

3.1.6.1. *Escenarios límites*

- Ruta sin bus asignado: el sistema no permite ventas ni consultas de disponibilidad hasta que el Líder de Línea active la ruta mediante asignación de bus.
- Consultas sin coincidencias: ante una ruta, horario o asiento inexistente, el sistema informa la inexistencia sin generar datos ni completar valores automáticamente.

- Reasignación en operación: si ocurre una incidencia y el bus se reasigna, el sistema actualiza disponibilidad y ventas en tiempo real sin perder el historial previo.

3.1.6.2. *Supuestos operativos*

- Usuarios internos autenticados: todos los roles deben iniciar sesión (vendedor, chofer, líder de línea, dueño) con credenciales válidas; las transacciones requieren PIN del vendedor o chofer.
- Parametrización inicial obligatoria: cada empresa proporciona rutas habilitadas, horarios, flota (modelo y numeración interna), precios base, vendedores por terminal y choferes autorizados.
- Disponibilidad de datos actualizada: el sistema opera únicamente con rutas activadas mediante asignación de bus; sin esa asignación no se habilitan ventas.
- Conectividad requerida: el sistema depende de la red corporativa y del servidor central; en ausencia de conexión no se realizan ventas ni sincronizaciones.
- Integridad de datos y trazabilidad: cada operación (venta, reembolso, reasignación, cambio de precio) debe registrarse con usuario, terminal, fecha y hora.
- Actualización centralizada: todo cambio (precios, disponibilidad, reasignaciones, ventas rápidas completadas) se propaga en tiempo real; no existen operaciones offline.

3.2. **SELECCIÓN DE HERRAMIENTAS PARA EL DESARROLLO**

En este apartado se fundamentan las decisiones tecnológicas que habilitan la solución propuesta. El objetivo no es enumerar herramientas, sino justificar una arquitectura coherente para un sistema de gestión operativa y comercial orientado a la venta de boletos mediante terminales POS, con disponibilidad en tiempo real, sincronización de estados, trazabilidad completa y soporte para transacciones seguras. Estas decisiones se presentan respaldadas por la información propuesta en el marco teórico, que aporta los conceptos, principios arquitectónicos y criterios de diseño empleados para sustentar la selección final.

La selección tecnológica abarca lenguajes y frameworks para backend y frontend, motores de base de datos y mensajería en tiempo real, servicios de autenticación, bibliotecas para procesamiento de pagos, así como metodologías de desarrollo apropiadas para un sistema que requiere confiabilidad transaccional y baja latencia.

La evaluación se presenta como tablas comparativas por dominio (backend, frontend, base de datos, tiempo real, infraestructura y despliegue), con criterios específicos como: rendimiento, escalabilidad, facilidad de mantenimiento, soporte comunitario, costo de operación, compatibilidad con POS, seguridad y experiencia de usuario. Para cada criterio se consigna una valoración cualitativa con evidencia (observaciones, pros/contras, riesgos, dependencias y supuestos), siguiendo las directrices conceptuales establecidas en el marco teórico y garantizando coherencia metodológica entre la teoría y la solución tecnológica adoptada.

3.2.1. BACKEND

El backend debe ofrecer un servicio GraphQL seguro con autenticación por correo y contraseña, control de permisos por rol. Debe integrarse con los módulos de disponibilidad en tiempo real, registro de eventos y gestión de incidencias, manteniendo trazabilidad completa. Para su implementación se comparan dos opciones tecnológicas: Spring Boot, dentro del ecosistema Java, y NestJS, sobre Node.js, evaluando su rendimiento, escalabilidad y facilidad de mantenimiento frente a los requerimientos del sistema.

3.2.1.1. Comparación de alternativas

Cuadro 3.3. Comparación de frameworks de backend

Criterio	Spring Boot	NestJS
Curva de aprendizaje / Facilidad	Curva de aprendizaje más pronunciada; requiere conocimiento previo de Java y del ecosistema Spring.	Curva más suave; aprovecha la sintaxis moderna de TypeScript y conceptos familiares de JavaScript.
Comunidad y documentación	Comunidad amplia, consolidada y con abundante documentación empresarial.	Comunidad activa y en crecimiento; documentación clara y orientada a desarrolladores web modernos.
Requerimientos de servidor	Requiere mayor consumo de recursos (RAM y CPU) debido a la JVM.	Ligero y eficiente; consumo reducido de recursos, ideal para despliegues en contenedores o nube.
Escalabilidad / Equipo	Panel admin listo para usar. Altamente escalable; ideal para sistemas grandes y distribuidos, con soporte robusto.	Escalable horizontalmente; adecuado para sistemas medianos y grandes con arquitectura orientada a servicios.

Criterio	Spring Boot	NestJS
Seguridad	Spring Security provee autenticación, autorización y protección ante ataques comunes (XSS, CSRF, JWT).	Alta productividad; compatible con Jest, Supertest y herramientas CI/CD de Node.js
Productividad	Incluye herramientas integradas para pruebas (JUnit, Mockito) y despliegue automatizado.	Muy alta. Idem, ideal como microservicio de IA.
Despliegue	Apto. Monolito modular o microservicios JVM; workers (batch, schedulers, colas) y escalado horizontal vía contenedores / Kubernetes.	Apto. Servicios ligeros y microservicios; workers (Bull, colas, lambdas) y mucha composición entre módulos..
Observabilidad	Sencilla. Actuator + Micrometer; integraciones estándar con Prometheus, Grafana, Zipkin, etc.	Variable. Buenas opciones (OpenTelemetry, loggers, APMs), pero más “a la carta”; depende de las decisiones del equipo.
Riesgos principales	Menores. Curva moderada-alta, plataforma muy estable; riesgo de sobre-configuración y consumo de recursos si no se afina.	Deriva. Heterogeneidad de librerías/estilos del ecosistema Node; fácil fragmentación entre proyectos y patrones.

Fuente: Elaboración propia, 2025.

3.2.1.2. *Decisión y justificación*

Se adopta NestJS como backend principal por su arquitectura modular, sintaxis moderna en TypeScript e integración nativa con GraphQL, lo que permite implementar de forma ordenada los servicios del sistema. La decisión se fundamenta en dos razones principales: (1) NestJS ofrece un entorno de desarrollo ágil con una estructura clara para organizar dominios, lo que facilita la evolución del sistema sin introducir complejidad innecesaria, y (2) proporciona una curva de aprendizaje favorable sin sacrificar robustez ni diseño arquitectónico.

Sus características técnicas, como el sistema de inyección de dependencias, la validación integrada, los middleware y controladores, junto con el soporte para arquitectura limpia, permiten implementar mecanismos de autenticación basados en roles, validación mediante PIN en operaciones sensibles y trazabilidad completa de ventas, reasignaciones y eventos operativos.

Si bien Spring Boot ofrece un entorno sólido y ampliamente probado para sistemas transaccionales, su adopción implicaría una infraestructura más pesada y un costo operativo mayor, mientras que NestJS mantiene un runtime único y ligero que reduce la complejidad y favorece la escalabilidad progresiva requerida por el sistema.

3.2.2. FRONTEND

El frontend se debe construir sobre un marco de componentes que permita modularidad, pruebas y evolución gradual. Se requiere soporte sólido para enrutamiento, manejo de estado, integración con GraphQL/WebSocket, técnicas de optimización (carga diferida, code splitting, virtualización de listas) y pautas de accesibilidad. A continuación, se presenta la comparación entre React y Vue.js.

3.2.2.1. Comparación de alternativas

Cuadro 3.4. Comparación de frameworks de frontend

Criterio	React	Vue.js
Adecuación general	Alta. Biblioteca declarativa, componentes reutilizables, hooks y gran adopción.	Alta. Framework progresivo, entrada rápida y curva suave.
Ecosistema y productividad	Muy alto. Amplio ecosistema (React Router, Redux Toolkit, Next.js, TanStack Query).	Alto. Ecosistema curado (Vue Router, Pinia, Nuxt) y SFC con tooling maduro.
Rendimiento y optimización	Muy alto. Virtual DOM, Suspense, lazy loading, virtualización de listas.	Alto. Reactividad eficiente, virtual DOM, lazy loading y code splitting.
Estado global y datos remotos	Muy alto. Redux Toolkit / Zustand / TanStack Query facilitan caché y sincronización.	Alto. Pinia ligera y clara; integración simple con Axios/Apollo.
Enrutamiento	Alto. React Router consolidado; patrones de rutas anidadas y protecciones.	Alto. Vue Router declarativo con guards y lazy loading.
Renderizado del lado del servidor (SSR)	Muy alto. Next.js y streaming para SSR/ISR y data fetching.	Alto. Nuxt con SSR/ISR y file-based routing.
Accesibilidad (a11y)	Muy alto. Amplia guía y linting; bibliotecas a11y maduras.	Alto. Buenas prácticas y tooling de la comunidad.

Criterio	React	Vue.js
Curva de aprendizaje	Media. Sintaxis sencilla; elección de librerías puede requerir criterio.	Baja–Media. Entrada rápida; Composition API añade potencia gradual.

Fuente: Elaboración propia, 2025.

3.2.2.2. *Decisión y justificación*

Se selecciona React como tecnología de frontend por dos motivos principales: (1) su ecosistema amplio y consolidado, que ofrece librerías maduras para enrutamiento, manejo de estado y comunicación con GraphQL, permitiendo desarrollar interfaces dinámicas y mantener el código de forma estable a lo largo del tiempo; y (2) su arquitectura declarativa basada en componentes y hooks, que facilita la gestión del estado, la reutilización de módulos y el manejo eficiente de pantallas con actualizaciones frecuentes, como la disponibilidad de asientos o el historial de ventas en un sistema. Si bien Vue representa una alternativa accesible y con buena curva de aprendizaje, React ofrece mayor madurez, un soporte comunitario más extenso y una flexibilidad que se adapta mejor a los requerimientos operativos del proyecto.

3.2.3. *BASES DE DATOS*

El diseño del modelo de datos del sistema requiere distinguir entre información transaccional, que sostiene las operaciones diarias de venta y control de disponibilidad, y datos orientados a consulta y análisis, utilizados para reportes y supervisión operativa. Por ello, la comparación de alternativas tecnológicas se plantea desde una perspectiva amplia, considerando categorías como bases de datos relacionales para garantizar consistencia, opciones no relacionales para estructuras flexibles y almacenes especializados para consultas de mayor volumen. La evaluación contempla la capacidad de cada enfoque para mantener integridad, ofrecer consultas expresivas, adaptarse al dominio operativo del transporte, soportar la carga prevista y minimizar riesgos relacionados con mantenimiento, escalabilidad y continuidad del servicio.

3.2.3.1. Tipos

a. Cuadro comparativo

Cuadro 3.5. Comparación de tipos de bases de datos

Criterio	Relacional	No relacional
Modelo e integridad	Esquema explícito, ACID, claves foráneas y restricciones.	Esquema flexible; consistencia eventual frecuente.
Consultas y relaciones	SQL expresivo, <i>joins</i> , agregaciones, vistas/materializadas.	Consultas por documento/colección; agregaciones específicas.
Adecuación al dominio	Muy alta para usuarios, roles, whitelist, historial, feedback, auditoría.	Útil para documentos heterogéneos o <i>logs</i> a gran escala.
Rendimiento típico	Óptimo en lecturas/escrituras transaccionales y reportes.	Óptimo en lecturas masivas/alta elasticidad por documento.
Operación y mantenimiento	Herramientas maduras; copias de seguridad, PITR, migraciones.	Muy elástico; requiere diseño cuidadoso de partición/índices.
Escalabilidad	Vertical + réplicas; <i>sharding</i> posible.	Horizontal natural (<i>sharding</i> /replicas).
Riesgos en el contexto	Bajos; modelo conocido y gobernable.	Posible complejidad en consistencia y duplicación de datos.

Fuente: Elaboración propia, 2025.

b. Decisión y justificación

Se adopta una estrategia combinada, se utilizará una base de datos relacional como almacén transaccional para identidad y acceso, historial de consultas, feedback y auditoría. La elección se fundamenta en la necesidad de integridad referencial, transacciones ACID, trazabilidad y reportabilidad con SQL, además de un ecosistema maduro de migraciones, respaldos y recuperación y también se utilizará almacenamiento vectorial para embeddings que habiliten la búsqueda semántica.

No se adopta una base no relacional como repositorio principal, dado que el dominio exige consistencias fuertes, controles relacionales y auditoría reproducible; su uso podría evaluarse

en fases posteriores para telemetría o logs a gran escala. Esta decisión mantiene el menor número de piezas posible, alinea gobernanza de datos con requisitos de seguridad y trazabilidad.

3.2.3.2. Bases de datos relacionales

Se comparan tres sistemas de gestión de bases de datos relacionales ampliamente adoptados (PostgreSQL y MySQL) considerando conformidad con estándares y propiedades ACID, extensibilidad y tipos avanzados,, capacidades para búsqueda semántica (índices y funciones vectoriales), operación (replicación, respaldo y recuperación), integración con ecosistemas de desarrollo, despliegue multiplataforma y licenciamiento/costo.

c. Cuadro comparativo

Cuadro 3.6. Comparación de tipos de bases de datos relacionales

Criterio	PostgreSQL	MySQL
Estándares y ACID	Alta conformidad SQL y transacciones ACID; robustez transaccional.	ACID con motores como InnoDB; estándar de facto en web.
Extensibilidad y tipos avanzados	Muy alta: tipos/operadores/funciones definidas por usuario; JSONB y ecosistema de extensiones.	Alta: JSON, ecosistema amplio; menor foco histórico en extensiones complejas.
Vector/semántica	pgvector para almacenamiento/consulta de embeddings e índices ANN.	VECTOR en versiones recientes y HeatWave Vector Store en nube.
Replicación, respaldo y recuperación	Replicación nativa, streaming, logical replication; PITR con WAL.	Replicación maestro–esclavo y maestro–maestro.
Integración con ecosistemas	Muy alta con Python (ORMs, controladores), R y Java.	Muy alta con PHP, Java, Python y Node.js.
Despliegue OS / contenedores	Multiplataforma; excelente soporte en contenedores y operators.	Multiplataforma; amplia oferta gestionada en nubes.
Licenciamiento/costo	Código abierto (licencia permisiva).	Código abierto; múltiples ediciones gestionadas.

Criterio	PostgreSQL	MySQL
Observaciones destacadas	Fuerte extensibilidad + pgvector unifican relacional y vectorial en un mismo plano operativo.	Amplia adopción en web; vector y tiendas vectoriales gestionadas emergen como opción.

Fuente: Elaboración propia, 2025.

d. Decisión y justificación

Se adopta PostgreSQL como motor relacional principal para el sistema, debido a su solidez en escenarios transaccionales, su soporte nativo para integridad referencial y su capacidad para manejar estructuras complejas como órdenes, tickets, asientos, rutas y viajes. La elección se fundamenta en su cumplimiento estricto del modelo ACID, su alta compatibilidad con SQL estándar, su flexibilidad para modelar relaciones y su facilidad para generar reportes operativos y financieros sin depender de herramientas externas. Estos atributos lo convierten en una opción adecuada para un sistema donde la consistencia, la trazabilidad y el control de estados son esenciales.

3.2.4. HERRAMIENTAS COMPLEMENTARIAS

El desarrollo del sistema incorpora un conjunto de herramientas que aportan solidez técnica, facilitan el trabajo colaborativo y garantizan la trazabilidad del proyecto. Postman se utiliza para construir y ejecutar pruebas sobre los endpoints a través de colecciones organizadas por entorno (desarrollo, pruebas y producción), lo que permite verificar el comportamiento funcional de los servicios.

Como herramienta principal para el control de versiones, GitHub sirve para centralizar el repositorio, administrar ramas de trabajo e implementar flujos de integración continua cuando es necesario. Por último, Visual Studio Code (VS Code) se adopta como editor de referencia debido a su compatibilidad con Git, su terminal integrada, la capacidad de depuración detallada y el amplio ecosistema de extensiones (formateo y análisis del código, compatibilidad con los

frameworks utilizados, snippets y herramientas de inspección), lo que contribuye a una mayor eficiencia y uniformidad en el código durante todo el ciclo de desarrollo.

3.2.5. ENFOQUE METODOLÓGICO ADOPTADO

A efectos de seleccionar la metodología de desarrollo más adecuada, se presenta una breve comparación entre los enfoques Cascada e Incremental, destacando sus implicaciones en gestión del cambio, entrega de valor, riesgo técnico, integración/pruebas, planificación y trazabilidad.

3.2.5.1. Comparación de metodologías

Cuadro 3.7. Comparación de metodologías

Criterio	ICONIX	Incremental
Enfoque	Proceso dirigido por casos de uso y modelo de dominio; énfasis fuerte en análisis/diseño antes de construir.	Entregas parciales y evolutivas; cada ciclo recorre análisis→diseño→construcción→pruebas→release.
Gestión del cambio	Cambios controlados: impacto evaluado en modelos (casos de uso, robustez, secuencia) antes del código.	Cambios asumibles entre incrementos; backlog vivo y priorización continua.
Entrega de valor	Valor visible al completar paquetes de casos de uso priorizados en cada ciclo.	Valor temprano y continuo por incrementos funcionales.
Riesgo técnico	Riesgos mitigados por el diseño basado en modelo, aunque la usabilidad real puede validarse más tarde.	Riesgos reducidos por validación temprana e iterativa.
Integración y pruebas	Pruebas guiadas por casos de uso y escenarios; integración alineada a “slices” de casos de uso.	Integración continua por incremento; feedback técnico rápido.
Planificación	Planificación por paquetes de casos de uso; hitos claros de análisis/diseño/implementación.	Roadmap por incrementos; replanificación ágil con métricas.
Trazabilidad y auditoría	Muy alta: Caso de uso → Robustez → Secuencia → Clases/código.	Alta si se gobierna con DoD, control de cambios y evidencias por incremento.
Adecuación al contexto	Óptimo en dominios complejos donde se requiere rigor sin caer en cascada puro.	Óptimo cuando hay descubrimiento, I+D y feedback de usuarios expertos.
Riesgos si se aplica mal	Burocracia o sobre-modelado; riesgo de análisis prolongado y modelos desactualizados.	Deuda técnica o deriva si no hay arquitectura evolutiva.

Criterio	ICONIX	Incremental
Dependencia de usuarios	Alta al inicio para validar casos de uso; riesgo de centrarse demasiado en análisis si baja la participación.	Alta y deseable: feedback temprano guía la evolución funcional.

Fuente: Elaboración propia, 2025.

3.2.5.2. *Elección de la metodología*

Se adopta el enfoque Incremental para este proyecto debido a la necesidad de entregar valor funcional temprano y obtener feedback continuo de usuarios operativos (vendedores, choferes, líderes de línea). El sistema POS requiere adaptación rápida a cambios operativos y validación temprana de funcionalidades críticas como venta de boletos, procesamiento de pagos e integraciones en tiempo real mediante WebSocket. Aunque ICONIX ofrece mayor rigor en análisis y diseño mediante casos de uso, su énfasis en completar fases de análisis/diseño antes de construir retrasaría la entrega de funcionalidades operativas esenciales. El enfoque incremental permite validar cada funcionalidad con usuarios reales en producción, gestionar cambios mediante backlog vivo y priorización continua, y reducir riesgos técnicos mediante integración continua e iteraciones cortas. Esta estrategia es adecuada para un sistema que debe evolucionar según necesidades operativas cambiantes y donde la usabilidad y el rendimiento en tiempo real requieren validación temprana y continua.

3.3. DISEÑO DEL SISTEMA

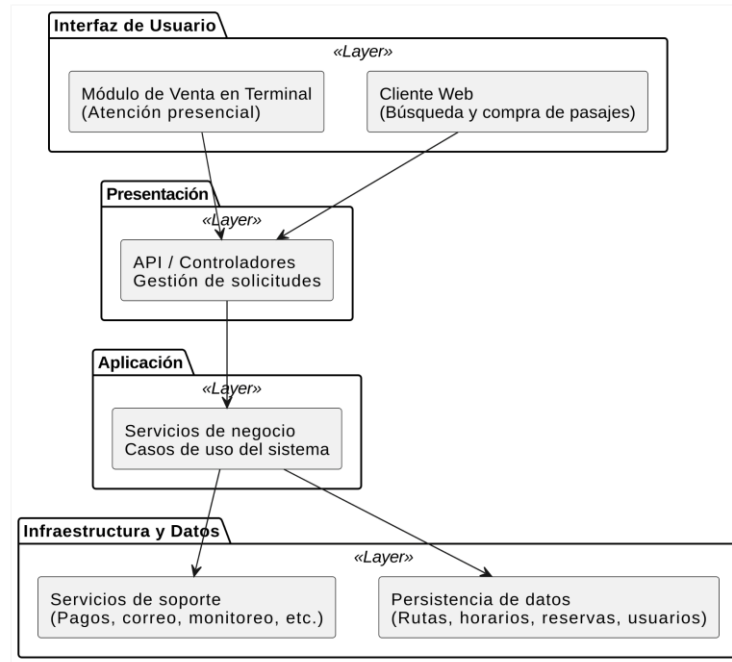
3.3.1. *ARQUITECTURA DE LA SOLUCION*

El sistema se organiza mediante una arquitectura en capas, lo que permite separar claramente las responsabilidades, controlar las dependencias y facilitar la evolución del proyecto. Cada capa cumple un propósito específico y se comunica únicamente con la capa inmediatamente superior o inferior. La estructura general puede resumirse en Capa de Interfaz → Capa de Exposición → Capa de Aplicación → Capa de Infraestructura

- **Capa Interfaz (React/Nextjs):** La capa de interfaz corresponde a la aplicación web que interactúa directamente con el usuario. Su función es presentar la información, capturar acciones y enviar solicitudes hacia el sistema mediante JSON sobre HTTPS, sin acceder a datos internos ni ejecutar lógica de negocio.

- Capa de Exposición (API): La capa de exposición actúa como intermediaria entre la interfaz y las operaciones internas. Recibe solicitudes como búsqueda de viajes, creación de reservas, autenticación de usuarios o validación de pagos; verifica que los datos cumplan los formatos establecidos y aplica los controles básicos de acceso antes de remitir la solicitud a la capa de aplicación. Su responsabilidad es exclusivamente gestionar comunicación y validaciones iniciales.
- Capa de Aplicación: En esta capa se encuentran los casos de uso del sistema definidos en el análisis de requerimientos: gestión de rutas, cálculo de disponibilidad, creación de reservas, confirmación de compras, cancelaciones, registro y administración de usuarios. Aquí se coordinan los procesos necesarios para completar la compra de un pasaje y se aplican las reglas operativas, garantizando coherencia entre la información mostrada al cliente y las políticas de la empresa.
- Capa Infraestructura / Datos: La capa de infraestructura y datos proporciona los servicios técnicos que sostienen el funcionamiento del sistema: base de datos para almacenar usuarios, rutas, horarios, reservas y transacciones; servicios de pago; almacenamiento de comprobantes; mecanismos de caché para mejorar tiempos de respuesta; y herramientas de monitoreo. Su propósito es asegurar persistencia, integridad de los datos y disponibilidad para las operaciones críticas del proceso de venta de pasajes.

Ilustración 5 Diagrama de arquitectura en capas de la solución



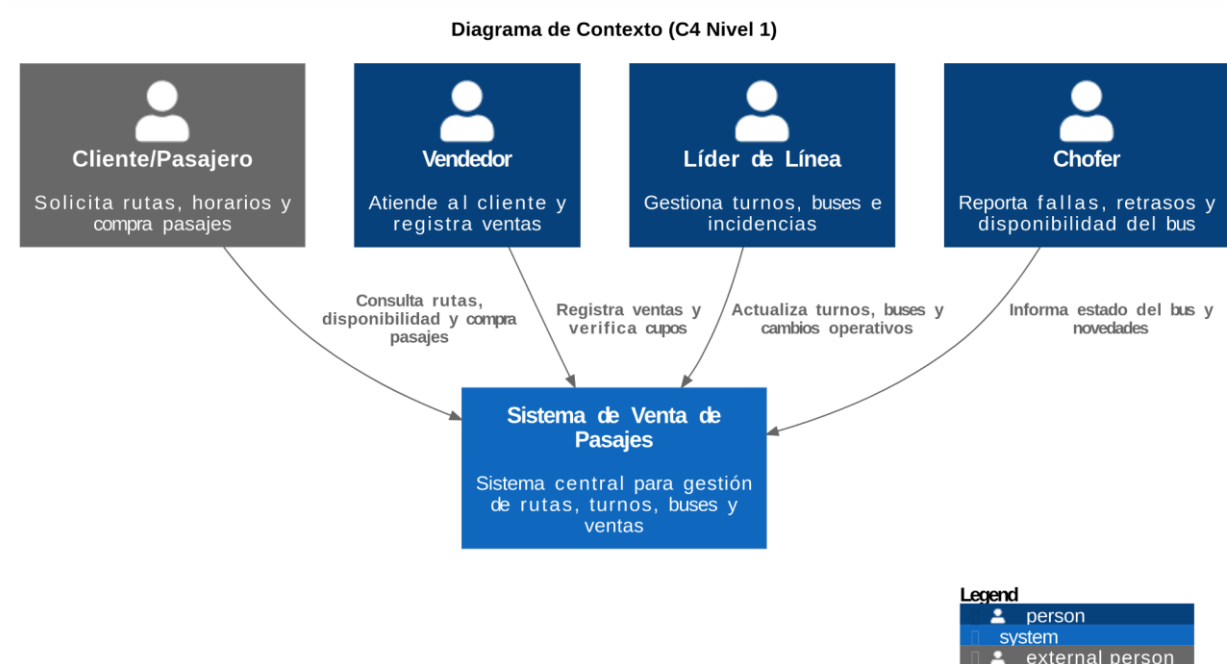
Fuente: Elaboración propia, 2025.

A partir de esta base se generarán las vistas del modelo C4 correspondientes al Nivel 1 (Contexto) y Nivel 2 (Contenedores), en las que se describirán las interacciones externas del sistema y se detallara cómo se organizan sus componentes técnicos.

3.3.1.1. Vista de contexto

La figura 3.5 presenta la vista de contexto del sistema, donde se identifican los actores humanos, los sistemas externos y las interacciones de alto nivel.

Ilustración 6 Diagrama de contexto (C4 – Nivel 1) de la solución

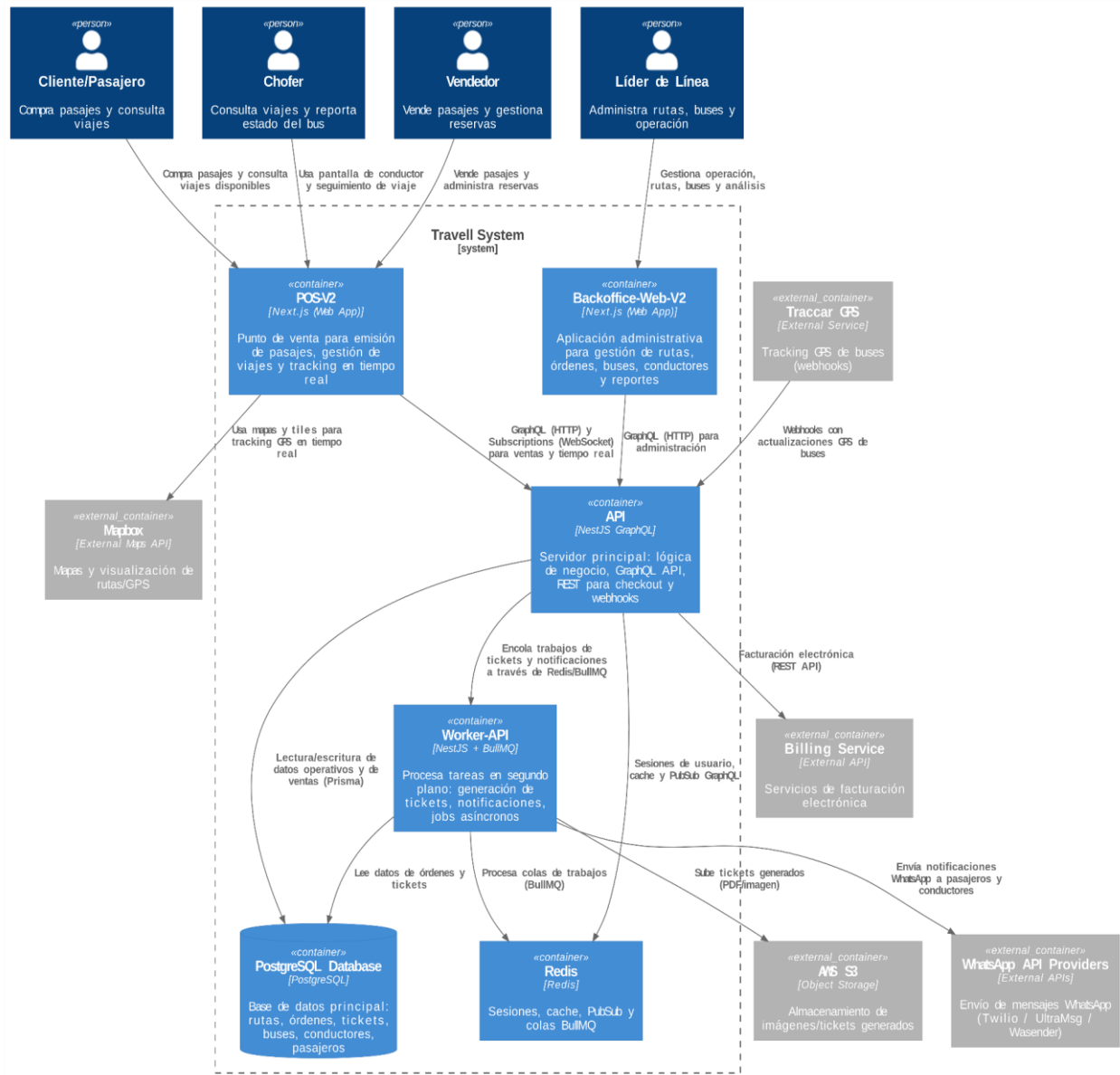


Fuente: Elaboración propia, 2025.

3.3.1.2. Vista de contenedores

La figura 3.6 muestra los contenedores que componen Alexandria, sus responsabilidades y los protocolos de interacción. El límite del sistema (línea discontinua) agrupa frontend, backend, workers de ingesta, base de datos, almacenamiento de objetos y el servicio de modelos. Fuera del límite se ubican las fuentes externas (TCP/TSJ) y el servicio de correo.

Ilustración 7 Diagrama de contenedores (C4 – Nivel 2) de la solución



Fuente: Elaboración propia, 2025.

3.3.1.3. Paquetes y organización del código

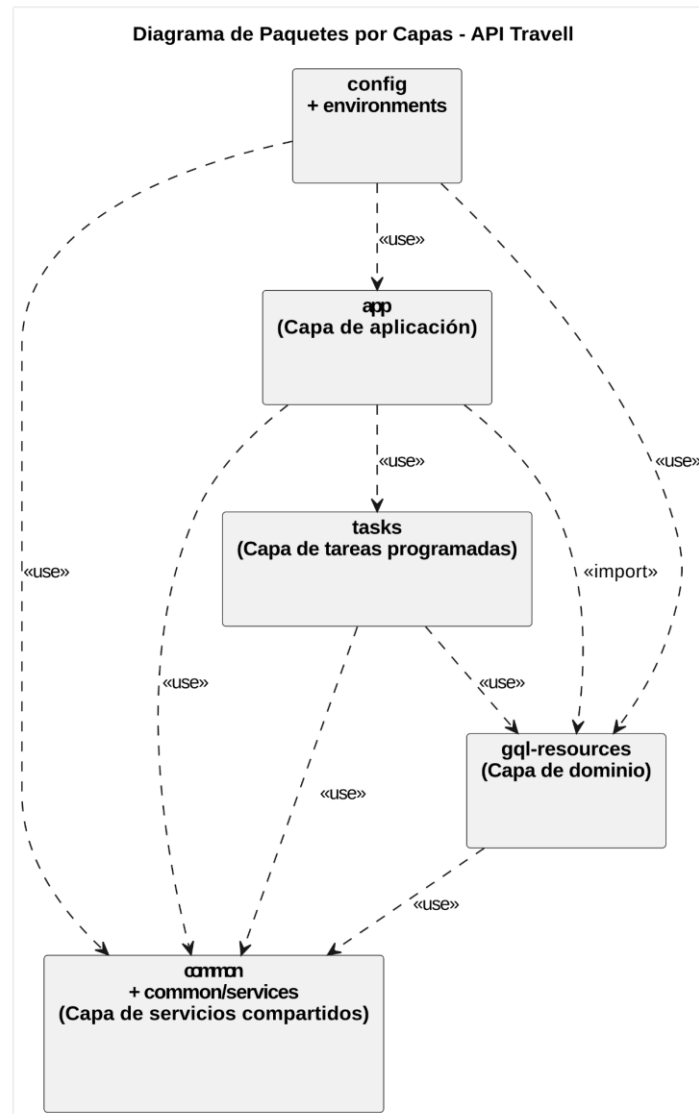
a. Api

La API de Travell se propone implementar sobre el framework NestJS, siguiendo una arquitectura modular orientada a dominios y expuesta principalmente mediante GraphQL. La

organización por módulos busca minimizar el acoplamiento, favorecer la reutilización de servicios compartidos y facilitar la evolución independiente de cada área funcional. A alto nivel, la solución se estructura en las siguientes capas lógicas:

- Capa de aplicación: arranque y configuración global (app/).
- Capa de dominio (GraphQL): módulos de negocio orientados a recursos (gql-resources/).
- Capa de servicios compartidos: servicios técnicos reutilizables (common/services/).
- Capa de tareas programadas: procesos en segundo plano y cron jobs (tasks/).
- Capa de infraestructura y configuración: filtros, guards, configuración y entorno (common/, config/, environments/).

Ilustración 8 Diagrama de paquetes generales del backend



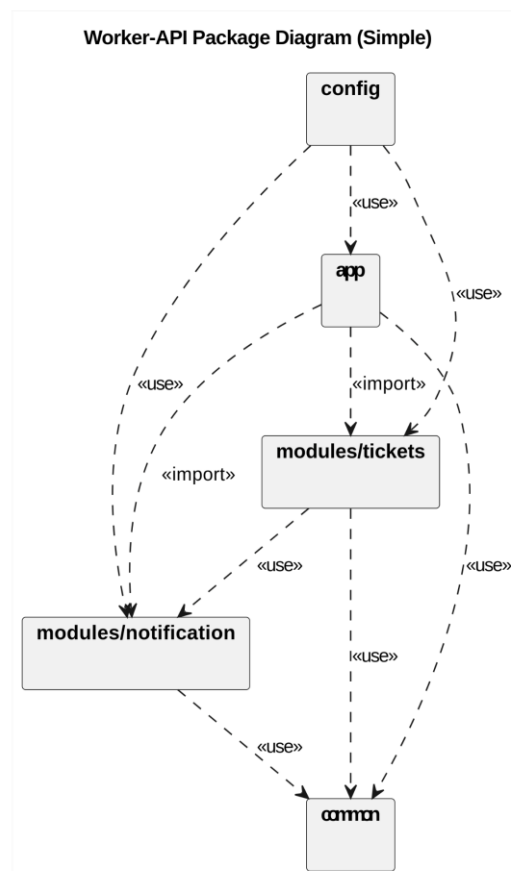
Fuente: Elaboración propia, 2025.

b. Worker

La Worker-API de Travell se plantea como un servicio independiente basado en NestJS, orientado al procesamiento de trabajos en segundo plano mediante colas BullMQ (sobre Redis). Su finalidad es desacoplar de la API principal aquellas tareas que son pesadas, lentas o que requieren reintentos controlados, tales como la generación de tickets en formato de imagen o el envío de notificaciones por WhatsApp, SMS o email. La arquitectura de este servicio se estructura en los siguientes paquetes principales:

- El paquete app/ contiene el módulo raíz y la configuración global del servicio
- El paquete modules/tickets/ representa el módulo de dominio responsable de procesar trabajos de generación de tickets y disparar notificaciones.
- El paquete modules/notification/ encapsula la lógica para consumir trabajos de notificación y enviar mensajes por WhatsApp:
- Paquete common/ (Utilidades compartidas del worker): Este paquete agrupa elementos transversales a los módulos de tickets y notificaciones.

Ilustración 9 Diagrama de paquetes generales del backend



Fuente: Elaboración propia, 2025.

c. Pos

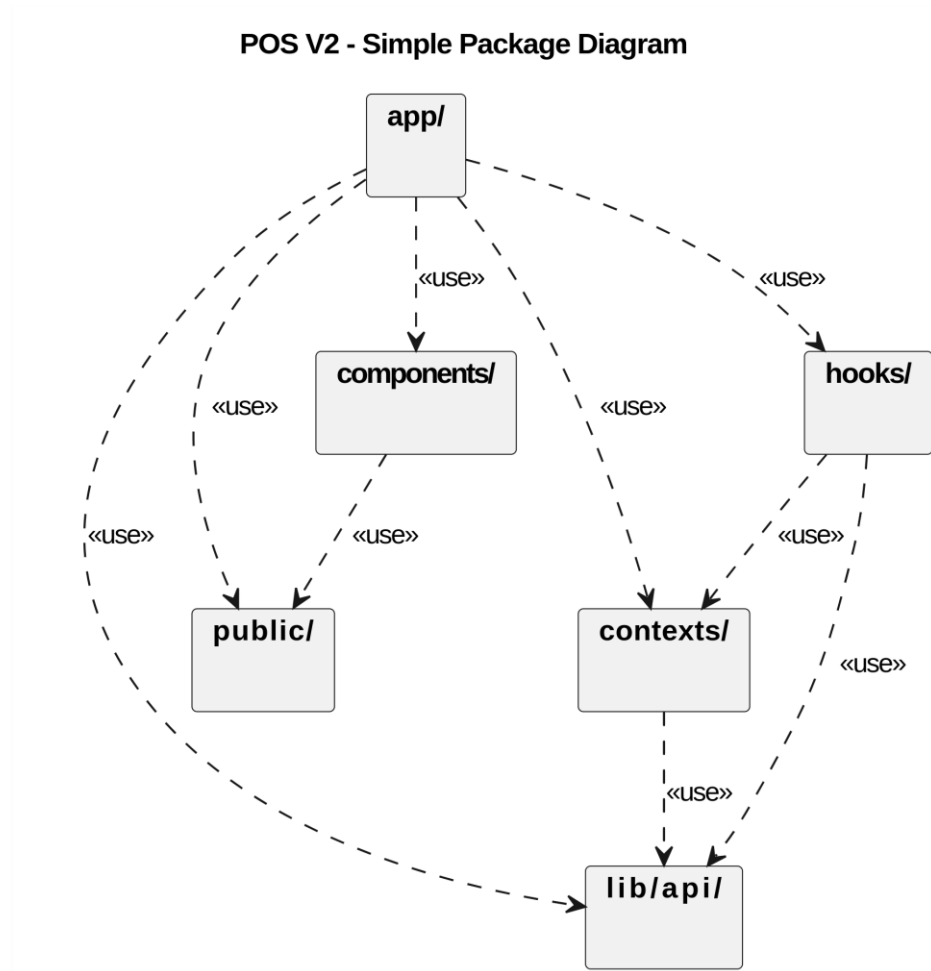
El Pos seguirá una arquitectura modular basada en el App Router de Next.js, donde las rutas y layouts viven en src/app/, mientras que la UI reutilizable se concentra en src/components/ y la

lógica de datos queda aislada en `src/lib/api/`. El estado global (contextos y stores), junto a hooks personalizados, se organiza en paquetes separados para mantener la lógica desacoplada de la presentación. Las utilidades, tipos e internacionalización se agrupan como capas compartidas para toda la aplicación. Este diseño favorece claridad, modularidad y cortes verticales simples por funcionalidad.

- `app/` — Rutas, layouts y server components.
- `components/` — Componentes UI reutilizables.
- `lib/api/` — Cliente GraphQL y funciones por dominio.
- `contexts/` — Contextos globales
- `hooks/` — Hooks personalizados para datos y UX.
- `public/`: assets estáticos (imágenes, iconos, fuentes).

El criterio general es seguir la convención de Next.js, separar UI (`components/`) de lógica de aplicación (`lib/`) y centralizar el acceso a datos en `lib/` para mantener coherencia, tipado y manejo uniforme de errores.

Ilustración 10 Diagrama de paquetes generales del frontend



Fuente: Elaboración propia, 2025.

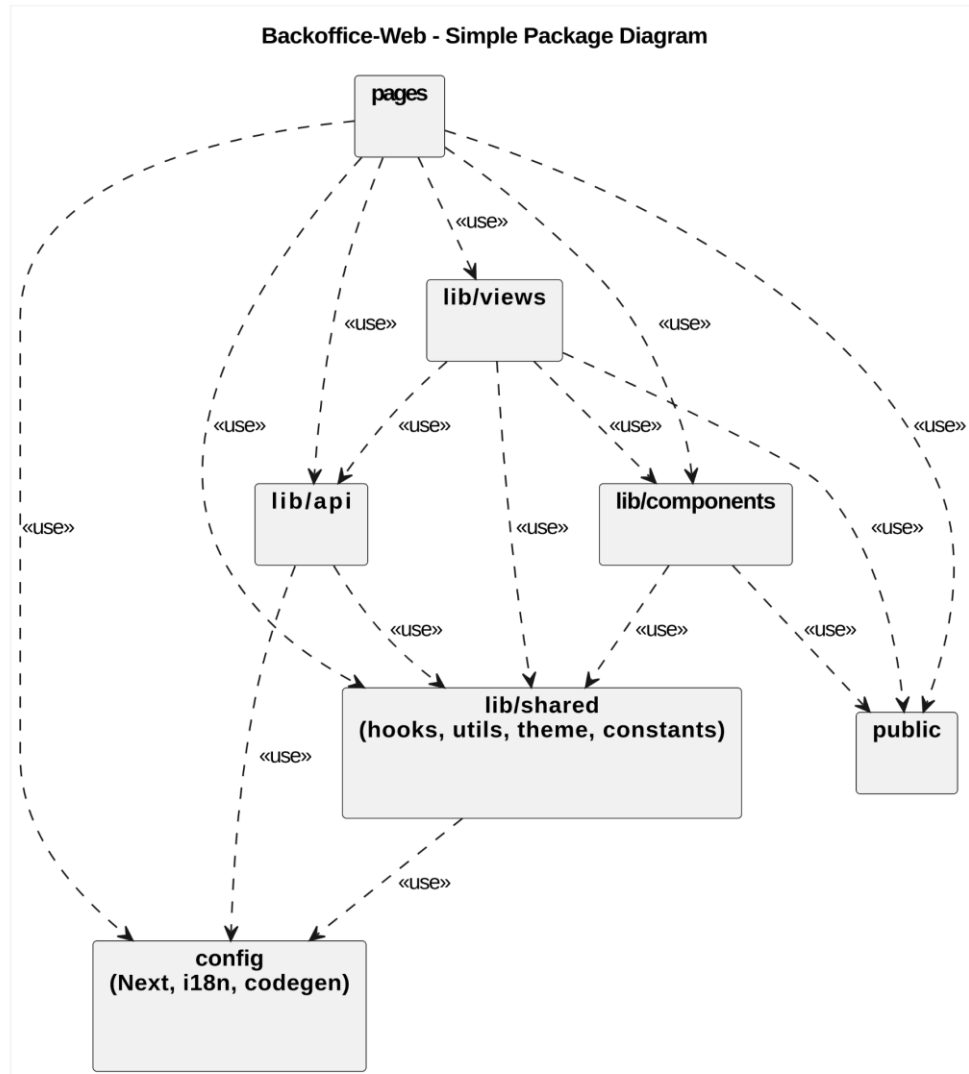
d. Backoffice

El frontend del Backoffice sigue una arquitectura modular basada en la convención del Pages Router de Next.js, priorizando la separación por responsabilidades y la composición progresiva de UI, datos y lógica. La estructura general organiza páginas, vistas, componentes, cliente de datos y utilidades compartidas de forma que permita construir verticalmente cada funcionalidad del sistema (por ejemplo: Buses → página + vista + componentes + queries), manteniendo una frontera clara con la API y reduciendo complejidad accidental.

La organización por paquetes busca que cada parte de la aplicación cumpla un propósito específico:

- `pages/` actúa como capa de entrada, donde se definen rutas, se aplica SSR/SSG, y se coordinan layouts globales.
- `lib/views/` agrupa la lógica de presentación por dominio (buses, rutas, órdenes, notificaciones, etc.), concentrando la composición de datos + UI sin sobrecargar las páginas.
- `lib/api/` establece la frontera con el backend, integrando Apollo Client, React Query y los tipos generados desde GraphQL, garantizando un acceso unificado a los datos.
- `lib/hooks/`, `lib/utils/`, `lib/theme/` y `lib/constants/` proveen capas de lógica transversal, utilidades y configuraciones que permiten mantener el código consistente y encapsulado.
- `lib/components/` funciona como una biblioteca interna de componentes reutilizables, abstraídos de la lógica de negocio y diseñados para uso transversal.
- `public/` y los archivos de configuración complementan la estructura con recursos estáticos, traducciones, middlewares y definiciones del entorno.

Ilustración 11 Diagrama de paquetes generales del frontend



Fuente: Elaboración propia, 2025.

3.3.2. MODELO DE DATOS

3.3.2.1. Esquema de base de datos relacional

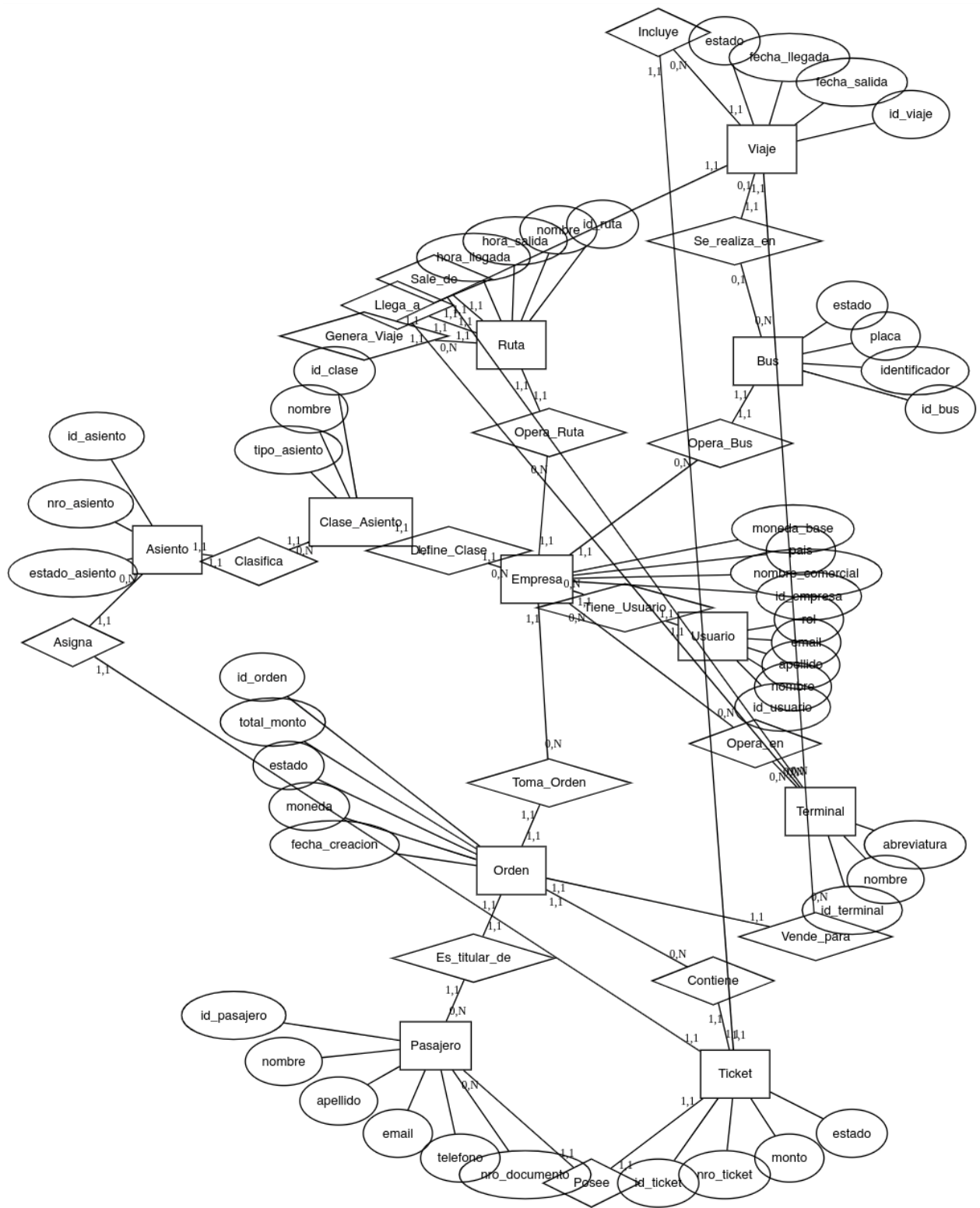
En este apartado se realiza el diseño de la base de datos relacional, tomando como referencia el modelo de datos lógico y los requisitos priorizados; se definen entidades, relaciones y restricciones de integridad (claves primarias y foráneas, unicidad y opcionalidad), junto con criterios de normalización orientados a reducir redundancia y preservar consistencia. Este esquema funcionará como contrato de información para los módulos del sistema y guiará la construcción por incrementos, permitiendo incorporar, en cada ciclo, tablas y relaciones

pertinentes al alcance liberado, manteniendo trazabilidad con casos de uso y facilitando la evolución controlada del modelo hacia su implementación física.

a. Modelo entidad-relación

El modelo entidad-relación a nivel conceptual se presenta en la figura 3.10.

Ilustración 12 Modelo entidad-relación del sistema



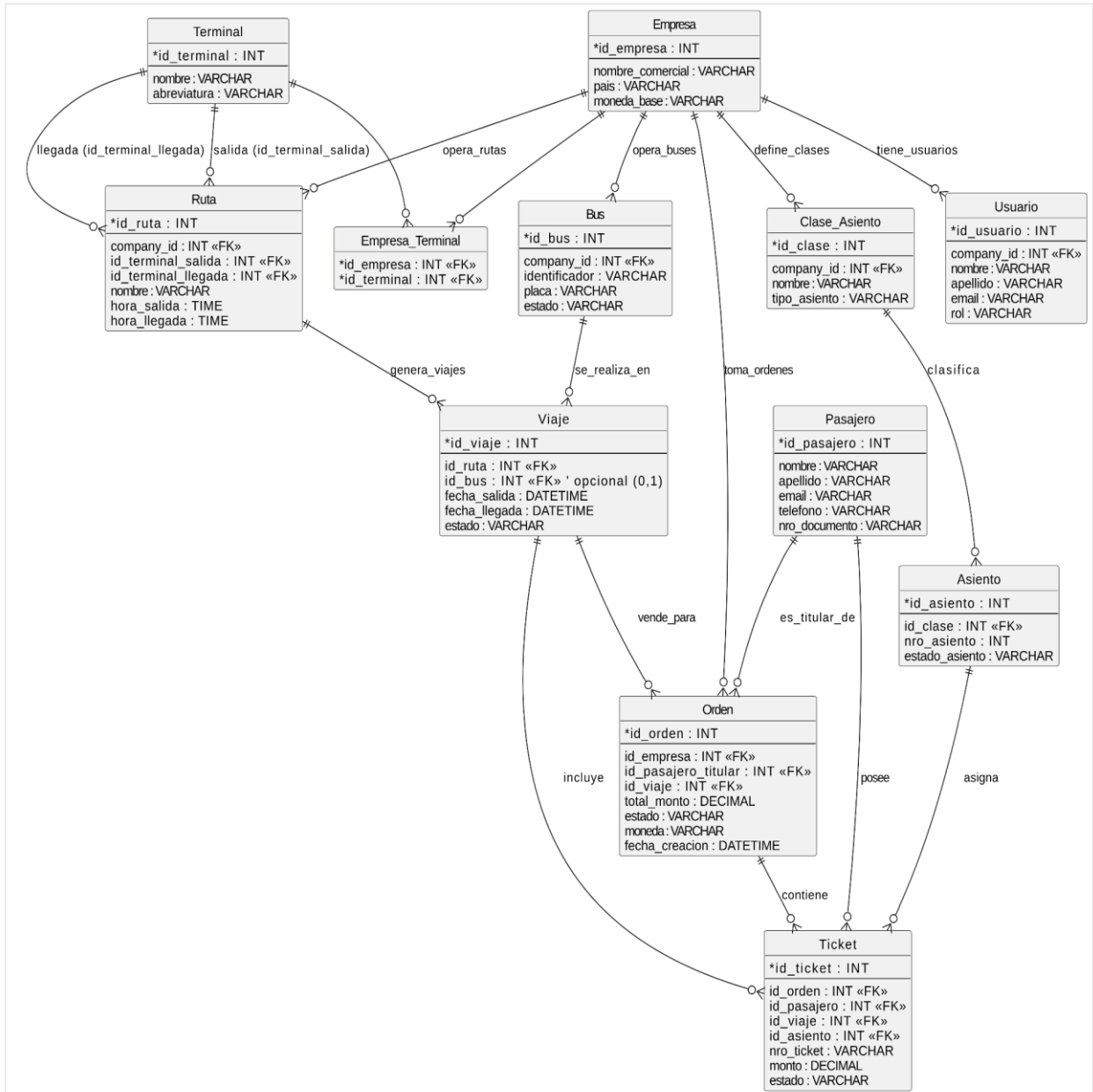
Fuente: Elaboración propia, 2025.

b. Modelo de datos lógico

A continuación se presenta el modelo de datos lógico del sistema, cuyo objetivo es describir, de forma independiente de cualquier base de datos o framework, las entidades de negocio y sus relaciones que soportarán los procesos operativos y transaccionales.

El modelo organiza las áreas clave del dominio: identidad y control de acceso (User, Role, Permission, asociaciones), registro y seguridad (PendingRegistration, PasswordResetTicket), gestión documental (Document, Chunk), interacción y trazabilidad de búsquedas (ChatSession, ChatMessage, SearchHit, Feedback), y gobernanza de accesos internos (StaffWhitelist). Esta estructuración favorece bajo acoplamiento y alta cohesión, y facilita la evolución incremental del sistema, sirviendo como contrato de referencia para el diseño físico posterior y la implementación de servicios y APIs.

Ilustración 13 Modelo de datos lógico del sistema



Fuente: Elaboración propia, 2025.

3.3.3. INTERACCIÓN USUARIO-SISTEMA

Esta sección describe cómo el vendedor, el pasajero, el líder de línea y el dueño interactúan con el sistema a través de diferentes interfaces como el punto de venta, las aplicaciones web y los paneles operativos. Cada acción iniciada por estos actores, desde consultar un viaje hasta registrar una operación o gestionar recursos, debe tener una correspondencia clara con los

procesos internos y las entidades del sistema, lo que asegura coherencia entre la experiencia de uso y las reglas de negocio.

Los flujos se representan mediante diagramas de secuencia que muestran el recorrido desde la acción del usuario hasta la respuesta generada por el sistema. Estos diagramas incluyen validaciones, consultas, actualizaciones y rutas de error para manejar situaciones como falta de disponibilidad, permisos insuficientes o datos inconsistentes. Con ello se garantiza un funcionamiento estable y una experiencia de usuario predecible.

3.3.3.1. Identificación de actores

En el sistema intervienen cinco actores principales: el vendedor, el pasajero, el líder de línea, el chofer y el dueño, cada uno utilizando diferentes superficies como el punto de venta, el panel administrativo o las herramientas móviles. El sistema opera como plataforma centralizada que asiste la venta de boletos en terminales físicas, gestiona la disponibilidad de asientos, registra operaciones con trazabilidad completa, procesa incidencias reportadas por el chofer, aplica reasignaciones ejecutadas por el líder de línea y actualiza la información en tiempo real para todos los usuarios. El dueño accede a reportes consolidados de ventas por terminal y periodo, mientras que el vendedor consulta disponibilidad y registra ventas bajo credenciales y parámetros definidos por el equipo técnico, en un entorno donde toda la operación depende de rutas habilitadas con bus asignado, conectividad activa y datos consistentes.

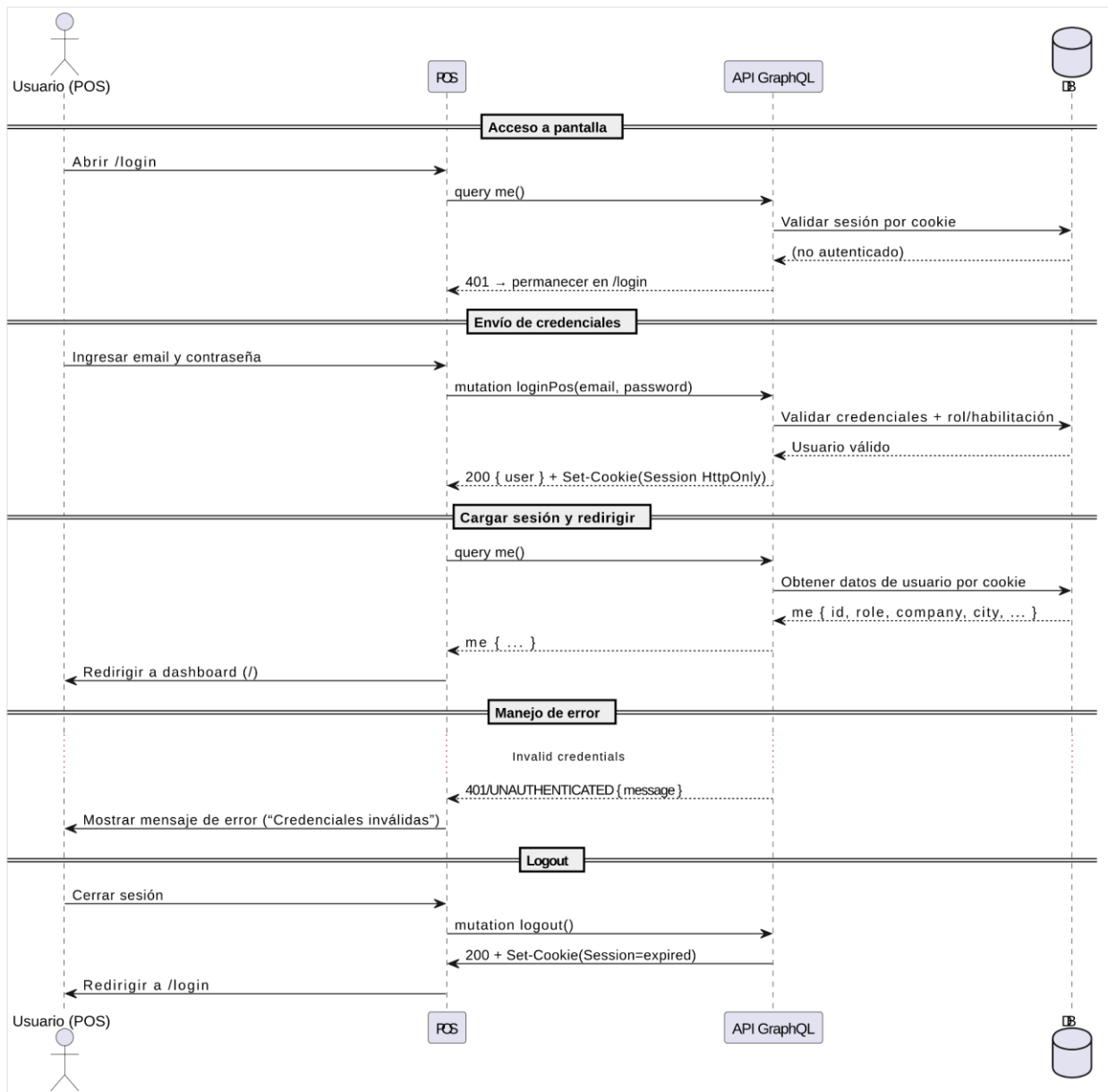
3.3.3.2. Diagramas de secuencia

Los diagramas de secuencia de esta sección describen, de extremo a extremo, el intercambio de mensajes entre la interfaz de usuario y los servicios internos para cada flujo funcional relevante. Cada diagrama detalla participantes, llamadas síncronas, respuestas y bifurcaciones de control (éxitos, errores, timeouts), así como fallbacks operativos cuando un componente no está disponible. Esta representación favorece la verificación de requisitos no funcionales (latencia, seguridad, trazabilidad) y sirve como contrato operativo entre frontend y backend, alineando la futura implementación con la arquitectura propuesta.

a. Gestión inicio de sesión (Usuario POS → Sistema)

- POS UI → RDS → API GraphQL → DB

Ilustración 14 Diagrama de secuencia de gestión de sesión

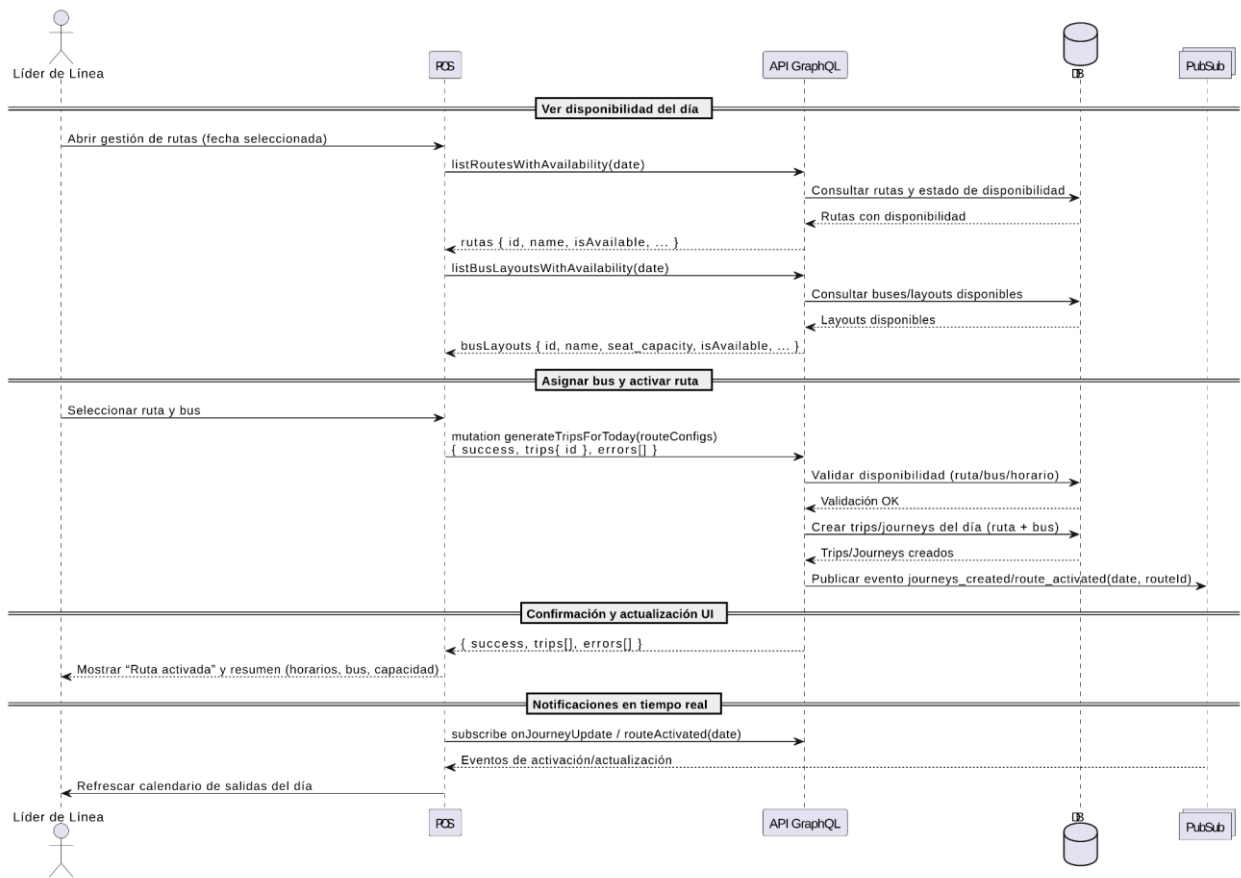


Fuente: Elaboración propia, 2025.

b. Disponibilidad y Asignación de Rutas (Lider de Linea)

- POS → RDS → API GraphQL → PubSub/DB

Ilustración 15 Diagrama de secuencia de Disponibilidad y Asignación de Rutas

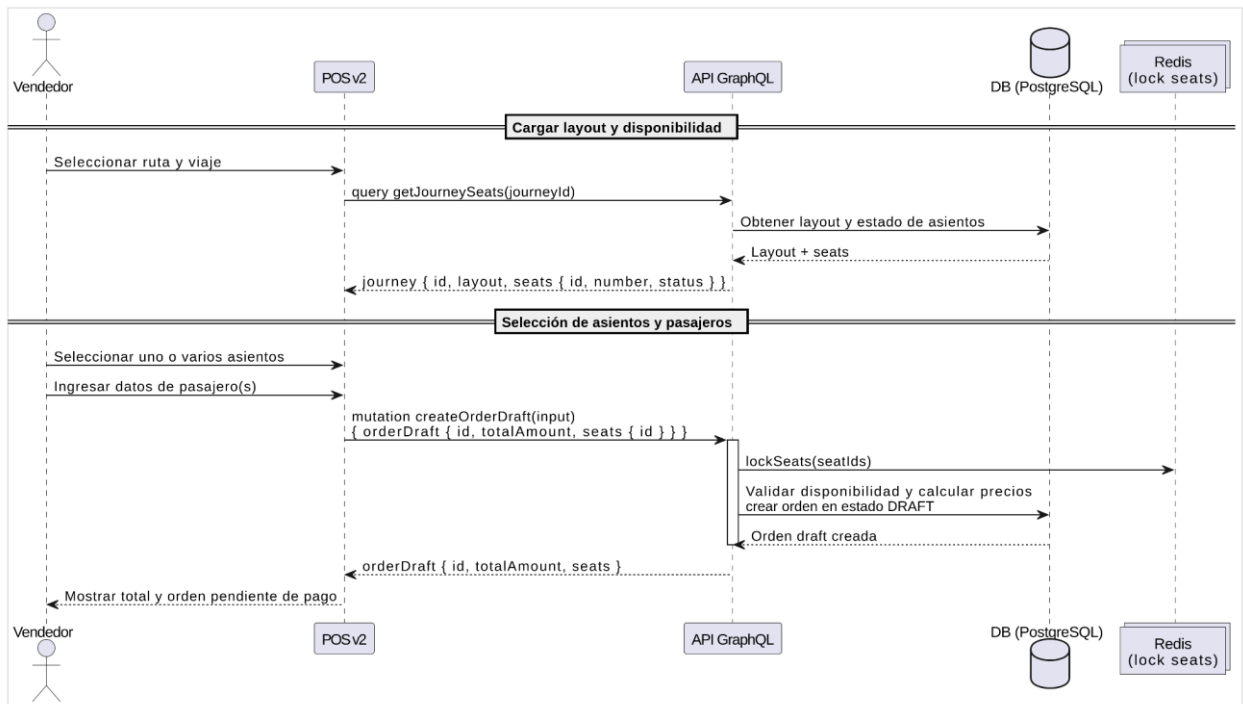


Fuente: Elaboración propia, 2025.

c. Selección de Asientos y Creación de Orden (Vendedor POS)

- POS UI → API GraphQL → DB (PostgreSQL) → Redis (lock seats)

Ilustración 16 Diagrama de secuencia de Selección de Asientos y Creación de Orden

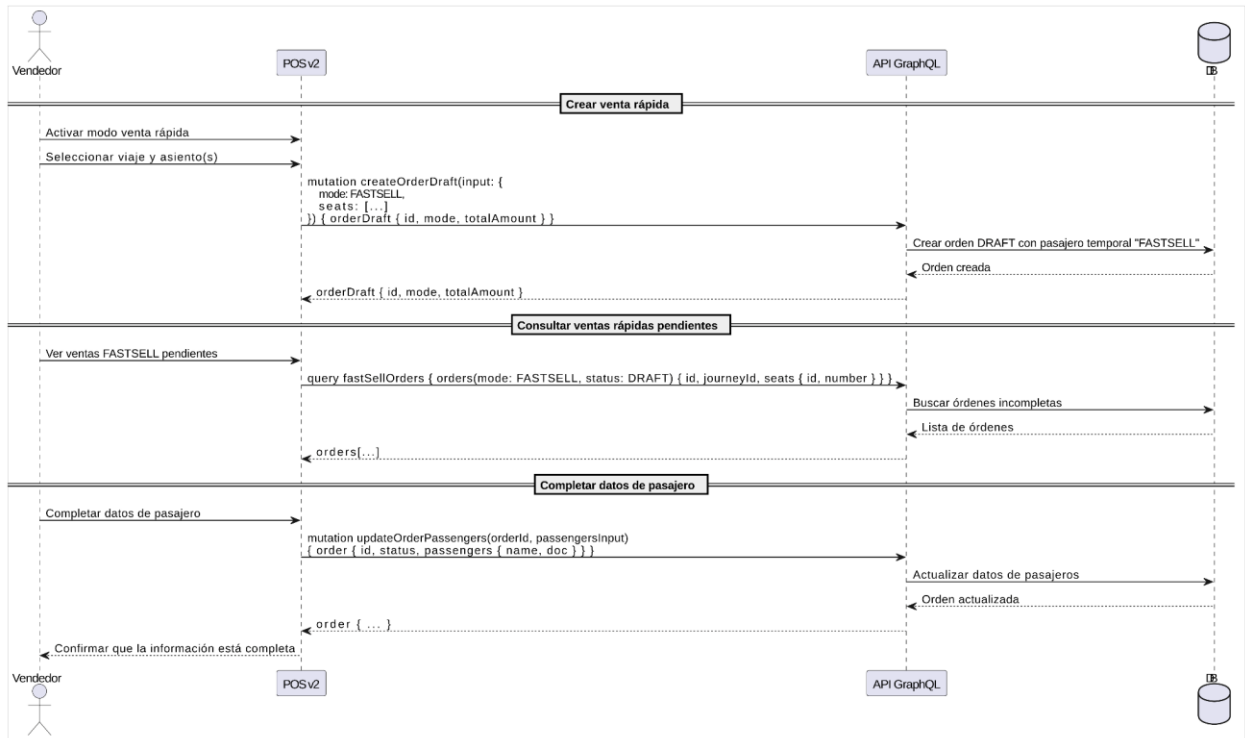


Fuente: Elaboración propia, 2025.

d. Creación de Venta Rápida

- POS UI → API GraphQL → DB

Ilustración 17 Diagrama de secuencia de Creación de Venta Rápida

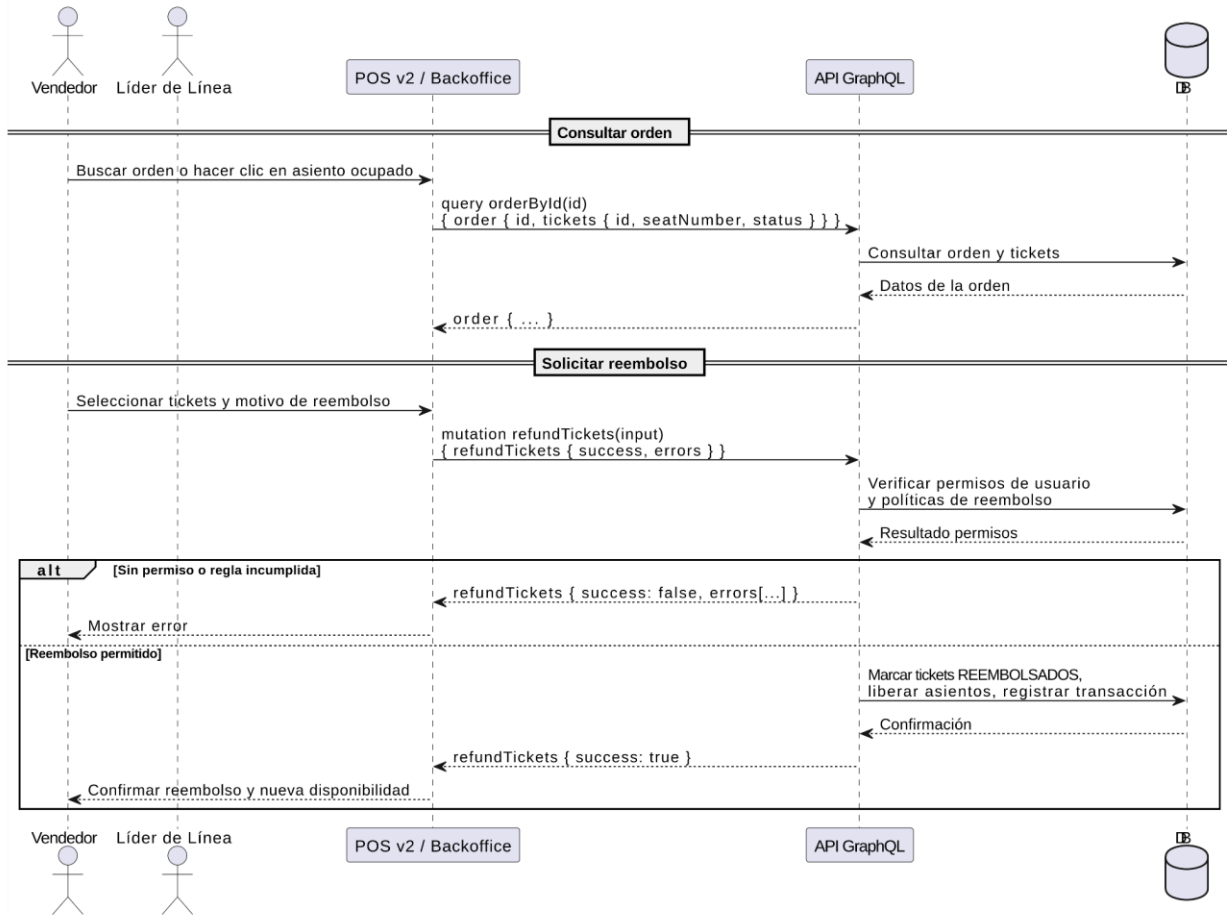


Fuente: Elaboración propia, 2025.

e. Reembolso de Tickets

- POS v2 / Backoffice → API GraphQL → DB

Ilustración 18 Diagrama de secuencia de Reembolso de Tickets

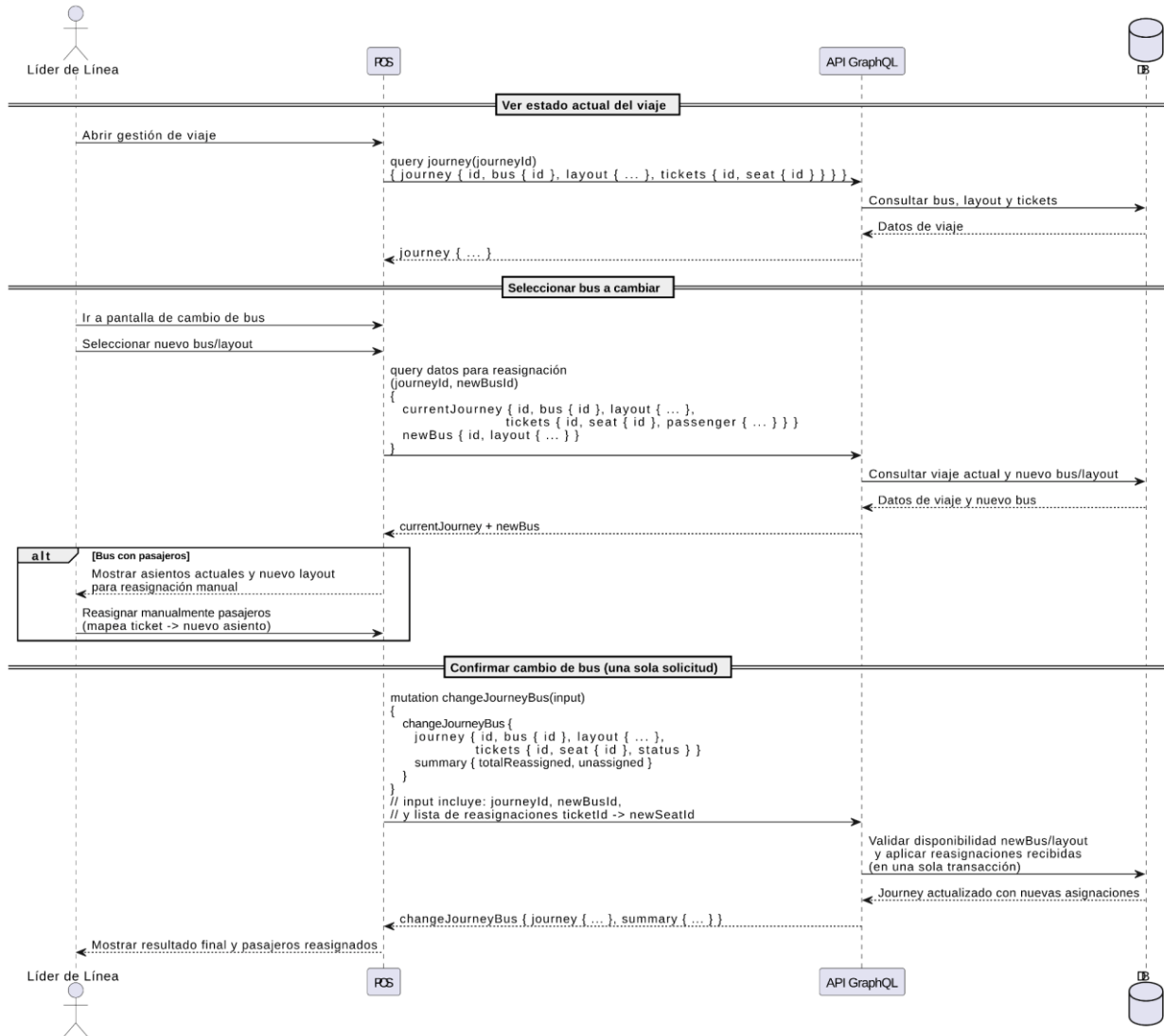


Fuente: Elaboración propia, 2025.

f. reasignacion de bus y pasajeros

- POS UI → API GraphQL → DB

Ilustración 19 Diagrama de gestión de Cambio de Bus en un Viaje

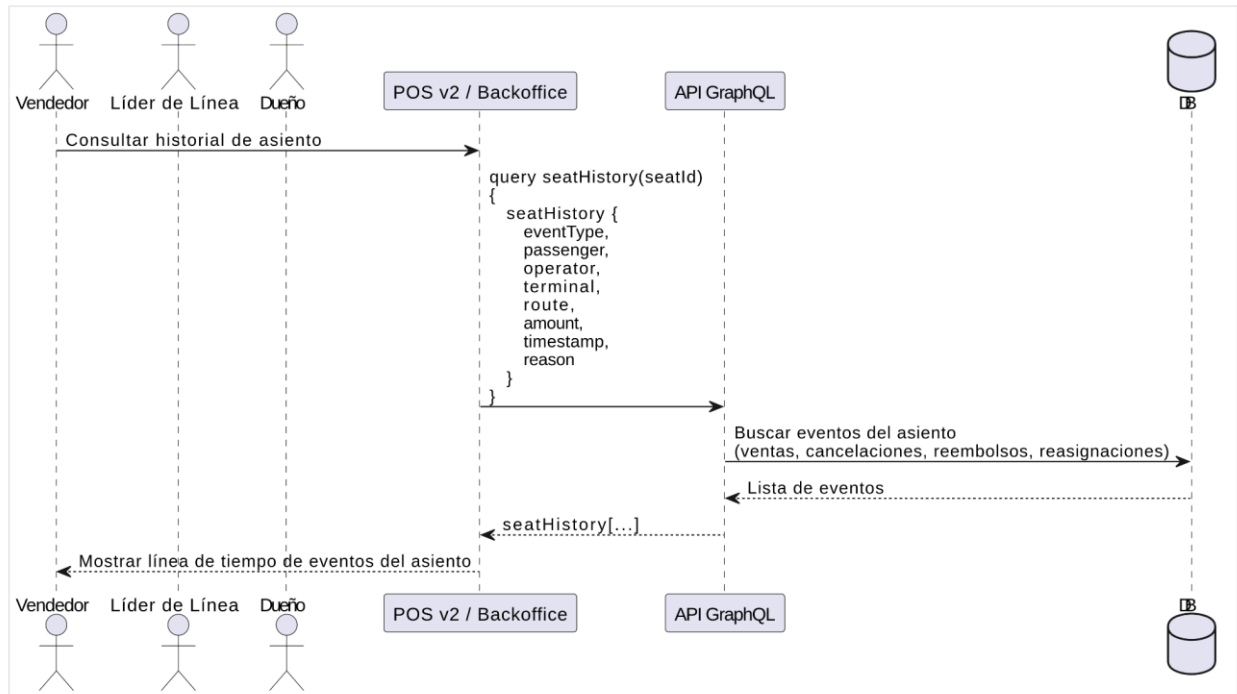


Fuente: Elaboración propia, 2025.

g. historial de Asiento

- POS v2 / Backoffice → API GraphQL → DB

Ilustración 20 Diagrama de gestión de Historial de Asiento

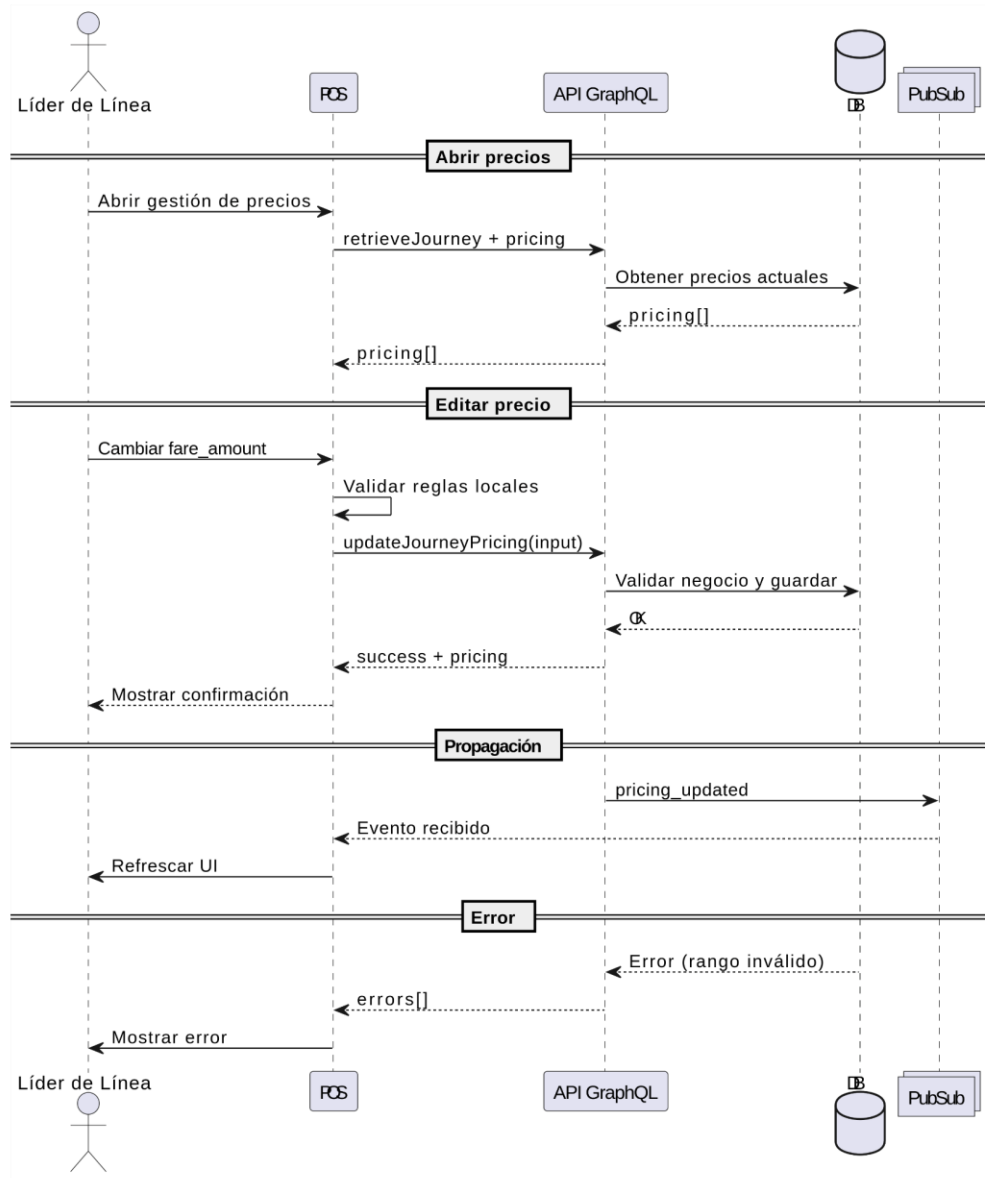


Fuente: Elaboración propia, 2025.

h. edicion de precio (Staff)

- POS UI → API GraphQL → DB → PubSub

Ilustración 21 Diagrama de Actualización de Precios



Fuente: Elaboración propia, 2025.

3.3.4. SEGURIDAD Y CUMPLIMIENTO

El sistema adopta una postura de seguridad por diseño que abarcará gestión de identidades y secretos, controles de acceso coherentes con los flujos API y mecanismos de trazabilidad

operativa. A falta de requisitos regulatorios específicos, se prioriza la defensa en profundidad, el principio de mínimo privilegio y la observabilidad de eventos críticos.

3.3.4.1. Gestión de seguridad de acuerdo a roles

a. Identidades y autenticación

Se implementa autenticación JWT Bearer para el consumo de endpoints protegidos; el access token incluirá claims mínimos (p. ej., role, email, name), y se habilita un flujo de refresh para renovación controlada del acceso.

b. Gestión de secretos

Las claves de firma JWT, credenciales de base de datos y las llaves de acceso a almacenamiento de objetos se custodian fuera del repositorio de código (variables de entorno y secret stores del entorno de despliegue). Se rotan secretos sensibles (incluida la clave de firma JWT) de acuerdo con ventanas de mantenimiento, con invalidez explícita de refresh tokens anteriores cuando aplique.

c. Higiene de credenciales

Las contraseñas de usuarios no se almacenan en texto plano; se usa hashing con salt y factor de trabajo ajustable. Se establecen límites de intentos (login) y throttling por IP/email para proteger ataques de fuerza bruta.

3.3.5. PLANIFICACIÓN DE INCREMENTOS PARA LA CONSTRUCCIÓN DEL SISTEMA

Esta sección detalla la estrategia de ejecución del proyecto bajo un enfoque incremental, tal como se definió en la selección metodológica, dividiendo la construcción en cuatro incrementos que se implementan y validan de forma secuencial. Cada incremento establece objetivos claros, alcance funcional acotado, entregables verificables y criterios de cierre (pruebas, documentación y aceptación), manteniendo trazabilidad explícita con los requerimientos funcionales. Con ello se reduce el riesgo técnico, se facilita la retroalimentación temprana de

usuarios y se entrega valor de manera progresiva, permitiendo ajustes informados antes de avanzar al siguiente incremento.

3.3.5.1. *Alcance y objetivo por incremento*

a. Incremento 1: Identidad, accesos y configuración base

- Objetivo: Habilitar el acceso controlado al sistema y la gestión centralizada de usuarios y datos base operativos, administrados por el equipo técnico.
- Autenticación por correo + contraseña (login/logout).
- Gestión de usuarios por parte del equipo técnico.
- Gestión base de configuración:
 - Carga inicial de rutas, turnos, buses, capacidad, precios base y reglas operativas mínimas (sin aún toda la lógica dinámica).
- Modelo básico de roles:
 - Distinción mínima entre Equipo Técnico, Líder de Línea, Vendedor, Chofer, Dueño.
- RF cubiertos:
 - RF-01 Autenticación de usuario
 - RF-02 Administración de usuarios y accesos
- Entregables:
 - Endpoints de autenticación (/login, /logout).
 - Pantalla de login.

b. Incremento 2: Activación operativa y disponibilidad

- Objetivo: habilitar la operación diaria básica: activación de rutas, gestión de precios sobre rutas activas, consulta de disponibilidad en tiempo real e incidencias de operación.
- Alcance:
 - Asignación de bus a ruta y turno (activación operativa):
 - Ajuste de precios por ruta activa
 - Consulta de disponibilidad de asientos
 - Gestión de incidencias
 - Reasignación de bus a ruta activa por el Líder de Línea.

- Actualización de disponibilidad y datos visibles tras reasignación.
- RF cubiertos:
 - RF-03 Asignación de bus a ruta y turno (Activación operativa)
 - RF-04 Ajuste de precios por ruta activa
 - RF-05 Consulta de disponibilidad de asientos
 - RF-06 Gestión de incidencias
- c. Incremento 3: Venta de boletos, pagos y reembolsos
 - Objetivo: Soportar el flujo completo de venta de boletos con selección de asientos, modos de venta normal y rápida, procesamiento de pagos y reembolsos, impactando en tiempo real la disponibilidad.
 - Alcance:
 - Venta con selección de asientos:
 - Visualización del layout del bus con asientos disponibles/ocupados.
 - Selección múltiple de asientos en una ruta/turno activo.
 - Registro de datos de pasajero por asiento (nombre, apellido, CI, teléfono).
 - Modo de venta rápida:
 - Venta con datos mínimos (por ejemplo: solo asiento + identificador temporal).
 - Gestión posterior de actualización de datos de pasajero.
 - Listado de boletos pendientes de completar información.
 - Procesamiento de pagos:
 - Selección de método de pago (efectivo / QR)
 - Autenticación del vendedor mediante PIN antes del cobro.
 - Generación de orden de compra.
 - Actualización inmediata de disponibilidad.
 - Reembolso de boletos
 - RF cubiertos
 - RF-09 Venta de Boletos con Selección de Asientos
 - RF-10 Modo de Venta Rápida
 - RF-11 Procesamiento de Pagos
 - RF-12 Reembolso de Boletos

d. Incremento 4: Reportes y trazabilidad histórica

- Objetivo: Brindar herramientas de control y análisis al Dueño, Líder de Línea y vendedor, con información histórica de ventas, asientos y montos consolidados por terminal
- Alcance:
 - Liquidaciones y control de ventas
 - Montos consolidados por terminal
 - Historial de asientos
- RF cubiertos
 - RF-07 Liquidaciones y control de ventas
 - RF-08 Visualización de montos consolidados por terminal
 - RF-13 Historial de Asientos

3.3.5.2. *Resumen de cobertura por incremento*

Cuadro 3.12. Trazabilidad de incrementos y requerimientos funcionales

Incremento	Alcance clave	RF asociados	Entregables principales
I1 – Identidad, accesos y configuración base	Autenticación; gestión de usuarios; gestión base de rutas, turnos, buses, capacidades y precios iniciales.	RF-01, RF-02	Queries de usuarios y configuración; mutations de login/logout; mutations de alta/edición/habilitación de usuarios; carga inicial de datos operativos.
I2 – Activación operativa y disponibilidad	Creación de Horarios, visualización de disponibilidad de asientos por horario y bus disponible	RF-03, RF-04, RF-05, RF-06	Queries de rutas activas y disponibilidad; mutations para asignar/reasignar bus; mutations de ajuste de precios; mutations de incidencias.
I3 – Venta de boletos, pagos y reembolsos	Ventas Grupales con información de pasajeros, ventas rápidas y liberación de asientos en reembolsos	RF-09, RF-10, RF-11, RF-12	Queries de layout/asientos; mutations de venta y venta rápida; mutation de pago; mutation de reembolso y liberación de asiento.

I4 – Reportes, trazabilidad histórica y venta web	Reportes de facturación y ventas, así como la liquidación diaria. portal simple de compra online para el pasajero	RF-07, RF-08, RF-13	Queries de reportes y consolidados; queries de historial de asiento; validación de consistencia histórica.
--	---	---------------------	--

Fuente: Elaboración propia, 2025.

3.3.5.3. Criterios de cierre de incrementos

a. Criterios generales

- Funcionalidad completa respecto a los RF asignados a cada incremento.
- Pruebas: Pruebas de integración sobre queries, mutaciones clave y prueba exploratoria de UI.
- Seguridad y cumplimiento:
 - I1: autenticación segura, manejo de sesiones correcto, roles aplicados en todos los endpoints administrativos.
 - I2–I4: todos los endpoints incorporados respetan roles y permisos definidos.
- Aceptación: Demostración funcional y checklist firmado por equipo técnico y rol operativo correspondiente.

b. Criterios específicos por incremento

- I1 (Identidad, accesos y configuración base):
 - Autenticación operando extremo a extremo (login, logout).
 - Configuración base cargada correctamente (rutas, turnos, buses, precios iniciales).
- I2 (Activación operativa y disponibilidad):
 - Asignación de bus a ruta/turno operativa y reflejada de inmediato.
 - Precios activos actualizados correctamente en rutas asignadas.
 - Disponibilidad de asientos consultable en tiempo real desde el estado centralizado.
 - reasignación, actualización de disponibilidad
- I3 (Venta, pagos y reembolsos):
 - Selección de asientos funcionando (ocupado/libre actualizado en tiempo real).

- o Venta normal y “venta rápida” completan flujo completo sin duplicidad de asiento.
- o Autenticación por PIN para pagos verificada.
- o Procesamiento de pago (efectivo/QR) confirmando orden y cambiando estados correctamente.
- o Reembolso libera asiento, registra evento y mantiene consistencia de datos.
- I4 (Reportes y trazabilidad histórica):
 - o Reportes de ventas por viaje, vendedor, chofer y terminal generados correctamente.
 - o Consolidados por terminal (diario/semanal/mensual)
 - o Historial completo por asiento disponible

3.4. CONSTRUCCIÓN DEL SISTEMA

3.4.1. INCREMENTO 1: IDENTIDAD, ACCESOS Y PARAMETRIZACIÓN INICIAL

Esta sección describe la construcción del primer incremento del sistema, cuyo propósito es establecer la capa fundamental de autenticación, administración de usuarios y carga base operativa. Este incremento implementa la autenticación mediante correo y contraseña, la gestión del ciclo de sesión y el control de accesos por roles; además incorpora la administración técnica de usuarios y la carga inicial de entidades operativas esenciales (rutas, turnos, buses, capacidades y precios). Se abarcan tanto los servicios de backend (modelos, reglas de negocio y APIs) como el soporte mínimo de interfaz requerido para permitir el acceso seguro y la administración técnica inicial. Con ello se garantiza que solo personal autorizado pueda operar el sistema y que la estructura operativa mínima se encuentre disponible para los incrementos siguientes (activación operativa, disponibilidad, ventas y reportes).

3.4.1.1. Alcance del incremento

El objetivo de este incremento es habilitar el ingreso controlado al sistema mediante whitelist y verificación por OTP, autenticación JWT, recuperación de contraseña y gobierno básico de accesos (RBAC admin/user). Se cubrirán los requerimientos detallados en el cuadro 3.12. Se contempla el desarrollo completo de las funcionalidades tanto en el backend como el desarrollo de las interfaces de usuario.

3.4.1.2. Plan de trabajo y estimaciones

A continuación, se presenta el plan de trabajo de este incremento, detallando cada tarea con su estimación de horas.

Cuadro 3.13. Plan de trabajo del primer incremento

#	Tarea	Descripción breve	Estimación (h)
1	Modelo de usuarios y roles	Tablas, estados, timestamps, relación con auditoría	6
2	administración de usuarios (CRUD técnico)	Crear/editar/habilitar/deshabilitar usuarios; validaciones y reglas	8
3	Servicio de autenticación	Login, verificación de contraseña, emisión	6
4	Creación/activación de usuario	Alta condicionada por OTP válido; asignación de rol desde whitelist; auditoría	6
5	Gestión del ciclo de sesión	Endpoints /me, refresh rotativo, logout e invalidación de sesión	4
6	RBAC mínimo	Guards/policies para roles técnicos y roles operativos; protección de endpoints sensibles	8
7	RBAC mínimo	Guards/policies para roles técnicos y roles operativos; protección de endpoints sensibles	4
8	Carga base operativa (rutas, turnos, buses, capacidades, precios)	Modelo + APIs técnicas para registrar/editar entidades base	12
9	UI mínima de acceso	Formularios de login y visualización de errores/estados	8
10	Pruebas	Tests de APIs + flujos E2E de autenticación	12

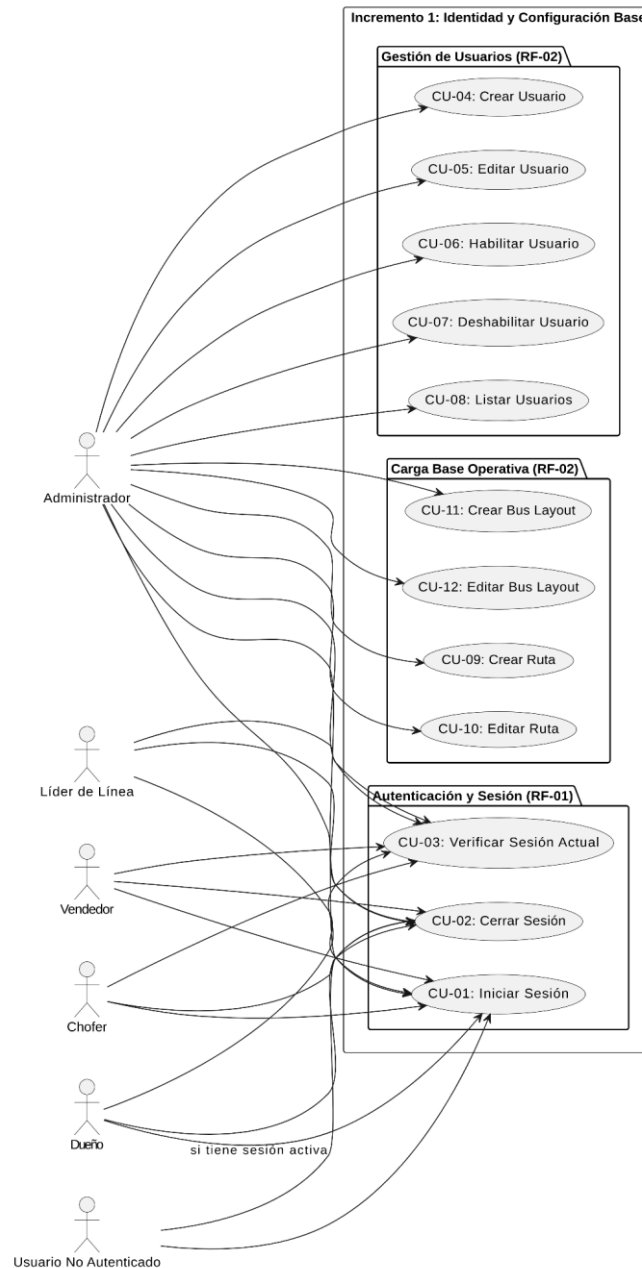
Fuente: Elaboración propia, 2025.

El esfuerzo total estimado asciende a 74 horas, incluyendo el desarrollo backend para autenticación, administración de usuarios, RBAC y carga base operativa; el desarrollo frontend mínimo para el acceso y el panel técnico; la ejecución de pruebas de API y validación funcional. Esta estimación parte de la existencia de un entorno de base de datos ya configurado, claves y secretos iniciales definidos.

3.4.1.3. Análisis funcional detallado

Esta sección desarrolla el análisis funcional detallado del incremento refinando los casos de uso para precisar precondiciones, postcondiciones y flujo principal. En la figura 3.21 se observan los casos de uso cubiertos en este incremento.

Ilustración 22 Casos de uso cubiertos en el primer incremento



Fuente: Elaboración propia, 2025.

A continuación, se detalla el catálogo de casos de uso con sus propósitos, actores asociados, precondiciones, postcondiciones y flujo principal. Entre paréntesis se incluye su requerimiento asociado para facilitar la trazabilidad.

CU-01	Iniciar sesión (RF-01)
Propósito	Gestionar el acceso seguro para usuarios habilitados.
Actores	Usuario autenticado (todos los roles)
Precondiciones	Usuario existente y habilitado.
Postcondiciones	Sesión activa; registro de auditoría.
Flujo principal	(1) Ingresar credenciales (2) Validar usuario y estado (3) Emitir cookie/ sesión (4) Redirigir según rol

CU-02	Cerrar sesión (RF-01)
Propósito	Finalizar la sesión del usuario de forma segura.
Actores	Usuario autenticado
Precondiciones	Sesión activa
Postcondiciones	Sesión invalidada y cookie eliminada; auditoría registrada.
Flujo principal	(1) Solicitar cierre (2) Invalidar sesión (3) Redirigir a login

CU-03	Verificar sesión actual (RF-01)
Propósito	Usuario autenticado.
Actores	Cookie de sesión presente.
Precondiciones	Contexto cargado o usuario redirigido a login.
Postcondiciones	Cuenta activa con credenciales válidas.
Flujo principal	(1) Consultar /me (2) Validar sesión (3) Retornar datos del usuario

CU-04	Crear usuario (RF-02)
Propósito	Registrar un nuevo usuario interno con rol asignado.
Actores	Equipo técnico.
Precondiciones	Administrador autenticado; email no existente
Postcondiciones	Usuario creado en estado deshabilitado; auditoría registrada.
Flujo principal	(1) Capturar datos (2) Validar unicidad y rol (3) Crear usuario (4) Confirmar creación

CU-05	Editar usuario (RF-02)
Propósito	Modificar datos generales y rol de un usuario.

Actores	Equipo técnico
Precondiciones	Usuario destino existente
Postcondiciones	Cambios actualizados y auditados
Flujo principal	(1) Seleccionar usuario (2) Editar campos (3) Validar cambios (4) Guardar edición

CU-06	Habilitar usuario (RF-02)
Propósito	Permitir acceso al sistema a un usuario existente.
Actores	Administrador
Precondiciones	Usuario deshabilitado.
Postcondiciones	Usuario habilitado; auditoría registrada.
Flujo principal	(1) Seleccionar usuario deshabilitado (2) Confirmar acción (3) Cambiar estado a habilitado

CU-07	Deshabilitar usuario (RF-02)
Propósito	Bloquear el acceso de un usuario al sistema.
Actores	Administrador
Precondiciones	Usuario habilitado.
Postcondiciones	Usuario deshabilitado; sesiones terminadas; auditoría registrada.
Flujo principal	(1) Seleccionar usuario (2) Confirmar deshabilitación (3) Persistir cambios e invalidar sesiones

CU-08	Listar usuarios (RF-02)
Propósito	Visualizar usuarios con búsqueda y filtros.
Actores	Administrador
Precondiciones	Administrador autenticado.
Postcondiciones	Listado mostrado.
Flujo principal	(1) Consultar lista (2) Mostrar resultados

CU-09	Crear ruta (RF-02)
Propósito	Registrar una ruta con terminales, horarios y layout base.
Actores	Equipo técnico.
Precondiciones	Terminales y layout existentes.
Postcondiciones	Ruta creada y disponible;
Flujo principal	(1) Ingresar datos (2) Validar terminales/horarios/layout (3) Crear ruta

CU-10	Editar ruta (RF-02)
--------------	---------------------

Propósito	Actualizar atributos de una ruta existente.
Actores	Administrador
Precondiciones	Ruta existente.
Postcondiciones	Cambios aplicados y auditados.
Flujo principal	(1) Seleccionar ruta (2) Editar campos (3) Validar (4) Guardar cambios

CU-11	Crear bus layout (RF-02)
Propósito	Registrar la estructura de asientos de un bus.
Actores	Administrador
Precondiciones	Administrador autenticado.
Postcondiciones	Layout disponible para buses y rutas.
Flujo principal	(1) Definir estructura (2) Validar (3) Crear layout

CU-12	Editar bus layout (RF-02)
Propósito	Actualizar estructura de un layout existente.
Actores	Administrador
Precondiciones	Layout existente.
Postcondiciones	Cambios vigentes y auditados.
Flujo principal	(1) Seleccionar layout (2) Editar estructura (3) Guardar cambios

3.4.1.4. *Diseño de solución*

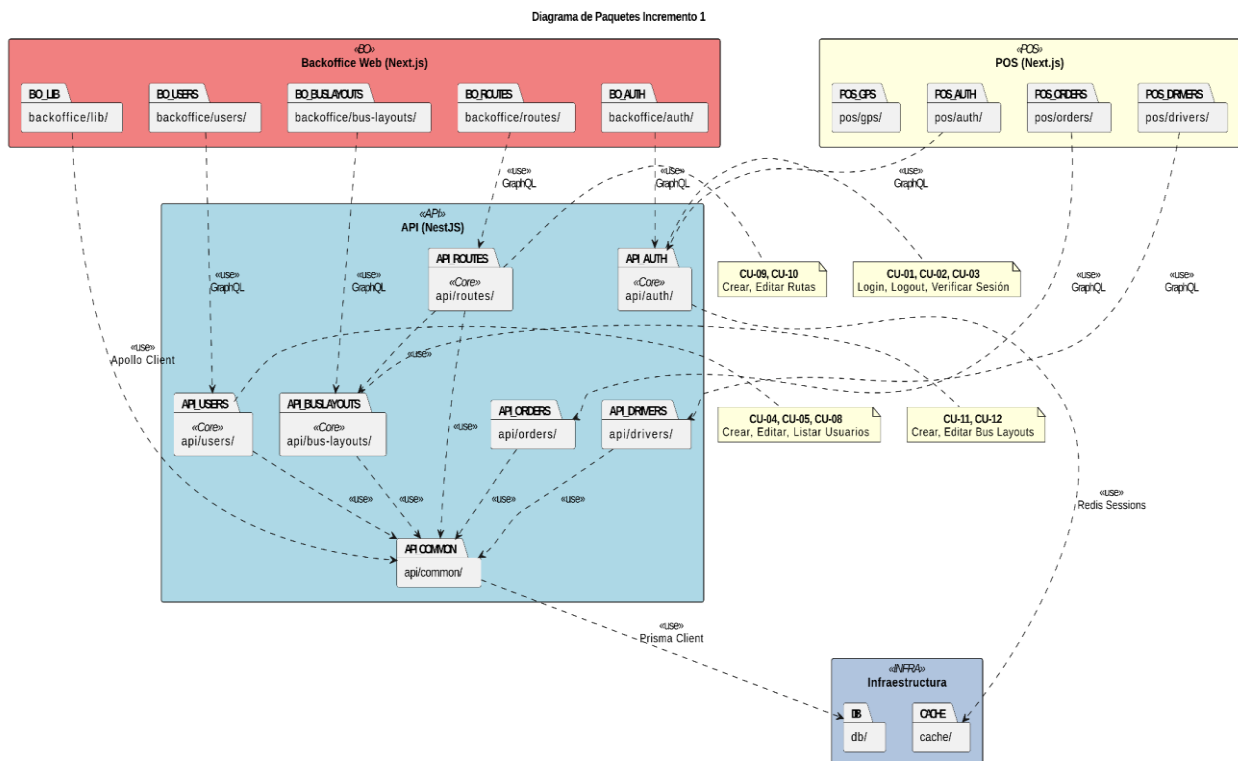
a. Aplicaciones

La solución se implementa como una plataforma modular basada en NestJS y GraphQL, donde cada subdominio funcional se encapsula en un módulo independiente. Este diseño sigue principios de separación de responsabilidades, bajo acoplamiento y alta cohesión, permitiendo evolucionar cada módulo sin afectar el resto del sistema.

La arquitectura se compone de:

- Módulos GraphQL por recurso: Cada entidad de dominio (usuarios, rutas, layouts de bus, autenticación) se implementa en un módulo propio dentro de apps/api/src/gql-resources/, cada uno con su resolver, servicio, DTOs y entidades.
- Sistema de permisos basado en CASL: El control de acceso se implementa mediante acciones (CREATE, READ, UPDATE, DELETE) y sujetos (USER, ROUTE, BUS_LAYOUT).
- Autenticación basada en sesiones: Las sesiones se gestionan con Passport.js y se almacenan en Redis, garantizando persistencia, expiración segura y compatibilidad con el despliegue distribuido.
- ORM con Prisma: Prisma Client gestiona el acceso y la manipulación de datos, con validaciones, tipado fuerte y relaciones complejas (stops, segments, pricing, elementos de bus).
- Frontend Backoffice en Next.js
- Frontend POS en Next.js

Ilustración 23 Diagrama de paquetes de la aplicación en el primer incremento



Fuente: Elaboración propia, 2025.

b. Modelo de datos y ORM

El modelo de datos se implementa con Prisma ORM, alineado a los subdominios definidos en la arquitectura. Las principales entidades para este incremento son:

- User
 - o id
 - o email (único)
 - o password (hash)
 - o first_name, last_name
 - o role (enum: UserRole)
 - o company_id (FK)
 - o terminalId (FK opcional)
 - o employee_of_id, director_of_id (relaciones jerárquicas con Company)
 - o posCode (opcional)
 - o company (Relacion)
 - o terminal (Relacion)
 - o asociaciones laborales (employee_of, director_of)
 - o auditoría automática mediante createdAt, updatedAt
- Company
 - o id
 - o display_name (único)
 - o legal_name (opcional)
 - o contact_phone, contact_email (opcionales)
 - o country
 - o default_currency (enum CurrencyCode)
 - o createdAt
 - o Tiene muchos User.
 - o Se asocia a Route, Segment, SeatClass, BusLayout.
- BusTerminal
 - o id
 - o name (único)
 - o abbreviation (opcional)
 - o address_id (FK)

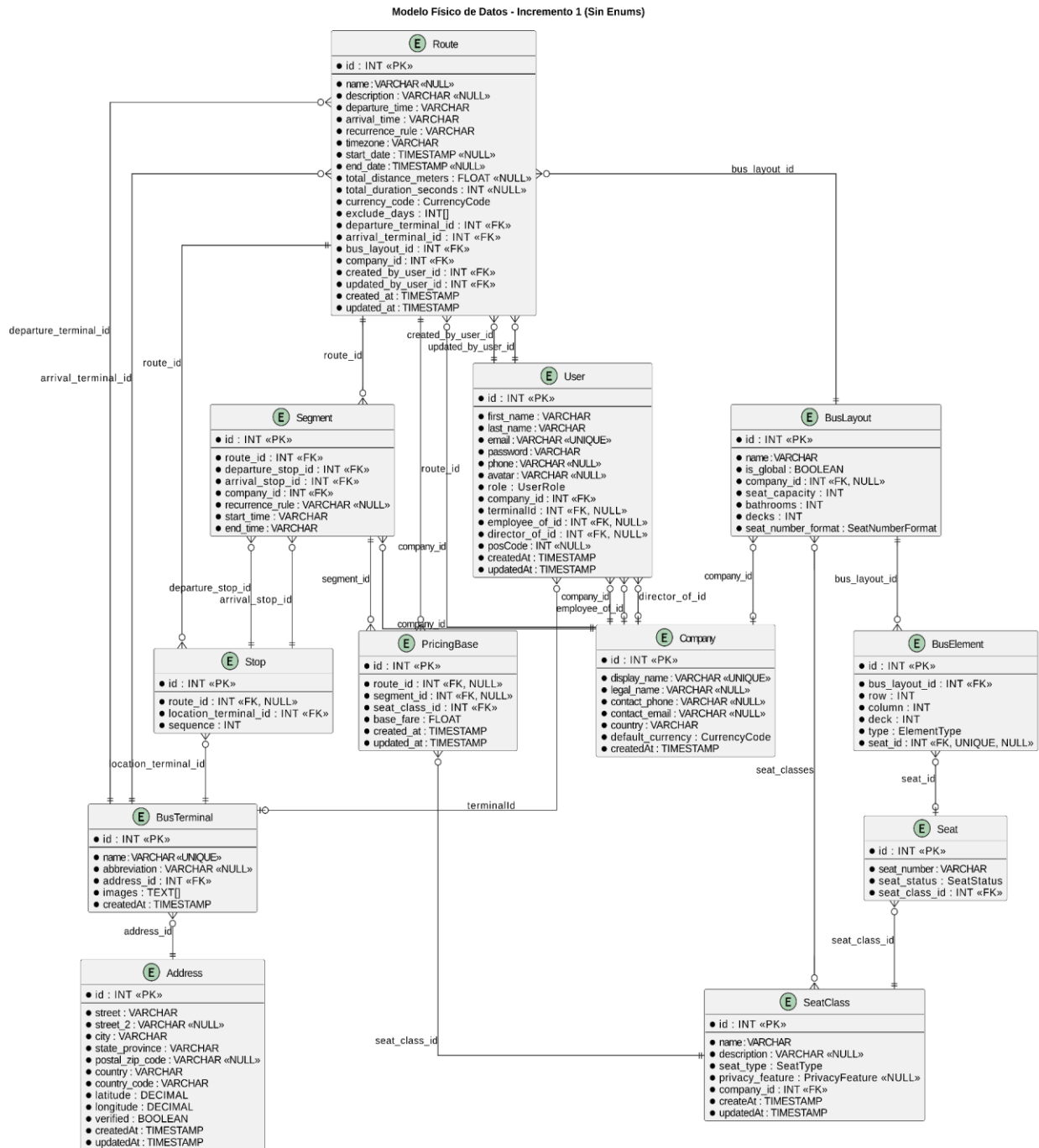
- o images (colección de URLs)
 - o createdAt
 - o Relación 1–1 con Address.
 - o Usado en rutas (Route) como terminal de salida y llegada.
 - o Usado en Stop como terminal de parada intermedia.
- Address
 - o id
 - o street, street_2 (esta última opcional)
 - o city, state_province
 - o postal_zip_code (opcional)
 - o country, country_code
 - o latitude, longitude
 - o verified (boolean)
 - o createdAt, updatedAt
 - o Asociada a BusTerminal mediante address_id.
- Route
 - o name, description
 - o departure_terminal_id, arrival_terminal_id
 - o departure_time, arrival_time
 - o recurrence_rule, timezone
 - o start_date, end_date
 - o bus_layout_id
 - o total_distance_meters, total_duration_seconds
 - o currency_code
 - o terminales de salida y llegada
 - o bus_layout
 - o stops y segments
 - o pricing_base
 - o usuario creador y actualizador (created_by_user_id, updated_by_user_id)
 - o company
- PricingBase
 - o id
 - o route_id (FK opcional)

- o segment_id (FK opcional)
 - o seat_class_id (FK)
 - o base_fare
 - o created_at, updated_at
- BusLayout
 - o name
 - o decks
 - o seat_capacity
 - o bathrooms
 - o is_global
 - o company_id
 - o seat_number_format
- BusElement
 - o type (enum: ElementType: SEAT, AISLE, BATHROOM, DOOR, etc.)
 - o row, column, deck
 - o bus_layout_id
 - o seat_id (FK opcional, si el elemento corresponde a un asiento)
- Seat
 - o seat_number
 - o seat_status (enum: SeatStatus)
 - o seat_class_id
- SeatClass
 - o name, description
 - o seat_type (enum: SeatType)
 - o privacy_feature (enum: PrivacyFeature)
 - o company_id

Por convención, Prisma genera tablas con nombres directamente derivados del modelo (User, Route, BusLayout, etc.). Las relaciones se generan automáticamente mediante claves foráneas.

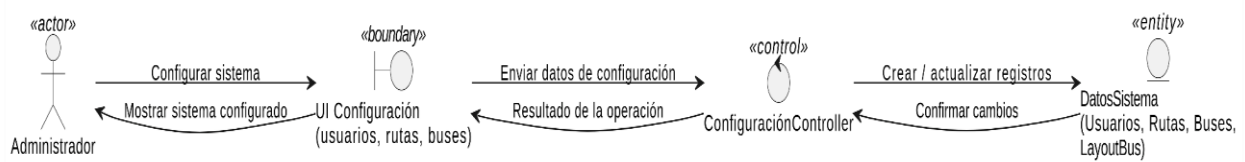
En la figura 3.23 se expone el modelo de datos físico para el primer incremento, especificando los tipos de datos y límites de caracteres si aplica.

Ilustración 24 Modelo de datos físico para el primer incremento



Fuente: Elaboración propia, 2025.

Ilustración 25 Diagrama de Robustez principal del Incremento



Fuente: Elaboración propia, 2025.

c. Calidad y seguridad

- Sistema de control de acceso basado en roles (RBAC): Sistema CASL con roles y decoradores `@Permission` en resolvers GraphQL. Endpoints públicos solo para login, logout, verificación de sesión y recuperación de contraseña; resto requiere autenticación y permisos específicos.
- Protecciones activas: Autenticación con Passport.js (estrategias local y POS separadas) y sesiones Redis con cookies `httpOnly`, `secure` y `sameSite` configurado.
- Criptografía y tokens: Hash de contraseñas con `bcrypt` (`SALT_ROUNDS = 10`) y sesiones almacenadas en Redis con cookies seguras (`httpOnly`, `secure` en producción, `maxAge` 2 meses).
- Privacidad y logging: Contraseñas nunca retornadas (middleware `passwordMiddleWear`), emails normalizados a `lowercase`, y mensajes de error no enumerativos.
- Confiabilidad: Transacciones Prisma para operaciones complejas (creación de rutas con stops, segments, pricing_bases) y validaciones de integridad (secuencia de stops, terminales existentes).

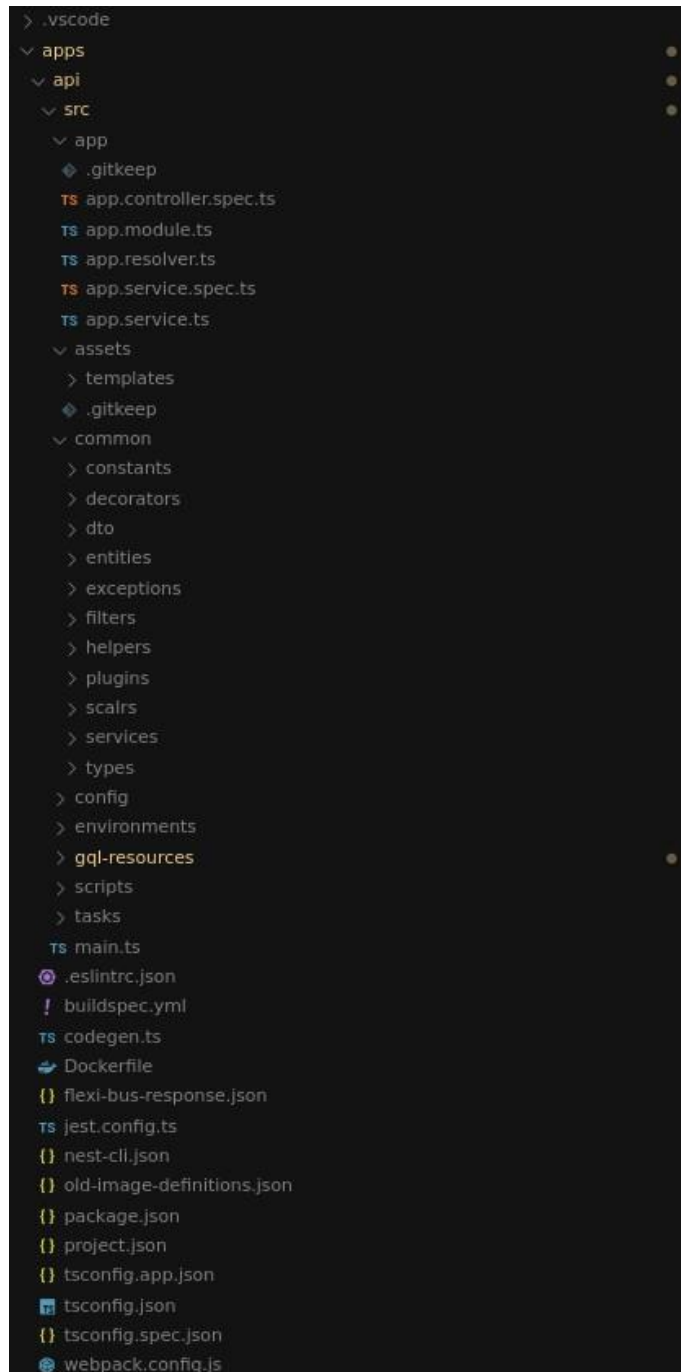
3.4.1.5. Desarrollo del Incremento

a. Backend

La implementación del incremento se desarrolló sobre una arquitectura modular en NestJS + GraphQL, siguiendo la separación por recursos auth, users, routes y bus-layouts. En el backend se construyeron los servicios de dominio (validación de credenciales con `bcrypt`, gestión de sesiones en Redis, reglas de RBAC mediante CASL y validaciones de negocio), los modelos ORM en Prisma y los resolvers/DTOs que exponen los contratos GraphQL para autenticación

(login, logout, verificación de sesión y recuperación de contraseña), gestión de usuarios (creación, edición, búsqueda y listado con filtros por permisos), administración de rutas (creación con validaciones de stops, cálculo de distancias y persistencia transaccional) y layouts de buses (creación y edición con elementos anidados). Se añadieron mecanismos de normalización de datos (lowercase para emails, middleware para ocultar contraseñas), validaciones estructurales con class-validator y reglas de acceso dinámico mediante UserAbilityFactory. La configuración sensible (secretos de sesión, URL de Redis, SALT_ROUNDS, políticas de cookies y claves de servicios externos) se externalizó en variables de entorno para asegurar portabilidad y cumplimiento de buenas prácticas.

Ilustración 26 Organización del primer incremento por aplicaciones de domino



Fuente: Elaboración propia, 2025.

En la figura 3.25 se muestra el servicio `AuthService.login()` que implementa la validación de credenciales. El método busca el usuario por email normalizado a lowercase, compara la contraseña con bcrypt mediante la función `compare()`, y lanza errores genéricos ("Invalid

password please try again") para evitar enumeración de usuarios. El servicio retorna el usuario completo si las credenciales son válidas, permitiendo que el guard de Passport.js establezca la sesión.

Ilustración 27 Servicio de Autenticación

```
import { ApiError } from '@mipasaje/api/shared/exceptions'
import { renderEmailTemplate } from '@mipasaje/api/shared/utils-email-templates'
import { HttpStatus, Injectable } from '@nestjs/common'
import { compare } from 'bcrypt'
import { randomBytes } from 'crypto'
import { DateTime } from 'luxon'
import { EmailService } from '../../../common/services'
import { PrismaService } from '../../../common/services/database/prisma.service'
import { UsersService } from '../users/users.service'
import { LoginInput } from './dto/login.input'
import { RequestPasswordResetInput } from './dto/request-password-reset.input'
import { ResetPasswordInput } from './dto/reset-password.input'
import { ValidatePasswordResetTokenInput } from './dto/validate-password-reset-token.input'
import { EventEmitter2 } from '@nestjs/event-emitter'
import { AUTH_EVENTS } from '../constants/event.constants'
import { RequestPasswordResetEvent } from '../events/request-password-reset.event'

@Injectable()
export class AuthService {
  constructor(
    private readonly prisma: PrismaService,
    private readonly emailService: EmailService,
    private readonly userService: UsersService,
    private readonly eventEmitter: EventEmitter2
  ) {}

  > async login(loginInput: LoginInput) {--
  }

  > async requestPasswordReset(resetPasswordInput: RequestPasswordResetInput) {--
  }

  > async validatePasswordResetToken( --
  }

  > async resetPasswordWithToken(resetPasswordInput: ResetPasswordInput) {--
  }
}
```

Fuente: Elaboración propia, 2025.

Ilustración 28 Servicio login Asincrono

```
28  async login(loginInput: LoginInput) {
29    const user = await this.prisma.user.findUnique({
30      where: { email: loginInput.email?.toLowerCase() },
31    })
32
33    if (!user)
34      throw new ApiError(`User ${loginInput.email} not found`, {
35        statusCode: HttpStatus.UNAUTHORIZED,
36        type: 'authenticate_error',
37      })
38
39    const matchingPassword = await compare(loginInput.password, user.password)
40
41    if (!matchingPassword)
42      throw new ApiError('Invalid password please try again', {
43        statusCode: HttpStatus.UNAUTHORIZED,
44        type: 'authenticate_error',
45      })
46
47    return user
48  }
```

Fuente: Elaboración propia, 2025.

En la figura 3.27 se muestra la configuración de sesiones Redis. La configuración establece cookies httpOnly (previene acceso desde JavaScript), secure en producción (solo HTTPS), sameSite según ambiente ('none' en producción, 'lax' en desarrollo para protección CSRF), y maxAge de 2 meses (60 días). El almacenamiento en Redis mediante `connect-redis` permite escalabilidad y persistencia de sesiones. La configuración utiliza variables de entorno para secretos y URLs de Redis.

Ilustración 29 Exportación de session de Redis

```

1 import RedisStoreClient from 'connect-redis'
2 import * as session from 'express-session'
3 import RedisClient from 'ioredis'
4 import { UserEntity } from '../gql-resources/users/entities/user.entity'
5 import { BACKOFFICE_HOST, GRAPHQL_API_URL } from '@travell/utlis-constants'
6
7 const client = new RedisClient(process.env['REDIS_URL'] as string, {})
8
9 const store = new RedisStoreClient({
10   client: client,
11 })
12
13 const isDevelopment = process.env['NODE_ENV'] === 'development'
14 const isProduction = process.env['NODE_ENV'] === 'production'
15
16 const isStaging = process.env['APP_ENV'] === 'staging'
17
18 const DOMAIN = isStaging ? '.travell.dev' : '.travell.app'
19
20 // Set secure to true in production, false in development
21 const secure = isProduction ? true : false
22 // Use domain in production only
23 const domain = isProduction ? DOMAIN : undefined
24 // Use 'none' for cross-site requests in production, 'lax' in development
25 const sameSite = isProduction ? 'none' : 'lax'
26 // 2 months in milliseconds
27 const maxAge = 1000 * 60 * 60 * 24 * 30 * 2
28
29 export const redisSession = session({
30   secret: process.env['SESSION_SECRET'] as string,
31   resave: false,
32   saveUninitialized: false,
33   proxy: true,
34   cookie: {
35     domain,
36     secure,
37     sameSite,
38     httpOnly: true,
39     maxAge,
40   },
41   store: store,
42 })
43
44 declare module 'express' {
45   interface Request {
46     user: UserEntity
47   }
48 }

```

Fuente: Elaboración propia, 2025.

A continuación se muestra la figura 3.28 se muestra el factory de permisos `UserAbilityFactory` que implementa RBAC mediante CASL (Code Access Security Library). El factory genera reglas de capacidad según el rol del usuario (OWNER, ADMIN, POS, TRAVELL_ADMIN, GUEST), permitiendo control granular de acceso a recursos. El método `createForUser()` evalúa el rol y delega a métodos específicos que definen las capacidades permitidas para cada rol, utilizando `AbilityBuilder` y `createPrismaAbility` para generar las reglas CASL.

Ilustración 30 Método createForUser() con switch de roles

```

createForUser(user: UserEntity | 'bypass' | null) {
  if (user === 'bypass' || user === null) return this.createByPassAbility()

  if (user.role === 'OWNER') return this.createForOwner(user)
  if (user.role === 'ADMIN') return this.createForAdmin(user)
  if (user.role === 'POS') return this.createForPos(user)
  if (user.role === 'TRAVELL_ADMIN') return this.createForTravellAdmin(user)

  return this.createForGuest(user)
}

private actionResolver() {
  const actionAlias = createAliasResolver({
    modify: [Action.CREATE, Action.UPDATE],
  })

  return actionAlias
}

private createForOwner(user: UserEntity) {
  const builder = new AbilityBuilder<AppAbility>(createPrismaAbility)

  builder.can(Action.MANAGE, Subject.USER,)
  builder.can(Action.MANAGE, Subject.BUS, { company_id: user.company_id })
  builder.can(Action.MANAGE, Subject.BUS_LAYOUT, { company_id: user.company_id })
  builder.can(Action.READ, Subject.BUS_LAYOUT, {
    company_id: null,
  })
  builder.can(Action.MANAGE, Subject.COMPANY, { id: user.company_id })
  builder.cannot(Action.DELETE, Subject.COMPANY)
  builder.can(Action.MANAGE, Subject.ROUTE, { company_id: user.company_id })
  builder.can(Action.MANAGE, Subject.JOURNEY, { company_id: user.company_id })
  builder.can(Action.MANAGE, Subject.TRIP, { route: { company_id: user.company_id } })
  builder.can(Action.MANAGE, Subject.DRIVER, { company_id: user.company_id })
  builder.can(Action.MANAGE, Subject.STOP, { route: { company_id: user.company_id } })
  builder.can(Action.MANAGE, Subject.SEGMENT, { company_id: user.company_id })
  builder.can(Action.MANAGE, Subject.ORDER, { company_id: user.company_id })
  builder.can(Action.READ, Subject.NOTIFICATION, { company_id: user.company_id })
  builder.can(Action.UPDATE, Subject.NOTIFICATION, { company_id: user.company_id })
  builder.can(Action.MANAGE, Subject.SEAT_CLASS, { company_id: user.company_id })
  builder.can(Action.READ, Subject.PASSENGER)
  builder.can(Action.CREATE, Subject.PASSENGER, )
  builder.can(Action.MANAGE, Subject.TICKET)

  return builder.build({ resolveAction: this.actionResolver() })
}

```

Fuente: Elaboración propia, 2025.

En la Figura 3.29 se detalla el método `create()` del `RoutesService`, encargado de la creación de rutas. Antes de persistir los datos, el servicio ejecuta diversas validaciones de negocio: verifica la secuencia de los stops mediante el helper `getIsValidSequenceOrder()`, comprueba la consistencia de las ubicaciones con `getIsValidStopLocations()` y obtiene la distancia y duración estimadas de la ruta mediante la integración con la API de Google Maps. Una vez superadas las validaciones, se realiza una operación transaccional con Prisma que crea la ruta junto con sus relaciones anidadas (`stops`, `segments` y `pricing_bases`), y se calculan campos derivados como `total_distance_meters` y `total_duration_seconds`. Esta aproximación garantiza la atomicidad de la operación y la coherencia de los datos almacenados.

Ilustración 31 Servicio de Creación de Rutas

```
export class RoutesService {
  async create(
    const locationTerminals = await this.prisma.busTerminal.findMany({
      select: {
        address: {
          longitude: true,
        },
      },
    })

    > const distance = await this.distance.getRouteDistance({-
  })

    const route = await this.prisma.route.create({
      data: {
        name: createRouteDto.name,
        description: createRouteDto.description,
        departure_terminal_id: departureStop.location_terminal_id,
        arrival_terminal_id: arrivalStop.location_terminal_id,
        departure_time: departureTime,
        arrival_time: `${arrivalTime}${dayOffset ? `+${dayOffset}` : ''}`,
        bus_layout_id: createRouteDto.bus_layout_id,
        recurrence_rule: createRouteDto.recurrence_rule,
        timezone: createRouteDto.timezone,
        created_by_user_id: createRouteDto.created_by_user_id,
        updated_by_user_id: createRouteDto.updated_by_user_id,
        company_id: createRouteDto.company_id,
        total_distance_meters: distance.totalDistanceInMeters,
        total_duration_seconds: distance.totalTimeInSeconds,
        pricing_bases: createRouteDto.pricing_bases
        ? {
          createMany: {
            data: createRouteDto.pricing_bases.map((pricing) => ({
              base_fare: pricing.fare,
              seat_class_id: pricing.seat_class_id,
            })),
          },
          : undefined,
        },
        stops: {
          createMany: {
            data: [...stops],
          },
        },
      },
      include: {
        stops: { include: { location_terminal: { include: { address: true } } } },
      },
    })

    const segments = await this.prisma.$transaction(
    > createRouteDto.segments.map((segment) => {-
  })
  )
}
```

Fuente: Elaboración propia, 2025.

La Figura 3.30 presenta el método `create()` del `BusLayoutsService`, responsable de la creación de layouts de buses. A partir de la estructura recibida en el input (`layout`), el servicio transforma cada elemento (asientos, pasillos, baños, etc.), calcula métricas agregadas como el número de baños (`bathrooms`), la capacidad total de asientos (`seat_capacity`) y la cantidad de pisos (`decks`), y finalmente persiste el layout con todos sus elementos asociados (`bus_elements`) y las clases de asiento (`seat_classes`). Este enfoque permite definir layouts complejos de forma declarativa desde el frontend, mientras que el backend se encarga de derivar y almacenar la información necesaria para posteriores procesos de venta de tickets y asignación de asientos.

Ilustración 32 Servicio de Creación de Rutas

```

export class BusLayoutsService {
  async create(
    createBusLayoutInput: CreateBusLayoutInput,
    params: { user: UserEntity | null }
  ): Promise<BusLayoutEntity> {
    const { layout } = createBusLayoutInput

    const busElements = layout.map(this.transformElements)

    // Collect seat class IDs
    const busLayoutSeatClassIds = layout.reduce((acc: Set<number>, element) => {
      if (element.seat?.seat_class_id) acc.add(element.seat.seat_class_id)
      return acc
    }, new Set<number>())

    // Accumulate bathroom and seat counts
    const elementCount = layout.reduce(
      (acc: { bathrooms: number; seat_capacity: number }, element) => {
        if (element.element_type === 'BATHROOM') acc.bathrooms++
        if (element.element_type === 'SEAT') acc.seat_capacity++
        return acc
      },
      { bathrooms: 0, seat_capacity: 0 } // Initialize with 0 to avoid undefined values
    )

    // Determine number of decks, fallback to 0 if undefined
    const decks = Math.max(...layout.map((element) => element.deck || 0))

    // Create the bus layout in the database
    const busLayout = await this.prisma.busLayout.create({
      data: {
        name: createBusLayoutInput.name || `Layout ${Math.floor(Math.random() * 10000)}`,
        decks: decks,
        company_id: params.user?.company_id, // Ensure this is valid
        bathrooms: elementCount.bathrooms,
        seat_capacity: elementCount.seat_capacity,
        bus_elements: {
          create: busElements,
        },
        seat_classes: {
          connect: Array.from(busLayoutSeatClassIds).map((seatClassId) => ({
            id: seatClassId,
          })),
        },
      },
      include: {
        bus_elements: {
          include: { seat: { include: { seat_class: true } } },
        },
      },
    })

    return busLayout
  }
}

```

Fuente: Elaboración propia, 2025.

Antes de avanzar con el desarrollo de las interfaces de usuario en el frontend, se realizaron pruebas unitarias de los módulos correspondientes del backend para garantizar la estabilidad y el correcto funcionamiento de la lógica de negocio. Se implementaron suites de pruebas para los módulos críticos del incremento 1: users (creación, edición, listado y validación de

permisos), auth (login, logout, verificación de sesión y validación de credenciales), routes (creación con validaciones de stops, cálculo de distancia mediante Google Maps API, y edición parcial) y bus-layouts (creación con cálculo automático de capacidad, bathrooms, decks y seat_capacity, y edición con manejo de elementos anidados). Las pruebas unitarias utilizaron Jest como framework de testing, implementaron mocks completos de PrismaService y servicios externos (DistanceService, EmailService), y cubrieron tanto casos de éxito como escenarios de error y validaciones de negocio. Esta estrategia de testing-first permitió identificar y corregir problemas en la lógica de negocio antes de integrar con el frontend, reduciendo la complejidad de debugging y asegurando que los contratos de API GraphQL funcionarán correctamente cuando se consumieran desde las interfaces de usuario.

Ilustración 33 Reporte de pruebas unitarias de Servicio Usuarios

```
alml@pop-os:~/Projects/travell-apps/Travell-MonoRepo$ nx test api --testPathPattern=users.service.spec --no-coverage
> nx run api:test --testPathPattern=users.service.spec --no-coverage

PASS api apps/api/src/gql-resources/users/users.service.spec.ts
  UserService
    create
      ✓ should create a user with hashed password (12 ms)
      ✓ should use admin user company_id when creating user (3 ms)
    update
      ✓ should update user successfully (2 ms)
      ✓ should hash password when provided in update (2 ms)
      ✓ should throw error if user does not exist (7 ms)
      ✓ should throw ForbiddenError if user does not have permission (4 ms)
    list
      ✓ should list users with pagination and permissions (3 ms)
    enableUser (to be implemented)
      ✓ should enable a user by setting is_active to true (2 ms)
    disableUser (to be implemented)
      ✓ should disable a user by setting is_active to false (3 ms)

Test Suites: 1 passed, 1 total
Tests:       9 passed, 9 total
Snapshots:   0 total
Time:        1.507 s, estimated 4 s
Ran all test suites matching /users.service.spec/i.

NX Successfully ran target test for project api (3s)

With additional flags:
  --testPathPattern=users.service.spec
  --coverage=false
```

Fuente: Elaboración propia, 2025.

En las pruebas unitarias de ****users**** se implementaron 9 casos de prueba que validan la creación de usuarios con hash de contraseña mediante bcrypt, la actualización de usuarios incluyendo el hash de contraseña cuando se proporciona, la validación de errores cuando el

usuario no existe, la verificación de permisos mediante CASL que lanza `ForbiddenError` cuando el usuario no tiene autorización para realizar la operación, y el listado de usuarios con paginación y filtros de permisos basados en roles. En las pruebas unitarias de `**auth**` se ejecutaron 11 pruebas distribuidas entre el servicio y el resolver: el `AuthService` valida el login con credenciales válidas e inválidas, normaliza emails a minúsculas para evitar duplicados, y emite mensajes de error genéricos para evitar enumeración de cuentas; el `AuthResolver` verifica que el login retorna el usuario autenticado por los guards (`LocalGqlAuthGuard` y `LocalAuthGuard`), que el logout destruye la sesión correctamente y maneja errores de forma elegante, y que la consulta ``me`` retorna el usuario actual o lanza una excepción si no hay sesión activa.

Ilustración 34 Reporte de pruebas unitarias de Servicios de Autenticación

```

alm1@pop-os:~/Projects/travell-apps/Travell-MonoRepo$ nx test api --testPathPattern="auth\.(service|resolver)\.spec" --no-coverage
> nx run api:test [existing outputs match the cache, left as is]

PASS api apps/api/src/gql-resources/auth/auth.service.spec.ts
[Nest] 1771454 - 11/17/2025, 7:06:11 PM ERROR [AuthResolver] Something went wrong
PASS api apps/api/src/gql-resources/auth/auth.resolver.spec.ts

Test Suites: 2 passed, 2 total
Tests: 11 passed, 11 total
Snapshots: 0 total
Time: 4.124 s
Ran all test suites matching /auth\.(service|resolver)\.spec/i.

NX Successfully ran target test for project api (49ms)

With additional flags:
--testPathPattern=auth\.(service|resolver)\.spec
--coverage=false

```

Fuente: Elaboración propia, 2025.

En las pruebas unitarias de `auth` se ejecutaron 11 pruebas distribuidas entre el servicio y el resolver: el `AuthService` valida el login con credenciales válidas e inválidas, normaliza emails a minúsculas para evitar duplicados, y emite mensajes de error genéricos para evitar enumeración de cuentas; el `AuthResolver` verifica que el login retorna el usuario autenticado por los guards (`LocalGqlAuthGuard` y `LocalAuthGuard`), que el logout destruye la sesión correctamente y maneja errores de forma elegante, y que la consulta ``me`` retorna el usuario actual o lanza una excepción si no hay sesión activa.

Ilustración 35 Reporte de pruebas unitarias de Servicios de Rutas

```

alm1@pop-os:~/Projects/travell-apps/Travell-MonoRepo$ nx test api --testPathPattern="routes\.service\.spec" --no-coverage 2>&1 | ta
l -30

> nx run api:test --testPathPattern=routes\.service\.spec --no-coverage

PASS api apps/api/src/gql-resources/routes/routes.service.spec.ts
  RoutesService
    create
      ✓ should create a route successfully with valid input (9 ms)
      ✓ should throw error if stop sequence is invalid (8 ms)
      ✓ should throw error if stop locations are invalid (2 ms)
      ✓ should calculate distance and duration from Google Maps API (1 ms)
    update
      ✓ should update route successfully (2 ms)
      ✓ should exclude stops from update data (2 ms)
  Test Suites: 1 passed, 1 total
  Tests: 6 passed, 6 total
  Snapshots: 0 total
  Time: 2.733 s
  Ran all test suites matching /routes\.service\.spec/i.

NX Successfully ran target test for project api

```

Fuente: Elaboración propia, 2025.

En las pruebas unitarias de routes se desarrollaron 6 casos de prueba que validan la creación exitosa de rutas con entrada válida, el cálculo de distancia y duración desde la API de Google Maps, la validación de secuencia de stops lanzando errores cuando la secuencia no es consecutiva, la validación de ubicaciones de stops verificando que los terminales existen, y la actualización de rutas excluyendo stops del payload de actualización para evitar conflictos en la estructura de datos.

Ilustración 36 Reporte de pruebas unitarias de Servicios de bus

```

alm1@pop-os:~/Projects/travell-apps/Travell-MonoRepo$ nx test api --testPathPattern="bus-layouts\.service\.spec" --no-coverage

> nx run api:test --testPathPattern=bus-layouts\.service\.spec --no-coverage

PASS   api  apps/api/src/gql-resources/bus-layouts/bus-layouts.service.spec.ts
BusLayoutsService
  create
    ✓ should create bus layout with calculated capacity (10 ms)
    ✓ should calculate seat_capacity correctly (2 ms)
    ✓ should calculate bathrooms correctly (1 ms)
    ✓ should calculate decks correctly (1 ms)
    ✓ should connect seat classes from layout elements (1 ms)
    ✓ should use company_id from authenticated user (1 ms)
  update
    ✓ should update bus layout with new elements (2 ms)
    ✓ should handle deletion of elements (2 ms)

Test Suites: 1 passed, 1 total
Tests:      8 passed, 8 total
Snapshots:  0 total
Time:       2.241 s
Ran all test suites matching /bus-layouts\.service\.spec/i.

NX   Successfully ran target test for project api (3s)

With additional flags:
  --testPathPattern=bus-layouts\.service\.spec
  --coverage=false

```

Fuente: Elaboración propia, 2025.

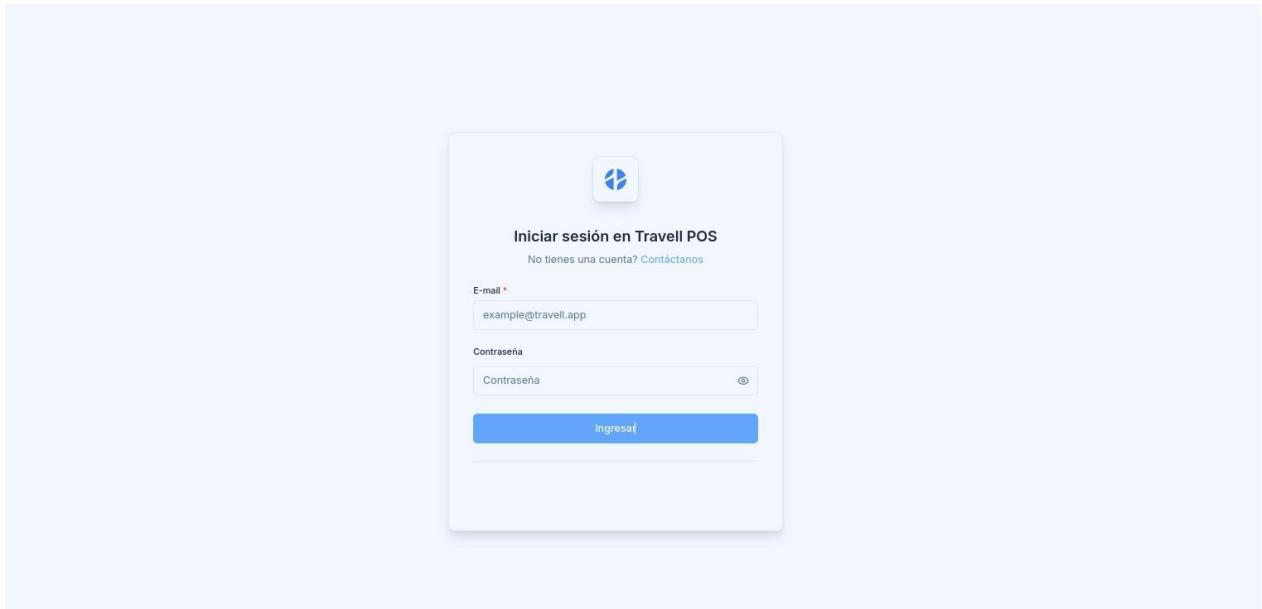
En las pruebas unitarias de bus-layouts se implementaron 8 casos de prueba que verifican la creación de layouts con capacidad calculada automáticamente (asientos, baños y pisos), el cálculo correcto de seat_capacity contando solo elementos de tipo SEAT, el cálculo de bathrooms contando elementos de tipo BATHROOM, el cálculo de decks determinando el número máximo de pisos, la conexión de seat classes desde los elementos del layout mediante relaciones Prisma usando `connect`, el uso del company_id del usuario autenticado, y la actualización de layouts con nuevos elementos y manejo de eliminación de elementos existentes.

b. Interfaces de usuario

En frontend se desarrollaron las pantallas de inicio de sesión, cierre de sesión, manejo de credenciales inválidas, así como los módulos para crear un bus layout, registrar rutas y visualizar rutas creadas. Todas las interfaces incluyen validaciones en cliente, estados de carga y un manejo consistente de errores. La UI se integró con la API mediante llamadas autenticadas con Bearer Token, gestionando también la renovación automática del token (refresh token) en

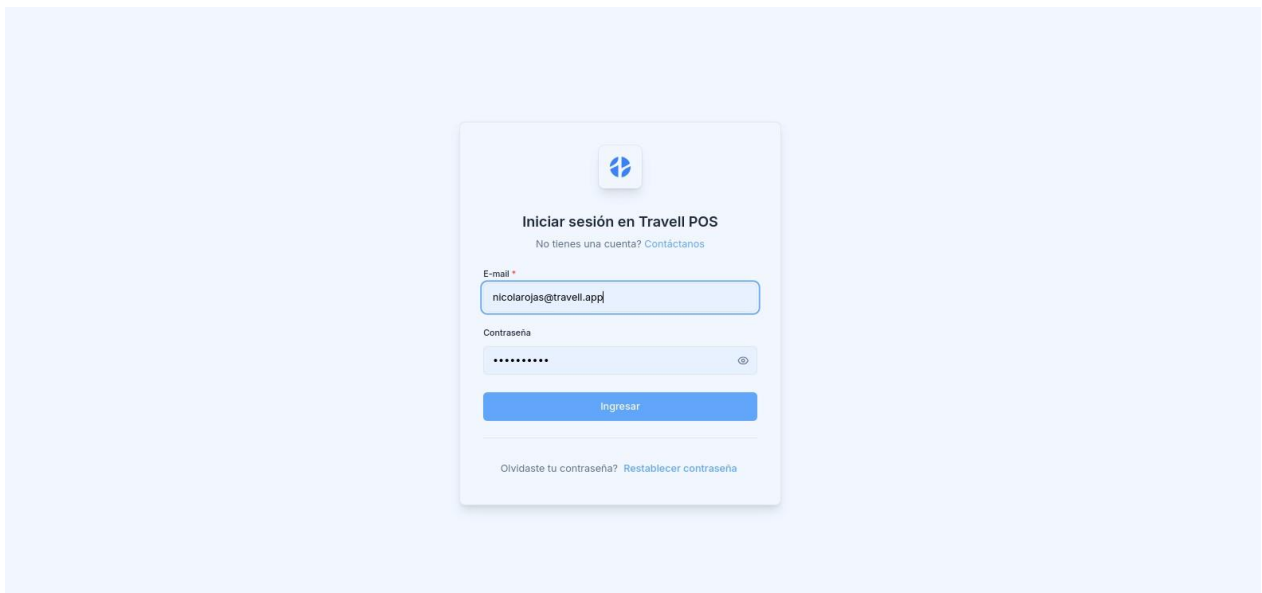
segundo plano. Se ejecutaron pruebas de extremo a extremo para validar el funcionamiento correcto del sistema y su integración con el backend.

Ilustración 37 Pantalla de inicio de sesión en la interfaz web



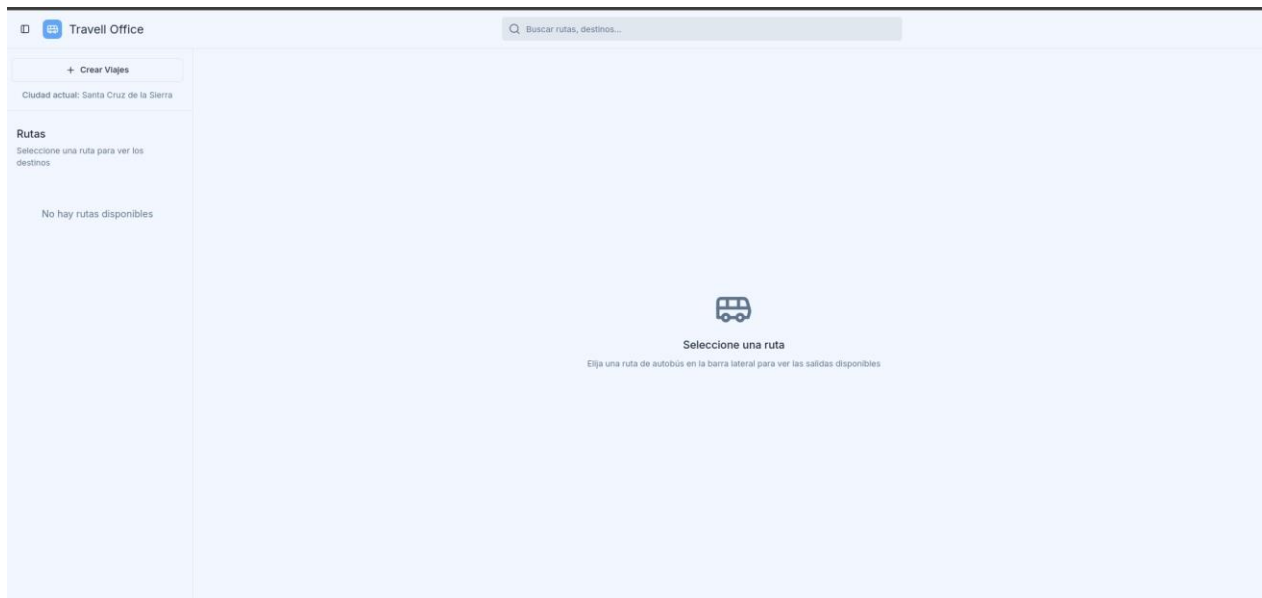
Fuente: Elaboración propia, 2025.

Ilustración 38 Pantalla de inicio de sesión con Credenciales en la interfaz web



Fuente: Elaboración propia, 2025.

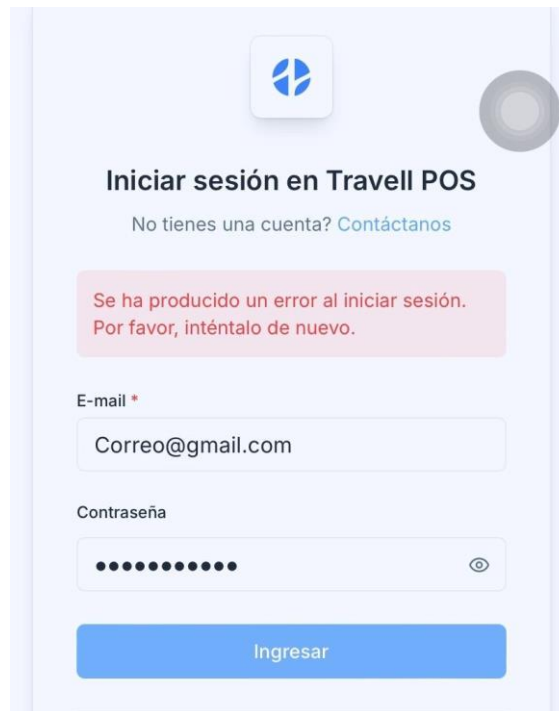
Ilustración 39 Redirección de Éxito de login



Fuente: Elaboración propia, 2025.

Se realizaron pruebas ingresando credenciales incorrectas (correo no registrado, contraseña errónea y combinaciones inválidas). El sistema respondió con mensajes neutrales y códigos HTTP adecuados (401/422), evitando la exposición de información sensible o pistas sobre la existencia de cuentas.

Ilustración 40 Credenciales inválidas en la interfaz web



The screenshot displays the login interface for 'Travell POS'. At the top, there is a logo consisting of a blue circle with a white stylized 'T' inside. Below the logo, the title 'Iniciar sesión en Travell POS' is centered. Underneath the title, a link 'No tienes una cuenta? Contáctanos' is provided. A red error message box is prominently displayed, stating: 'Se ha producido un error al iniciar sesión. Por favor, inténtalo de nuevo.' Below the error message, there are two input fields: 'E-mail *' with the placeholder 'Correo@gmail.com' and 'Contraseña' with a masked password '.....'. To the right of the password field is an eye icon for toggling visibility. At the bottom, there is a blue button labeled 'Ingresar'.

Fuente: Elaboración propia, 2025.

Además, se implementó y probó el módulo de gestión de usuarios, accesible únicamente para cuentas con rol administrativo. Desde este panel es posible listar usuarios, agregar nuevos registros, editar información existente y deshabilitar o habilitar usuarios según las necesidades operativas del sistema. La pantalla principal muestra una tabla de usuarios con opciones de búsqueda y filtrado por nombre, correo, rol y estado (activo/inactivo), así como ordenamiento por columnas. Esto facilita la administración cuando el número de usuarios crece y permite localizar rápidamente un registro específico.

Ilustración 41 Listado de usuarios en la interfaz web

Home > Usuarios

Usuarios

Actualmente hay 4 usuarios

Search for anything...

RolNombre

Filters 0Columns

ID	NOMBRE	CORREO	TELÉFONO	ROL	CREADO	ACCIONES
#0001	J John Doe john.doe@example.com	john.doe@exampl	+591 7 123 4567	Admin	Nov 17, 2025	...
#0002	J Jane Smith jane.smith@example.com	jane.smith@exampl	+591 7 234 5678	Owner	Nov 17, 2025	...
#0003	B Bob Johnson bob.johnson@example.com	bob.johnson@exar	N/A	Guest	Nov 17, 2025	...
#0004	A Alice Williams alice.williams@example.com	alice.williams@exa	+591 7 345 6789	Pos	Nov 17, 2025	...

Fuente: Elaboración propia, 2025.

Para agregar un usuario, se dispone de un formulario donde se registran datos básicos (nombre, correo electrónico, rol, estado inicial, etc.). Se validan campos obligatorios, formato de correo y posibles duplicados para evitar inconsistencias. Al confirmar la creación, el nuevo usuario aparece inmediatamente en el listado, reflejando el impacto en tiempo real sobre la base de datos.

Ilustración 42 Formulario de creación de usuario en la interfaz web

Editar Usuario

Información General

Nombre * Apellido *

John Doe

Correo Electrónico *

john.doe@example.com

Teléfono

+591 71234567

Rol *

admin

Seleccione el rol del usuario para determinar sus permisos

Next

Fuente: Elaboración propia, 2025.

Ilustración 43 Cambio de estado (habilitar/deshabilitar) de un usuario en la interfaz web

Edit User

General Information

First Name * Last Name *

matias bueno

Email *

matiasbueno@travell.app

Phone

+1 Phone number

Role *

POS

Terminal

Terminal de Buses Santa Cruz

POS Code (4 digits)

1238

Must be a 4-digit number

User Status

Active ☒

Disabled users cannot log in to the system

Submit

Fuente: Elaboración propia, 2025.

Se probó la creación de un bus layout desde el panel administrativo. El formulario permite definir características como nombre, cantidad de filas, columnas, disposición de asientos y

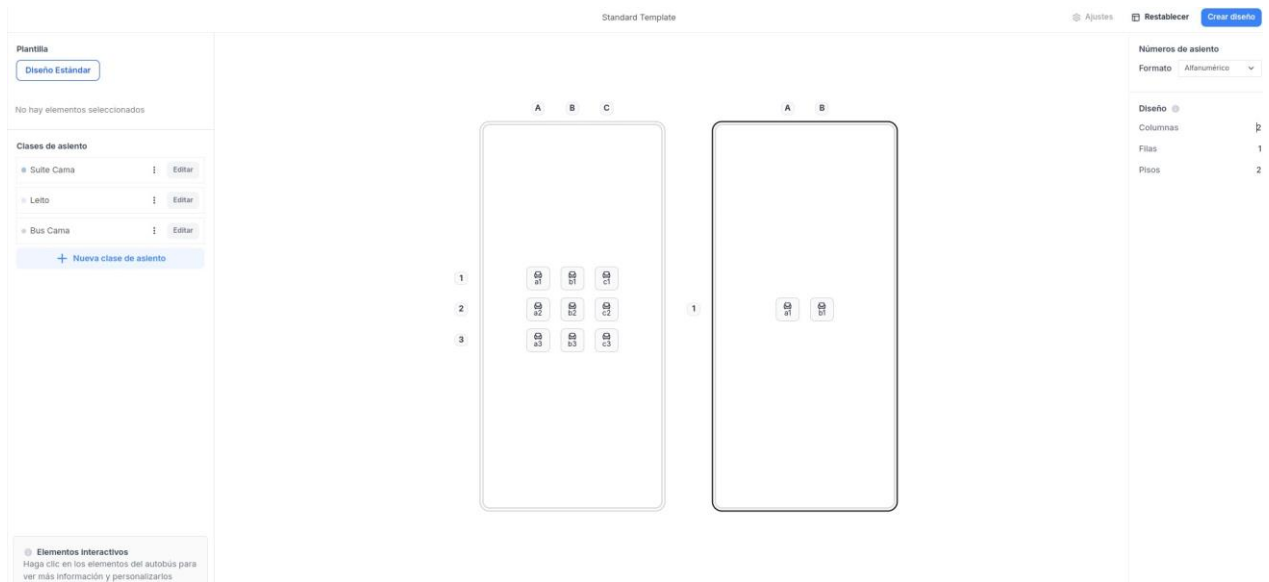
zonas especiales (por ejemplo, conductor o pasillo). Se validaron campos obligatorios, restricciones de diseño y vista previa del layout. En la Figura 3.42 se muestra la pantalla para la creación de un nuevo bus layout.

Ilustración 44 Listado bus layouts

TABLES.ID	TABLES.LAYOUTS.LA	TABLES.LAYOUTS.DE	TABLES.LAYOUTS.TY
#0039	io Garcia Letto (Co	2	Custom
#0033	tor Escobar (Copy	2	Custom
#0032	do Espinoza (Cope	2	Custom
#0031	rano Robore (Cops	1	Custom
#0038	ilo Canaza (Copy)	2	Custom
#0040	ummy Letto (Copy)	2	Custom
#0041	anuel Ayala (Copy)	2	Custom
#0042	ose Ayala (Copy)	2	Custom
#0043	ber romero (Copy)	2	Custom

Fuente: Elaboración propia, 2025.

Ilustración 45 Creación de un nuevo bus layout



Fuente: Elaboración propia, 2025.

Una vez guardado, se comprobó que el layout se visualiza correctamente en el listado general y que puede editarse o eliminarse según permisos. Posteriormente se validó el módulo de creación de rutas, que permite seleccionar un layout, asignar origen/destino, horarios, paradas y otros parámetros operativos. Se verificó la correcta interacción con la API y que las validaciones evitan datos incompletos o inconsistentes. La Figura 3.44 muestra el formulario utilizado para registrar una nueva ruta.

Ilustración 46 Creación de Ruta

Paradas
 Selecciona todas las paradas que tomará esta ruta. Usa el interruptor para hacer que una parada sea solo cosmética

Terminal: Select an option
 Hora de salida: --:--:--

La Paz
 Santa Cruz de la Sierra
 Cochabamba
 Sucre
 Potosí
 Oruro
 Quíjarro
 Puerto Suarez
 Yacuses

Precio De Leito
 Bs 0,20\$
 Precio para la clase de asiento leito. Esto se reflejará en todos los viajes

Horario
 Pricing Rules

No hay paradas seleccionadas
 Agrega paradas arriba para ver el horario generado

Back Next

Fuente: Elaboración propia, 2025.

3.4.1.6. Entrega y validación

La liberación del primer incremento se realizó mediante la ejecución de las migraciones de base de datos con Prisma y la aplicación de un smoke test a los flujos críticos del sistema (login, logout, verificación de sesión, gestión de usuarios, creación y edición de rutas y administración de bus layouts). Para considerar el incremento como concluido, se establecieron los siguientes criterios: (1) cobertura completa de los requerimientos funcionales del incremento —RF-01 (autenticación y sesiones) y RF-02 (gestión de usuarios, rutas y layouts de buses)—; (2) pruebas unitarias completas para los módulos core, con un total de 34 pruebas distribuidas entre users, auth, routes y bus-layouts; (3) RBAC operativo mediante CASL, con roles diferenciados 3y validación estricta de permisos en todas las operaciones críticas; (4) autenticación basada en sesiones almacenadas en Redis y gestionadas mediante cookies httpOnly/secure; y (5) integración con la API de Google Maps para el cálculo automático de distancias y duraciones en las rutas. el incremento quedó aceptado y listo como base para los siguientes módulos (véase Anexo A.10).

3.4.2. INCREMENTO 2: ACTIVACIÓN OPERATIVA Y DISPONIBILIDAD EN TIEMPO REAL

Esta sección describe la construcción del segundo incremento, cuyo propósito es habilitar la operación diaria del sistema, permitiendo asignar buses a rutas y turnos, ajustar precios operativos, consultar disponibilidad de asientos en tiempo real y gestionar incidencias que pueden afectar la operación. Este incremento constituye el puente entre la configuración inicial del sistema (incremento 1) y los procesos transaccionales de venta y pagos incorporados en el incremento siguiente.

El incremento implementa los modelos, reglas de negocio y APIs necesarias para representar la activación operativa: asignaciones de ruta, bus, turno, precios vigentes, estado centralizado de disponibilidad, y registro y resolución de incidencias. Esto incluye tanto la lógica del backend como las interfaces operativas mínimas para que el Líder de Línea y Chofer interactúen con el sistema. Con este incremento, la operación diaria puede comenzar: el sistema conoce qué rutas están activas, qué bus corresponde a cada turno, cuánto cuesta cada viaje y cuál es el estado actual de asientos para cada combinación ruta–turno.

3.4.2.1. Alcance del incremento

El objetivo de este incremento es habilitar el inicio y la operación del servicio de transporte en tiempo real, permitiendo la asignación de buses a rutas y turnos (RF-03), el ajuste inmediato de precios en rutas activas para su visualización por vendedores y choferes (RF-04), la consulta en tiempo real de disponibilidad de asientos (RF-05) y la gestión de incidencias junto con la reasignación operativa de buses (RF-06). Con estas capacidades se cubren los requisitos operativos necesarios para comenzar la venta de boletos en incrementos posteriores. Quedan fuera de alcance en este incremento el proceso de venta de boletos y selección de asientos (incremento 3), los reportes, consolidados y trazabilidad histórica (incremento 4), el layout gráfico de buses y la lógica de pagos, así como las políticas avanzadas de reasignación automática.

3.4.2.2. Plan de trabajo y estimaciones

A continuación se presenta el plan de trabajo de este incremento, detallando cada tarea con su estimación de horas.

Cuadro 3.14. Plan de trabajo del segundo incremento

#	Tarea	Descripción breve	Estimación (h)
1	Obtención de documentos	Modelo de asignaciones ruta–bus–turno	8
2	API de activación operativa	Mutations para asignar, modificar y desactivar asignaciones; validaciones lógicas	8
3	Modelo de precios activos	Definición de precio vigente por ruta/turno/bus y reglas de actualización	5
4	API de ajuste de precios	Mutations para actualizar precios activos; propagación inmediata	4
5	Servicio de disponibilidad centralizada	almacenamiento del estado de asientos por viaje; estructura base	8
6	Modelo y API de incidencias	Endpoint para consultar asientos libres por ruta/turno en tiempo real	4
7	Modelo y API de incidencias	Registro por chofer, tipos de incidencia, mensajes de contexto	6
8	Reasignación operativa	Mutations para reasignar bus; actualización automática de disponibilidad	6
9	UI operativa	Pantallas para: activar ruta, ajustar precios	12
10	Pruebas	Tests de disponibilidad, activación, reasignación	12

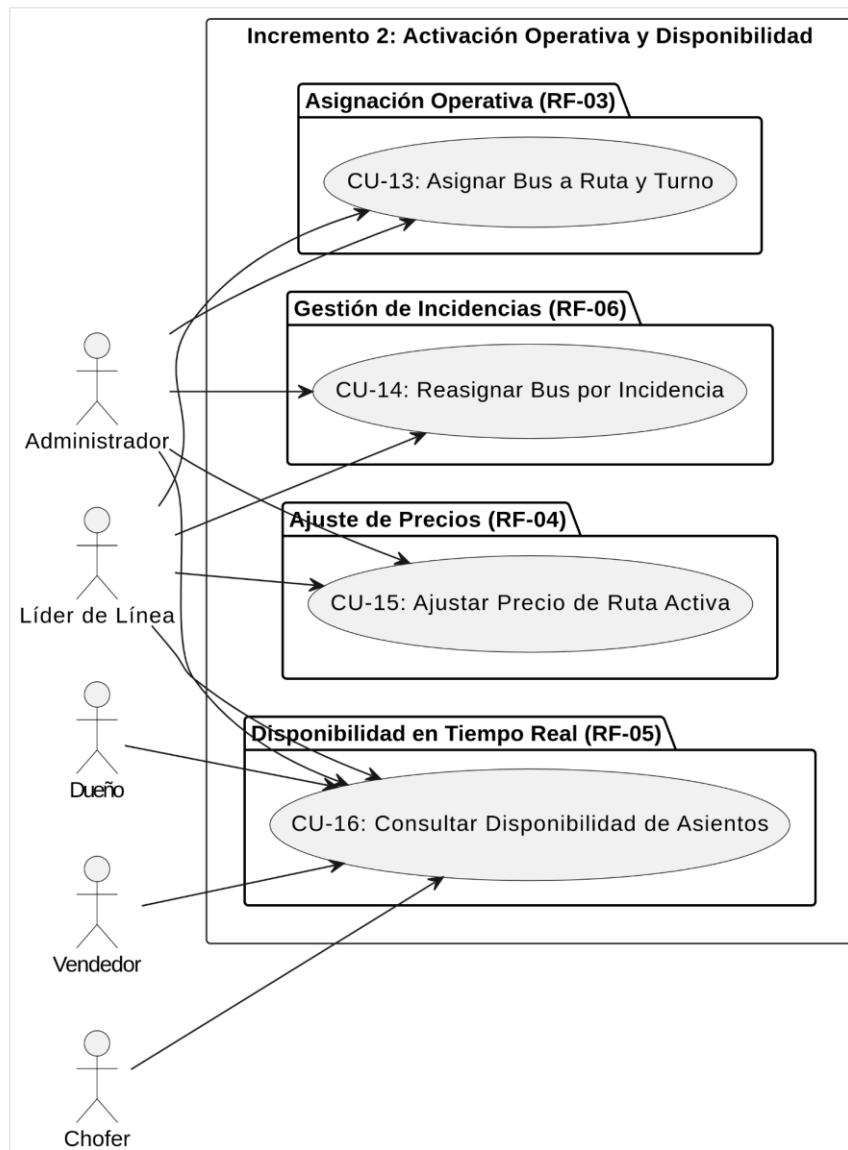
Fuente: Elaboración propia, 2025.

El esfuerzo total estimado para este incremento asciende a 73 horas, considerando el desarrollo backend para gestionar asignaciones, precios, disponibilidad e incidencias, la creación de interfaces operativas mínimas para el Líder de Línea y el Chofer, la ejecución de pruebas de integración y E2E para asegurar la coherencia operativa, y el manejo de la consistencia en las actualizaciones en tiempo real. Esta estimación parte del supuesto de que los modelos base (rutas, turnos y buses) ya se encuentran cargados en el sistema desde el incremento 1 y que existe un mecanismo centralizado de almacenamiento de disponibilidad.

3.4.2.3. Análisis funcional detallado

Esta sección profundiza el análisis funcional del incremento, afinando los casos de uso para definir con precisión las precondiciones, las postcondiciones y el flujo principal. La figura 3.35 muestra los casos de uso incluidos en este incremento encerrados.

Ilustración 47 Casos de uso cubiertos en el segundo incremento



Fuente: Elaboración propia, 2025.

A continuación, se detalla el catálogo de casos de uso con sus propósitos, actores asociados, precondiciones, postcondiciones y flujo principal, además se incluye entre paréntesis su requerimiento asociado para facilitar la trazabilidad.

CU-13	Asignar Bus a Ruta (RF-03)
Propósito	Activar operativamente una ruta asignándole un bus
Actores	Líder de Línea, Administrador
Precondiciones	Rutas existentes y buses cargados y disponibles
Postcondiciones	La ruta queda activada para operación
Flujo principal	(1) Seleccionar ruta existente (2) Seleccionar bus disponible (3) Confirmar activación operativa

CU-14	Reasignar Bus por Incidencia (RF-06)
Propósito	Permitir reemplazar en tiempo real el bus asociado a una ruta activa debido a una incidencia
Actores	Líder de Línea, Administrador
Precondiciones	Buses disponibles para reasignación
Postcondiciones	Bus nuevo queda asignado a la ruta
Flujo principal	(1) Seleccionar ruta (2) Consultar buses disponibles (3) Seleccionar bus reemplazo (4) Confirmar reasignación

CU-15	Ajustar Precio de Ruta (RF-04)
Propósito	Permitir reemplazar en tiempo real el bus asociado a una ruta activa debido a una incidencia
Actores	Líder de Línea, Administrador
Precondiciones	Ruta activa con bus asignado
Postcondiciones	Precio actualizado, Ventas futuras usan el nuevo precio
Flujo principal	(1) Seleccionar ruta activa (2) Ingresar nuevo precio (3) Confirmar actualización

CU-16	Consultar Disponibilidad de Asientos (RF-05)
Propósito	Obtener la disponibilidad actual de asientos en tiempo real para una ruta activa
Actores	Vendedor, Chofer, Líder de Línea, Dueño, Administrador
Precondiciones	Ruta activa con bus asignado
Postcondiciones	Se devuelve la lista de asientos libres/ocupados
Flujo principal	(1) Seleccionar ruta activa. (2) Consultar disponibilidad (3) Mostrar asientos libres/ocupados (4) ctualizar vista si cambian los estados

3.4.2.4. *Diseño de solución*

El Incremento 2 se centra en la activación operativa del sistema, habilitando la administración en tiempo real de rutas, viajes, buses y disponibilidad de asientos. El diseño de la solución evoluciona la arquitectura inicial hacia un sistema más robusto, orientado a servicios y con flujos transaccionales capaces de responder a cambios dinámicos en la operación del transporte.

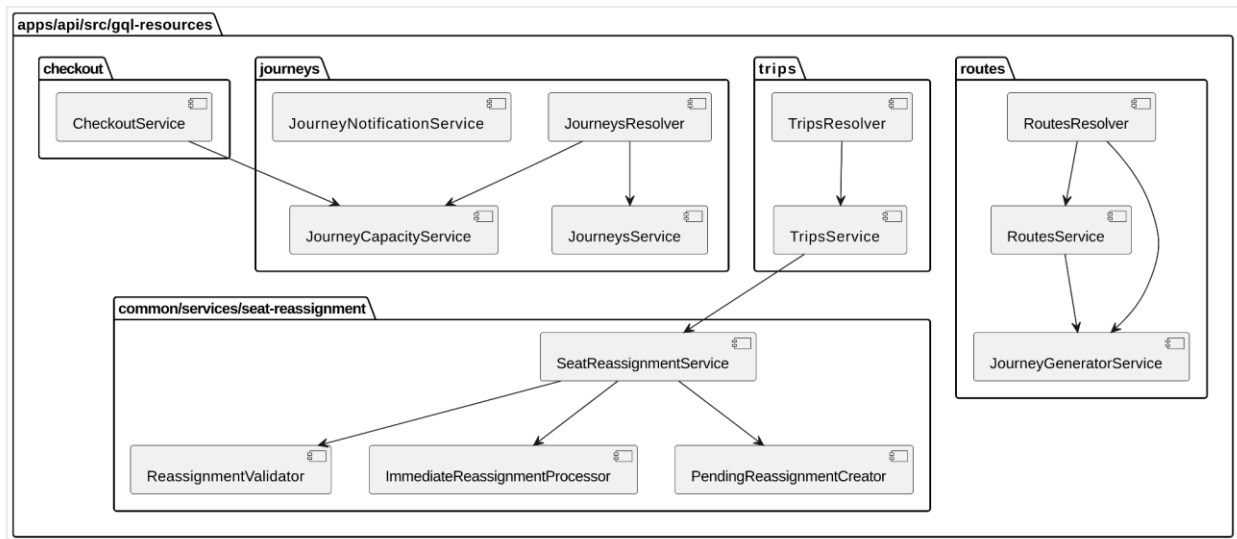
a. *Arquitectura General de API*

El núcleo del sistema está organizado en una arquitectura modular basada en GraphQL. Cada módulo encapsula casos de uso específicos y mantiene fronteras claras entre funcionalidades, lo que incrementa la cohesión y permite una evolución independiente. Los módulos principales son:

- Routes Module: gestión de rutas, generación de viajes y asignación de buses.
- Journeys Module: administración de viajes individuales, precios y disponibilidad.
- Trips Module: coordinación de cambios de bus y manejo de reasignaciones de asientos.
- Seat Reassignment Service: motor especializado para reasignaciones operativas.

Todos estos componentes interactúan mediante un conjunto estructurado de Resolvers, Servicios de Dominio y Modelos Prisma, definidos en el archivo de diseño del incremento.

Ilustración 48 Diagrama de paquetes de api del segundo incremento



Fuente: Elaboración propia, 2025.

b. Modelo de datos y ORM

El diseño del modelo físico utiliza Prisma ORM, estructurado para reflejar las relaciones entre rutas, viajes, buses, precios y reasignaciones. Las entidades más relevantes incluyen:

- Route
- Journey
- Trip
- JourneyPricing
- SeatReassignment

En la figura 3.37 se expone el modelo de datos físico para el segundo incremento, especificando los tipos de datos y límites de caracteres si aplica. Se incluyen además las tablas del primer incremento.

c. Diseño por Módulos

c.1. Módulo de Rutas (Routes)

Este módulo centraliza la gestión de viajes individuales y su disponibilidad. Destaca el método `retrieveJourney`, que retorna la información completa del viaje junto a un `bus_layout_custom`, calculado con base en tickets activos, viajes superpuestos y reasignaciones pendientes

- `RoutesResolver`: punto de entrada GraphQL
- `RoutesService`: reglas de negocio para rutas
- `JourneyGeneratorService`: motor que crea Trips y Journeys a partir de reglas de recurrencia

c.2. Módulo de Viajes (Journeys)

Este módulo centraliza la gestión de viajes individuales y su disponibilidad, ofreciendo funciones clave como la recuperación de datos del viaje y la estructura del bus. Su pieza fundamental es el método `retrieveJourney`, encargado de retornar la información completa del viaje junto con un `bus_layout`, generado a partir del análisis de tickets activos, posibles viajes superpuestos y reasignaciones pendientes, garantizando así una representación precisa y actualizada de la ocupación y disponibilidad.

c.3. Módulo de Trips

El módulo de Trips coordina múltiples journeys dentro de un viaje completo y permite gestionar cambios de bus a nivel global. Su caso de uso principal, CU-14: Reasignar Bus por Incidencia (RF-06), se sostiene en el método `changeBusAndReassignSeats`, que habilita el reemplazo en tiempo real del bus asociado a un trip y ejecuta procesos automáticos de reasignación de asientos. Este flujo contempla la validación del nuevo layout, el procesamiento de reasignaciones inmediatas, la generación de reasignaciones pendientes para otras terminales y la notificación en tiempo real a los clientes suscritos, garantizando continuidad operativa y mínima afectación al pasajero.

c.4. Servicio de Reasignación de Asiento

El `SeatReassignmentService` implementa la lógica de reasignación bajo condiciones de cambio de bus o layout. Su diseño contempla:

- Validaciones estrictas de disponibilidad y consistencia.
- Agrupación de reasignaciones por terminal.
- Procesamiento dentro de transacciones para evitar condiciones de carrera.
- Uso de índices optimizados para búsquedas rápidas de asientos ocupados o reservados.

c.5. Servicio de Capacidad (`JourneyCapacityService`)

La disponibilidad de asientos depende no solo de un viaje individual, sino de todos los viajes superpuestos dentro del mismo trip. Este servicio:

- Identifica viajes superpuestos.
- Calcula ocupación consolidada considerando tickets activos.
- Evalúa asientos reservados por reasignaciones pendientes.
- Expone la capacidad final para procesos como checkout, consulta y asignación.

3.4.2.5. *Desarrollo del Incremento*

a. Flujo de procesos

a.1. Flujo de Activación de Ruta

El flujo de activación de ruta permite al Líder de Línea activar operativamente una ruta mediante la generación automática de viajes (journeys) para un rango de fechas determinado. Este proceso involucra la asignación del bus layout a cada journey generado, el cálculo de precios iniciales basados en las reglas de negocio configuradas, y la creación de trips que agrupan múltiples journeys relacionados que comparten el mismo bus físico. La implementación se realiza mediante la mutación GraphQL ``generateTripsForToday`` que delega la lógica de negocio

al servicio `JourneyGeneratorService`, garantizando la consistencia de datos mediante transacciones de base de datos y validaciones previas a la creación de registros. La Figura 3.48 muestra la estructura completa del flujo desde la recepción de la petición hasta la creación de los registros en base de datos.

Ilustración 51 Metodo de Generacion de Ruta

```
@Public()
@Mutation(() => GenerateJourneysResponse, { nullable: true })
async generateTripsForToday([
  @Args('routeConfigs', { type: () => [RouteConfigInput], nullable: true })
  routeConfigs?: RouteConfigInput[],

  @Args('generateDayRange', { type: () => Int, nullable: true })
  generateDayRange?: number,

  @Args('targetDateISO', { type: () => String, nullable: true })
  targetDateISO?: string,

  @Args('dayOffset', { type: () => Int, nullable: true })
  dayOffset?: number,

  @Args('skipExistingCheck', { type: () => Boolean, nullable: true })
  skipExistingCheck?: boolean,
]) {
  return this.journeyGeneratorService.generateTripsForToday({
    routeConfigs,
    generateDayRange,
    targetDateISO,
    dayOffset,
    skipExistingCheck,
  })
}

@Permission({ action: Action.UPDATE, subject: Subject.ROUTE })
@Mutation(() => RouteEntity)
updateRoute(
  @Args('updateRouteInput') updateRouteInput: UpdateRouteInput,
  @Context('req') request: Request
) {
  throw new ApiError('Unimplemented')

  return this.routesService.update(updateRouteInput.id, {
    ...updateRouteInput,
    updated_by_user_id: request.user.id,
  })
}
```

Fuente: Elaboración propia, 2025.

La activación de ruta comienza con la recepción de parámetros en el resolver GraphQL `RoutesResolver.generateTripsForToday()`, ubicado en `apps/api/src/gql-resources/routes/routes.resolver.ts` líneas 159-183. Este endpoint acepta configuraciones opcionales como `routeConfigs` para especificar rutas específicas con layouts

personalizados, `generateDayRange` para definir el rango de días a generar, `targetDateISO` para establecer una fecha objetivo, y `dayOffset` para ajustar la fecha base. El servicio primero ejecuta `updateRoutesWithCustomLayouts()` para actualizar los layouts de bus personalizados cuando se especifican en las configuraciones, permitiendo que diferentes rutas utilicen layouts distintos según las necesidades operativas. Posteriormente, construye una consulta condicional para obtener las rutas activas junto con toda su información relacionada: segmentos, paradas, bases de precios, condiciones de precio, y el bus layout completo con sus elementos y clases de asiento. La Figura 3.49 ilustra el código que implementa la recepción de parámetros y la construcción de la consulta con todas las relaciones necesarias.

Ilustración 52 Creación Transaccional de Trips y Journeys con Logging

```

async generateJourneysV2(params?: GenerateJourneysParams) {
  const generateDayRange = params?.generateDayRange || this.DEFAULT_GENERATE_DAY_RANGE
  const BOLIVIA_TIMEZONE = 'America/La_Paz'

  const routes = await this.prisma.route.findMany({
    include: {
      segments: {
        include: {
          arrival_stop: true,
          departure_stop: true,
          pricing_bases: true,
          pricing_conditions: {
            include: {
              journey_pricing: true,
              seat_classes: true,
            },
          },
          journeys: true,
        },
      },
      stops: {
        orderBy: {
          sequence: 'asc',
        },
      },
      pricing_bases: {
        include: {
          seat_class: true,
        },
      },
      bus_layout: {
        include: {
          bus_elements: {
            include: {
              seat: {
                select: { seat_class_id: true }
              }
            }
          }
        },
      },
    },
  })

  const generatedJourneys = routes.reduce((routeAcc: any[], route) => {
    routeAcc.push(route)
  }, [])

  if (!generatedJourneys.length) return

  const createdTrips = await this.prisma.$transaction(async () => {
    this.logger.debug(
      `Generated ${createdTrips.length} trips totaling ${generatedJourneys.length} journeys`
    )
    return { generated_count: generatedJourneys.length }
  })
}

```

Fuente: Elaboración propia, 2025.

Antes de integrar la funcionalidad en la interfaz gráfica, se desarrolló una suite completa de pruebas unitarias para validar el flujo de activación de ruta y la generación automática de journeys. Estas pruebas cubren desde la recepción de parámetros en el resolver GraphQL hasta la creación transaccional de trips y journeys, incluyendo la validación de layouts personalizados, reglas de recurrencia con rrule, cálculos de precios con PricingCalculatorService y asignación correcta de bus_layout_id. También verifican la

atomicidad de las transacciones, la conversión de fechas entre zonas horarias y UTC, el manejo de casos edge y la prevención de duplicados con `skipExistingCheck`. Usando mocks de `PrismaService` y `PricingCalculatorService`, se garantiza el aislamiento de la lógica y la fiabilidad del sistema. La Figura 3.50 muestra la ejecución exitosa de todas las pruebas, confirmando que el sistema está listo para su integración con la interfaz gráfica.

Ilustración 53 Ejecución de Pruebas Unitarias de Activación de Ruta

```

$ cd /home/auml/Projects/travell-apps/Travell-MonoRepo && npx jest apps/api/src/gql-resources/route
4.4. Creación Transaccional de Trips y Journeys
  ✓ debe crear trips y journeys en una transacción atómica (4 ms)
  ✓ debe usar connectOrCreate para trips permitiendo múltiples journeys compartir el mismo trip (4 ms)
  ✓ debe generar un tripIndex único usando nanoid para agrupar journeys relacionados (4 ms)
  ✓ debe retornar el conteo de journeys generados (3 ms)
  ✓ debe manejar errores en la transacción y hacer rollback (12 ms)
Validación de Datos y Casos Edge
  ✓ debe filtrar pricing bases que no corresponden a seat classes del bus layout (3 ms)
  ✓ debe saltar journeys que ya existen cuando skipExistingCheck es false (2 ms)

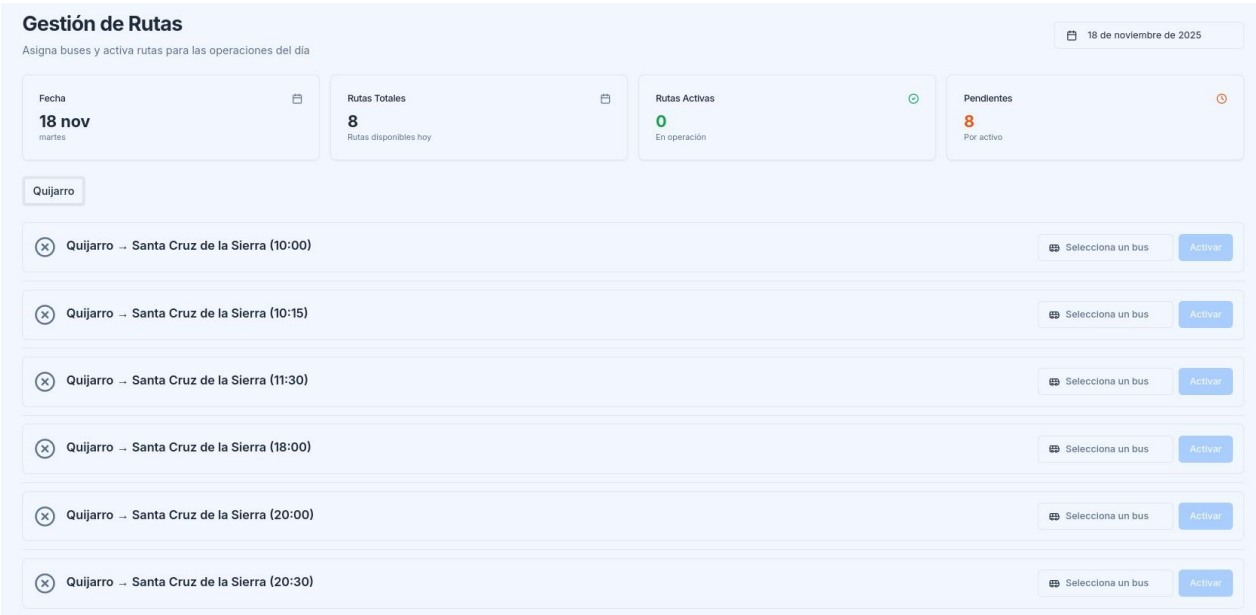
Test Suites: 1 passed, 1 total
Tests:       18 passed, 18 total
Snapshots:   0 total
Time:        3.368 s, estimated 4 s

```

Fuente: Elaboración propia, 2025.

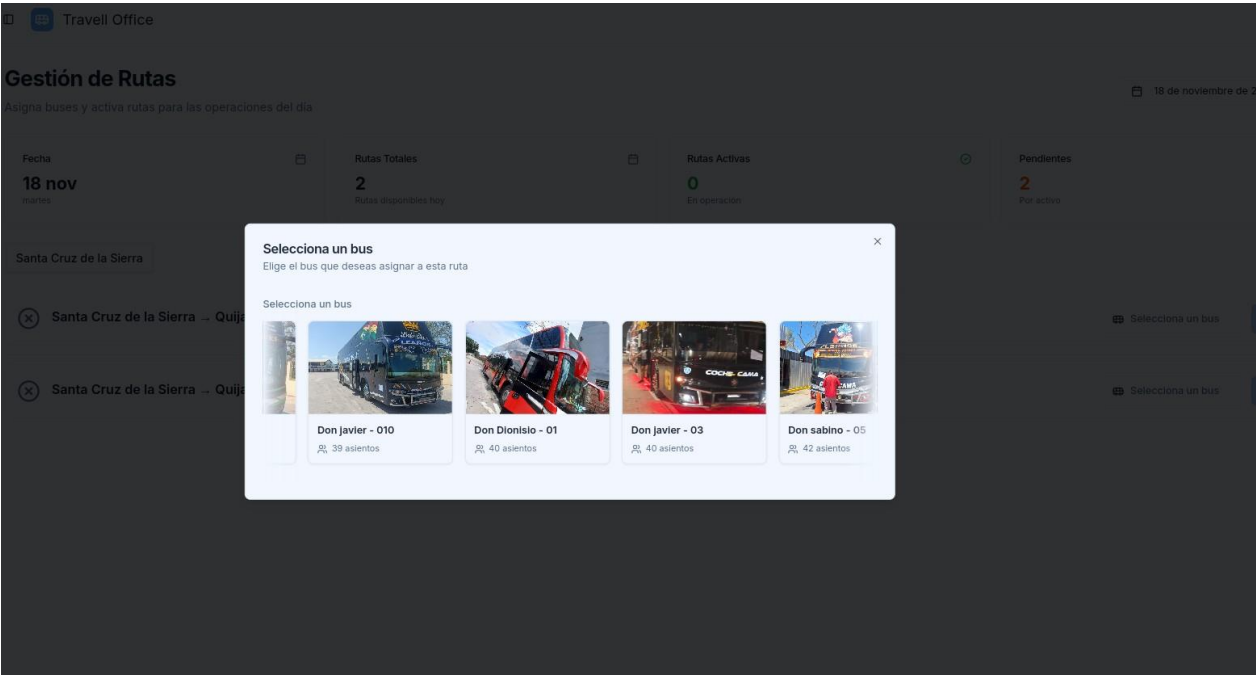
La funcionalidad de asignación de bus a ruta se implementa mediante una interfaz de gestión de rutas que permite al Líder de Línea visualizar todas las rutas disponibles para una fecha específica, agruparlas por terminal de origen, y asignar buses disponibles a cada ruta para activarlas operativamente. La pantalla principal muestra un resumen de rutas totales, activas y pendientes mediante tarjetas de estadísticas, seguido de una lista agrupada por terminal donde cada ruta se presenta como una tarjeta con su información (origen, destino, horario) y controles para seleccionar un bus y activar la ruta. El proceso de activación incluye validaciones en tiempo real: verificación de disponibilidad del bus seleccionado para la fecha/hora, validación de que la ruta no esté ya activa, y confirmación mediante un diálogo antes de ejecutar la mutación. Una vez confirmada, el sistema crea los viajes (trips) y jornadas (journeys) correspondientes, actualiza el estado de disponibilidad del bus, y propaga el evento de activación mediante PubSub para que otros usuarios vean la actualización inmediatamente.

Ilustración 54 Interfaz Grafica de Activación de Rutas



Fuente: Elaboración propia, 2025. Ilustración 55

Selección de Bus para Activar Ruta



Fuente: Elaboración propia, 2025.

a.2. Flujo de Reasignación Operativa

El flujo de reasignación operativa permite al Líder de Línea cambiar el bus layout de un trip y reasignar automáticamente los asientos de los pasajeros cuando se produce un cambio de bus o una incidencia operativa. Este proceso se implementa mediante la mutación GraphQL `changeBusAndReassignSeats` que recibe el identificador del trip, el nuevo bus layout, y un array de reasignaciones de asientos especificando el ticket, el asiento antiguo y el nuevo asiento para cada pasajero. El sistema procesa las reasignaciones de forma diferenciada según la terminal de salida: para los pasajeros que parten desde la terminal del usuario que realiza la operación, las reasignaciones se aplican inmediatamente actualizando el ticket con el nuevo asiento y creando un registro de auditoría; para los pasajeros de otras terminales, se crean registros pendientes en la tabla `SeatReassignment` con estado `PENDING` que serán aplicados posteriormente por el operador correspondiente. La Figura 3.53 muestra la estructura completa del sistema, incluyendo la separación entre procesamiento inmediato y pendiente, la validación de disponibilidad de asientos, y la gestión transaccional de los cambios.

Ilustración 56 Servicio de Reasignación de Bus y Asientos

```

export class SeatReassignmentService {
  async changeBusAndReassignSeats(
    // 2. Pre-fetch trip context and permissions outside transaction
    const tripContextPreview = await this.getTripContext(this.prisma, request.tripId);
    const permissions = await this.terminalPermissionService
      .getUserJourneyPermissions(user, tripContextPreview.journeys);

    return await this.prisma.$transaction(async (tx) => {
      try {
        // 3. Get fresh trip context within transaction (for consistency)
        const tripContext = await this.getTripContext(tx, request.tripId);

        // 3.5. 🟡 MEDIUM: Optimistic locking check
        if (request.expectedUpdatedAt) {
        }

        // 4. Separate immediate vs pending reassignments
        const reassignmentGroups = this.groupReassignmentsByPermission(...);

        this.logger.debug(...);

        // 5. Process immediate reassignments (user's terminal journeys)
        const immediateResults = await this.immediateProcessor.process(...);

        // 6. Create pending reassignments for other terminals (explicit reassignments)
        const pendingResults = await this.pendingCreator.create(...);

        // 6.5. Automatically create pending reassignments for existing tickets from other terminals
        const autoPendingResults = await this.createAutomaticPendingReassignments(...);

        // 7. Update ALL journey layouts in the same trip and route
        await this.updateAllJourneyLayoutsInTripAndRoute(...);

        const allResults = [...immediateResults, ...pendingResults, ...autoPendingResults];
        const successCount = allResults.filter(r => r.status !== 'FAILED').length;

        this.logger.log(
          `Seat reassignment completed: ${successCount}/${allResults.length} successful`
        );

        // 8. Refresh trip context for response
        const updatedTripContext = await this.getTripContext(tx, request.tripId);

        // Note: Real-time notifications for pending reassignments are handled in the resolver

        return this.buildResponse(updatedTripContext.trip, allResults);
      } catch (error: any) {
        this.logger.error('Error during seat reassignment:', error);
        throw error;
      }
    }, {
      timeout: 15000, // 15 seconds max for Prisma Accelerate compatibility
      maxWait: 20000 // Maximum wait time including queue time
    });
  }
}

```

Fuente: Elaboración propia, 2025.

El proceso de reasignación aplica validaciones esenciales para garantizar la integridad de datos y evitar conflictos de asientos. El sistema confirma que el nuevo asiento esté disponible en el journey, verificando tickets activos y reasignaciones pendientes mediante `validateSeatAvailabilityInTransaction`, ejecutado dentro de una transacción para prevenir condiciones de carrera. También valida la compatibilidad entre el asiento original y el nuevo según las reglas de negocio. Al ejecutar `applyPendingReassignmentsOnly`, se localizan las reasignaciones pendientes, se vuelve a validar la disponibilidad, se actualiza el ticket con el nuevo asiento y se marca la reasignación como completada, registrando el usuario y la fecha de aplicación. La Figura 3.54 muestra las validaciones y la aplicación transaccional de las reasignaciones.

Ilustración 57 Metodos de Validaciones

```

/**
 * Enhanced seat availability validation for transaction context
 * Checks both existing tickets and pending reassignments
 */
async validateSeatAvailabilityInTransaction(--
)

private async validateSeatCompatibility(--
)

private async validateJourneyMatch(--
)

async validateBusLayoutChange(
  tripId: string,
  currentBusLayoutId: number,
  newBusLayoutId: number
): Promise<void> {
  // Validate that the trip exists and has the expected current layout
  const trip = await this.prisma.trip.findUnique({
    where: { id: tripId },
    include: {
      journeys: {
        select: {
          bus_layout_id: true
        },
        take: 1
      }
    }
  });

  if (!trip) {
    throw new ValidationError('Trip not found');
  }

  // Validate new bus layout exists
  const newBusLayout = await this.prisma.busLayout.findUnique({
    where: { id: newBusLayoutId }
  });

  if (!newBusLayout) {
    throw new ValidationError('New bus layout not found');
  }
}

```

Fuente: Elaboración propia, 2025.

Antes de proseguir con la integración a la interfaz gráfica, se implementó una suite completa de pruebas unitarias para validar el flujo de reasignación operativa en todos sus componentes críticos. Las pruebas cubren la creación de reasignaciones inmediatas y pendientes, la validación de disponibilidad de asientos considerando tanto tickets activos como reasignaciones pendientes, la compatibilidad entre asientos de diferentes clases, y la aplicación de reasignaciones pendientes con sus respectivas validaciones transaccionales. La suite de pruebas utiliza mocks de `PrismaService` para aislar la lógica de negocio y garantizar que cada componente funcione correctamente de forma independiente, incluyendo casos de éxito, validación de datos, manejo de errores, y casos edge como asientos no disponibles, reasignaciones duplicadas, y conflictos de concurrencia. Todas las pruebas fueron ejecutadas exitosamente antes de proceder con la implementación de la interfaz gráfica, asegurando que la funcionalidad core del sistema de reasignación esté completamente validada y lista para la integración.

Ilustración 58 Ejecución de Pruebas Unitarias de Reasignación

```

a1m1@pop-os:~/Projects/travell-apps/Travell-MonoRepo$ cd /home/a1m1/Projects/travell-apps/Travell-MonoRepo && npx jest apps/api/src/common/services
[Nest] 744449 - 11/18/2025, 9:22:25 PM ERROR [SeatReassignmentService] Failed to apply reassignment for ticket 1:
[Nest] 744449 - 11/18/2025, 9:22:25 PM ERROR [SeatReassignmentService] TypeError: Cannot read properties of undefined (reading 'first_name')
[Nest] 744449 - 11/18/2025, 9:22:25 PM ERROR [SeatReassignmentService] Failed to apply reassignment for ticket 1:
[Nest] 744449 - 11/18/2025, 9:22:25 PM ERROR [SeatReassignmentService] TypeError: Cannot read properties of undefined (reading 'first_name')
[Nest] 744449 - 11/18/2025, 9:22:25 PM ERROR [SeatReassignmentService] Error during seat reassignment:
[Nest] 744449 - 11/18/2025, 9:22:25 PM ERROR [SeatReassignmentService] Error: Trip was modified by another user at 2024-01-02T00:00:00.000Z. Exp
n.
PASS  api apps/api/src/common/services/seat-reassignment/seat-reassignment.service.spec.ts
SeatReassignmentService - Flujo de Reasignación Operativa
3.3.1. Arquitectura del Flujo de Reasignación Operativa
  ✓ debe procesar reasignaciones inmediatas y pendientes correctamente (17 ms)
  ✓ debe separar reasignaciones inmediatas de pendientes según permisos de terminal (4 ms)
3.3.2. Proceso de Validación y Aplicación de Reasignaciones
  ✓ debe validar disponibilidad de asientos considerando tickets activos y reasignaciones pendientes (3 ms)
  ✓ debe validar compatibilidad entre asiento antiguo y nuevo (6 ms)
  ✓ debe aplicar reasignaciones pendientes con validación transaccional (5 ms)
  ✓ debe manejar errores cuando el asiento no está disponible (4 ms)
Validación de Datos y Casos Edge
  ✓ debe validar que el ticket existe y no está cancelado (3 ms)
  ✓ debe manejar bloqueo optimista cuando el trip fue modificado (10 ms)
  ✓ debe crear reasignaciones automáticas para tickets de otras terminales (3 ms)

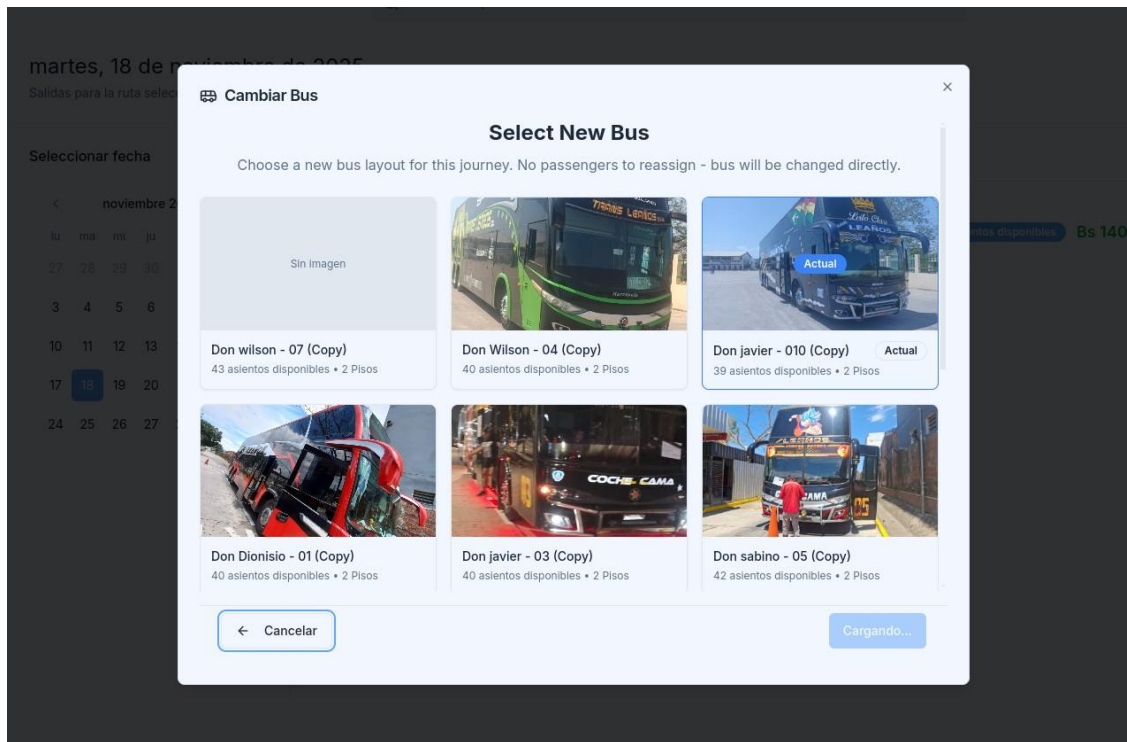
Test Suites: 1 passed, 1 total
Tests: 9 passed, 9 total
Snapshots: 0 total
Time: 1.599 s, estimated 5 s
Ran all test suites matching /apps/api/src/common/services/seat-reassignment/seat-reassignment.service.spec.ts/i.

```

Fuente: Elaboración propia, 2025.

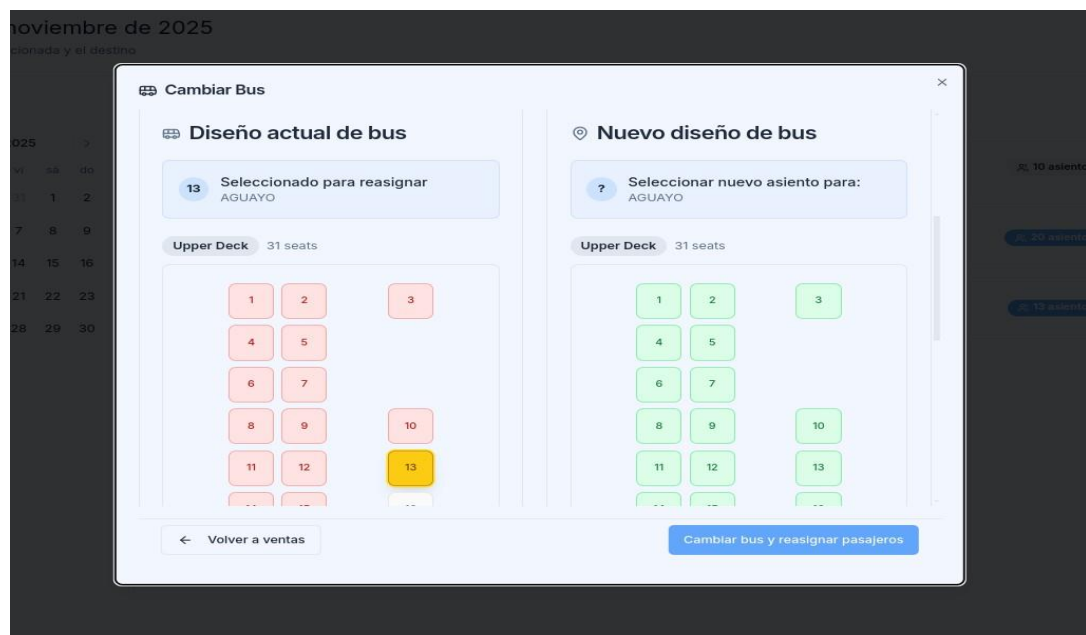
La funcionalidad de asignación de bus a ruta se implementa mediante una interfaz de gestión de rutas que permite al Líder de Línea visualizar todas las rutas disponibles para una fecha específica, agruparlas por terminal de origen, y asignar buses disponibles a cada ruta para activarlas operativamente. La pantalla principal muestra un resumen de rutas totales, activas y pendientes mediante tarjetas de estadísticas, seguido de una lista agrupada por terminal donde cada ruta se presenta como una tarjeta con su información (origen, destino, horario) y controles para seleccionar un bus y activar la ruta. El proceso de activación incluye validaciones en tiempo real: verificación de disponibilidad del bus seleccionado para la fecha/hora, validación de que la ruta no esté ya activa, y confirmación mediante un diálogo antes de ejecutar la mutación. Una vez confirmada, el sistema crea los viajes (trips) y jornadas (journeys) correspondientes, actualiza el estado de disponibilidad del bus, y propaga el evento de activación mediante PubSub para que otros usuarios vean la actualización inmediatamente.

Ilustración 59 Interfaz de Reasignación de Bus



Fuente: Elaboración propia, 2025. Ilustración 60

Interfaz de Reasignación de asientos



Fuente: Elaboración propia, 2025.

a.3. Flujo de Consulta de Disponibilidad y Actualización de precio

El flujo de consulta de bus layout por journey permite a los usuarios (vendedores, choferes, líderes de línea) obtener la información completa del layout del bus asignado a un journey específico, incluyendo la disponibilidad en tiempo real de cada asiento. Este proceso se implementa mediante la query GraphQL `retrieveJourney` que recibe el identificador del journey y retorna toda la información del viaje junto con el `bus_layout_custom`, un objeto que contiene el layout del bus enriquecido con información de disponibilidad calculada dinámicamente. El método `buildBusLayoutCustom` en el `JourneysService` es responsable de construir este objeto, consultando primero el journey con su bus layout asociado, luego obteniendo la capacidad disponible mediante el `JourneyCapacityService` que calcula los asientos reservados considerando journeys superpuestos del mismo trip, y finalmente marcando cada asiento con su estado de disponibilidad (`isAvailable`, `isReserved`) basado en tickets activos y órdenes pendientes. La Figura 3.58 muestra la estructura completa del sistema, incluyendo la consulta del journey con sus relaciones, el cálculo de disponibilidad considerando journeys superpuestos, y la construcción del objeto `bus_layout_custom` con información en tiempo real.

Ilustración 61 Método de Consulta de Viaje

```

@Public()
@Query(() => RetrieveJourneyGraphQLType, { name: 'retrieveJourney' })
async retrieveJourney(@Args() query: RetrieveJourneyQueryDto) {}
// Fetch the journey with the associated bus layout
const journey = await this.prisma.journey.findUniqueOrThrow({
  where: { id: query.id },
  include: {
    departure_terminal: {
      include: {
        address: true
      }
    },
    arrival_terminal: {
      include: {
        address: true
      }
    },
    company: true,
    pricing: {
      include: {
        seat_class: true,
      }
    },
    bus_layout: {
      include: {
        seat_classes: true,
        images: true,
        bus_elements: {
          include: {
            seat: {
              include: {
                seat_class: true,
              }
            },
          },
        },
      }
    },
  },
});

```

Fuente: Elaboración propia, 2025.

Ilustración 62 Metodo de Búsqueda de Viaje

```

/**
 * Public facing function used to search for journeys
 */
async search(params: SearchJourneyQueryDto) {
  const searchDepartureDate: DateTime = DateTime.fromJSDate(params.departure_date)

  const journeys = await this.prisma.journey.findMany({
    where: {
      status: 'SCHEDULED',
      departure_terminal: {
        address: { city: { mode: 'insensitive', equals: params.departure_city } },
      },
      arrival_terminal: {
        address: { city: { mode: 'insensitive', equals: params.arrival_city } },
      },
      departure_date: {
        // TODO: Also need to adjust for timezone, similar to how I'm doing on generated instances
        // TODO: Temporary fix to display journeys since they have invalid times
        gte: searchDepartureDate.startOf('day').toJSDate(),
        lte: searchDepartureDate.endOf("day").plus({hours: 4}).toJSDate(),
      },
    },
    ...getPageParams({
      page: params.page,
      limit: params.limit,
    }),
    orderBy: { departure_date: 'asc' },
  })

  return journeys
}

```

Fuente: Elaboración propia, 2025.

El proceso de construcción del bus layout custom incluye validaciones y cálculos complejos para garantizar la precisión de la información de disponibilidad. El sistema utiliza el método `getOverlappingJourneyIds` del `JourneyCapacityService` para identificar todos los journeys que comparten el mismo bus físico (mismo trip), ya que los asientos se comparten entre estos journeys y un asiento reservado en un journey afecta la disponibilidad en los demás. Una vez identificados los journeys superpuestos, se consultan todos los tickets activos (excluyendo cancelados y reembolsados) y órdenes pendientes de estos journeys para determinar qué asientos están ocupados o reservados. El resultado final es un objeto `bus_layout_custom` que contiene todos los elementos del bus (asientos, pasillos, escaleras, etc.) con información detallada de cada asiento incluyendo su número, clase, estado operativo, y disponibilidad en tiempo real, permitiendo a los usuarios visualizar exactamente qué asientos están disponibles para la venta en un momento dado. La Figura 3.59 ilustra el flujo detallado de consulta de journeys superpuestos, cálculo de asientos reservados, y construcción del layout customizado con estados de disponibilidad.

Ilustración 63 Servicio de Consulta de Disponibilidad de Asientos

```

import { PrismaService } from '../../common/services/database/prisma.service'
import { SeatEntity } from '../../bus-layouts/entities/seat.entity'
import { GetOrderCapacityEntity } from '../../orders/entities/order-capacity.entity'

/**
 * Service responsible for gathering bus layout capacity for a journey
 */
@Injectable()
export class JourneyCapacityService {
  private readonly logger = new Logger(JourneyCapacityService.name)

  constructor(private readonly prisma: PrismaService) {}

  /**
   * Get all journey IDs that overlap with the given journey in the same trip.
   * This is the single source of truth for determining overlapping journeys.
   * @param journeyId - The journey to check overlaps for
   * @returns Array of journey IDs that overlap (including the current journey)
   */
  async getOverlappingJourneyIds(journeyId: number): Promise<number[]> { ...
  }

  /**
   * Get available capacity for a journey. Uses the centralized getOverlappingJourneyIds
   * to determine which orders/tickets affect seat availability.
   * @param params
   * @returns Capacity of bus with booked seats from overlapping journeys
   */
  async getAvailableBusCapacity(params: {
    journeyId: number
  }): Promise<GetOrderCapacityEntity & { seats_booked: Seat[] }> {
    // Get basic journey info for capacity calculation
    const journey = await this.prisma.journey.findUniqueOrThrow({
      where: { id: params.journeyId },
      select: {
        id: true,
        bus_layout: {
          select: {
            id: true,
            seat_capacity: true,
          },
        },
      },
    })
  }
}

```

Fuente: Elaboración propia, 2025.

El ajuste de precios se implementa mediante un modal accesible desde la vista de gestión de viajes, permitiendo al Líder de Línea modificar el precio vigente de una ruta activa. El modal muestra los precios actuales por clase de asiento (si aplica) o un precio único, con campos editables que validan en tiempo real, como se puede observar en la figura 3.60. El sistema previene precios negativos, verifica que no excedan el precio máximo configurado, y muestra el impacto del cambio (diferencia con precio anterior). Cuando se intenta ingresar un precio que excede el máximo permitido, el sistema muestra un mensaje de error claro y resalta el campo, tal como se muestra en la figura 3.60. Al confirmar el ajuste, el sistema actualiza los precios activos en la base de datos, propaga el cambio mediante PubSub para que vendedores y choferes vean el nuevo precio inmediatamente, y registra el evento en auditoría con valores anteriores y nuevos.

Ilustración 64 Metodo de Edición y Sync de Precios

```

@Permission({ action: Action.UPDATE, subject: Subject.JOURNEY })
@Mutation(() => [JourneyPricingEntity])
updateJourneyPricing(
  @Args('updateJourneyPricingInput') updateJourneyPricingInput: BatchUpdatePricingInput,
  @GqlUser() user: UserEntity
) {
  return this.journeysService.batchUpdateJourneyPricing(updateJourneyPricingInput)
}

@Permission({ action: Action.UPDATE, subject: Subject.JOURNEY })
@Mutation(() => SyncJourneyPricingResponse, {
  description: 'Syncs journey pricing based on the actual bus layout. Updates all journeys in the same trip.'
})
async syncJourneyPricing(
  @Args('input') input: SyncJourneyPricingInput
): Promise<SyncJourneyPricingResponse> {
  return this.journeysService.syncJourneyPricing(input.journeyId);
}

```

Fuente: Elaboración propia, 2025.

Antes de proseguir con la integración a la interfaz gráfica, se implementó una suite completa de pruebas unitarias para validar tanto el flujo de consulta de bus layout por journey como el flujo de edición de pricing. Las pruebas cubren la construcción correcta del objeto `bus\_layout\_custom` con información de disponibilidad en tiempo real, el cálculo de asientos reservados considerando journeys superpuestos, la actualización de precios a nivel de journey y trip, la sincronización automática de precios basada en bus layout, y el manejo de casos edge como journeys sin layout y clases de asiento sin precios base. La suite utiliza mocks de `PrismaService` y `JourneyCapacityService` para aislar la lógica de negocio y todas las pruebas fueron ejecutadas exitosamente, asegurando que la funcionalidad core esté completamente validada y lista para la integración. La Figura 3.61 muestra la ejecución exitosa de todas las pruebas.

Ilustración 65 Ejecución de Pruebas Unitarias Consulta y Edición de Precio

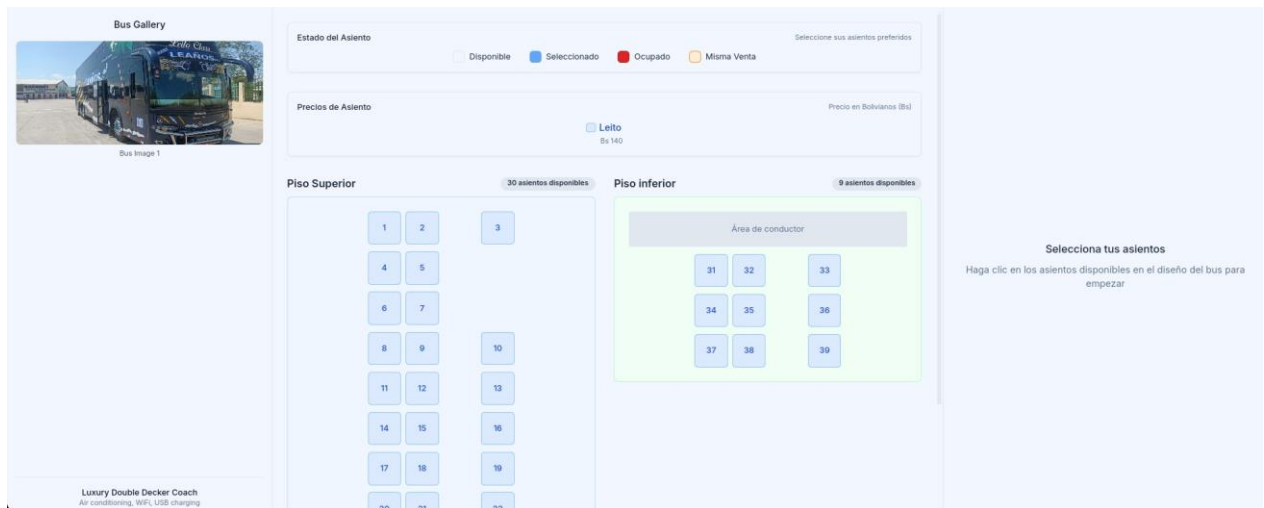
```
PASS api apps/api/src/gql-resources/journeys/journeys.service.spec.ts
JourneysService - Flujo de Consulta de Bus Layout y Edición de Pricing
3.4.1. Flujo de Consulta de Bus Layout por Journey
✓ debe construir bus layout custom con información de disponibilidad (18 ms)
✓ debe retornar null cuando no hay bus layout (2 ms)
✓ debe manejar errores y retornar layout básico sin disponibilidad (5 ms)
✓ debe calcular correctamente asientos reservados considerando journeys superpuestos (3 ms)
3.4.2. Flujo de Edición de Pricing
✓ debe actualizar precios de journeys individuales correctamente (4 ms)
✓ debe ejecutar actualizaciones dentro de una transacción (4 ms)
3.4.3. Flujo de Sincronización Automática de Pricing
✓ debe sincronizar precios basándose en el bus layout actual (2 ms)
✓ debe eliminar precios de clases de asiento que ya no existen en el layout (3 ms)
✓ debe crear precios para nuevas clases de asiento en el layout (3 ms)
✓ debe usar precio por defecto cuando no hay pricing base en segmento ni ruta (3 ms)
✓ debe manejar journeys sin bus layout (3 ms)

Test Suites: 1 passed, 1 total
Tests:      11 passed, 11 total
Snapshots:  0 total
Time:       4.285 s
```

Fuente: Elaboración propia, 2025.

La vista de viaje se implementa mediante una página dedicada que muestra toda la información relevante del viaje seleccionado, incluyendo detalles de la ruta (origen, destino, horarios), información del bus asignado, y el layout visual completo del bus con su configuración de asientos. La interfaz presenta el bus layout de forma destacada, mostrando la estructura física del vehículo con sus elementos organizados por filas, columnas y pisos, tal como se muestra en la figura 3.62. El layout incluye imágenes del bus cuando están disponibles, información técnica (capacidad, número de pisos, clases de asientos), y una representación visual interactiva donde cada asiento es un elemento clicable que muestra su estado actual. La vista permite al usuario comprender rápidamente la configuración del bus asignado al viaje, facilitando la toma de decisiones operativas y proporcionando contexto visual para las operaciones de venta y gestión de asientos que se realizarán posteriormente.

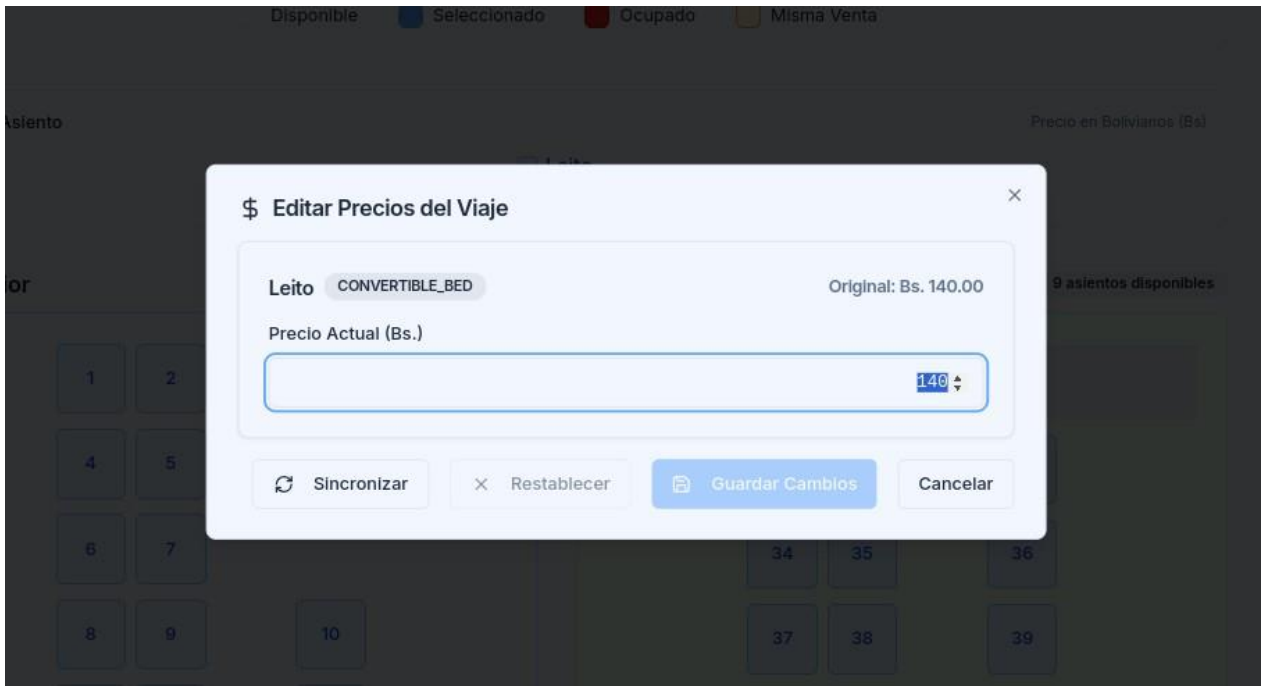
Ilustración 66 Interfaz de Consulta de Asientos Disponibles



Fuente: Elaboración propia, 2025.

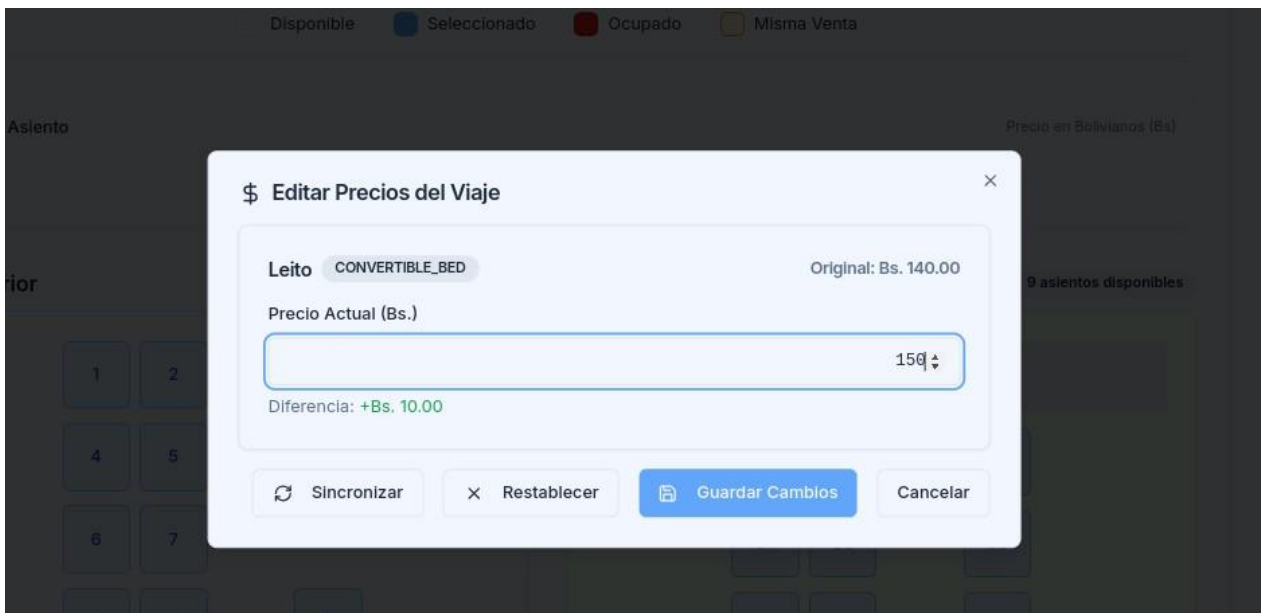
El proceso de sincronización de pricing incluye validaciones y cálculos complejos para garantizar que los precios estén siempre alineados con la configuración actual del bus layout. Cuando se ejecuta `syncJourneyPricing`, el sistema obtiene el journey con su trip completo, incluyendo todos los journeys relacionados, sus bus layouts con elementos y asientos, y sus precios actuales. Para cada journey, el sistema identifica las clases de asiento que tienen asientos físicos en el bus layout actual, compara esta lista con los precios existentes, y determina qué precios deben eliminarse (clases que ya no tienen asientos) y qué precios deben crearse (clases nuevas con asientos). Al crear nuevos precios, el sistema intenta obtener el precio base desde el segmento, luego desde la ruta, y finalmente utiliza un valor por defecto si no encuentra ninguna configuración. Todas las operaciones de creación y eliminación se ejecutan dentro de una transacción para garantizar la consistencia, y el sistema retorna un resumen detallado de los cambios realizados incluyendo cuántos journeys fueron actualizados y cuántos precios fueron agregados o eliminados en cada uno. La Figura 3.63 ilustra el flujo detallado de análisis del bus layout, comparación con precios existentes, y ejecución de operaciones de creación y eliminación dentro de transacciones.

Ilustración 67 Interfaz de Edición de Precios



Fuente: Elaboración propia, 2025.

Ilustración 68 Aumento de Precio en Interfaz



Fuente: Elaboración propia, 2025.

Ilustración 69 Actualización de Precio en Interfaz



Fuente: Elaboración propia, 2025.

3.4.2.6. Entrega y validación

Las funcionalidades de asignación de bus a ruta, consulta de disponibilidad por journey y actualización dinámica de precios se desplegaron en un entorno de preproducción con datos representativos para validar el flujo completo. Se ejecutó un recorrido extremo a extremo que incluyó la visualización de rutas, la activación operativa con validaciones de disponibilidad, la consulta del layout con información en tiempo real mediante `retrieveJourney` y la edición de precios con validaciones y propagación inmediata por PubSub. El criterio de cierre consideró el funcionamiento integral de estos módulos, la coherencia de datos y la correcta propagación de eventos entre sesiones simultáneas.

La validación funcional se realizó mediante pruebas guiadas que confirmaron la trazabilidad entre backend y frontend, así como la precisión del cálculo de disponibilidad, la construcción del objeto `bus_layout_custom` y la consistencia del pricing después de la sincronización automática. Las pruebas de desempeño mostraron tiempos aceptables en las operaciones críticas. Con la evidencia compilada y la conformidad del cliente, las funcionalidades fueron aceptadas (véase Anexo A.6).

3.4.3. INCREMENTO 3: HISTORIAL DE SESIONES DE BÚSQUEDA

Este incremento introduce las funcionalidades centrales del negocio orientadas a la venta de boletos, incluyendo selección de asientos, registro de pasajeros, modos de venta rápida, procesamiento de pagos, generación de órdenes, y mecanismos de reembolso. Constituye el corazón transaccional de la plataforma, sustentado en la activación operativa construida en el Incremento 2. El incremento implementa el motor de ventas con control de disponibilidad en tiempo real, integración del layout del bus para selección visual de asientos, registro seguro de datos de pasajeros y procesamiento de pagos con PIN del vendedor. Además, se incorpora la capacidad de reembolsar boletos, liberando los asientos y dejando trazabilidad transaccional. Con la entrega de este incremento, el sistema queda preparado para operar comercialmente de manera completa.

3.4.3.1. Alcance del incremento

El objetivo de este incremento es ofrecer todas las funcionalidades necesarias para completar la venta de un boleto, desde la selección del asiento hasta la confirmación del pago y su posible reembolso. El alcance incluye la selección visual y múltiple de asientos en el layout del bus (RF-09), un modo de venta rápida que permite realizar operaciones con datos mínimos del pasajero (RF-10), el procesamiento de pagos en efectivo o mediante QR con autenticación por PIN (RF-11), y la gestión de reembolsos, registro de transacciones y liberación inmediata de asientos (RF-12). Este incremento contempla el desarrollo de backend, el frontend operativo para vendedores y choferes, las validaciones de disponibilidad, la consistencia transaccional y el registro detallado del flujo de venta. Quedan fuera de alcance los reportes consolidados (incremento 4), el historial detallado de asientos y eventos operativos (incremento 4), la integración con sistemas de facturación electrónica.

3.4.3.2. Plan de trabajo y estimaciones

A continuación se presenta el plan de trabajo de este incremento, detallando cada tarea con su estimación de horas.

Cuadro 3.15. Plan de trabajo del tercer incremento

#	Tarea	Descripción breve	Estimación (h)
1	Integración con layout de bus	Lectura del layout, mapeo de asientos, estados (ocupado/libre/reservado)	10
2	Query de disponibilidad enriquecida	Capa para mostrar estado por asiento con sincronización en tiempo real	6
3	Selección múltiple de asientos	Validaciones de concurrencia, reservas temporales, expiración	8
4	Registro de pasajeros	Datos completos o mínimos; validaciones; reutilización de pasajeros existentes	6
5	Creación de órdenes	Mutations para generar la orden, validar estado y asociar asientos	6
6	Modo de venta rápida	Identificador temporal, lógica de actualización posterior de datos	6
7	Procesamiento de pagos	Selección de método, autenticación PIN, marcación de orden como pagada	8
8	Generación de boletos	Creación del ticket individual por asiento, numeración, timestamps	6
9	Reembolso	Mutations de liberación de asiento, registro de reembolso, trazabilidad	8
10	UI para vendedores	Pantalla de layout, flujo de venta rápida, flujo de registro completo	16
11	UI de reembolsos	Consulta de orden, ejecución de reembolso y validaciones	10
12	Pruebas	Integración: disponibilidad, pagos, concurrencia, reembolsos	14

Fuente: Elaboración propia, 2025.

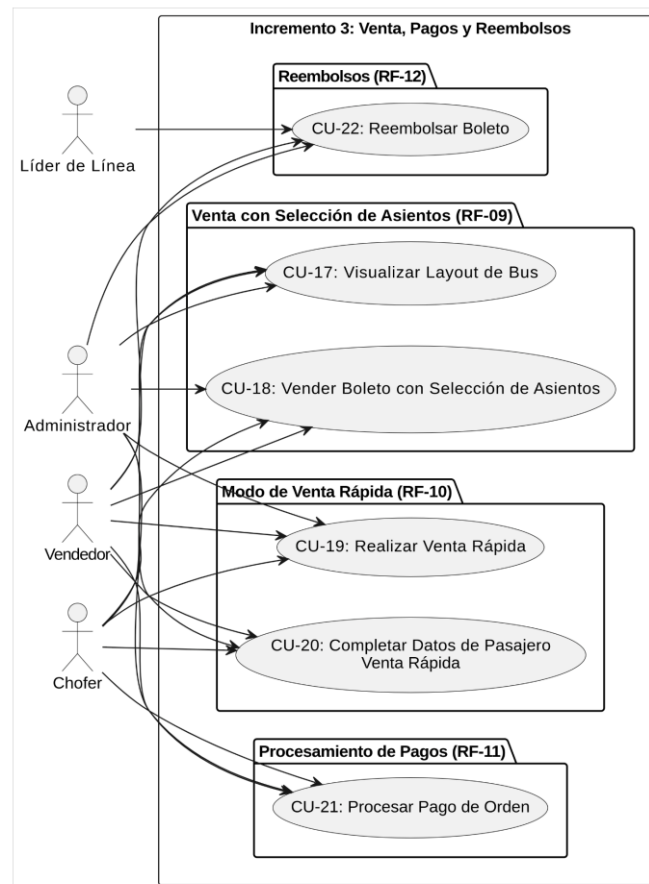
El esfuerzo total del incremento asciende a 108 horas, abarcando el desarrollo del motor completo de venta y reembolso, la implementación del layout visual con selección de asientos

en la interfaz de usuario, los flujos para múltiples pasajeros junto con los procesos de pago y generación de órdenes o tickets, además de pruebas exhaustivas de concurrencia para detectar conflictos de asiento, validaciones de pagos y manejo de reversos. Esta estimación asume un backend transaccional consistente, una sincronización continua de disponibilidad y una experiencia fluida para los vendedores incluso en contextos de alta demanda.

3.4.3.3. *Análisis funcional detallado*

Esta sección amplía el análisis funcional del incremento, refinando los casos de uso para especificar con claridad las precondiciones, las postcondiciones y el flujo principal. La figura 3.50 destaca el caso de uso incorporado en este incremento y, además, referencia los casos ya implementados en los incrementos previos.

Ilustración 70 Casos de uso cubiertos en el tercer incremento



Fuente: Elaboración propia, 2025.

A continuación, se detalla el caso de uso con sus propósitos, actores asociados, precondiciones, postcondiciones y flujo principal, además se incluye entre paréntesis su requerimiento asociado para facilitar la trazabilidad.

CU-17	Visualizar Layout de Bus (RF-09)	
Propósito	Permitir que el operador vea el layout del bus y el estado de cada asiento	
Actores	Vendedor, Chofer	
Precondiciones	Ruta/turno activos con bus asignado.	
Postcondiciones	Se muestra el layout del bus con el estado actualizado de los asientos según condiciones	
Flujo principal	(1) El actor selecciona ruta, fecha y turno. (2) El sistema valida que la ruta esté activa y tenga un bus asignado. (3) El sistema obtiene el layout del bus correspondiente (4) El sistema presenta el layout al actor con los asientos y su estado.	

CU-18	Vender Boleto con Selección de Asientos (RF-09)	
Propósito	Registrar la venta de uno o varios boletos asociando asientos específicos y datos completos de pasajeros.	
Actores	Vendedor, Chofer	
Precondiciones	Ruta/turno activos con bus asignado. Layout y disponibilidad de asientos cargados (CU-17 ejecutado).	
Postcondiciones	Se crea una orden de venta con uno o más boletos. Los asientos seleccionados quedan marcados como ocupados.	
Flujo principal	(1) El actor selecciona la ruta, fecha y turno. (2) El sistema muestra el layout del bus con asientos disponibles. (3) El actor selecciona uno o varios asientos libres. (4) El actor ingresa los datos de pasajero por asiento (nombre, apellido, CI, teléfono). (5) El sistema calcula el precio total según la ruta activa y cantidad de asientos. (6) El sistema registra la orden (7) El sistema actualiza la disponibilidad marcando los asientos como ocupados.	

CU-19	Realizar Venta Rápida (RF-10)	
Propósito	Permitir vender boletos con información mínima de pasajero, para completar los datos más adelante.	
Actores	Vendedor, Chofer	
Precondiciones	Asientos disponibles.	
Postcondiciones	Se crea una orden y boletos en modo venta rápida, con identificador temporal.	

	Los asientos quedan ocupados.
Flujo principal	(1) Seleccionar ruta (2) El sistema muestra el layout del bus y la disponibilidad. (3) El actor selecciona uno o varios asientos libres. (4) El actor indica "venta rápida" para esos asientos. (5) El sistema asigna un identificador temporal a cada boleto y omite datos completos del pasajero. (6) El sistema registra la orden y los boletos como venta rápida. (7) El sistema actualiza la disponibilidad marcando los asientos como ocupados.

CU-20	Completar Datos de Pasajero de Venta Rápida (RF-10)
Propósito	Permitir completar o corregir los datos de pasajero de boletos previamente vendidos en modo rápido.
Actores	Vendedor, Chofer
Precondiciones	Existen boletos marcados como "venta rápida" pendientes de completar datos.
Postcondiciones	Los boletos quedan con datos de pasajero completos.
Flujo principal	(1) El actor accede al listado de boletos/órdenes con venta rápida (2) El actor ingresa o actualiza nombre, apellido, CI y teléfono del pasajero (3) El sistema actualiza el registro de pasajero y del boleto (4) El sistema marca el boleto como información completa

CU-21	Procesar Pago de Orden (RF-11)
Propósito	Registrar el pago de una orden mediante efectivo o código QR, autenticando al vendedor mediante PIN.
Actores	Vendedor, Chofer
Precondiciones	Existen boletos seleccionados y asociados a una orden (CU-19 o CU-20)
Postcondiciones	La orden queda con estado de pago actualizado (pagada o error).
Flujo principal	(1) El actor selecciona el método de pago (efectivo o QR) (2) El sistema solicita el PIN del vendedor/operador. (3) El actor ingresa el PIN (4) El sistema valida el PIN y procesa el pago según el método elegido. (5) El sistema marca la orden como pagada y confirma la operación.

CU-22	Reembolsar Boleto (RF-12)
Propósito	Permitir reembolsar uno o varios boletos, liberando los asientos y registrando la transacción de reembolso.
Actores	Vendedor, Chofer, Líder de Línea
Precondiciones	Existe un boleto vendido asociado a una orden identificable
Postcondiciones	El boleto queda marcado como reembolsado. El asiento asociado vuelve a estar disponible.

Flujo principal	(1) El actor busca y selecciona la orden (2) El sistema muestra detalles de la orden (pasajeros, asientos, montos) (3) El actor confirma el reembolso. (4) El sistema marca los boletos como reembolsados y libera los asientos.
------------------------	---

3.4.3.4. *Diseño de solución*

El Incremento 3 incorpora los elementos necesarios para completar el flujo de venta de boletos, procesamiento de pagos, validación de pasajeros, emisión de órdenes y gestión del ciclo de vida de tickets. Este incremento consolida la plataforma operativa diseñada en incrementos previos, integrando capacidades críticas como creación de órdenes, métodos de pago múltiples, integración con proveedores externos y mecanismos de reembolso.

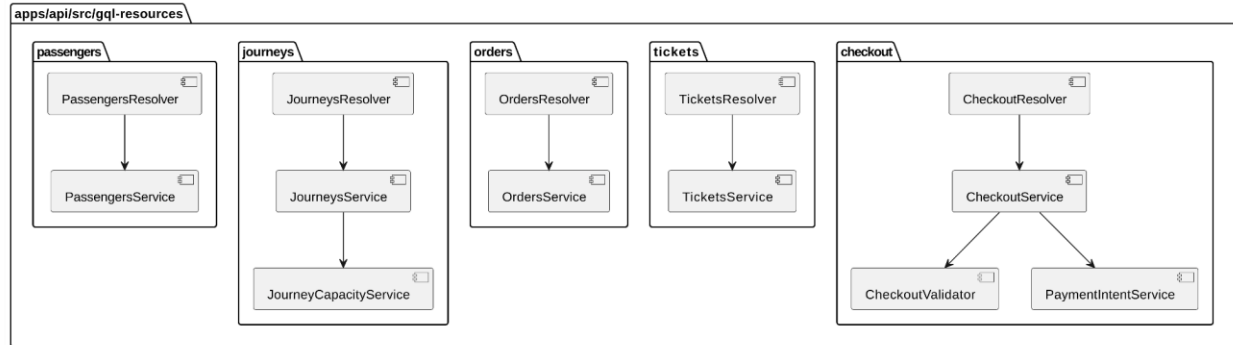
a. *Api*

El sistema se estructura en torno a módulos de dominio especializados, expuestos mediante resolvers GraphQL. El diagrama de paquetes incluido en el archivo describe esta organización con precisión. Los módulos principales son:

- Checkout Module: Procesa ventas, valida disponibilidad, genera órdenes, inicializa cobros y se comunica con proveedores de pago.
- Tickets Module: Gestiona el ciclo de vida del ticket (creación, verificación, reembolso).
- Orders Module: Administra órdenes, totales, estados y sincronización con tickets.
- Passengers Module: Gestiona creación, reutilización y actualización de pasajeros.
- Journeys Module: Provee disponibilidad de asientos en tiempo real

Todos estos componentes interactúan mediante un conjunto estructurado de Resolvers, Servicios de Dominio y Modelos Prisma, definidos en el archivo de diseño del incremento. Visualice el diagrama de paquetes del incremento en la Figura 3.36.

Ilustración 71 Diagrama de paquetes de api del tercer incremento



Fuente: Elaboración propia, 2025.

b. Modelo de datos y ORM

El Incremento 3 introduce un conjunto de ampliaciones significativas en el modelo de datos, orientadas a soportar de forma completa el flujo de venta de boletos, la gestión de pasajeros, la creación de órdenes y la validación de pagos. Estas ampliaciones permiten dotar al sistema de una estructura sólida que respalda operaciones transaccionales críticas como la selección de asientos, la generación de tickets, la confirmación de pagos y el registro de reembolsos.

La entidad Order constituye el eje central de cualquier transacción comercial y representa la compra como un todo. Su propósito es agrupar todos los elementos relacionados a una venta: datos del comprador, información del método de pago, asociación con el viaje seleccionado y vínculo con los pasajeros involucrados. En este sentido, la orden no solo facilita la organización de la información, sino que también actúa como punto de control para confirmar o revertir transacciones. Representa una transacción de compra. Contiene:

- Cantidad total de pasajeros
- Montos económicos
- Datos de contacto del comprador
- Método de pago
- Relación directa con un journey
- Relación con pasajero principal
- Estados del ciclo de compra (PENDING, CONFIRMED, REFUNDED)

La entidad **Ticket** es el elemento fundamental y más granular de la venta: cada ticket corresponde a un asiento asignado para un pasajero específico dentro de un viaje. Su función es permitir el control detallado de la ocupación, el estado del asiento y la trazabilidad del proceso de pago asociado a cada pasajero. Es el elemento central de la venta::

- Tiene asiento asignado
- Posee estados operativos: PENDING_PAYMENT, PENDING_SCAN, VERIFIED, REFUNDED
- Almacena datos opcionales del pasajero en modo “venta rápida”
- Está vinculado a orden, journey, pasajero y asiento

La entidad Passenger está diseñada para representar a las personas asociadas a cada ticket vendido. La estructura del modelo permite tanto la creación tradicional de pasajeros completos como la creación automática de registros mínimos cuando se procesa una venta rápida. Gracias a esto, el sistema es flexible para distintos tipos de operación sin comprometer la calidad de los datos. Permite:

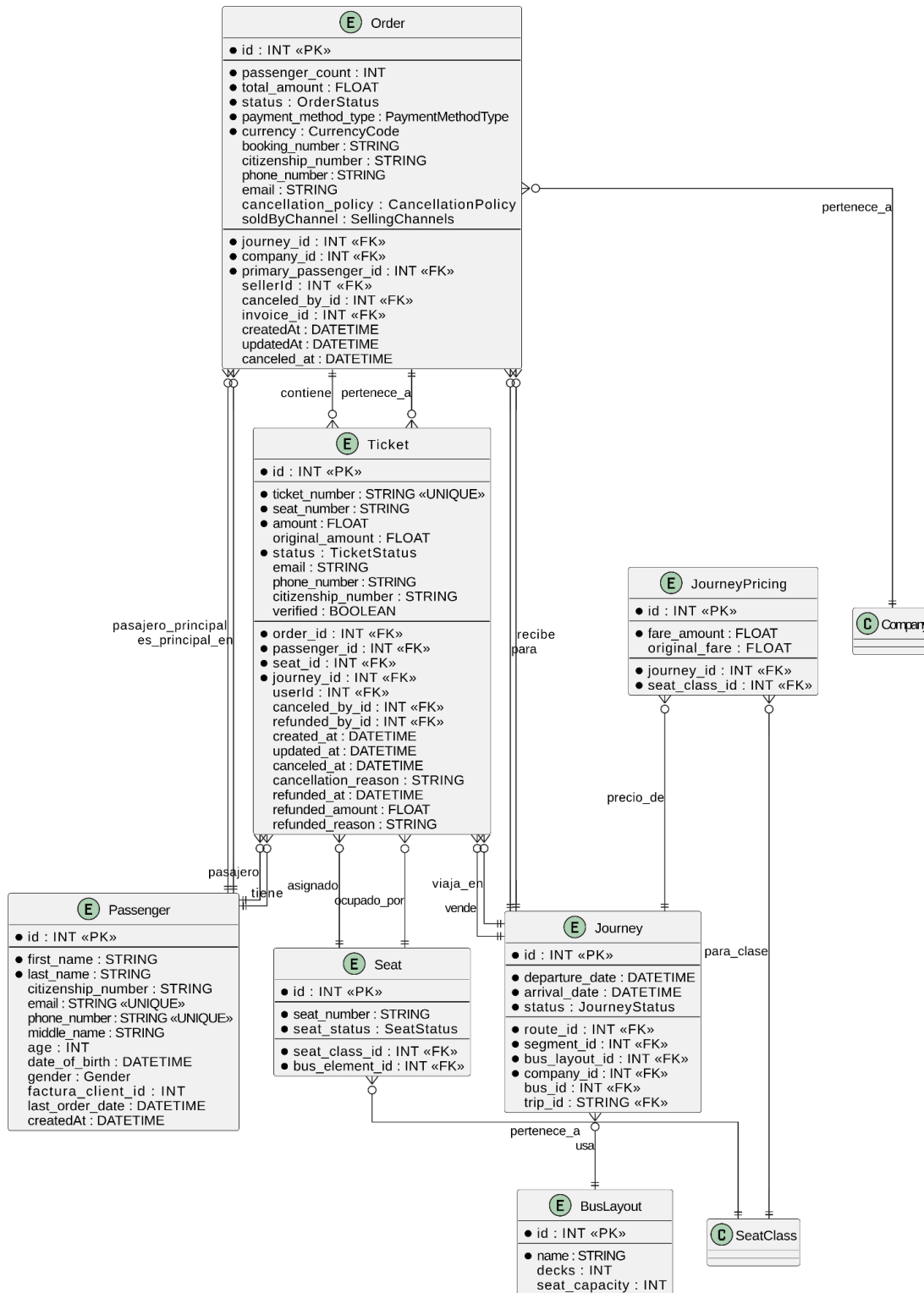
- Reutilización por teléfono o CI
- Creación automática en ventas rápidas
- Actualización posterior de datos

El conjunto de entidades introducidas y extendidas en este incremento necesita integrarse con los componentes estructurales del sistema definidos en incrementos anteriores. En particular, se establece una relación estrecha con Journey, Seat y JourneyPricing, las cuales determinan la disponibilidad real de los asientos y los precios asociados. El modelo integra::

- Precios por clase de asiento
- Capacidad dependiente de viajes superpuestos
- Restricciones fuertes por layout

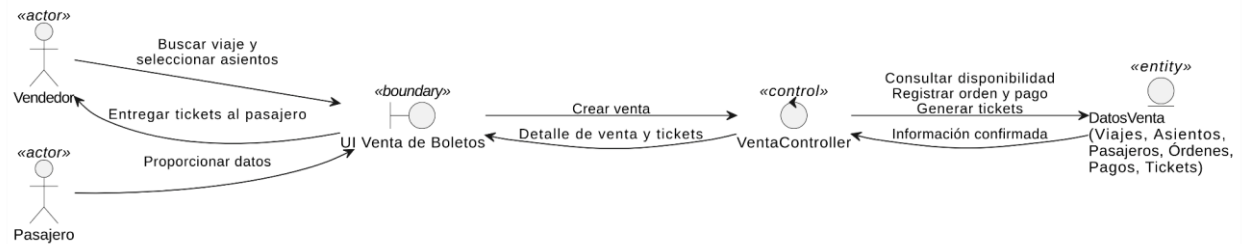
En la figura 3.37 se expone el modelo de datos físico para el segundo incremento, especificando los tipos de datos y límites de caracteres si aplica.

Ilustración 72 Modelo de datos físico para el tercer incremento



Fuente: Elaboración propia, 2025.

Ilustración 73 Diagrama de Robustez del Incremento 3



Fuente: Elaboración propia, 2025.

3.4.3.5. Desarrollo del Incremento

El flujo de venta de boletos con selección de asientos permite a los vendedores y choferes registrar la venta de uno o varios boletos asociando asientos específicos y datos completos de pasajeros. Este proceso se implementa mediante la mutación GraphQL checkout que recibe el identificador del journey, un array de tickets con información de pasajeros y asientos seleccionados, y el método de pago. El sistema valida primero que el journey esté activo y tenga un bus layout asignado, luego consulta la disponibilidad de asientos mediante el JourneyCapacityService que calcula los asientos ocupados considerando journeys superpuestos del mismo trip, y verifica que todos los asientos seleccionados estén disponibles. Una vez validada la disponibilidad, el sistema procesa los pasajeros en una transacción, reutilizando pasajeros existentes cuando se encuentra coincidencia por `phone\_number` o citizenship_number, o creando nuevos pasajeros si no existen. Finalmente, se crea la orden con todos los tickets asociados, calculando el precio total según la clase de asiento de cada ticket, y se actualiza la disponibilidad de asientos marcándolos como ocupados. La Figura 3.68 muestra la estructura completa del sistema, incluyendo la validación de disponibilidad, el procesamiento de pasajeros, y la creación transaccional de órdenes y tickets.

Ilustración 74 Checkout Resolver

```

@Resolver(() => CheckoutEntity)
export class CheckoutResolver {
  private readonly logger = new Logger(CheckoutResolver.name)

  constructor(
    private readonly checkoutService: CheckoutService,
    private readonly prisma: PrismaService,
    @Inject('PUB_SUB') private readonly pubSub: RedisPubSub
  ) {}

  @Public()
  @Mutation(() => OrderEntity, { name: 'checkout' })
  async checkout(
    @Args('checkoutInput') checkoutInput: CheckoutInput,
    @GqlUser() user: UserEntity
  ): Promise<OrderEntity> {
    return await this.checkoutService.checkout(checkoutInput, user);
  }

  @Public()
  @Mutation(() => CheckoutOutput)
  async checkoutOnline(...
  }

  /**
   * 🟡 HIGH: Additional rate limiting and input validation
   */
  private validateCheckoutInput(input: CheckoutOnlineInput): void {
  }

  @Public()
  @Query(() => PaymentStatusOutput)
  async getPaymentStatus(...
  }

  @Public()
  @Subscription(() => OrderEntity, {
  })
  paymentStatusUpdated(...
  }

```

Fuente: Elaboración propia, 2025.

Ilustración 75 Checkout Service

```

export class CheckoutService {
  async checkout(checkoutInput: CheckoutInput, user: UserEntity) {
    this.logger.debug('Creating ticket data...');
    const tickets: Prisma.TicketCreateWithoutOrderInput[] = checkoutInput.tickets.map(
      (ticket, index) => {
        const passenger = passengerTransaction[index];

        const ticketSeat = seats.find((seat) => seat.id === ticket.seat_id);
        const price =
          journey.pricing.find((pricing) => pricing.seat_class_id === ticketSeat?.seat_class_id)
            ?.fare_amount ||
          journey.pricing?.[0]?.original_fare ||
          0;

        return {
          amount: ticket.amount,
          original_amount: price,
          status: checkoutInput.isReserved ? 'PENDING_PAYMENT' : 'PENDING_SCAN',
          journey: { connect: { id: checkoutInput.journey_id } },
          seat: { connect: { id: ticket.seat_id } },
          seat_number: seats.find((seat) => seat.id === ticket.seat_id)?.seat_number || '',
          passenger: {
            connect: { id: passenger.id },
          },
        };
      }
    );
    this.logger.debug('Ticket data created successfully.');
```

```

    const totalAmount = tickets.reduce((acc, ticket) => acc + ticket.amount, 0);
  }
}

```

Fuente: Elaboración propia, 2025.

El flujo de venta rápida permite a los vendedores y choferes vender boletos con información mínima de pasajero, utilizando un identificador temporal que puede completarse posteriormente. Este proceso se implementa mediante la misma mutación GraphQL checkout pero con el parámetro `isReserved` establecido en `'true'`, lo que indica que la venta es rápida y los datos del pasajero pueden completarse más adelante. El sistema crea la orden y los tickets con estado `PENDING_PAYMENT` en lugar de `PENDING_SCAN`, permitiendo que los tickets queden marcados como pendientes de completar información. Los pasajeros se crean con datos mínimos (solo nombre y apellido si están disponibles, o un identificador temporal si no se proporciona información), y el sistema asigna un `ticket_number` único a cada ticket para su identificación posterior. La orden se crea con el método de pago especificado (efectivo o QR), y los asientos quedan ocupados inmediatamente, evitando que otros usuarios los seleccionen mientras se completa la información del pasajero. La Figura 3.70 los argumento de entrada, incluyendo la creación de tickets con estado pendiente, la asignación de identificadores temporales, y la reserva inmediata de asientos.

Ilustración 76 Argumentos de entrada venta rapida

```

@InputType()
export class CheckoutInput {
  @Field(() => Int)
  @IsNumber()
  journey_id!: number

  @Field(() => Boolean)
  @IsBoolean()
  isReserved!: boolean

  @Field(() => String)
  @IsString()
  primary_passenger_citizenship_number!: string

  @Field(() => [CheckoutTicketInput])
  @IsArray()
  @ValidateNested({ each: true })
  @Type(() => CheckoutTicketInput)
  tickets!: CheckoutTicketInput[]

  @IsString()
  @IsPhoneNumber()
  @IsOptional()
  @Field(() => String, { nullable: true })
  phone_number?: string

  @Field(() => PaymentMethodType)
  @IsEnum(PaymentMethodType)
  payment_method_type!: PaymentMethodType

  @IsString()
  @IsEmail()
  @IsOptional()
  @Field(() => String, { nullable: true })
  email?: string

  @IsNumber()
  @IsOptional()
  @Field(() => Int, { nullable: true })
  sellerId?: number
}

```

Fuente: Elaboración propia, 2025.

Ilustración 77 funcion de Creacion de orden

```

this.logger.debug('Creating order...');
const order = await this.prisma.order.create({
  data: {
    total amount: totalAmount,
    email: checkoutInput.email,
    journey: { connect: { id: checkoutInput.journey_id } },
    company: {
      connect: { id: journey.company_id },
    },
    SoldByEmployee: {
      connect: {
        id: checkoutInput.sellerId ? checkoutInput.sellerId : user.id,
      }
    },
    status: checkoutInput.isReserved ? 'PENDING' : 'CONFIRMED',
    tickets: { create: tickets },
    payment_method_type: checkoutInput.payment_method_type,
    phone_number: checkoutInput.phone_number,
    primary_passenger: { connect: { id: primaryPassengerId } },
    passenger_count: checkoutInput.tickets.length,
  },
  include: {
    tickets: {
      include: {
        passenger: true,
      },
    },
    primary_passenger: true,
  },
});
this.logger.debug('Order created successfully.');
```

Fuente: Elaboración propia, 2025.

El flujo de completar datos de pasajero de venta rápida permite actualizar la información de pasajeros previamente vendidos en modo rápido, completando o corrigiendo los datos faltantes. Este proceso se implementa mediante la mutación GraphQL `updatePassenger` que recibe el identificador del pasajero y los datos actualizados (nombre, apellido, CI, teléfono, email). El sistema busca el pasajero por su identificador, valida que exista, y actualiza sus datos en la base de datos. Adicionalmente, se puede actualizar la información del ticket asociado mediante la mutación `updateTicket` que permite modificar los campos ``email``, ``phone_number``, y `citizenship_number` del ticket sin afectar el pasajero principal. Una vez completados los datos, el sistema marca el ticket como listo para procesamiento, y si el ticket estaba en estado `PENDING_PAYMENT`, puede procederse con el pago. La Figura 3.72 ilustra el flujo de actualización de información de pasajeros y tickets, incluyendo las validaciones y la sincronización de datos entre pasajeros y tickets.

Ilustración 78 Servicio de Actualización de información de Pasajeros

```

export class TicketsService {
  async bulkUpdateTicketPassengers(
    await this.prisma.$transaction(async (tx) => {
      try {
        // First, get the current ticket with its passenger
        const currentTicket = await tx.ticket.findUnique({
          where: { id: ticketInput.ticket_id },
          include: { passenger: true }
        });

        if (!currentTicket) {
          this.logger.warn('Ticket not found: ${ticketInput.ticket_id}');
          continue;
        }

        // Check if passenger exists by phone number or citizenship number
        let existingPassenger = null;

        if (ticketInput.phone_number) {
          existingPassenger = await tx.passenger.findUnique({
            where: { phone_number: ticketInput.phone_number }
          });
        }

        if (!existingPassenger && ticketInput.citizenship_number) {
          existingPassenger = await tx.passenger.findFirst({
            where: { citizenship_number: ticketInput.citizenship_number }
          });
        }

        let passenger;
        let passengerCreated = false;

        if (existingPassenger) {
          // Update existing passenger
          passenger = await tx.passenger.update({--
        });
        passengersUpdated++;
        } else {
          // Create new passenger
          passenger = await tx.passenger.create({--
        });
        passengerCreated = true;
        passengersCreated++;
        }

        // Update the ticket to point to the passenger
        const updatedTicket = await tx.ticket.update({--
        });

        results.push({
          ticket: updatedTicket,
          passenger: passenger,
          passenger_created: passengerCreated
        });
      }
    })
  );
}

```

Fuente: Elaboración propia, 2025.

El flujo de reembolso permite devolver uno o varios boletos, liberando sus asientos y registrando la transacción para mantener trazabilidad. Se implementa con la mutación GraphQL `refundTicket`, que recibe el ID del boleto, el monto y opcionalmente la razón. El sistema valida permisos con CASL, obtiene el ticket y verifica que no haya sido reembolsado antes. Luego lo marca como REFUNDED, registra fecha, monto, razón y usuario, y actualiza el total de la orden, marcándola como REFUNDED si todos sus boletos lo están. Finalmente, se notifican los cambios de disponibilidad de asientos mediante GraphQL Subscriptions para actualizar las interfaces en tiempo real.

Ilustración 79 Servicio de Reembolso de Tickets

```

async cancelAllTickets(order_id : number, employee : UserEntity, reason? : string): Promise < TicketEntity[] > {
  const ability = this.userAbility.createForUser(employee)

  ForbiddenError.from(ability).throwUnlessCan(Action.MANAGE, Subject.TICKET)

  const tickets = await this.prisma.ticket.findMany(
    {where: {
      order_id
    }}
  );

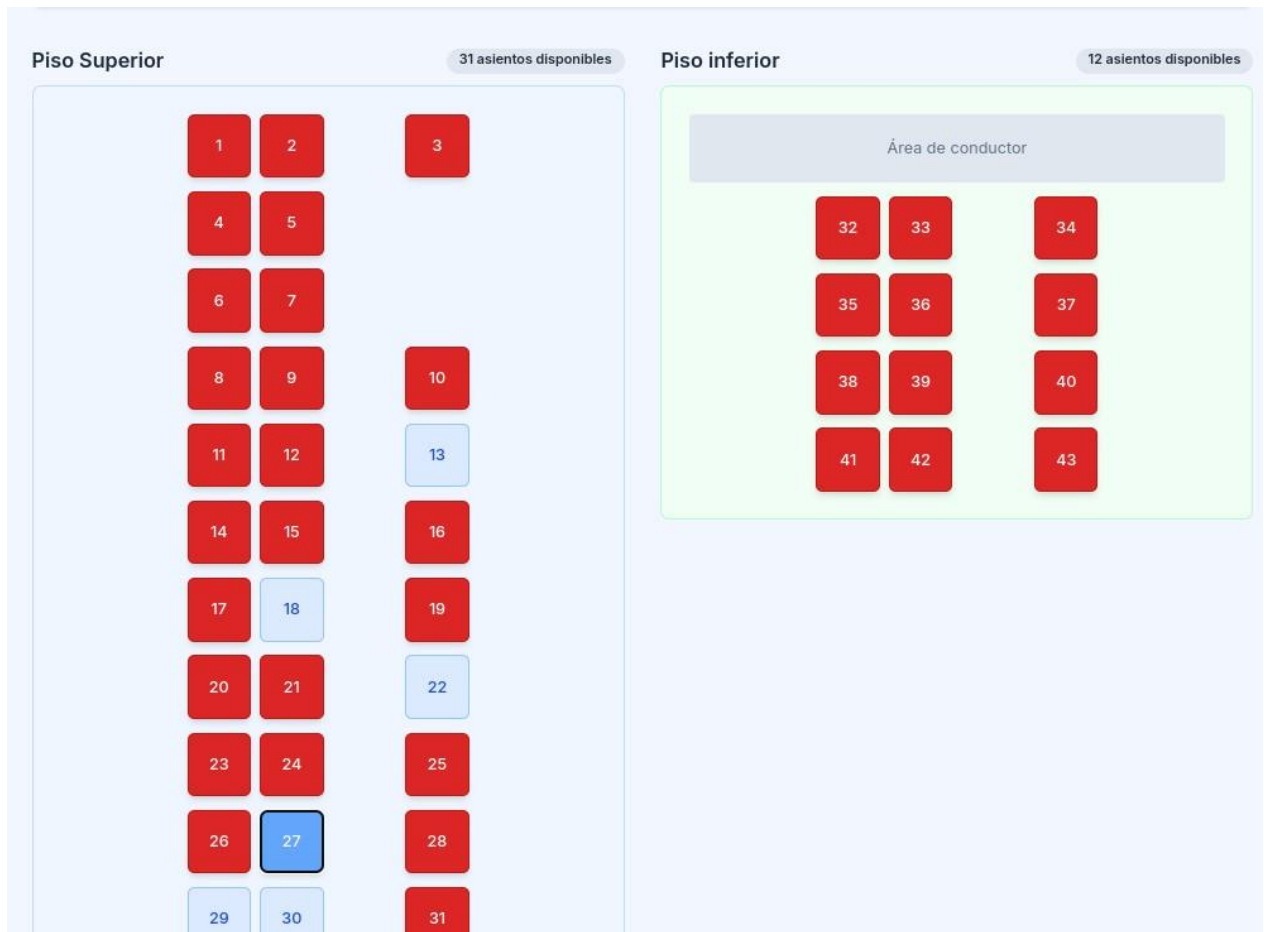
  const updatedTickets = await this.prisma.$transaction(tickets.map((ticket) => this.prisma.ticket.update({
    where: {
      id: ticket.id
    },
    data: {
      status: 'CANCELLED',
      canceled_at: DateTime.now().toJSDate(),
      cancellation_reason: reason || null,
      canceled_by: {
        connect: {
          id: employee.id
        }
      }
    }
  })));
}

```

Fuente: Elaboración propia, 2025.

La venta de boletos con selección de asientos se realiza mediante una interfaz visual donde el vendedor elige asientos del layout del bus como se muestra en la figura 3.74. Luego se presenta un formulario para ingresar los datos de cada pasajero, calculando automáticamente el total según la ruta, tal como se ve en la figura 3.75. Al confirmar, la mutación checkout valida la disponibilidad en tiempo real, crea la orden y los tickets de forma atómica, marca los asientos como ocupados y envía actualizaciones por PubSub. Finalmente, se muestra un resumen con la orden y los tickets generados, como en la figura 3.76.

Ilustración 80 Bus Layout con asiento 27 seleccionado



Fuente: Elaboración propia, 2025.

Ilustración 81 Formulacion de Información de Pasajero

Información del pasajero

 Seat 27

 **Voz**  Buscar por voz

Comandos de ejemplo:

- "Carnet 8905881"
- "Teléfono 75002909"
- "Cédula 12345678"

 Hable claro y espere unos segundos para mejores resultados

C.I. *

Enter C.I. number 

Número de teléfono

 +591  Enter phone number

Nombre * **Apellido ***

Enter first name Enter last name

Fuente: Elaboración propia, 2025.

Ilustración 82 Orden Procesada con generación de Tickets



¡Reserva exitosa!

Los Boleto han sido reservados correctamente.

Detalles de compra

Id de Venta	30974
Asientos:	27
Pago:	EFFECTIVO
Salida:	Terminal de Buses Santa Cruz
Fecha y Hora:	2025-11-19 at 10:30 - Santa Cruz de la Sierra to Quijarro
Total pagado:	Bs 120.00

 Crear factura 

Nueva Venta  Imprimir Boleto

 Compartir via WhatsApp

Fuente: Elaboración propia, 2025.

La venta rápida permite vender con datos mínimos del pasajero para acelerar la atención. Al activar el modo "Venta Rápida", el vendedor selecciona asientos igual que en la venta normal, pero usa un formulario simplificado, como se muestra en la figura 3.77. El sistema genera identificadores temporales

Ilustración 83 Modo de venta rapida activado

Detallada **Venta rápida**

⚡ Modo de venta rápida Venta rápida

Ahora puede vender asientos rápidamente. Puede añadir información de pasajeros más adelante para imprimir listas de pasajeros.

\$ Configuración de precio

Precio unitario (Bs)
 Precio original: Bs 120.00 (Maximum allowed)

\$ 120

🪑 Seleccionar Asientos:	13
Cantidad:	1
Precio por asiento:	Bs 120.00
Cantidad total:	Bs 120.00

Fuente: Elaboración propia, 2025.

Ilustración 84 Modo de venta rápida activado

Actualizar información de Pasajeros					
20	MARIA TEREZA	SEGOVIA	5888290	Bs 70.00	Completado
21	MARIA TEREZA	SEGOVIA	5888290	Bs 70.00	Completado
22				Bs 120.00	Venta rápida
23	anoela	rivera	1080574	Bs 70.00	Completado

Fuente: Elaboración propia, 2025.

El procesamiento de pago se realiza mediante una interfaz modal donde el vendedor elige el método de pago y autentica la operación con su PIN. Tras crear una orden o desde su detalle, se muestra el resumen de la venta, como en la figura 3.79. El vendedor selecciona efectivo (ingresando el monto recibido) o QR . Luego ingresa su PIN, con validación y límite de intentos, tal como se observa en la figura 3.80. Al confirmar, la mutación processOrderPayment marca la orden como pagada, registra la transacción y notifica a las demás interfaces.

Ilustración 85 Modo de venta rapida activado

 **Detalles de pago**

Precio unitario (Bs)
Precio original : Bs 120.00 (Maximum allowed) - Haga clic para editar

\$ 120

Seleccionar Asientos:	13
Cantidad:	1
Precio original por asiento:	Bs 120.00
Precio actual por asiento:	Bs 120.00

Cantidad total: **Bs 120.00**

Método de pago

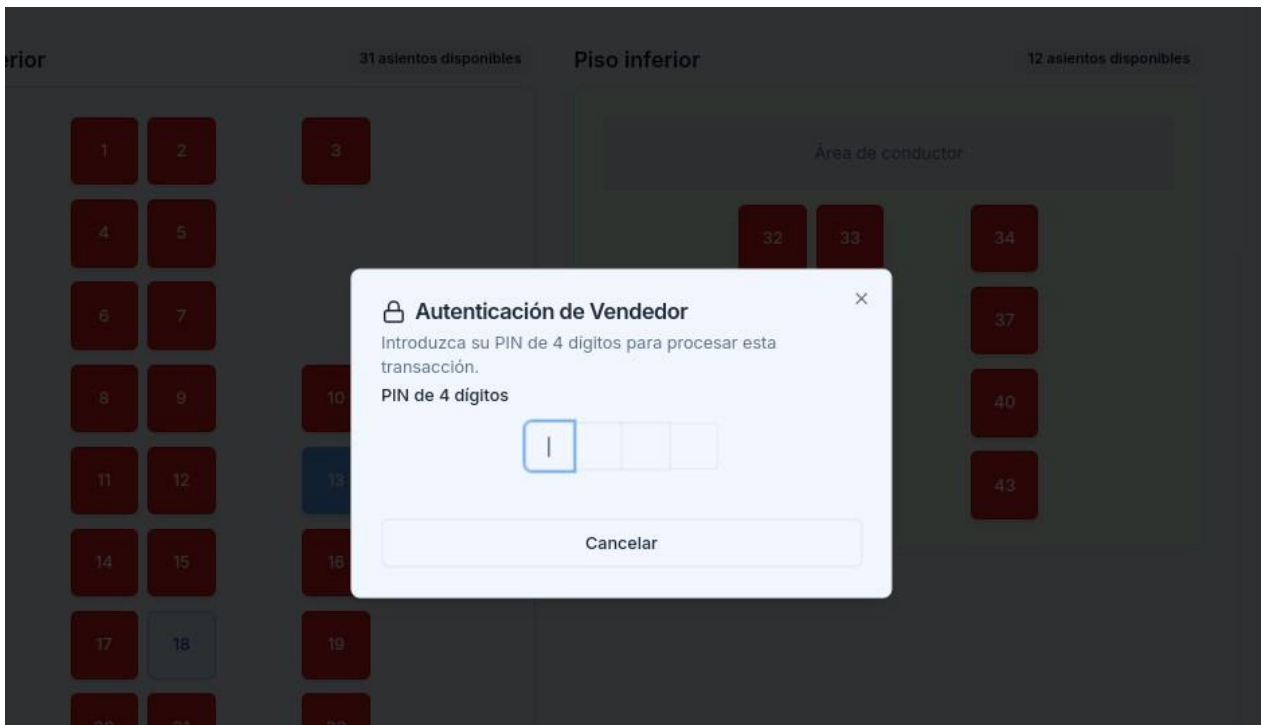
☒  **Pago en efectivo**
Pagar en efectivo en el mostrador

☐  **QR pago**
Pagar usando aplicaciones de pago móvil

 **Completar pago - Bs 120.00**

Fuente: Elaboración propia, 2025.

Ilustración 86 Modo de venta rápida activado



Fuente: Elaboración propia, 2025.

3.4.3.6. *Entrega del Incremento*

La entrega del incremento se validó mediante una demo guiada enfocada en el ciclo de vida del historial. Junto con el cliente se ejercitaron los escenarios prioritarios: creación de sesión desde la UI, renombrado mediante menú contextual, eliminación lógica, así como la carga del hilo al reabrir sesiones pasadas. Se verificó además que cada usuario solo accede a sus propias sesiones y que las eliminadas dejan de aparecer en el listado. En paralelo se revisó la consistencia con el backend.

3.4.4. *INCREMENTO 4: REPORTES Y LIQUIDACIONES*

Este incremento consolida las capacidades analíticas y de trazabilidad del sistema, permitiendo obtener información estructurada sobre ventas, montos, desempeño por terminal y el historial completo de eventos asociados a cada asiento. Constituye la capa final del sistema, enfocada en análisis, control operativo y soporte a decisiones tanto para dueños como para líderes de línea y personal operativo. El incremento incorpora reportes agregados por viaje, por terminal y

por periodo; visualizaciones consolidadas; así como historiales detallados de cada asiento, integrando eventos de venta, cancelación, reembolso y reasignación. Este conjunto de funcionalidades cierra el ciclo operativo, ofreciendo visibilidad total sobre la operación.

3.4.4.1. Alcance del incremento

El objetivo de este incremento es habilitar la visualización integral del estado financiero y operativo de la empresa mediante reportes consolidados y trazabilidad completa de asientos. El alcance incluye el reporte de ventas por viaje, vendedor y chofer (RF-07), los montos consolidados por terminal en períodos seleccionados (RF-08) y el historial detallado de cada asiento, con todos sus eventos y la información asociada a pasajeros, operadores y montos (RF-13). Para ello se desarrollarán consultas especializadas, agregaciones, filtros por periodo, endpoints optimizados y vistas de interfaz dirigidas a dueños, líderes y operadores autorizados. Quedan fuera de alcance las integraciones externas con sistemas de contabilidad o ERP, los dashboards de BI avanzados y cualquier tipo de predicciones o modelos analíticos.

3.4.4.2. Plan de trabajo y estimaciones

A continuación se presenta el plan de trabajo de este incremento, detallando cada tarea con su estimación de horas.

Cuadro 3.16. Plan de trabajo del cuarto incremento

#	Tarea	Descripción breve	Estimación (h)
1	Reportes por viaje	Totales por viaje, vendedor, chofer; agrupación y métricas básicas	10
2	Consolidados por terminal	Agregaciones por terminal diario/semanal/mensual;	8
3	Historial de asiento	Query por asiento; eventos (venta, cancelación, reembolso)	12
4	UI de reportes	Visualización de tablas y totales	14
5	UI de historial de asiento	Detalles completos de cada evento, operador, monto y timestamp	12

#	Tarea	Descripción breve	Estimación (h)
6	UI venta online web	permitir a los pasajeros la compra por internet	35

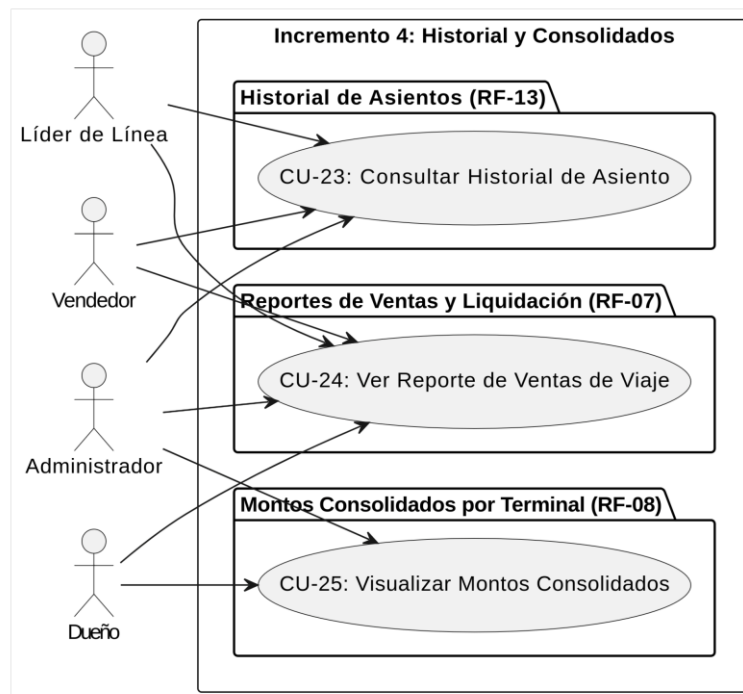
Fuente: Elaboración propia, 2025.

El esfuerzo total estimado para este incremento es 91 horas. Incluye la captura de feedback extremo a extremo, su agregación en señales útiles para el ranking, el entrenamiento de pesos de señales de ranking.

3.4.4.3. *Análisis funcional detallado*

Esta sección expone el análisis funcional del incremento, analizando el caso de uso de este incremento para especificar con claridad las precondiciones, las postcondiciones y el flujo principal. La figura 3.81 destaca el caso de uso incorporado en este incremento.

Ilustración 87 Casos de uso cubiertos en el cuarto incremento



Fuente: Elaboración propia, 2025.

A continuación, se detallan los casos de uso con sus propósitos, actores asociados, precondiciones, postcondiciones y flujo principal, además se incluye entre paréntesis su requerimiento asociado para facilitar la trazabilidad.

CU-23	Consultar Historial de Asiento (RF-13)	
Propósito	Permitir visualizar el historial completo asociado a un asiento específico ventas y reembolsos	
Actores	Vendedor, Chofer, Líder de Línea, Administrador	
Precondiciones	Asiento asociado a una ruta previamente activada	
Postcondiciones	Se muestra el historial completo del asiento.	
Flujo principal	(1) Seleccionar asiento (2) Visualizar eventos asociados	

CU-24	Ver Reporte de Ventas de Viaje (RF-07)	
Propósito	Generar un reporte consolidado de ventas por viaje, incluyendo totales por vendedor, chofer y terminal.	
Actores	Vendedor, Chofer, Líder de Línea, Administrador	
Precondiciones	Ventas registradas	
Postcondiciones	Reporte generado con totales	
Flujo principal	(1) Seleccionar viaje (2) Recuperar ventas asociadas	

CU-25	Visualizar Montos Consolidados por Terminal (RF-08)	
Propósito	Permitir al dueño o administrador ver los montos consolidados diarios, semanales o mensuales por terminal.	
Actores	Dueño, Administrador	
Precondiciones	Ventas históricas registradas.	
Postcondiciones	Consolidados mostrados por período y terminal.	
Flujo principal	(1) Seleccionar período (día, semana, mes). (2) Recuperar registros de ventas correspondientes. (3) Mostrar tabla comparativa	

CU-26	Compra de Boletos Web por Pasajero con Pago QR	
Propósito	Permitir que el pasajero realice la compra de boletos a través de la plataforma web	
Actores	pasajero	
Precondiciones	Existen rutas activas y asientos disponibles.	

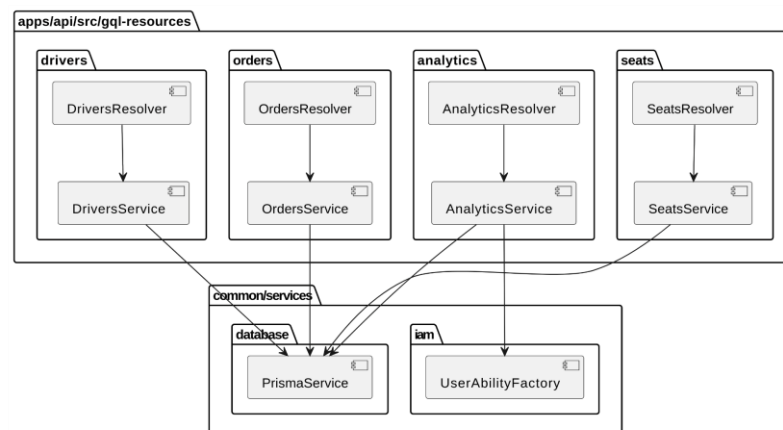
Postcondiciones	La transacción queda registrada en el sistema.
Flujo principal	(1) El pasajero ingresa al portal web de compra. (2) Selecciona el origen y el destino desde la lista de rutas disponibles. (3) Selecciona la fecha y horario del viaje. (4) El sistema calcula el monto total y muestra el resumen de compra. (5) El pasajero confirma y selecciona “Pagar por QR”. (6) El sistema genera y muestra el código QR de pago. (7) El pasajero realiza el pago escaneando el QR. (8) El sistema de pagos confirma la transacción. (9) El sistema registra la venta y genera los boletos electrónicos.

3.4.4.4. Diseño de solución

a. Aplicaciones

El Incremento 4 expande las capacidades del sistema hacia funciones de analítica operacional, control de ventas y trazabilidad financiera, integrando módulos de reporting orientados a vendedores, administradores y conductores. Esta fase se apoya en los modelos transaccionales creados en los incrementos previos (Orders, Tickets, Journey, Passenger), añadiendo servicios especializados para la consulta histórica, la generación de reportes consolidados y el análisis financiero por terminal.

Ilustración 88 Diagrama de paquetes del cuarto incremento.



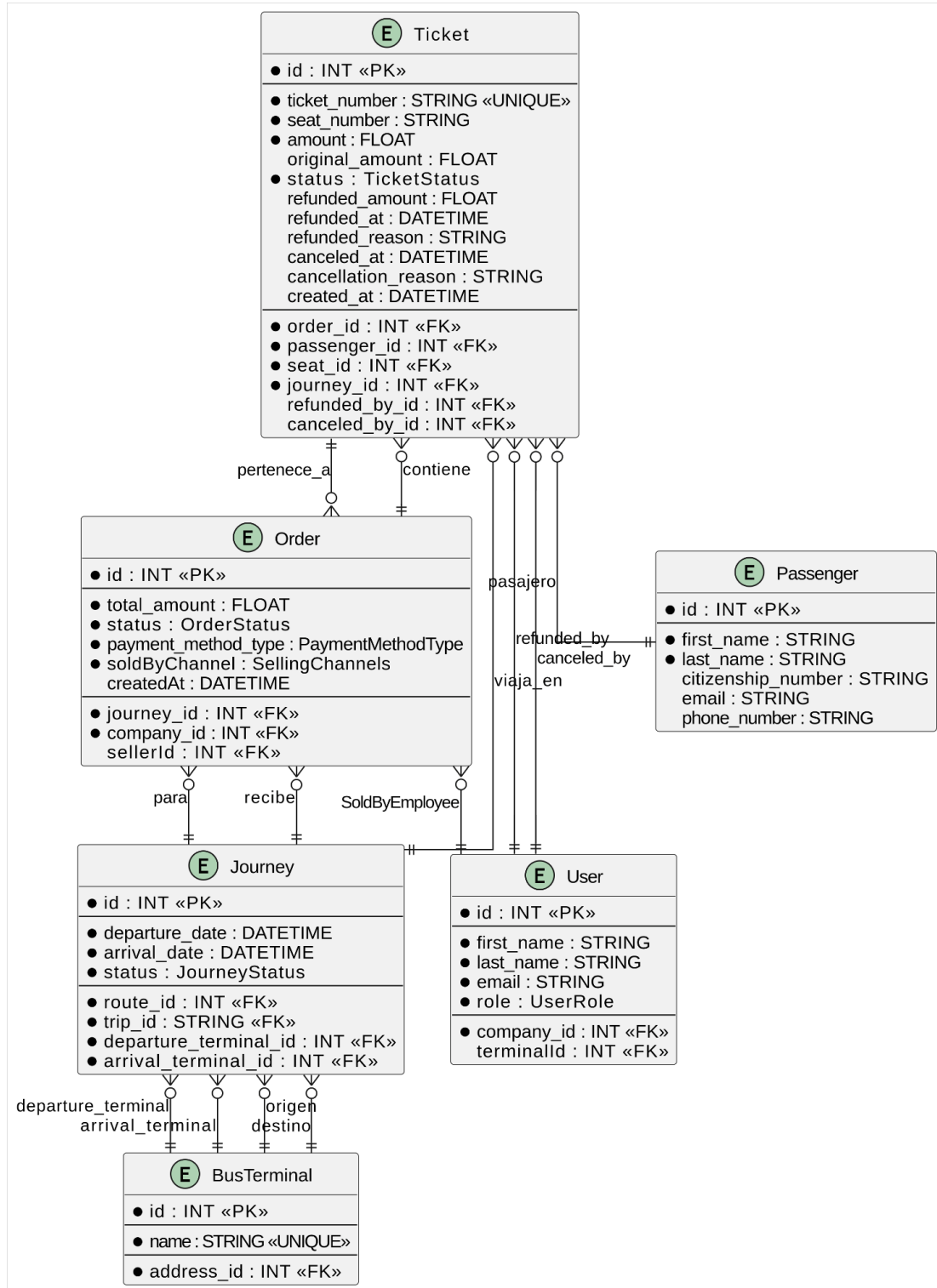
Fuente: Elaboración propia, 2025.

b. Modelo de datos y ORM

El Incremento 3 introduce un conjunto de ampliaciones significativas en el modelo de datos, orientadas a soportar de forma completa el flujo de venta de boletos, la gestión de pasajeros, la creación de órdenes y la validación de pagos. Estas ampliaciones permiten dotar al sistema de una estructura sólida que respalda operaciones transaccionales críticas como la selección de asientos, la generación de tickets, la confirmación de pagos y el registro de reembolsos.

En la figura 3.56 se expone el modelo de datos físico para el cuarto incremento, especificando los tipos de datos y límites de caracteres si aplica. Se incluye además las tablas de los anteriores incrementos.

Ilustración 89 Modelo de datos físico del cuarto incremento

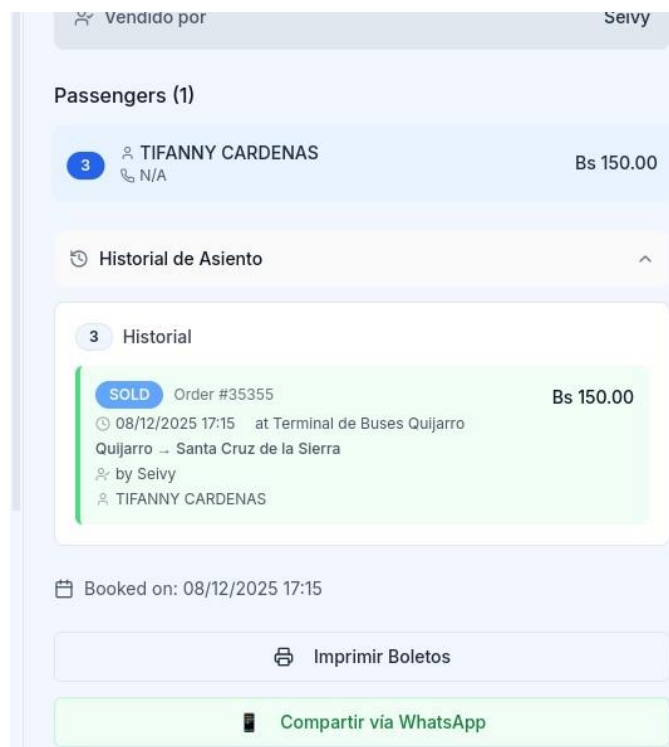


Fuente: Elaboración propia, 2025.

3.4.4.5. Desarrollo del incremento

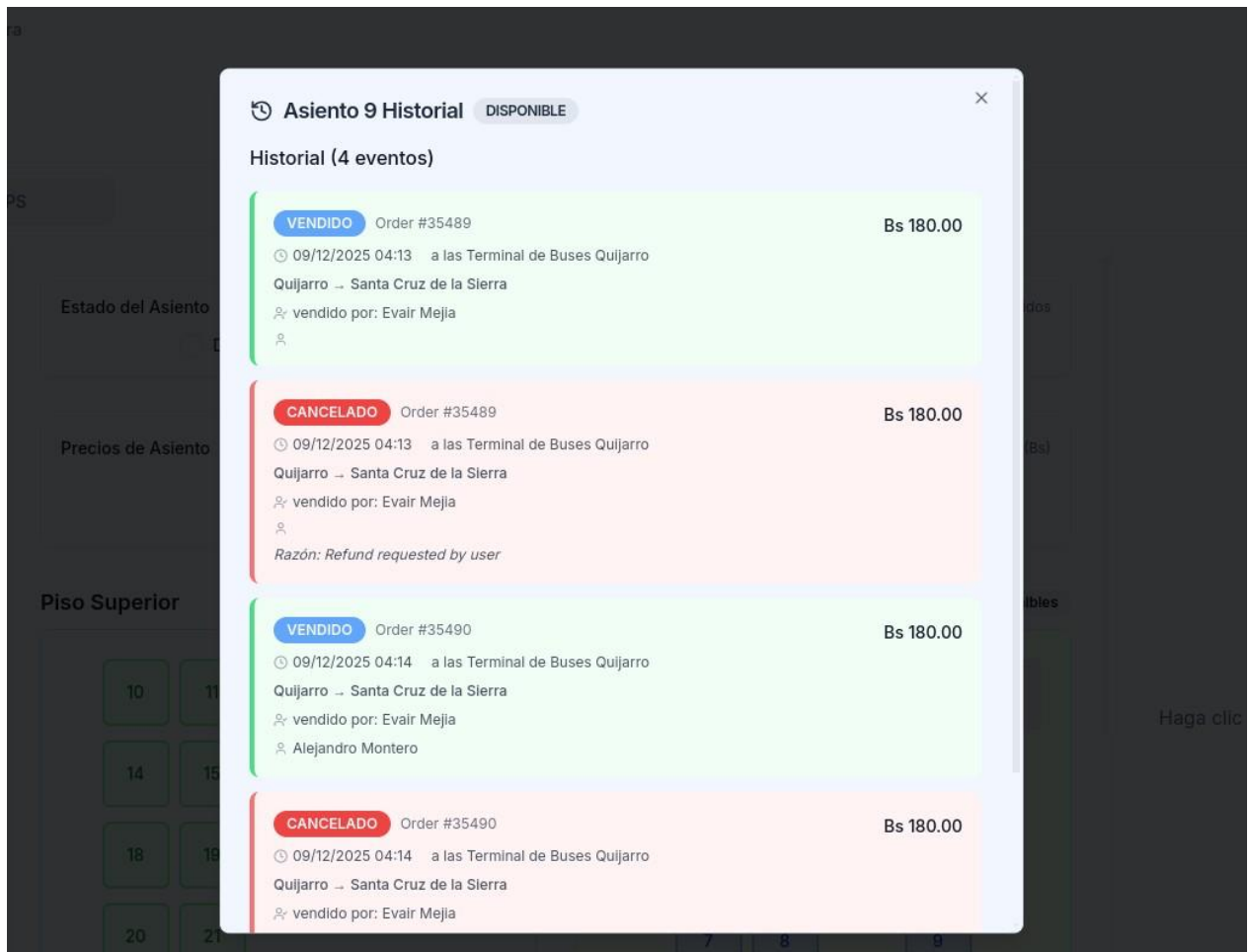
El sistema permite consultar el historial completo de un asiento específico desde la vista de detalles de orden (Order Details). Cuando el vendedor, chofer o administrador visualiza una orden activa, puede acceder al historial del asiento para revisar todos los eventos asociados a ese asiento en particular, incluyendo ventas previas, reembolsos, cambios de asiento y cualquier otra transacción histórica. Este flujo se implementa mediante un modal que se activa al hacer clic en el botón de historial junto al asiento seleccionado, el cual consulta el servicio SeatHistoryService que recupera todos los registros de la tabla seat_history asociados al seat_number y journey_id correspondiente. La Figura 3.84 muestra la interfaz de Order Details con el botón de acceso al historial, mientras que la Figura 3.85 presenta el modal de historial desplegado con la lista cronológica de eventos del asiento.

Ilustración 90 Historial en detalle de Venta



Fuente: Elaboración propia, 2025.

Ilustración 91 Modal de historial desplegado con la lista cronológica



Fuente: Elaboración propia, 2025.

Adicionalmente, el sistema ofrece un acceso rápido al historial mediante clic derecho sobre cualquier asiento en la visualización del layout del bus. Esta funcionalidad de menú contextual permite a los usuarios consultar el historial sin necesidad de navegar primero a los detalles de la orden, agilizando significativamente el flujo de trabajo cuando se requiere verificar el estado histórico de múltiples asientos. Al hacer clic derecho sobre un asiento, se despliega un menú contextual con opciones que incluyen "Ver Historial", "Cambiar Asiento" y "Reembolsar", según los permisos del usuario y el estado actual del asiento. La opción "Ver Historial" abre el mismo modal de SeatHistoryModal utilizado en Order Details, manteniendo consistencia en la experiencia de usuario. La Figura 3.86 ilustra el menú contextual desplegado sobre un asiento seleccionado en el layout del bus.

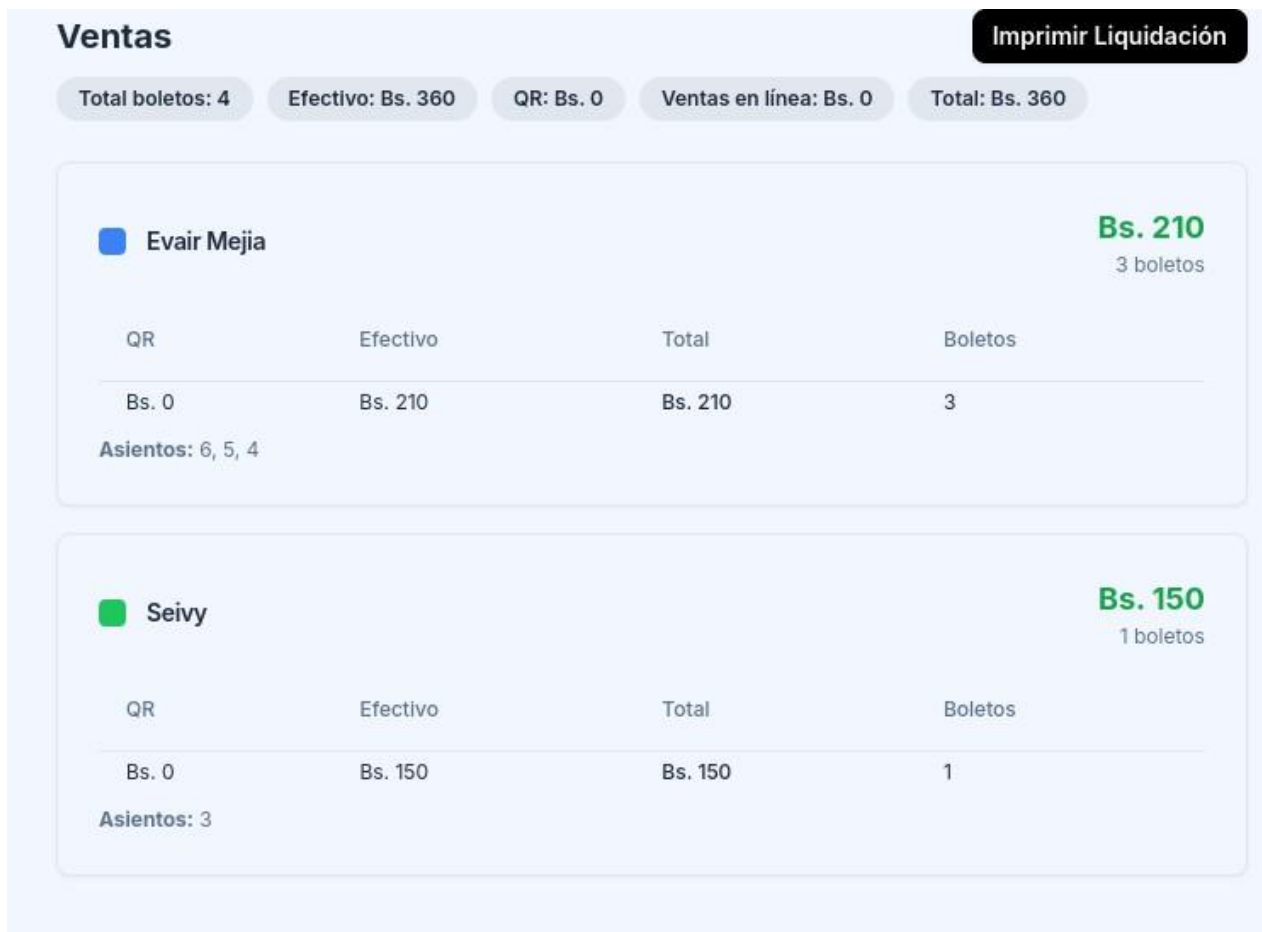
Ilustración 92 Argumentos de entrada venta rapida



Fuente: Elaboración propia, 2025.

El módulo de liquidación proporciona reportes consolidados de ventas por viaje, permitiendo a vendedores, choferes, líderes de línea y administradores visualizar el desglose completo de ingresos asociados a cada journey. Este reporte se implementa mediante el componente LiquidationSummaryCards que consulta la query GraphQL `getJourneyLiquidation`, la cual recupera todas las órdenes asociadas al journey seleccionado y calcula los totales agrupados por vendedor, chofer y terminal de origen. El sistema presenta tarjetas (cards) con métricas clave como total de boletos vendidos, ingresos totales, promedio por boleto, y desglose por método de pago (efectivo vs QR). Adicionalmente, se incluyen tablas detalladas implementadas en `LiquidationSellersTable` y `LiquidationTerminalsTable` que muestran el rendimiento individual de cada vendedor y los montos consolidados por terminal. La Figura 3.87 presenta las tablas detalladas de liquidación por vendedorl.

Ilustración 93 Tarjetas de resumen



Fuente: Elaboración propia, 2025.

El sistema de backoffice incorpora un módulo integral de reportes de facturación que permite a los administradores y propietarios consultar y analizar el historial completo de viajes facturados organizados por período mensual. Esta funcionalidad se accede desde la sección "Reportes" del menú lateral, específicamente en la opción "Reporte de Facturación", la cual está restringida exclusivamente a usuarios con rol OWNER por razones de seguridad y control financiero. El módulo implementa un sistema de filtrado avanzado que permite seleccionar el mes y año deseado mediante selectores desplegables, así como aplicar filtros adicionales por nombre de bus para segmentar la información según las necesidades del análisis. La consulta se realiza mediante el query GraphQL `getTripsByMonth`, el cual recupera únicamente las órdenes que poseen un `invoice_id` asociado dentro del rango de fechas especificado, filtrando

automáticamente por la compañía del usuario autenticado para garantizar la segregación de datos entre empresas.

El reporte presenta una vista tabular detallada que incluye información crítica de cada viaje facturado: identificador del viaje, fecha de salida, nombre de la ruta, bus asignado, empresa operadora, cantidad de tickets vendidos y monto total facturado. Adicionalmente, el sistema calcula y muestra estadísticas agregadas en tiempo real, incluyendo el número total de viajes facturados, la suma total de tickets emitidos y el monto acumulado de facturación para el período seleccionado, ajustándose dinámicamente cuando se aplican filtros. El módulo ofrece capacidades de exportación robustas mediante dos formatos: PDF, generado utilizando las librerías jsPDF y jspdf-autotable con formato profesional que incluye encabezados, resúmenes estadísticos y tablas estructuradas; y Excel, implementado con la librería xlsx que genera hojas de cálculo con datos tabulares completos y secciones de resumen para facilitar análisis posteriores. La Figura 3.88 muestra la interfaz principal del reporte de facturación con los controles de filtrado.

Ilustración 94 Reporte Mensual de Facturación

Home > Reportes > Facturación

Reporte Mensual de Viajes

diciembre 2025

Periodo
Diciembre 2025

Total Viajes
32

Total Tickets
91

Monto Facturado
Bs. 8.420

Filtros y Exportación

Excel PDF

Mes
Diciembre

Año
2025

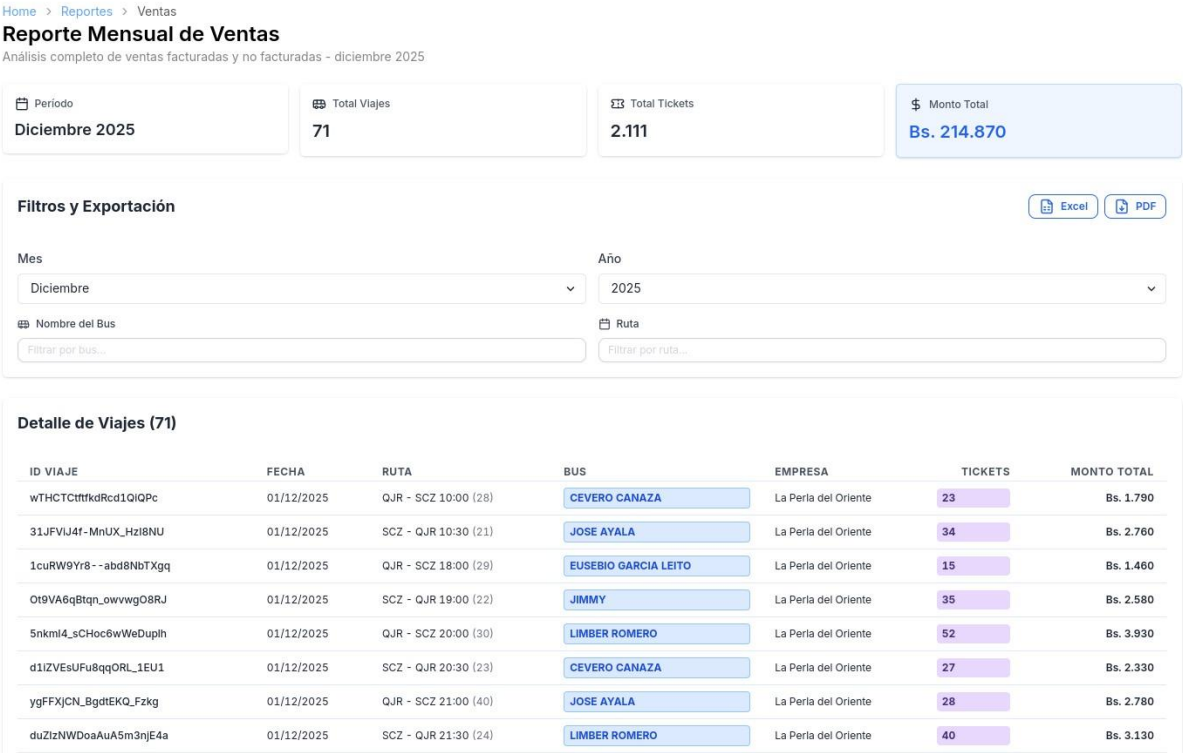
Nombre del Bus

ID VIAJE	FECHA	RUTA	BUS	EMPRESA	TICKETS	MONTO FACTURADO
1cuRW9Yr8 - abd8NbTXgq	01/12/2025	QJR - SCZ 18:00 (29)	EUSEBIO GARCIA LEITO	La Perla del Oriente	2	Bs. 160
0t9VA6qBtqn_owvvg08RJ	01/12/2025	SCZ - QJR 19:00 (22)	JIMMY	La Perla del Oriente	2	Bs. 160
d1IZVEsUFu8qqORL_1EU1	01/12/2025	SCZ - QJR 20:30 (23)	CEVERO CANAZA	La Perla del Oriente	1	Bs. 80
du2izNWDoaAuA5m3njE4a	01/12/2025	SCZ - QJR 21:30 (24)	LIMBER ROMERO	La Perla del Oriente	3	Bs. 240
USKn8pQlnmuaKExgthVIV	02/12/2025	QJR - SCZ 10:00 (28)	CEVERO CANAZA	La Perla del Oriente	4	Bs. 240
a5hdDeuT1Kym1uikvQJNJ	02/12/2025	SCZ - QJR 10:30 (21)	JOSE AYALA	La Perla del Oriente	2	Bs. 140
uTZ66fyG3zxGKHnyJVK9h	02/12/2025	QJR - SCZ 18:00 (29)	EUSEBIO GARCIA LEITO	La Perla del Oriente	1	Bs. 80
JqNq4*1g7IBxHQPh0You	02/12/2025	SCZ - QJR 20:30 (23)	CEVERO CANAZA	La Perla del Oriente	2	Bs. 160

Fuente: Elaboración propia, 2025.

El módulo de reportes de ventas mensuales proporciona una visión comprehensiva de todas las transacciones comerciales realizadas en un período específico, independientemente de su estado de facturación, permitiendo a los propietarios analizar el desempeño comercial global de la operación. A diferencia del reporte de facturación que se enfoca exclusivamente en órdenes facturadas, este módulo consulta todas las ventas confirmadas mediante el query GraphQL `getAllTripsByMonth`, el cual recupera tanto órdenes facturadas como no facturadas, proporcionando métricas diferenciadas para cada categoría. El sistema implementa filtros combinados por mes, año, nombre de bus y ruta, permitiendo análisis granulares del rendimiento comercial por diferentes dimensiones operativas. La interfaz presenta estadísticas agregadas que incluyen el total de viajes realizados, cantidad de tickets vendidos, monto total de ventas, desglose de montos facturados versus no facturados, y conteo de órdenes en cada categoría, calculados dinámicamente mediante hooks de React que procesan los datos en el cliente para optimizar la experiencia de usuario. El módulo ofrece las mismas capacidades de exportación que el reporte de facturación (PDF y Excel), pero con columnas adicionales que reflejan la naturaleza más comprehensiva de los datos, incluyendo diferenciación visual entre ventas facturadas y no facturadas mediante códigos de color y badges informativos. La Figura 3.89 ilustra la interfaz del reporte de ventas con los filtros aplicados y las métricas calculadas en tiempo real.

Ilustración 95 Reporte Mensual de Ventas

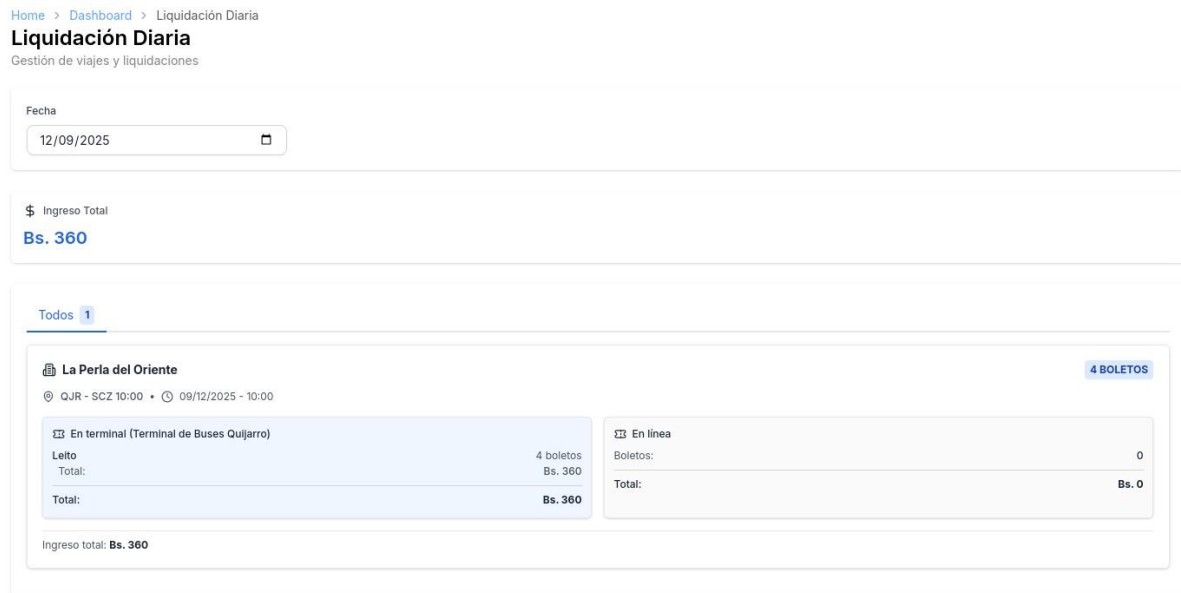


Fuente: Elaboración propia, 2025.

La interfaz de liquidación presenta tarjetas (cards) individuales para cada viaje con ventas registradas, mostrando información operativa clave como empresa operadora, ruta, horarios, terminal de salida, y un desglose financiero detallado que incluye: ventas en terminal organizadas por clase de asiento con totales por categoría, ventas en línea con listado de asientos vendidos y pasajeros asociados, cálculo automático del monto a liquidar basado en el precio acordado (agreedPrice) multiplicado por la cantidad de boletos en línea, y el ingreso total del viaje sumando todas las fuentes. Para usuarios con rol OWNER o ADMIN, el sistema habilita un botón de acción "Liquidar" en cada tarjeta de viaje pendiente, el cual al ser activado registra la liquidación en la base de datos actualizando el campo settlementStatus a "liquidated" y almacenando la fecha y hora de la transacción, generando además un registro auditable del usuario que ejecutó la acción. El módulo incluye estadísticas agregadas en la parte superior que muestran el ingreso total del día, el monto pendiente de liquidación, y la cantidad de viajes que requieren atención inmediata, proporcionando indicadores visuales mediante badges de color

que alertan sobre viajes con liquidaciones pendientes que superan umbrales temporales establecidos. La Figura 3.90 muestra la vista general de liquidación con las métricas de liquidación por ruta del día.

Ilustración 96 Modo de venta rapida activado

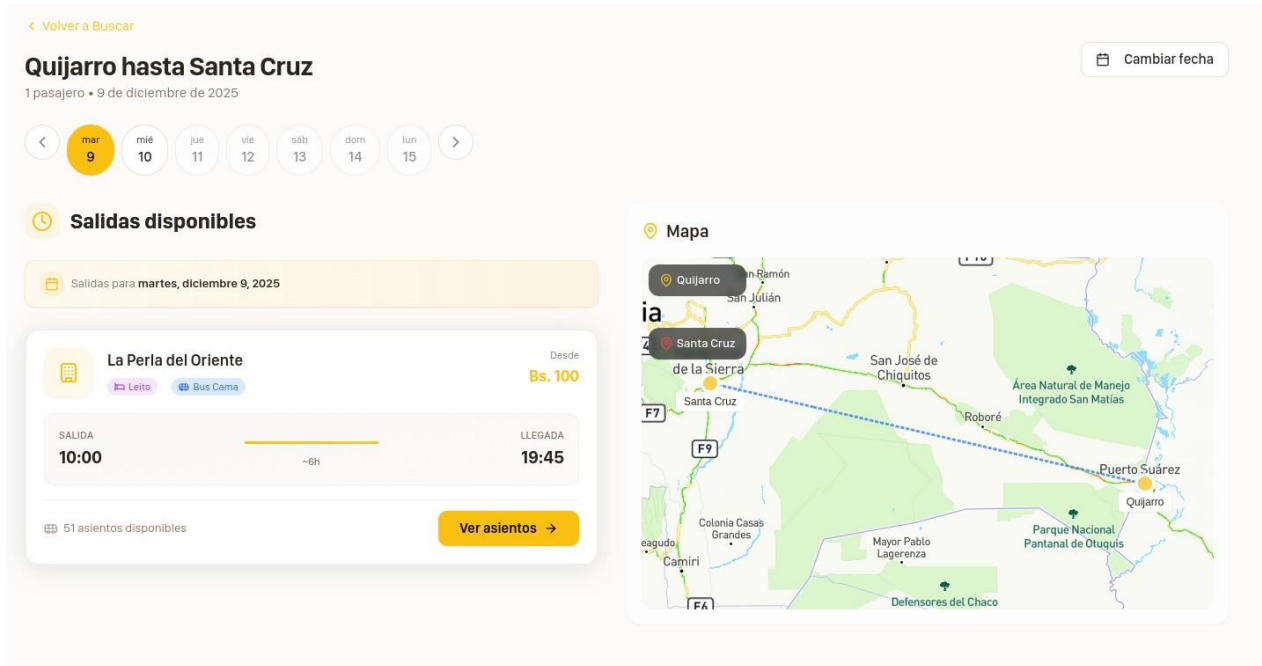


Fuente: Elaboración propia, 2025.

El sistema de selección de ruta permite a los usuarios buscar viajes disponibles mediante un formulario que solicita origen, destino, fecha de viaje y número de pasajeros, ejecutando una consulta GraphQL que recupera los viajes disponibles con precios dinámicos y opciones de descuento. La interfaz presenta una lista de salidas organizadas por horario, mostrando para cada viaje el precio base, precio original (si aplica descuento), porcentaje de descuento, tipo de clase de asiento, duración estimada del trayecto y disponibilidad de asientos, permitiendo al usuario filtrar resultados para mostrar únicamente viajes con promociones activas. Al seleccionar una salida, el sistema valida la disponibilidad, calcula el precio final según la cantidad de pasajeros y la clase seleccionada, y redirige automáticamente a la pantalla de selección de asientos pasando todos los parámetros del viaje seleccionado mediante URLSearchParams, generando además notificaciones toast que confirman la selección y proporcionan retroalimentación visual sobre el estado de la búsqueda. La Figura 3.91 muestra

la interfaz de resultados de búsqueda con la lista de salidas disponibles y el mapa interactivo de la ruta seleccionada.

Ilustración 97 Salidas disponible portal web

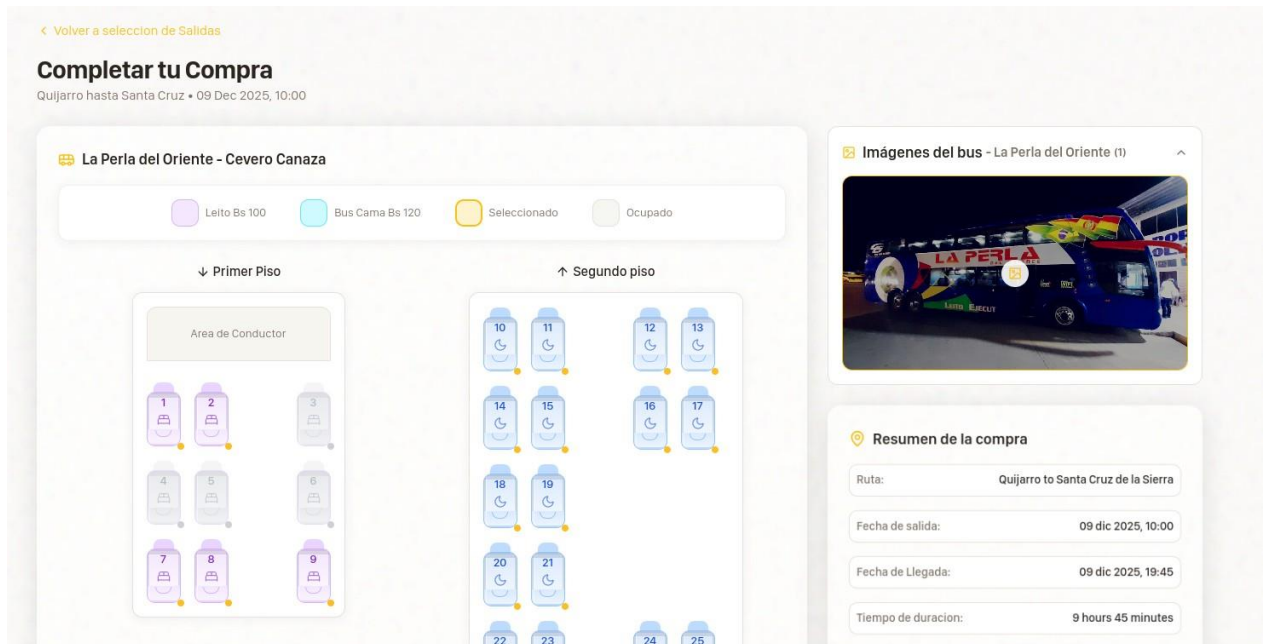


Fuente: Elaboración propia, 2025.

El módulo de selección de asientos presenta un layout interactivo del autobús que muestra la distribución de asientos en múltiples pisos, utilizando un sistema de colores para indicar el estado de cada asiento: disponibles (verde), ocupados (gris), y seleccionados (azul), permitiendo a los usuarios hacer clic directamente sobre los asientos para seleccionarlos o deseccionarlos. El sistema valida en tiempo real la disponibilidad de asientos mediante consultas GraphQL que recuperan el estado actualizado de cada viaje, previniendo conflictos de reserva y actualizando automáticamente la disponibilidad cuando se detectan cambios, mientras que un resumen de reserva flotante muestra el total de asientos seleccionados, el precio por asiento según su clase (Ejecutivo, Premium, etc.), y el monto total a pagar. Una vez seleccionados los asientos, el sistema solicita los datos de cada pasajero mediante formularios dinámicos que se generan según la cantidad de asientos elegidos, validando campos obligatorios como nombre completo, número de documento de identidad, y número de teléfono, con la opción de guardar pasajeros frecuentes para agilizar reservas futuras. La Figura 3.92

muestra el layout interactivo del autobús con los asientos seleccionados resaltados y el formulario de detalles de pasajeros desplegado en el panel lateral.

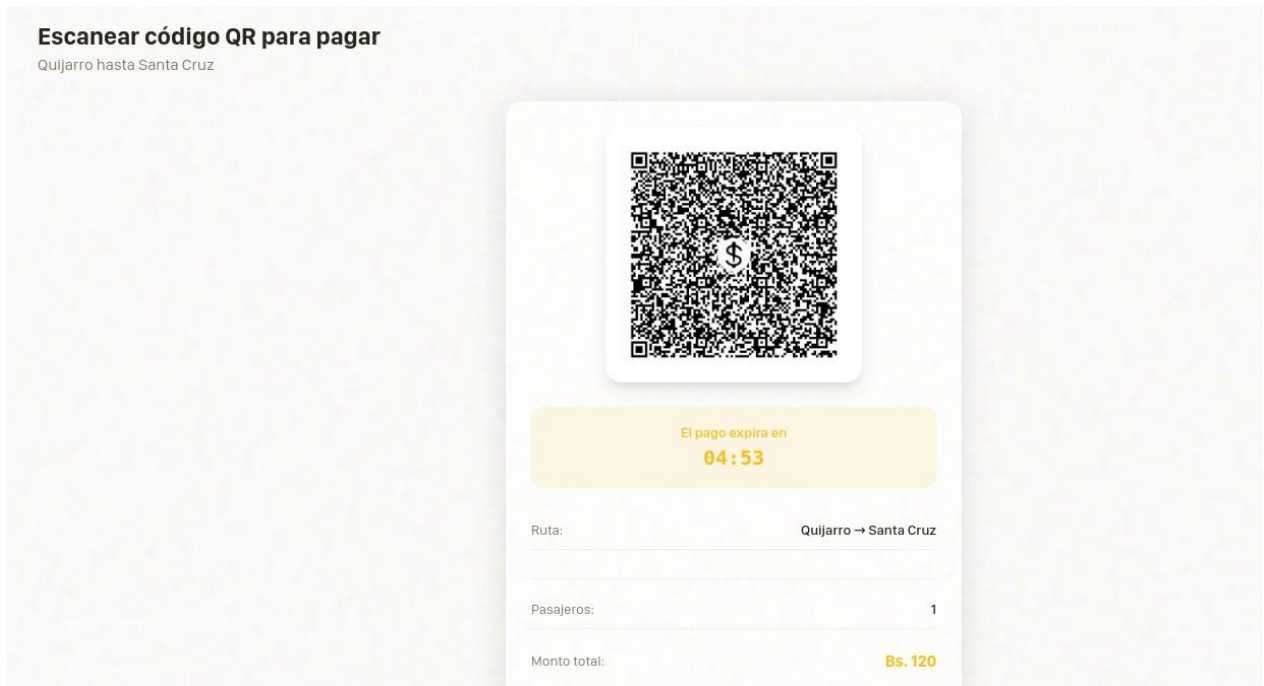
Ilustración 98 Selección de Asientos portal web



Fuente: Elaboración propia, 2025.

El módulo de selección de asientos presenta un layout interactivo del autobús que muestra la distribución de asientos en múltiples pisos, utilizando un sistema de colores para indicar el estado de cada asiento: disponibles (verde), ocupados (gris), y seleccionados (azul), permitiendo a los usuarios hacer clic directamente sobre los asientos para seleccionarlos o deseccionarlos. El sistema valida en tiempo real la disponibilidad de asientos mediante consultas GraphQL que recuperan el estado actualizado de cada viaje, previniendo conflictos de reserva y actualizando automáticamente la disponibilidad cuando se detectan cambios, mientras que un resumen de reserva flotante muestra el total de asientos seleccionados, el precio por asiento según su clase (Ejecutivo, Premium, etc.), y el monto total a pagar. Una vez seleccionados los asientos, el sistema solicita los datos de cada pasajero mediante formularios dinámicos que se generan según la cantidad de asientos elegidos, validando campos obligatorios como nombre completo, número de documento de identidad, y número de teléfono, con la opción de guardar pasajeros frecuentes para agilizar reservas futuras. La Figura 3.92 muestra el layout interactivo del autobús con los asientos seleccionados resaltados y el formulario de detalles de pasajeros desplegado en el panel lateral.

Ilustración 99 Pago mediante QR portal web



Fuente: Elaboración propia, 2025.

3.5. EVALUACIÓN DEL SISTEMA

3.5.1. PRUEBAS FUNCIONALES

Las pruebas funcionales se planificaron y ejecutaron de manera iterativa durante los cuatro incrementos de desarrollo, abarcando tanto la API como la interfaz de usuario. Para la API se empleó una colección Postman parametrizada (entorno, credenciales, rutas críticas), mientras que la UI fue validada mediante escenarios de usuario alineados a los flujos priorizados de cada incremento. Esta estrategia permitió asegurar el cumplimiento integral de los casos de uso (CU) y la consistencia funcional antes del paso a pruebas técnicas.

3.5.1.1. Cobertura de casos de uso por incremento

La cobertura funcional se garantizó iterativamente, alineando para cada incremento los casos de uso activos con los entregables validados:

- I1 – Configuración inicial y gestión de rutas: Se validó la creación y gestión de rutas, horarios con reglas RRULE, construcción de segmentos con paradas, asignación de layouts de bus, configuración de precios base y administración de elementos y asientos. También se verificaron validaciones de fecha, zonas horarias e integridad referencial entre rutas, terminales, segmentos y buses.
- I2 – Activación operativa y disponibilidad en tiempo real: Se ejercitaron los CU de activación de rutas, generación automática de journeys, asignación de buses por terminal y turno, y consulta de disponibilidad en tiempo real considerando journeys superpuestos
- I3 – Venta de boletos y procesamiento de pagos: Se validó la selección múltiple de asientos, creación de órdenes y tickets (incluyendo venta rápida),
- I4 – Retroalimentación y mejora continua: Se probó la generación de reportes de ventas por viaje, consolidado por terminal, y visualización del historial completo de asientos con todos los eventos

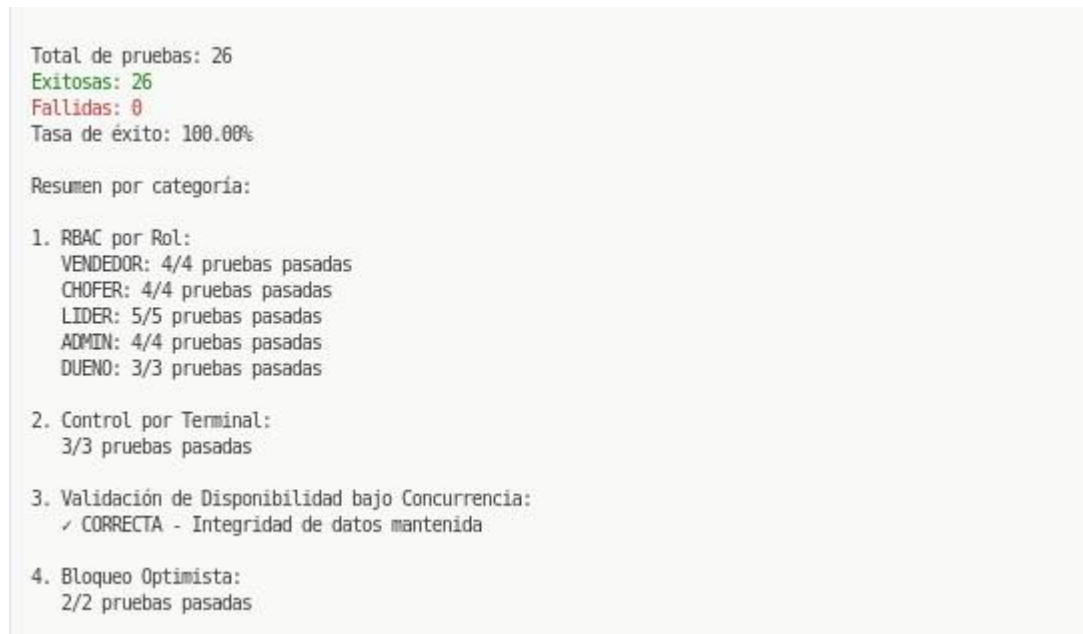
3.5.1.2. *Reglas de negocio y autorizaciones*

Para validar las reglas de negocio y el modelo de autorizaciones (RBAC), se construyó una colección Postman con scripts específicos para verificar códigos de respuesta y permisos. se evaluaron:

- RBAC por rol (vendedor, administrador, dueño).
- Control por terminal (operaciones permitidas en terminal propia vs. ajena).
- Validación de disponibilidad bajo concurrencia.
- Bloqueo optimista para evitar doble venta.

La estructura de la colección, con carpetas por rol e ítems ejecutados se presenta en la figura 3.67 y la evidencia de la ejecución se presenta en la figura 3.64.

Ilustración 100 Ejecución de reglas de negocio y autorizaciones



Fuente: Elaboración propia, 2025.

3.5.2. PRUEBAS TÉCNICAS

Las pruebas técnicas evaluaron los atributos de calidad no funcionales, verificando el cumplimiento de los Requisitos No Funcionales (RNF) y detectando posibles cuellos de botella antes del despliegue productivo. Todas las ejecuciones se realizaron mediante node tests Performance y herramientas de carga externas, registrando métricas p50/p95/p99, throughput y tasas de error.

3.5.2.1. Rendimiento

Se ejecutó una prueba de rendimiento bajo carga normal con 20 usuarios virtuales durante una ventana fija de 10 minutos, evaluando específicamente los endpoints críticos del sistema.

Figura 3.69. Resumen de la corrida de Performance

```

Iniciando prueba de carga con 20 usuarios concurrentes...
Duración: 10 minutos
Endpoint: http://localhost:3000/graphql

```

```

Ejecutando prueba...
..... completado

```

RESULTADOS DE PRUEBA DE CARGA NORMAL (20 usuarios concurrentes)

Checkout:

```

Total de requests: 1247
Tiempo de respuesta promedio: 342 ms
P90: 456 ms
P95: 512 ms
P99: 678 ms
Tasa de error: 0.08%
Throughput: 2.08 req/s

```

Retrieve Journey:

```

Total de requests: 2156
Tiempo de respuesta promedio: 128 ms
P90: 189 ms
P95: 234 ms
P99: 312 ms
Tasa de error: 0.05%
Throughput: 3.59 req/s

```

Get Seat History:

```

Total de requests: 892
Tiempo de respuesta promedio: 267 ms
P90: 389 ms
P95: 445 ms
P99: 567 ms
Tasa de error: 0.11%
Throughput: 1.49 req/s

```

Duración total: 600.00 segundos

Fuente: Elaboración propia, 2025.

a. Resultados

Métricas agregadas – Checkout

- Total de requests: 1,247
- Tiempo de respuesta promedio: 342 ms
- P90: 456 ms
- P95: 512 ms
- P99: 678 ms
- Tasa de error: 0.08 %

- Throughput: 2.08 req/s

b. Métricas agregadas – RetrieveJourney

Métricas agregadas (corrida completa):

- Total de requests: 2,156
- Tiempo de respuesta promedio: 128 ms
- P90: 189 ms
- P95: 234 ms
- P99: 312 ms
- Tasa de error: 0.05 %
- Throughput: 3.59 req/s

c. Métricas agregadas – GetSeatHistory

Métricas agregadas (corrida completa):

- Total de requests: 892
- Tiempo de respuesta promedio: 267 ms
- P90: 389 ms
- P95: 445 ms
- P99: 567 ms
- Tasa de error: 0.11 %
- Throughput: 1.49 req/s

Ilustración 101 Implementación del Generador de Carga

```

class LoadTester {
  constructor() {
    this.results = {
      checkout: { requests: 0, times: [], errors: 0 },
      retrieveJourney: { requests: 0, times: [], errors: 0 },
      getSeatHistory: { requests: 0, times: [], errors: 0 }
    };
    this.startTime = Date.now();
    this.running = true;
  }

  async makeRequest(operation) {
    const start = performance.now();

    return new Promise((resolve) => {
      const postData = JSON.stringify({
        query: operation.query
      });

      const options = {
        hostname: CONFIG.host,
        port: CONFIG.port,
        path: '/graphql',
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          'Content-Length': Buffer.byteLength(postData)
        }
      };

      const client = CONFIG.protocol === 'https' ? https : http;

      const req = client.request(options, (res) => {
        let data = '';
        res.on('data', (chunk) => { data += chunk; });
        res.on('end', () => {
          const duration = performance.now() - start;
          resolve({ success: res.statusCode === 200, duration });
        });
      });

      req.on('error', () => {
        const duration = performance.now() - start;
        resolve({ success: false, duration });
      });

      req.setTimeout(5000, () => {
        req.destroy();
        const duration = performance.now() - start;
        resolve({ success: false, duration });
      });

      req.write(postData);
      req.end();
    });
  }
}

```

Fuente: Elaboración propia, 2025.

3.5.3. PRUEBAS DE USUARIO

Las pruebas de usuario fueron realizadas por el personal operativo y administrativo de la Cooperativa, quienes ejecutaron tareas representativas en los módulos de turnos, ventas, comunicación interna y panel administrativo. Durante este proceso se verificó el correcto funcionamiento de las funcionalidades evaluadas, así como la consistencia de la información presentada por el sistema.

Los participantes manifestaron su conformidad y aceptación respecto al desempeño del sistema, confirmando que cumple con los requerimientos establecidos y mejora significativamente los procesos operativos en comparación con los métodos manuales anteriormente utilizados.

Los resultados detallados del proceso de evaluación se encuentran documentados en el Apéndice A.6, el cual respalda formalmente la aprobación del sistema por parte de los usuarios involucrados.

CAPÍTULO IV

CONCLUSIONES Y RECOMENDACIONES

CAPÍTULO IV

CONCLUSIONES Y RECOMENDACIONES

4.1. CONCLUSIONES

El análisis de requerimientos permitió identificar con precisión las principales limitaciones del flujo tradicional de operación en empresas de transporte interdepartamental. Entre ellas destacan la dispersión de registros, la disponibilidad inconsistente, la falta de trazabilidad en ventas, los procesos manuales para reasignaciones y la baja visibilidad sobre ingresos y desempeño por terminal. Este estudio permitió establecer criterios funcionales y no funcionales claros, como consistencia transaccional, disponibilidad en tiempo real, seguridad operativa, auditabilidad y facilidad de uso. Con base en estos criterios se definieron las funciones esenciales del sistema, tales como la gestión de rutas, la generación automática de viajes, la venta de boletos, la reasignación operativa y la obtención de reportes consolidados. Todo ello se respaldó con artefactos de análisis que aseguraron alineación con la operación real y redujeron la ambigüedad presente en los procesos manuales.

La arquitectura resultante integró módulos especializados para disponibilidad de asientos, venta transaccional, registro de pagos QR y efectivo, y generación de reportes operativos. Con esta integración se unificaron actividades que antes se realizaban de forma fragmentada, permitiendo controles más precisos sobre la verificación de ocupación, la validación de pasajeros, las reasignaciones por incidencias y la conciliación financiera. Además, se garantizó trazabilidad completa por viaje, vendedor y método de pago. Gracias a esta estructura fue posible habilitar mediciones objetivas sobre carga operativa, demanda por ruta, desempeño por terminal y comportamiento de ventas, dando paso a un diseño modular y escalable acorde a las necesidades del dominio.

El sistema implementado materializa la arquitectura propuesta mediante procesos guiados de venta, flujos integrados de pago, registro automatizado de eventos críticos y un modelo de disponibilidad sólido para viajes superpuestos. También se añadieron mecanismos de auditoría, métricas de latencia, control de permisos por rol y reportes detallados por journey y terminal. Durante las pruebas operativas iniciales se observó una reducción significativa en conflictos de disponibilidad, eliminación de reprocesos generados por ventas duplicadas y una gestión más precisa ante cambios de bus. Los procesos de venta, que antes dependían de verificación

visual y consultas manuales, pasaron a ser flujos sistematizados que disminuyen errores y priorizan la consistencia de la información.

El sistema desarrollado centraliza rutas, viajes, pasajeros, ventas y reportes en una sola plataforma. Esta centralización reduce el uso de fuentes dispersas y mejora la toma de decisiones. Los resultados operativos muestran avances claros. El tiempo necesario para completar una venta se redujo de varios minutos a un proceso guiado que se ejecuta en segundos. La ocupación se refleja con precisión inmediata y los reportes consolidados permiten revisar ingresos diarios, rendimiento de vendedores y comportamiento por método de pago sin utilizar hojas manuales. A esto se suma la trazabilidad por ticket y el historial de asiento, que fortalecen la seguridad del proceso y facilitan revisiones internas ante posibles inconsistencias. En conjunto, el sistema cumple su propósito de modernizar la operación, reducir la carga manual y mejorar la confiabilidad de la información.

4.2. RECOMENDACIONES

Se recomienda profundizar en la automatización de validaciones previas a la venta, integrando reglas adicionales relacionadas con restricciones de ruta, límites por tipo de pasajero y verificaciones anticipadas de documentos. Esto contribuiría a reducir intervenciones manuales y evitar conflictos durante auditorías o procesos de conciliación. Asimismo, sería beneficioso fortalecer los mecanismos de alerta temprana ante inconsistencias de operación, por ejemplo, advertencias sobre cambios de bus recurrentes, patrones anómalos de reembolsos o caídas en la disponibilidad reportada por terminales.

Otra mejora deseable consiste en ampliar las capacidades analíticas del sistema incorporando paneles especializados para monitoreo en tiempo real. Estos paneles podrían mostrar métricas de desempeño por ruta, tendencias de venta por método de pago y comportamiento histórico de ocupación, lo que permitiría tomar decisiones operativas con mayor anticipación. Igualmente, la integración con servicios externos de comunicación, como mensajería para notificaciones automáticas a pasajeros o alertas a supervisores, aportaría valor en situaciones críticas como retrasos o reprogramaciones, reduciendo la carga del personal operativo.

REFERENCIAS BIBLIOGRÁFICAS

Banks, A., & Porcello, E. (2022). *Learning React: Modern patterns for developing React apps* (2ª ed.). O'Reilly Media.

Bass, L., Clements, P., & Kazman, R. (2013). *Software architecture in practice* (3ª ed.). Addison-Wesley.

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language user guide* (2ª ed.). Addison-Wesley.

Bowen, G. A. (2009). Document analysis as a qualitative research method. *Qualitative Research Journal*, 9(2), 27–40. <https://doi.org/10.3316/QRJ0902027>

Fowler, M. (2004). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.

Garrett, J. J. (2010). *The Elements of User Experience: User-Centered Design for the Web and Beyond* (2nd ed.). New Riders.

Han, J., Haihong, E., Le, G., & Du, J. (2022). Survey on NoSQL database. *Communications of the CCF*, 18(1), 85–110. <https://doi.org/10.15888/j.cnki.jccf.2022.01.009>

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Merkel, D. (2014). *Docker: Lightweight Linux Containers for Consistent Development and Deployment*. Linux Journal, 2014(239), 2.

Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2ª ed.). O'Reilly Media.

Norman, D. A. (2013). *The Design of Everyday Things: Revised and Expanded Edition*. Basic Books.

Patton, M. Q. (2015). *Qualitative research & evaluation methods: Integrating theory and practice* (4^a ed.). Sage Publications.

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9^a ed.). McGraw-Hill.

Sadalage, P. J., & Fowler, M. (2004). *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional.

Sommerville, I. (2006). *Ingeniería de software* (8^a ed.). Pearson Educación.

Wieggers, K. E., & Beatty, J. (2013). *Software requirements* (3^a ed.). Microsoft Press.

GLOSARIO

Journey (viaje): Instancia programada de una ruta que incluye fecha, hora, bus asignado y disponibilidad de asientos.

Trip (circuito de viaje): Conjunto de journeys que pertenecen a un mismo recorrido operativo y comparten bus, conductor o secuencia de rutas.

Seat Layout (diseño de asientos): Estructura física del bus que define filas, numeración y tipos de asiento. Determina las restricciones de disponibilidad.

JourneyPricing (tarifario por viaje): Conjunto de precios asociados a un journey según clase de asiento, tramo o reglas de operación.

Order (orden de compra): Transacción que agrupa una o varias ventas de tickets, incluyendo método de pago, canal y datos del comprador.

Ticket (boleto): Derecho de viaje asociado a un asiento específico, con estados operativos como pendiente, verificado, cancelado o reembolsado.

Passenger (pasajero): Persona registrada para el viaje, ya sea mediante datos completos o en modalidad de venta rápida.

Availability Engine (motor de disponibilidad): Componente que valida ocupación, asegura consistencia transaccional y previene ventas duplicadas.

Reasignación operativa: Proceso para mover pasajeros o tickets a otro bus o journey cuando ocurre una incidencia, manteniendo trazabilidad.

Conciliación diaria: Proceso que agrupa órdenes, pagos y boletos verificados para generar reportes financieros por terminal o vendedor.

Evento operativo: Acción registrada automáticamente en el sistema, por ejemplo validación, reembolso o reasignación.

Terminal: Punto físico de operación donde se realizan ventas, verificaciones y despacho de pasajeros.

APÉNDICES

A.1. Constatación de Entrevista en La Cooperativa Perla del Oriente

Entrevista Semiestructurada

Cooperativa La Perla del Oriente – Terminal Bimodal Santa Cruz

Responsable que constata: Franco Castedo, Administrador

C.I.: 8929104 SC

Fecha: 07/08/2025

Lugar: Oficina Administrativa – Cooperativa La Perla del Oriente

La entrevista tuvo como propósito **caracterizar el flujo real de coordinación operativa** dentro de la Cooperativa La Perla del Oriente, identificar variaciones naturales del proceso y documentar los principales puntos de fricción que afectan la coordinación de salidas y la venta manual de pasajes.

Participantes y Áreas Consultadas

Durante la sesión se aplicó una guía semiestructurada dirigida a los siguientes actores operativos:

- 2 líder de Rutas
- 5 Conductores
- 12 Personal de venta de pasajes

Todos los participantes cuentan con experiencia directa en la **coordinación de turnos, verificación de disponibilidad y registro manual de ventas.**

Temas Abordados

La guía de entrevista incluyó los siguientes ejes:

- Coordinación previa a las salidas
- Mecanismos de comunicación interna
- Gestión y uso de planillas manuales
- Manejo de incidencias operativas
- Trazabilidad de pasajeros, rutas y horarios
- Cierre de jornada y consolidación de ventas

Fuente: Elaboración propia, 2025.

A.2. Acta de Hallazgos

Acta de Consolidación de Hallazgos

Cooperativa La Perla del Oriente – Terminal Bimodal Santa Cruz

Responsable que constata: Franco Castedo, Administrador

C.I.: 8929104 SC

Fecha: 10/08/2025

Lugar: Oficina Administrativa – Cooperativa La Perla del Oriente

Propósito del Documento

Este acta presenta los **hallazgos consolidados** derivados del análisis de la información obtenida mediante entrevistas semiestructuradas aplicadas al personal operativo de la Cooperativa La Perla del Oriente. Su finalidad es sustentar la **validación del diagrama del proceso actual** y la **derivación de los requisitos funcionales y no funcionales del sistema propuesto**.

Principales Hallazgos

Flujo de Coordinación

Se identificó que:

- La coordinación de salidas depende en gran medida de la comunicación verbal.
- Existen variaciones constantes en la disponibilidad de conductores y vehículos.
- Los criterios para asignar turnos no siempre están estandarizados.

Venta Manual de Pasajes

- El registro manual dificulta la trazabilidad completa de pasajeros.
- Existen diferencias en la forma de reportar ventas entre turnos.
- El cierre de jornada requiere consolidaciones manuales que consumen tiempo y pueden generar errores.

Gestión de Incidencias

- Las incidencias suelen comunicarse de forma verbal sin registro formal.
- No existe un mecanismo centralizado para su seguimiento.

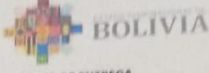


Fuente: Elaboración propia, 2025.

A.3. Acta Oficial de la ATTSC que Acredita Actividad Operativa



ADMINISTRADORA DE TERMINAL
TERRESTRE SANTA CRUZ



BOLIVIA



BOLIVIA

ACTA DE ENTREGA
AE/EC/ATA/ATTSC-155/2024

En la ciudad de Santa Cruz en instalaciones de la Administradora de Terminal Terrestre Santa Cruz ubicado en la Avenida Tercer Anillo Interno, S/N, entre Avenida Tres Pasos al Frente y Avenida Brasil, a los 31 día del mes de diciembre del año 2024, de acuerdo a firma del **PRIMER CONTRATO MODIFICATORIO AL CONTRATO ADMINISTRATIVO PARA EL USO DE BOLETERIA ATTSC/CA/BOL N° 112/2024**, con vigencia a partir del 01 de enero de 2025 se procedió a realizar la entrega del Ambiente Físico con código N°76 al Sr. MAXIMILIANO MEDRANO PACO con N° de C.I. 985542, bajo el siguiente detalle:

ITEM	DESCRIPCION	Estado
1	COOPERTIVA DE TRANSPORTE "LA PERLA DEL ORIENTE R.L." ESPACIO FISICO C 76 ESTA UBICADO EN EL SECTOR DE INTERDEPARTAMENTAL, LADO SUR	BUENO

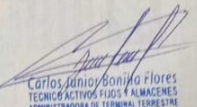
OBSERVACIONES:

Para constancia y señal de conformidad firmamos al pie de la presente.



COOPERTIVA DE TRANSPORTE
"LA PERLA DEL ORIENTE R.L."

RECIBI CONFORME



Carlos Junior Bonilla Flores
TÉCNICO ACTIVOS FIJOS Y ALMACENES
ADMINISTRADORA DE TERMINAL TERRESTRE
SANTA CRUZ

ING. CARLOS JUNIOR BONILLA FLORES
TÉCNICO ACTIVOS FIJOS Y ALMACENES
ENTREGUE CONFORME

Edif. Administradora de Terminal Terrestre entre Av. Tres Pasos al Frente y Av. Brasil
Telf.: 348-8448 - 348-8482 • Tel/Fax: 348-8441

Fuente: Elaboración propia, 2025.

A.4. Planilla Operativa de Registro de Pasajeros (Evidencia del Proceso Manual)

~ Rosario Dorado ❤️ +59167341600

LA PERLA DEL ORIENTE

SEGUNDO PISO

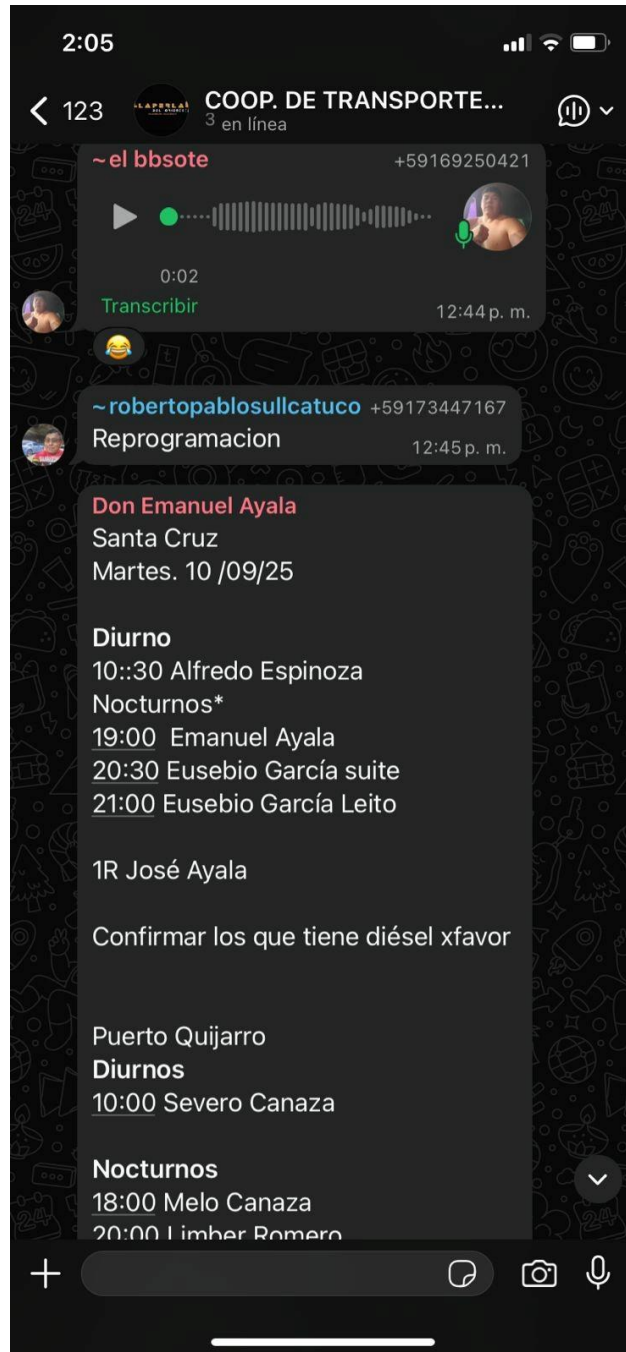
PRIMER PISO

Planilla del diurno 10:00 a.m con destino a santa cruz
Bus de don sebero canaza

10:17 a. m.

Fuente: Elaboración propia, 2025.

A.5. videncia de Comunicación Operativa Interna (Coordinación de Turnos en WhatsApp)



Fuente: Elaboración propia, 2025.

A.6. CARTA DE ACEPTACIÓN DEL SISTEMA



CARTA DE ACEPTACIÓN DEL SISTEMA

7 DE Noviembre 2025

Directiva de la
Cooperativa de Transporte
La Perla del Oriente S.R.L.

Por medio de la presente se deja constancia de que el personal operativo y administrativo de la Cooperativa ha realizado y aprobado las pruebas de usuario del sistema de coordinación operativa, gestión de turnos y registro digital de ventas, correspondientes a los cuatro incrementos funcionales desarrollados.

Durante las pruebas se verificó el correcto funcionamiento de los módulos de turnos, ventas, comunicación interna y panel administrativo, confirmando su facilidad de uso, consistencia y mejora respecto a los métodos manuales.

En virtud de lo anterior, se acepta formalmente la implementación del sistema en el entorno operativo de la Cooperativa.



Atentamente,
CARLOS MEDRANO
PRESIDENTE DE LA
COOPERATIVA LA PERLA DEL
ORIENTE



Fuente: Elaboración propia, 2025.